

Principles and Practices of Programming Languages

Principles and Practices of Programming Languages

Brian Wyvill

ETP
ED - TECH PRESS
www.edtechpress.co.uk



ED - TECH PRESS

www.edtechpress.co.uk

Published by ED-Tech Press,
71-75 Shelton Street, Covent Garden
London WC2H9JQ
United Kingdom

© 2025 by ED-Tech Press

Principles and Practices of Programming Languages
Brian Wyvill

ISBN: 978-1-83535-037-9

All rights reserved. No part of this publication may be reproduced, stored in retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd., Saffron House, 6-10 Kirby Street, London EC1N 8TS.

Trademark Notice: All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

Unless otherwise indicated herein, any third-party trademarks that may appear in this work are the property of their respective owners and any references to third-party trademarks, logos or other trade dress are for demonstrative or descriptive purposes only. Such references are not intended to imply any sponsorship, endorsement, authorization, or promotion of ED-Tech products by the owners of such marks, or any relationship between the owner and ED-Tech Press or its affiliates, authors, licensees or distributors.

British Library Cataloguing in Publication Data.
A catalogue record for this book is available from the British Library.

For more information regarding ED-Tech Press and its products, please visit the publisher's website
www.edtechpress.co.uk

TABLE OF CONTENTS

Preface

xiii

Chapter 1	Introduction	1
	Python Programming Language	1
	Programme	3
	Debugging	3
	Formal and Natural Languages	5
	Glossary	7
	Fruitful Functions	8
	Return Values	8
	Composition	9
	Boolean Functions	10
	More Recursion	10
	Leap of Faith	12
	Checking Types	12
	Conditionals and Recursion	13
	The Modulus Operator	13
	Boolean Expressions	14
	Logical Operators	14
	Conditional Execution	15
	Alternative Execution	15
	Chained Conditionals	16
	Nested Conditionals	17
	The Return Statement	17
	Recursion	18

Stack Diagrams for Recursive Functions	19
Infinite Recursion	19
Keyboard Input	20
Chapter 2 C Language Programming	21
Introduction	21
Programming Examples	22
First Example	22
Second Example	26
Statement and Control	27
Expression Statements	28
If Statements	29
Programme Structure	31
Chapter 3 Object Oriented Programming	35
Object Entity	35
Abstraction	35
Encapsulation	36
Inheritance	36
Polymorphism	36
Example	37
Variables	39
Annotation	40
Initialization	40
Objects	41
Object-Orientation	46
Types of Typing	47
Generic Classes	48
Inheritance	49
Feature Renaming	51
Method Overloading	51
Operator Overloading	51
High Order Functions	52
Language Implementation	53
Normal Access	54
Class Variables/Methods	54
Reflection	54
Access Control	55
Features of DBC	56
Object-Oriented Language Features	56
Why Object-Oriented Database	56
Strategy for Defining Immutable Objects	57
High Level Concurrency Objects	58

Lock Objects	59
Executors	61
Executor Interfaces	61
Thread Pools	62
Fork/Join	63
Multithreading	65
Regular Expressions	66
Pointer Arithmetic	66
Language Integration	66
Built-In Security	67
Capers Jones Language Level	67
Chapter 4 Functional Programming	68
Code Fragmentation	68
Flow of Execution	69
Parameters and Arguments	69
Variables and Parameters are Local	70
Stack Diagrams	71
Functions with Results	72
Statements	72
Evaluating Expressions	72
Operators and Operands	73
Order of Operations	74
Operations on Strings	74
Composition	75
Function Prototypes	75
Function Philosophy	76
Separate Compilation Logistics	77
Reading Lines	79
Reading Numbers	81
Formatted I/O	82
Output: Printf Family	82
Input Scanf	85
I/O Model	87
Text Streams	88
Binary Streams	88
The Stdio.h Header Dile	88
Functions of Streams	89
Opening	89
Closing	90
Buffering	90
Direct File Manipulation	90
Freopen	91

Closing Files	91
Setbuf, Setvbuf	92
Fflush	92
Chapter 5 Variables and Expressions	94
Values and Types	94
Variables	95
Variable Names and Keywords	96
Iteration	96
Multiple Assignment	96
The while Statement	97
Two-Dimensional Tables	99
Encapsulation and Generalization	99
More Encapsulation	100
Local Variables	100
More Generalization	101
Functions	103
Function Calls	103
Type Conversion	103
Type Coercion	104
Math Functions	104
Composition	105
Adding New Functions	105
Chapter 6 Structure of Programming	108
Identifiers	111
Variables	114
Strings	115
Comments	154
Variables	154
Scope of Variables	155
Initialization of Variables	155
Chapter 7 Statements and Language Programming	157
Characters	160
Strings	161
Names	161
Simple Input/Output	162
Annotation	162
Comments	163
Memory	164
Defined Data Types (Typedef)	165

Enumerations (Enum)	167
Classes (I)	168
Operators and Expressions	169
Arithmetic Operators	169
Relational Operators	170
Logical Operators	171
Bitwise Operators	172
Increment/Decrement Operators	173
Programme	173
The Argument Vector	173
Environment Variables	174
Special Library Functions and Macros	175
Checking Character	175
Character Identification	175
String Manipulation	178
Mathematical Functions	180
Maths Errors	182
Hidden Operators and Values	184
Expressions	184
Extended and Hidden =	185
Hidden ++ and –	186
Arrays, Strings and Hidden Operators	187
Cautions about Style	188
Data Types	190
Special Constant Expressions	190
Machine Level Operations	193
Bits and Bytes	193
Bit Patterns	193
Flags, Registers and Messages	193
Bit Operators d Assignments	194
Bit Operators	194
Shift Operations	195
Truth Tables and Masking	196
Files and Devices	199
Files Generally	199
File Positions	200
High Level File Handling Functions	200
Opening Files	201
Closing a File	202
Single Character I/O	203
Printer Output	205
Low Level Filing Operations	206
File Descriptors	206

Chapter 8	Integer Data Type	211
	Float Data Type	212
	Assignment Operator	212
	Conditional Operator	213
	Comma Operator	214
	Size of Operator	214
	Operator Precedence	215
	Simple Type Conversion	215
	Data Types and Operators	215
	Types	216
	Constants	217
	Declarations	218
	Variable Names	219
	Arithmetic Operators	220
	Assignment Operators	220
	Function Calls	221
	Constructors and Destructors	222
	Classes (II)	227
	Friendship and Inheritance	231
Chapter 9	Computer Programmes and Programming Languages	259
	Development of Computers	260
	Analog Computers	260
	Computer Programming	261
	Definition	261
	Overview	261
	History	262
	Modern Programming	264
	Programming Language	266
	Definitions	267
	Elements	268
	Design and Implementation	273
	Usage	274
	Taxonomies	276
	History	276
	History of Programming Languages	278
	Before 1940	278
	The 1940s	279
	The 1950s and 1960s	280
	1967-1978: Establishing Fundamental Paradigms	281
	The 1980s: Consolidation, Modules and Performance	282
	The 1990s: The Internet Age	283
	Current Trends	284

Prominent People in the History of Programming Languages	284
Programming Language Generations	285
Historical View of First Three Generations	285
Re-characterization of First Three Generations	287
Later Generations	288
<i>Bibliography</i>	289
<i>Index</i>	292

Preface

Using computer programming languages, we can communicate with a computer in a language that it can understand. There are numerous human-based languages, and programmers that can use a variety of computer programming languages to connect with a computer. A "binary" is the part of the language that a computer can comprehend. Compiling is the process of turning a programming language into a binary format. Though there are frequently similarities between programming languages, each language—from C to Python—has its own unique features.

A programming language is a made-up language used to convey calculations that a machine, especially a computer, can carry out. Programming languages can be used to build scripts that direct a machine's operation, precisely express algorithms, or as a mode of human communication. The earliest programming languages were used to control the behaviour of devices like Jacquard looms and player pianos before computers were even created. Programming languages have been developed into thousands of distinct varieties, mostly in the computer industry, and new ones are continually being developed.

One description of C is "high-level assembly language." Although some people mistakenly believe that to be an insult, it is actually a planned and important linguistic feature. If you have experience writing assembly language programmes, you will likely find C to be quite natural and pleasant (although if you continue to place a lot of emphasis on machine-level details, you'll likely produce programmes that are excessively nonportable). You may find C's lack of some higher-level features frustrating if you haven't programmed in assembly language.

This book is appropriate for an advanced undergraduate or introductory graduate course on programming language concepts. Instead of being divided into several languages, it is organised around concepts and paradigms.

Author

Introduction

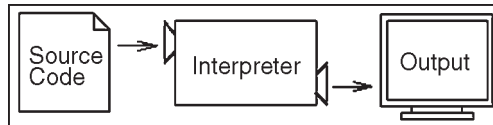
The single most important skill for a computer scientist is problem solving. Problem solving means the ability to formulate problems, think creatively about solutions, and express a solution clearly and accurately. As it turns out, the process of learning to programme is an excellent opportunity to practice problem-solving skills. That's why this chapter is called, "The way of the programme." On one level, you will be learning to programme, a useful skill by itself. On another level, you will use programming as a means to an end.

PYTHON PROGRAMMING LANGUAGE

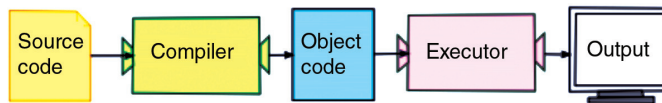
The programming language you will be learning is Python. Python is an example of a high-level language; other high-level languages you might have heard of are C, C++, Perl, and Java. As you might infer from the name "high-level language," there are also low-level languages, sometimes referred to as "machine languages" or "assembly languages." Loosely speaking, computers can only execute programmes written in low-level languages. Thus, programmes written in a high-level language have to be processed before they can run. This extra processing takes some time, which is a small disadvantage of high-level languages.

But the advantages are enormous. First, it is much easier to programme in a high-level language. Programmes written in a high-level language take less time to write, they are shorter and easier to read, and they are more likely to be correct. Second, high-level languages are portable, meaning that they can run on different kinds of computers with

few or no modifications. Low-level programmes can run on only one kind of computer and have to be rewritten to run on another. Due to these advantages, almost all programmes are written in high-level languages. Low-level languages are used only for a few specialized applications. Two kinds of programmes process high-level languages into low-level languages: Interpreters and compilers. An interpreter reads a high-level programme and executes it, meaning that it does what the programme says. It processes the programme a little at a time, alternately reading lines and performing computations.



A compiler reads the programme and translates it completely before the programme starts running. In this case, the high-level programme is called the source code, and the translated programme is called the object code or the executable. Once a programme is compiled, you can execute it repeatedly without further translation.



Many modern languages use both processes: They are first compiled into a lower level language, called byte code, and then interpreted by a program called a virtual machine. Python uses both processes, but because of the way programmers interact with it, it is usually considered an interpreted language.

There are two ways to use the Python interpreter: Shell mode and script mode. In shell mode, you type Python statements into the Python shell and the interpreter immediately prints the result:

```

$ python
Python 2.4.1 (#1, Apr 29 2005, 00:28:56) Type "help", "copyright",
"credits" or "license" for more information.
>>> print 1 + 1
2
  
```

The first line of this example is the command that starts the Python interpreter. The next two lines are messages from the interpreter. The third line starts with `>>>`, which is the prompt the interpreter uses to indicate that it is ready. We typed `print 1 + 1`, and the interpreter replied 2. Alternatively, you can write a programme in a file and use the interpreter to execute the contents of the file. Such a file is called a script.

For example, we used a text editor to create a file named `latoya.py` with the following contents:

```
print 1 + 1
```

By convention, files that contain Python programmes have names that end with `.py`.

To execute the programme, we have to tell the interpreter the name of the script:

```
$ python latoya.py
2
```


In other development environments, the details of executing programmes may differ. Also, most programmes are more interesting than this one. Most of the examples in this book are executed on the command line. Working on the command line is convenient for programme development and testing, because you can type programmes and execute them immediately. Once you have a working programme, you should store it in a script so you can execute or modify it in the future.

PROGRAMME

A programme is a sequence of instructions that specifies how to perform a computation. The computation might be something mathematical, such as solving a system of equations or finding the roots of a polynomial, but it can also be a symbolic computation, such as searching and replacing text in a document or (strangely enough) compiling a programme. The details look different in different languages, but a few basic instructions appear in just about every language:

- *Input*: Get data from the keyboard, a file, or some other device.
- *Output*: Display data on the screen or send data to a file or other device.
- *Math*: Perform basic mathematical operations like addition and multiplication.
- *Conditional execution*: Check for certain conditions and execute the appropriate sequence of statements.
- *Repetition*: Perform some action repeatedly, usually with some variation.

Believe it or not, that's pretty much all there is to it. Every programme you've ever used, no matter how complicated, is made up of instructions that look more or less like these. Thus, we can describe programming as the process of breaking a large, complex task into smaller and smaller subtasks until the subtasks are simple enough to be performed with one of these basic instructions.

DEBUGGING

Programming is a complex process, and because it is done by human beings, it often leads to errors. For whimsical reasons, programming errors are called bugs and the process of tracking them down and correcting them is called debugging. Three kinds of errors can occur in a programme: Syntax errors, runtime errors, and semantic errors. It is useful to distinguish between them in order to track them down more quickly.

Syntax Errors

Python can only execute a programme if the programme is syntactically correct; otherwise, the process fails and returns an error message. Syntax refers to the structure of a programme and the rules about that structure. For example, in English, a sentence must begin with a capital letter and end with a period. This sentence contains a syntax error. So does this one

For most readers, a few syntax errors are not a significant problem, which is why we can read the poetry of e. e. cummings without spewing error messages. Python is not so forgiving. If there is a single syntax error anywhere in your programme, Python will print an error message and quit, and you will not be able to run your programme. During the first few weeks of your programming career, you will probably spend a lot of time tracking down syntax errors. As you gain experience, though, you will make fewer errors and find them faster.

Runtime Errors

The second type of error is a runtime error, so called because the error does not appear until you run the programme. These errors are also called exceptions because they usually indicate that something exceptional (and bad) has happened. Runtime errors are rare in the simple programmes you will see in the first few chapters, so it might be a while before you encounter one.

Semantic Errors

The third type of error is the semantic error. If there is a semantic error in your programme, it will run successfully, in the sense that the computer will not generate any error messages, but it will not do the right thing. It will do something else. Specifically, it will do what you told it to do. The problem is that the programme you wrote is not the programme you wanted to write. The meaning of the programme (its semantics) is wrong. Identifying semantic errors can be tricky because it requires you to work backward by looking at the output of the programme and trying to figure out what it is doing.

Experimental Debugging

One of the most important skills you will acquire is debugging. Although it can be frustrating, debugging is one of the most intellectually rich, challenging, and interesting parts of programming. In some ways, debugging is like detective work. You are confronted with clues, and you have to infer the processes and events that led to the results you see. Debugging is also like an experimental science. Once you have an idea what is going wrong, you modify your programme and try again. If your hypothesis was correct, then you can predict the result of the modification, and you take a step closer to a working programme. If your hypothesis was wrong, you have to come up with a new one. As Sherlock Holmes pointed out, “When you have eliminated the impossible, whatever remains, however improbable, must be the truth.” (A. Conan Doyle, *The Sign of Four*).

For some people, programming and debugging are the same thing. That is, programming is the process of gradually debugging a programme until it does what you want. The idea is that you should start with a programme that does *something* and make small modifications, debugging them as you go, so that you always have a working

programme. For example, Linux is an operating system that contains thousands of lines of code, but it started out as a simple programme Linus Torvalds used to explore the Intel 80386 chip. According to Larry Greenfield, “One of Linus’s earlier projects was a programme that would switch between printing AAAA and BBBB.

FORMAL AND NATURAL LANGUAGES

Natural languages are the languages that people speak, such as English, Spanish, and French. They were not designed by people (although people try to impose some order on them); they evolved naturally.

Formal languages are languages that are designed by people for specific applications. For example, the notation that mathematicians use is a formal language that is particularly good at denoting relationships among numbers and symbols. Chemists use a formal language to represent the chemical structure of molecules. And most importantly:

Programming languages are formal languages that have been designed to express computations. Formal languages tend to have strict rules about syntax. For example, $3+3=6$ is a syntactically correct mathematical statement, but $3=+6\$$ is not. H_2O is a syntactically correct chemical name, but ${}_2\text{Zz}$ is not. Syntax rules come in two flavours, pertaining to tokens and structure. Tokens are the basic elements of the language, such as words, numbers, and chemical elements. One of the problems with $3=+6\$$ is that $\$$ is not a legal token in mathematics (at least as far as we know). Similarly, ${}_2\text{Zz}$ is not legal because there is no element with the abbreviation Zz .

The second type of syntax error pertains to the structure of a statement that is, the way the tokens are arranged. The statement $3=+6\$$ is structurally illegal because you can’t place a plus sign immediately after an equal sign. Similarly, molecular formulas have to have subscripts after the element name, not before.

As an exercise, create what appears to be a well-structured English sentence with unrecognizable tokens in it. Then write another sentence with all valid tokens but with invalid structure. When you read a sentence in English or a statement in a formal language, you have to figure out what the structure of the sentence is (although in a natural language you do this subconsciously). This process is called parsing.

For example, when you hear the sentence, “The other shoe fell,” you understand that “the other shoe” is the subject and “fell” is the predicate. Once you have parsed a sentence, you can figure out what it means, or the semantics of the sentence. Assuming that you know what a shoe is and what it means to fall, you will understand the general implication of this sentence. Although formal and natural languages have many features in common tokens, structure, syntax, and semantics there are many differences:

Ambiguity

Natural languages are full of ambiguity, which people deal with by using contextual clues and other information. Formal languages are designed to be nearly or completely

unambiguous, which means that any statement has exactly one meaning, regardless of context.

Redundancy

In order to make up for ambiguity and reduce misunderstandings, natural languages employ lots of redundancy. As a result, they are often verbose. Formal languages are less redundant and more concise.

Literalness

Natural languages are full of idiom and metaphor. If I say, “The other shoe fell,” there is probably no shoe and nothing falling. Formal languages mean exactly what they say. People who grow up speaking a natural language everyone often have a hard time adjusting to formal languages. In some ways, the difference between formal and natural language is like the difference between poetry and prose, but more so:

Poetry

Words are used for their sounds as well as for their meaning, and the whole poem together creates an effect or emotional response. Ambiguity is not only common but often deliberate.

Prose

The literal meaning of words is more important, and the structure contributes more meaning. Prose is more amenable to analysis than poetry but still often ambiguous.

Programmes

The meaning of a computer programme is unambiguous and literal, and can be understood entirely by analysis of the tokens and structure. Here are some suggestions for reading programmes (and other formal languages). First, remember that formal languages are much more dense than natural languages, so it takes longer to read them. Also, the structure is very important, so it is usually not a good idea to read from top to bottom, left to right. Instead, learn to parse the programme in your head, identifying the tokens and interpreting the structure. Finally, the details matter. Little things like spelling errors and bad punctuation, which you can get away with in natural languages, can make a big difference in a formal language.

The First Programme

Traditionally, the first programme written in a new language is called “Hello, World!” Because all it does is display the words, “Hello, World!” In Python, it looks like this:

```
print "Hello, World!"
```

This is an example of a print statement, which doesn't actually print anything on paper. It displays a value on the screen. In this case, the result is the words

Hello, World!

The quotation marks in the programme mark the beginning and end of the value; they don't appear in the result. Some people judge the quality of a programming language by the simplicity of the "Hello, World!" Programme. By this standard, Python does about as well as possible.

GLOSSARY

- *Problem solving*: The process of formulating a problem, finding a solution, and expressing the solution.
- *High-level language*: A programming language like Python that is designed to be easy for humans to read and write.
- *low-level language*: A programming language that is designed to be easy for a computer to execute; also called "machine language" or "assembly language."
- *Portability*: A property of a programme that can run on more than one kind of computer.
- *Interpret*: To execute a programme in a high-level language by translating it one line at a time.
- *Compile*: To translate a programme written in a high-level language into a low-level language all at once, in preparation for later execution.
- *Source code*: A programme in a high-level language before being compiled.
- *Object code*: The output of the compiler after it translates the programme.
- *Executable*: Another name for object code that is ready to be executed.
- *Script*: A programme stored in a file (usually one that will be interpreted).
- *Programme*: A set of instructions that specifies a computation.
- *Algorithm*: A general process for solving a category of problems.
- *Bug*: An error in a programme.
- *Debugging*: The process of finding and removing any of the three kinds of programming errors.
- *Syntax*: The structure of a programme.
- *Syntax error*: An error in a programme that makes it impossible to parse (and therefore impossible to interpret).
- *Runtime error*: An error that does not occur until the programme has started to execute but that prevents the programme from continuing.
- *Exception*: Another name for a runtime error.
- *Semantic error*: An error in a programme that makes it do something other than what the programmer intended.
- *Semantics*: The meaning of a programme.
- *Natural language*: Any one of the languages that people speak that evolved naturally.

- *Formal language*: Any one of the languages that people have designed for specific purposes, such as representing mathematical ideas or computer programmes; all programming languages are formal languages.
- *Token*: One of the basic elements of the syntactic structure of a programme, analogous to a word in a natural language.
- *Parse*: To examine a programme and analyse the syntactic structure.
- *Print statement*: An instruction that causes the Python interpreter to display a value on the screen.
- *Warning*: The HTML version of this document is generated from Latex and may contain translation errors. In particular, some mathematical expressions are not translated correctly.

FRUITFUL FUNCTIONS

RETURN VALUES

Some of the built-in functions we have used, such as the math functions, have produced results. Calling the function generates a new value, which we usually assign to a variable or use as part of an expression.

```
e = math.exp(1.0)
height = radius * math.sin(angle)
```

But so far, none of the functions we have written has returned a value. In this chapter, we are going to write functions that return values, which we will call fruitful functions, for want of a better name.

The first example is area, which returns the area of a circle with the given radius:

```
import math
def area(radius):
    temp = math.pi * radius**2
    return temp
```

We have seen the return statement before, but in a fruitful function the return statement includes a return value. This statement means: “Return immediately from this function and use the following expression as a return value.”

The expression provided can be arbitrarily complicated, so we could have written this function more concisely:

```
def area(radius):
    return math.pi * radius**2
```

On the other hand, temporary variables like temp often make debugging easier.

Sometimes it is useful to have multiple return statements, one in each branch of a conditional:

```
def absoluteValue(x):
    if x < 0:
        return -x
    else:
        return x
```

Since these return statements are in an alternative conditional, only one will be executed. As soon as one is executed, the function terminates without executing any subsequent statements.

Code that appears after a return statement, or any other place the flow of execution can never reach, is called dead code. In a fruitful function, it is a good idea to ensure that every possible path through the programme hits a return statement.

For example:

```
def absoluteValue(x):
    if x < 0:
        return -x
    elif x > 0:
        return x
```

This programme is not correct because if x happens to be 0, neither condition is true, and the function ends without hitting a return statement. In this case, the return value is a special value called `None`:

```
print absoluteValue(0)
None
```

As an exercise, write a compare function that returns 1 if $x > y$, 0 if $x == y$, and -1 if $x < y$.

COMPOSITION

As you should expect by now, you can call one function from within another. This ability is called composition. As an example, we'll write a function that takes two points, the centre of the circle and a point on the perimeter, and computes the area of the circle.

Assume that the centre point is stored in the variables `xc` and `yc`, and the perimeter point is in `xp` and `yp`. The first step is to find the radius of the circle, which is the distance between the two points.

Fortunately, there is a function, distance, that does that:

```
radius = distance(xc, yc, xp, yp)
```

The second step is to find the area of a circle with that radius and return it:

```
result = area(radius)
return result
```

Wrapping that up in a function, we get:

```
def area2(xc, yc, xp, yp):
    radius = distance(xc, yc, xp, yp)
    result = area(radius)
    return result
```

We called this function `area 2` to distinguish it from the `area` function defined earlier. There can only be one function with a given name within a given module.

The temporary variables `radius` and `result` are useful for development and debugging, but once the programme is working, we can make it more concise by composing the function calls:

```
def area2(xc, yc, xp, yp):
    return area(distance(xc, yc, xp, yp))
```

As an exercise, write a function `slope(x1, y1, x2, y2)` that returns the slope of the line through the points $(x1, y1)$ and $(x2, y2)$. Then use this function in a function called `intercept(x1, y1, x2, y2)` that returns the y-intercept of the line through the points $(x1, y1)$ and $(x2, y2)$.

BOOLEAN FUNCTIONS

Functions can return boolean values, which is often convenient for hiding complicated tests inside functions.

For example:

```
def isDivisible(x, y):
    if x % y == 0:
        return True
    else:
        return False
```

The name of this function is `isDivisible`. It is common to give boolean functions names that sound like yes/no questions. `isDivisible` returns either `True` or `False` to indicate whether the `x` is or is not divisible by `y`.

We can make the function more concise by taking advantage of the fact that the condition of the `if` statement is itself a boolean expression.

We can return it directly, avoiding the if statement altogether:

```
def isDivisible(x, y):
    return x % y == 0
```

This session shows the new function in action:

```
isDivisible(6, 4)
False
isDivisible(6, 3)
True
```

Boolean functions are often used in conditional statements:

```
if isDivisible(x, y):
    print "x is divisible by y"
else:
    print "x is not divisible by y"
```

It might be tempting to write something like:

```
if isDivisible(x, y) == True:
```

But the extra comparison is unnecessary. As an exercise, write a function `isBetween(x, y, z)` that returns `True` if $y \leq x \leq z$ or `False` otherwise.

MORE RECURSION

So far, you have only learned a small subset of Python, but you might be interested to know that this subset is a *complete* programming language, which means that anything that can be computed can be expressed in this language. Any programme ever written

could be rewritten using only the language features you have learned so far (actually, you would need a few commands to control devices like the keyboard, mouse, disks, etc., but that's all).

Proving that claim is a *non-trivial* exercise first accomplished by Alan Turing, one of the first computer scientists (some would argue that he was a mathematician, but a lot of early computer scientists started as mathematicians). Accordingly, it is known as the Turing Thesis. If you take a course on the Theory of Computation, you will have a chance to see the proof.

To give you an idea of what you can do with the tools you have learned so far, we'll evaluate a few recursively defined mathematical functions. A recursive definition is similar to a circular definition, in the sense that the definition contains a reference to the thing being defined.

A truly circular definition is not very useful: Frabjuous: An adjective used to describe something that is frabjuous.

If you saw that definition in the dictionary, you might be annoyed. On the other hand, if you looked up the definition of the mathematical function factorial, you might get something like this:

```
0! = 1
n! = n (n-1)!
```

This definition says that the factorial of 0 is 1, and the factorial of any other value, n , is n multiplied by the factorial of $n-1$.

So $3!$ is 3 times $2!$, which is 2 times $1!$, which is 1 times $0!$. Putting it all together, $3!$ equals 3 times 2 times 1 times 1, which is 6.

If you can write a recursive definition of something, you can usually write a Python programme to evaluate it. The first step is to decide what the parameters are for this function.

With little effort, you should conclude that factorial has a single parametre:

```
def factorial(n):
```

If the argument happens to be 0, all we have to do is return 1:

```
    def factorial(n):
        if n == 0:
            return 1
```

Otherwise, and this is the interesting part, we have to make a recursive call to find the factorial of $n-1$ and then multiply it by n :

```
    def factorial(n):
        if n == 0:
            return 1
        else:
            recurse = factorial(n-1)
            result = n * recurse
            return result
```

The flow of execution for this programme is similar to the flow of countdown in Section 4.9.

If we call factorial with the value 3:

Since 3 is not 0, we take the second branch and calculate the factorial of $n-1$...

Since 2 is not 0, we take the second branch and calculate the factorial of $n-1$...

Since 1 is not 0, we take the second branch and calculate the factorial of $n-1$...

Since 0 is 0, we take the first branch and return 1 without making any more recursive calls.

The return value (1) is multiplied by n , which is 1, and the result is returned.

The return value (1) is multiplied by n , which is 2, and the result is returned.

The return value (2) is multiplied by n , which is 3, and the result, 6, becomes the return value of the function call that started the whole process.

Here is what the stack diagram looks like for this sequence of function calls:

The return values are shown being passed back up the stack. In each frame, the return value is the value of `result`, which is the product of n and `recurse`. Notice that in the last frame, the local variables `recurse` and `result` do not exist, because the branch that creates them did not execute.

LEAP OF FAITH

Following the flow of execution is one way to read programmes, but it can quickly become labyrinthine. An alternative is what we call the “leap of faith.” When you come to a function call, instead of following the flow of execution, you *assume* that the function works correctly and returns the appropriate value. In fact, you are already practicing this leap of faith when you use built-in functions. When you call `math.cos` or `math.exp`, you don’t examine the implementations of those functions. You just assume that they work because the people who wrote the built-in functions were good programmers.

The same is true when you call one of your own functions. For example, in Section 5.4, we wrote a function called `isDivisible` that determines whether one number is divisible by another. Once we have convinced ourselves that this function is correct by testing and examining the code we can use the function without looking at the code again. The same is true of recursive programmes. When you get to the recursive call, instead of following the flow of execution, you should assume that the recursive call works (yields the correct result) and then ask yourself, “Assuming that I can find the factorial of $n-1$, can I compute the factorial of n ?” In this case, it is clear that you can, by multiplying by n . Of course, it’s a bit strange to assume that the function works correctly when you haven’t finished writing it, but that’s why it’s called a leap of faith!

CHECKING TYPES

What happens if we call `factorial` and give it 1.5 as an argument?

```
factorial (1.5)
RuntimeError: Maximum recursion depth
exceeded
```

It looks like an infinite recursion. But how can that be? There is a base case when $n == 0$. The problem is that the values of n *miss* the base case. In the first recursive call, the value of n is 0.5. In the next, it is -0.5 . From there, it gets smaller and smaller, but it will never be 0. We have two choices. We can try to generalize the factorial function to work with floating-point numbers, or we can make factorial check the type of its argument. The first option is called the gamma function and it's a little beyond the scope of this book. So we'll go for the second. We can use the built-in function `isinstance` to verify the type of the argument.

While we're at it, we also make sure the argument is positive:

```
def factorial (n):
    if not isinstance(n, int):
        print "Factorial is only defined for
        integers."
        return -1
    elif n < 0:
        print "Factorial is only defined for
        positive integers."
        return -1
    elif n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

Now we have three base cases. The first catches non-integers. The second catches negative integers. In both cases, the programme prints an error message and returns a special value, -1 , to indicate that something went wrong:

```
factorial ("fred")
Factorial is only defined for integers.
-1
factorial (-2)
Factorial is only defined for positive
integers.
-1
```

If we get past both checks, then we know that n is a positive integer, and we can prove that the recursion terminates. This programme demonstrates a pattern sometimes called a guardian. The first two conditionals act as guardians, protecting the code that follows from values that might cause an error. The guardians make it possible to prove the correctness of the code.

CONDITIONALS AND RECURSION

THE MODULUS OPERATOR

The modulus operator works on integers (and integer expressions) and yields the remainder when the first operand is divided by the second. In Python, the modulus operator is a per cent sign (%).

The syntax is the same as for other operators:

```
quotient = 7/3
print quotient
2
remainder = 7 % 3
>>> print remainder
1
```

So 7 divided by 3 is 2 with 1 left over.

The modulus operator turns out to be surprisingly useful. For example, you can check whether one number is divisible by another if $x \% y$ is zero, then x is divisible by y .

Also, you can extract the right-most digit or digits from a number. For example, $x \% 10$ yields the right-most digit of x (in base 10). Similarly $x \% 100$ yields the last two digits.

BOOLEAN EXPRESSIONS

A boolean expression is an expression that is either true or false. One way to write a boolean expression is to use the operator `==`, which compares two values and produces a boolean value:

```
5 == 5
True
5 == 6
False
```

In the first statement, the two operands are equal, so the value of the expression is `True`; in the second statement, 5 is not equal to 6, so we get `False`. `True` and `False` are special values that are built into Python.

The `==` operator is one of the comparison operators; the others are:

<code>x != y</code>	#	<code>x</code> is not equal to <code>y</code>
<code>x > y</code>	#	<code>x</code> is greater than <code>y</code>
<code>x < y</code>	#	<code>x</code> is less than <code>y</code>
<code>x ≥ y</code>	#	<code>x</code> is greater than or equal to <code>y</code>
<code>x ≤ y</code>	#	<code>x</code> is less than or equal to <code>y</code>

Although these operations are probably familiar to you, the Python symbols are different from the mathematical symbols. A common error is to use a single equal sign (`=`) instead of a double equal sign (`==`). Remember that `=` is an assignment operator and `==` is a comparison operator. Also, there is no such thing as `⇒` or `⇐`.

LOGICAL OPERATORS

There are three logical operators: and, or, and not. The semantics (meaning) of these operators is similar to their meaning in English.

For example, $x > 0$ and $x < 10$ is true only if x is greater than 0 *and* less than 10. $n \% 2 == 0$ or $n \% 3 == 0$ is true if *either* of the conditions is true, that is, if the number is divisible by 2 *or* 3. Finally, the not operator negates a boolean expression, so `not(x > y)` is true if $(x > y)$ is false, that is, if x is less than or equal to y .

Strictly speaking, the operands of the logical operators should be boolean expressions, but Python is not very strict. Any non-zero number is interpreted as “true.”

```
x = 5
x and 1
1
y = 0
y and 1
0
```

In general, this sort of thing is not considered good style. If you want to compare a value to zero, you should do it explicitly.

CONDITIONAL EXECUTION

In order to write useful programmes, we almost always need the ability to check conditions and change the behaviour of the programme accordingly. Conditional statements give us this ability.

The simplest form is the if statement:

```
if x > 0:
    print "x is positive"
```

The boolean expression after the if statement is called the condition. If it is true, then the indented statement gets executed. If not, nothing happens.

Like other compound statements, the if statement is made up of a header and a block of statements:

```
HEADER:
    FIRST STATEMENT
    ...
    LAST STATEMENT
```

The header begins on a new line and ends with a colon (:). The indented statements that follow are called a block. The first unindented statement marks the end of the block. A statement block inside a compound statement is called the body of the statement.

There is no limit on the number of statements that can appear in the body of an if statement, but there has to be at least one. Occasionally, it is useful to have a body with no statements (usually as a place keeper for code you haven’t written yet). In that case, you can use the pass statement, which does nothing.

ALTERNATIVE EXECUTION

A second form of the if statement is alternative execution, in which there are two possibilities and the condition determines which one gets executed.

The syntax looks like this:

```
if x%2 == 0:
    print x, "is even"
else:
    print x, "is odd"
```

If the remainder when x is divided by 2 is 0, then we know that x is even, and the programme displays a message to that effect. If the condition is false, the second set of statements is executed. Since the condition must be true or false, exactly one of the alternatives will be executed. The alternatives are called branches, because they are branches in the flow of execution.

As an aside, if you need to check the parity (evenness or oddness) of numbers often, you might “wrap” this code in a function:

```
def printParity(x):
    if x%2 == 0:
        print x, "is even"
    else:
        print x, "is odd"
```

For any value of x , printParity displays an appropriate message. When you call it, you can provide any integer expression as an argument.

```
>>> printParity(17)
17 is odd
>>> y = 17
>>> printParity(y+1)
18 is even
```

CHAINED CONDITIONALS

Sometimes there are more than two possibilities and we need more than two branches.

One way to express a computation like that is a chained conditional:

```
if x < y:
    print x, "is less than", y
elif x > y:
    print x, "is greater than", y
else:
    print x, "and", y, "are equal"
```

elif is an abbreviation of “else if.” Again, exactly one branch will be executed. There is no limit of the number of elif statements, but the last branch has to be an else statement:

```
if choice == 'A':
    functionA()
elif choice == 'B':
    functionB()
elif choice == 'C':
    functionC()
else:
    print "Invalid choice."
```

Each condition is checked in order. If the first is false, the next is checked, and so on. If one of them is true, the corresponding branch executes, and the statement ends. Even if more than one condition is true, only the first true branch executes.

As an exercise, wrap these examples in functions called compare(x , y) and dispatch(choice).

NESTED CONDITIONALS

One conditional can also be nested within another. We could have written the trichotomy example as follows:

```
if x == y:
    print x, "and", y, "are equal"
else:
    if x < y:
        print x, "is less than", y
    else:
        print x, "is greater than", y
```

The outer conditional contains two branches. The first branch contains a simple output statement. The second branch contains another if statement, which has two branches of its own. Those two branches are both output statements, although they could have been conditional statements as well.

Although the indentation of the statements makes the structure apparent, nested conditionals become difficult to read very quickly. In general, it is a good idea to avoid them when you can. Logical operators often provide a way to simplify nested conditional statements.

For example, we can rewrite the following code using a single conditional:

```
if 0 < x:
    if x < 10:
        print "x is a positive single digit."
```

The print statement is executed only if we make it past both the conditionals, so we can use the and operator:

```
if 0 < x and x < 10:
    print "x is a positive single digit."
```

These kinds of conditions are common, so Python provides an alternative syntax that is similar to mathematical notation:

```
if 0 < x < 10:
    print "x is a positive single digit."
```

This condition is semantically the same as the compound boolean expression and the nested conditional.

THE RETURN STATEMENT

The return statement allows you to terminate the execution of a function before you reach the end.

One reason to use it is if you detect an error condition:

```
import math
def printLogarithm(x):
    if x ≤ 0:
        print "Positive numbers only, please."
        return
    result = math.log(x)
    print "The log of x is", result
```

The function `printLogarithm` has a parameter named `x`. The first thing it does is check whether `x` is less than or equal to 0, in which case it displays an error message and then uses `return` to exit the function.

The flow of execution immediately returns to the caller, and the remaining lines of the function are not executed. Remember that to use a function from the `math` module, you have to import it.

RECURSION

We mentioned that it is legal for one function to call another, and you have seen several examples of that. We neglected to mention that it is also legal for a function to call itself. It may not be obvious why that is a good thing, but it turns out to be one of the most magical and interesting things a programme can do.

For example, look at the following function:

```
def countdown(n):
    if n == 0:
        print "Blastoff!"
    else:
        print n
        countdown(n-1)
```

`countdown` expects the parameter, `n`, to be a positive integer. If `n` is 0, it outputs the word, "Blastoff!" Otherwise, it outputs `n` and then calls a function named `countdown` itself passing `n-1` as an argument.

What happens if we call this function like this:

```
>>> countdown(3)
```

The execution of `countdown` begins with `n=3`, and since `n` is not 0, it outputs the value 3, and then calls itself...

The execution of `countdown` begins with `n=2`, and since `n` is not 0, it outputs the value 2, and then calls itself...

The execution of `countdown` begins with `n=1`, and since `n` is not 0, it outputs the value 1, and then calls itself...

The execution of `countdown` begins with `n=0`, and since `n` is 0, it outputs the word, "Blastoff!" and then returns.

The `countdown` that got `n=1` returns.

The `countdown` that got `n=2` returns.

The `countdown` that got `n=3` returns.

And then you're back in `__main__` (what a trip). So, the total output looks like this:

```
3
2
1
Blastoff!
```

As a second example, look again at the functions `newLine` and `threeLines`:

```
def newLine():
    print
```



```
def threeLines():
    newLine()
    newLine()
    newLine()
```

Although these work, they would not be much help if we wanted to output 2 newlines, or 106.

A better alternative would be this:

```
def nLines(n):
    if n > 0:
        print
        nLines(n-1)
```

This programme is similar to countdown; as long as n is greater than 0, it outputs one newline and then calls itself to output $n-1$ additional newlines. Thus, the total number of newlines is $1 + (n - 1)$ which, if you do your algebra right, comes out to n .

The process of a function calling itself is recursion, and such functions are said to be recursive.

STACK DIAGRAMS FOR RECURSIVE FUNCTIONS

Every time a function gets called, Python creates a new function frame, which contains the function's local variables and parameters. For a recursive function, there might be more than one frame on the stack at the same time.

As usual, the top of the stack is the frame for `__main__`. It is empty because we did not create any variables in `__main__` or pass any arguments to it.

The four countdown frames have different values for the parameter n . The bottom of the stack, where $n=0$, is called the base case. It does not make a recursive call, so there are no more frames.

As an exercise, draw a stack diagram for `nLines` called with $n=4$.

INFINITE RECURSION

If a recursion never reaches a base case, it goes on making recursive calls forever, and the programme never terminates. This is known as infinite recursion, and it is generally not considered a good idea. Here is a minimal programme with an infinite recursion:

```
def recurse():
    recurse()
```

In most programming environments, a programme with infinite recursion does not really run forever.

Python reports an error message when the maximum recursion depth is reached:

```
File "<stdin>", line 2, in recurse
(98 repetitions omitted)
File "<stdin>", line 2, in recurse
RuntimeError: Maximum recursion depth exceeded
```

This traceback is a little bigger than the one we saw in the previous chapter. When the error occurs, there are 100 recurse frames on the stack!

As an exercise, write a function with infinite recursion and run it in the Python interpreter.

KEYBOARD INPUT

The programmes we have written so far are a bit rude in the sense that they accept no input from the user. They just do the same thing every time. Python provides built-in functions that get input from the keyboard. The simplest is called `raw_input`. When this function is called, the programme stops and waits for the user to type something. When the user presses Return or the Enter key, the programme resumes and `raw_input` returns what the user typed as a string:

```
input = raw_input ()
What are you waiting for?
print input
What are you waiting for?
```

Before calling `raw_input`, it is a good idea to print a message telling the user what to input. This message is called a prompt.

We can supply a prompt as an argument to `raw_input`:

```
name = raw_input ("What...is your name?")
What...is your name? Arthur, King of the
Britons!
>>> print name
Arthur, King of the Britons!
```

If we expect the response to be an integer, we can use the `input` function:

```
prompt = "What...is the airspeed velocity of an unladen swallow?\n"
speed = input(prompt)
```

The sequence `\n` at the end of the string represents a newline, so the user's input appears below the prompt.

If the user types a string of digits, it is converted to an integer and assigned to `speed`.

Unfortunately, if the user types a character that is not a digit, the programme crashes:

```
speed = input (prompt)
What...is the airspeed velocity of an unladens swallow?
What do you mean, an African or a European swallow?
SyntaxError: invalid syntax
```

To avoid this kind of error, it is generally a good idea to use `raw_input` to get a string and then use conversion functions to convert to other types.

C Language Programming

INTRODUCTION

C is sometimes referred to as a “high-level assembly language.” Some people think that’s an insult, but it’s actually a deliberate and significant aspect of the language. If you have programmed in assembly language, you’ll probably find C very natural and comfortable (although if you continue to focus too heavily on machine-level details, you’ll probably end up with unnecessarily non-portable programmes). If you haven’t programmed in assembly language, you may be frustrated by C’s lack of certain higher-level features. In either case, you should understand why C was designed this way: so that seemingly-simple constructions expressed in C would not expand to arbitrarily expensive (in time or space) machine language constructions when compiled.

If you write a C programme simply and succinctly, it is likely to result in a succinct, efficient machine language executable. If you find that the executable programme resulting from a C programme is not efficient, it’s probably because of something silly you did, not because of something the compiler did behind your back which you have no control over. In any case, there’s no point in complaining about C’s low-level flavour: C is what it is.

A programming language is a tool, and no tool can perform every task unaided. If you’re building a house, and I’m teaching you how to use a hammer, and you ask how to assemble rafters and trusses into gables, that’s a legitimate question, but the answer has fallen out of the realm of “How do I use a hammer?” and into “How do I build a

house?”. In the same way, we’ll see that C does not have built-in features to perform every function that we might ever need to do while programming.

As mentioned above, C imposes relatively few built-in ways of doing things on the programmer. Some common tasks, such as manipulating strings, allocating memory, and doing input/output (I/O), are performed by calling on library functions. Other tasks which you might want to do, such as creating or listing directories, or interacting with a mouse, or displaying windows or other user-interface elements, or doing colour graphics, are not defined by the C language at all.

You can do these things from a C programme, of course, but you will be calling on services which are peculiar to your programming environment (compiler, processor, and operating system) and which are not defined by the C standard. Since this course is about portable C programming, it will also be steering clear of facilities not provided in all C environments.

Another aspect of C that’s worth mentioning here is that it is, to put it bluntly, a bit dangerous. C does not, in general, try hard to protect a programmer from mistakes. If you write a piece of code which will (through some oversight of yours) do something wildly different from what you intended it to do, up to and including deleting your data or trashing your disk, and if it is possible for the compiler to compile it, it generally will. You won’t get warnings of the form “Do you really mean to...” or “Are you sure you really want to...?”. C is often compared to a sharp knife: it can do a surgically precise job on some exacting task you have in mind, but it can also do a surgically precise job of cutting off your finger. It’s up to you to use it carefully.

This aspect of C is very widely criticized; it is also used (justifiably) to argue that C is not a good teaching language. C aficionados love this aspect of C because it means that C does not try to protect them from themselves: when they know what they’re doing, even if it’s risky or obscure, they can do it. Students of C hate this aspect of C because it often seems as if the language is some kind of a conspiracy specifically designed to lead them into booby traps.

This is another aspect of the language which it’s fairly pointless to complain about. If you take care and pay attention, you can avoid many of the pitfalls. These notes will point out many of the obvious (and not so obvious) trouble spots.

PROGRAMMING EXAMPLES

FIRST EXAMPLE

The best way to learn programming is to dive right in and start writing real programmes. This way, concepts which would otherwise seem abstract make sense, and the positive feedback you get from getting even a small programme to work gives you a great incentive to improve it or write the next one. Diving in with “real” programmes right away has another advantage, if only pragmatic: if you’re using a

conventional compiler, you can't run a fragment of a programme and see what it does; nothing will run until you have a complete (if tiny or trivial) programme.

You can't learn everything you'd need to write a complete programme all at once, so you'll have to take some things "on faith" and parrot them in your first programmes before you begin to understand them. (You can't learn to programme just one expression or statement at a time any more than you can learn to speak a foreign language one word at a time. If all you know is a handful of words, you can't actually *say* anything: you also need to know something about the language's word order and grammar and sentence structure and declension of articles and verbs.)

Besides the occasional necessity to take things on faith, there is a more serious potential drawback of this "dive in and programme" approach: it's a small step from learning-by-doing to learning-by-trial-and-error, and when you learn programming by trial-and-error, you can very easily learn many errors. When you're not sure whether something will work, or you're not even sure what you could use that might work, and you try something, and it does work, you do *not* have any guarantee that what you tried worked for the right reason. You might just have "learned" something that works only by accident or only on your compiler, and it may be very hard to unlearn it later, when it stops working.

Therefore, whenever you're not sure of something, be very careful before you go off and try it "just to see if it will work." Of course, you can never be absolutely sure that something is going to work before you try it, otherwise we'd never have to try things. But you should have an expectation that something is going to work before you try it, and if you can't predict how to do something or whether something would work and find yourself having to determine it experimentally, make a note in your mind that whatever you've just learned (based on the outcome of the experiment) is suspect.

The first example programme is the first example programme in any language: print or display a simple string, and exit. Here is "hello, world" programme:

```
#include <stdio.h>
main()
{
printf("Hello, world!\n");
return 0;
}
```

If you have a C compiler, the first thing to do is figure out how to type this programme in and compile it and run it and see where its output went. The first line is practically boilerplate; it will appear in almost all programmes we write. It asks that some definitions having to do with the "Standard I/O Library" be included in our programme; these definitions are needed if we are to call the library function `printf` correctly.

The second line says that we are defining a function named `main`. Most of the time, we can name our functions anything we want, but the function name `main` is special: it is the function that will be "called" first when our programme starts running. The empty pair of parentheses indicates that our `main` function accepts *no arguments*, that is, there

isn't any information which needs to be passed in when the function is called. The braces { and } surround a list of statements in C. Here, they surround the list of statements making up the function main.

The line

```
printf("Hello, world!\n");
```

is the first statement in the programme. It asks that the function printf be called; printf is a library function which prints formatted output. The parentheses surround printf's argument list: the information which is handed to it which it should act on. The semicolon at the end of the line terminates the statement. (printf's name reflects the fact that C was first developed when Teletypes and other printing terminals were still in widespread use. Today, of course, video displays are far more common. printf's "prints" to the *standard output*, that is, to the default location for programme output to go. Nowadays, that's almost always a video screen or a window on that screen.

If you do have a printer, you'll typically have to do something extra to get a programme to print to it.) printf's first (and, in this case, only) argument is the string which it should print. The string, enclosed in double quotes "", consists of the words "Hello, world!" followed by a special sequence: \n. In strings, any two-character sequence beginning with the backslash \ represents a single special character. The sequence \n represents the "new line" character, which prints a carriage return or line feed or whatever it takes to end one line of output and move down to the next. (This programme only prints one line of output, but it's still important to terminate it.)

The second line in the main function is return 0; In general, a function may return a value to its caller, and main is no exception. When main returns (that is, reaches its end and stops functioning), the programme is at its end, and the return value from main tells the operating system (or whatever invoked the programme that main is the main function of) whether it succeeded or not. By convention, a return value of 0 indicates success.

This programme may look so absolutely trivial that it seems as if it's not even worth typing it in and trying to run it, but doing so may be a big (and is certainly a vital) first hurdle. On an unfamiliar computer, it can be arbitrarily difficult to figure out how to enter a text file containing programme source, or how to compile and link it, or how to invoke it, or what happened after (if?) it ran.

The most experienced C programmers immediately go back to this one, simple programme whenever they're trying out a new system or a new way of entering or building programmes or a new way of printing output from within programmes. As Kernighan and Ritchie say, everything else is comparatively easy.

How *you* compile and run this (or any) programme is a function of the compiler and operating system you're using. The first step is to type it in, exactly as shown; this may involve using a text editor to create a file containing the programme text. You'll have to give the file a name, and all C compilers require that files containing C source end with the extension .c.

So you might place the programme text in a file called `hello.c`. The second step is to compile the programme. (Strictly speaking, compilation consists of two steps, compilation proper followed by linking, but we can overlook this distinction at first, especially because the compiler often takes care of initiating the linking step automatically.) On many Unix systems, the command to compile a C programme from a source file `hello.c` is

```
cc -o hello hello.c
```

You would type this command at the Unix shell prompt, and it requests that the `cc` (C compiler) programme be run, placing its output (*i.e.* the new executable programme it creates) in the file `hello`, and taking its input (*i.e.* the source code to be compiled) from the file `hello.c`. The third step is to run (execute, invoke) the newly-built `hello` programme. Again on a Unix system, this is done simply by typing the program's name:

```
hello
```

Depending on how your system is set up (in particular, on whether the current directory is searched for executables, based on the `PATH` variable), you may have to type

```
./hello
```

to indicate that the `hello` programme is in the current directory (as opposed to some "bin" directory full of executable programmes, elsewhere).

You may also have your choice of C compilers. On many Unix machines, the `cc` command is an older compiler which does not recognize modern, ANSI Standard C syntax. An old compiler will accept the simple programmes we'll be starting with, but it will not accept most of our later programmes. If you find yourself getting baffling compilation errors on programmes which you've typed in exactly as they're shown, it probably indicates that you're using an older compiler. On many machines, another compiler called `acc` or `gcc` is available, and you'll want to use it, instead. (Both `acc` and `gcc` are typically invoked the same as `cc`; that is, the above `cc` command would instead be typed, say, `gcc -o hello hello.c`.) (One final caveat about Unix systems: don't name your test programmes `test`, because there's already a standard command called `test`, and you and the command interpreter will get badly confused if you try to replace the system's `test` command with your own, not least because your own almost certainly does something completely different.)

Under MS-DOS, the compilation procedure is quite similar. The name of the command you type will depend on your compiler (*e.g.* `cl` for the Microsoft C compiler, `tc` or `bcc` for Borland's Turbo C, etc.). You may have to manually perform the second, linking step, perhaps with a command named `link` or `tlink`. The executable file which the compiler/linker creates will have a name ending in `.exe` (or perhaps `.com`), but you can still invoke it by typing the base name (*e.g.* `hello`). See your compiler documentation for complete details; one of the manuals should contain a demonstration of how to enter, compile, and run a small programme that prints some simple output, just as we're trying to describe here.

In an integrated or “visual” programming environment, such as those on the Macintosh or under various versions of Microsoft Windows, the steps you take to enter, compile, and run a programme are somewhat different (and, theoretically, simpler). Typically, there is a way to open a new source window, type source code into it, give it a file name, and add it to the programme (or “project”) you’re building. If necessary, there will be a way to specify what other source files (or “modules”) make up the programme.

Then, there’s a button or menu selection which compiles and runs the programme, all from within the programming environment. (There will also be a way to create a stand-alone executable file which you can run from outside the environment.) In a PC-compatible environment, you may have to choose between creating DOS programmes or Windows programmes.

If you have troubles pertaining to the `printf` function, try specifying a target environment of MS-DOS. Supposedly, some compilers which are targeted at Windows environments won’t let you call `printf`, because until you call some fancier functions to request that a window be created, there’s no window for `printf` to print to. Again, check the introductory or tutorial manual that came with the programming package; it should walk you through the steps necessary to get your first programme running.

SECOND EXAMPLE

Our second example is of little more practical use than the first, but it introduces a few more programming language elements:

```
#include <stdio.h>
/* print a few numbers, to illustrate a simple loop */
main()
{
  int i;
  for(i = 0; i < 10; i = i + 1)
    printf("i is %d\n", i);
  return 0;
}
```

As before, the line `#include <stdio.h>` is boilerplate which is necessary since we’re calling the `printf` function, and `main()` and the pair of braces `{ }` indicate and delineate the function named `main` we’re (again) writing.

The first new line is the line `/* print a few numbers, to illustrate a simple loop */` which is a *comment*. Anything between the characters `/*` and `*/` is ignored by the compiler, but may be useful to a person trying to read and understand the programme. You can add comments anywhere you want to in the programme, to document what the programme is, what it does, who wrote it, how it works, what the various functions are for and how *they* work, what the various variables are for, etc.

The second new line, down within the function `main`, is `int i`; which *declares* that our function will use a variable named `i`. The variable’s type is `int`, which is a plain integer.

Next, we set up a loop:

```
for(i = 0; i < 10; i = i + 1)
```

The keyword `for` indicates that we are setting up a “for loop.” A for loop is controlled by three expressions, enclosed in parentheses and separated by semicolons. These expressions say that, in this case, the loop starts by setting `i` to 0, that it continues as long as `i` is less than 10, and that after each iteration of the loop, `i` should be incremented by 1 (that is, have 1 added to its value).

Finally, we have a call to the `printf` function, as before, but with several differences. First, the call to `printf` is within the *body* of the for loop. This means that control flow does not pass once through the `printf` call, but instead that the call is performed as many times as are dictated by the for loop. In this case, `printf` will be called several times: once when `i` is 0, once when `i` is 1, once when `i` is 2, and so on until `i` is 9, for a total of 10 times.

A second difference in the `printf` call is that the string to be printed, “`i is %d`”, contains a per cent sign. Whenever `printf` sees a per cent sign, it indicates that `printf` is not supposed to print the exact text of the string, but is instead supposed to read another one of its arguments to decide what to print. The letter after the per cent sign tells it what type of argument to expect and how to print it. In this case, the letter `d` indicates that `printf` is to expect an int, and to print it in decimal. Finally, we see that `printf` is in fact being called with another argument, for a total of two, separated by commas. The second argument is the variable `i`, which is in fact an int, as required by `%d`. The effect of all of this is that each time it is called, `printf` will print a line containing the current value of the variable `i`:

```
i is 0
i is 1
i is 2
...
```

After several trips through the loop, `i` will eventually equal 9. After that trip through the loop, the third control expression `i = i + 1` will increment its value to 10. The condition `i < 10` is no longer true, so no more trips through the loop are taken. Instead, control flow jumps down to the statement following the for loop, which is the return statement. The main function returns, and the programme is finished.

STATEMENT AND CONTROL

Statements are the “steps” of a programme. Most statements compute and assign values or call functions, but we will eventually meet several other kinds of statements as well. By default, statements are executed in sequence, one after another.

We can, however, modify that sequence by using *control flow constructs* which arrange that a statement or group of statements is executed only if some condition is true or false, or executed over and over again to form a *loop*. (A somewhat different kind of control flow happens when we call a function: execution of the caller is suspended while the called function proceeds.)

My definitions of the terms *statement* and *control flow* are somewhat circular. A statement is an element within a programme which you can apply control flow to; control flow is how you specify the order in which the statements in your programme are executed. (A weaker definition of a statement might be “a part of your programme that does something,” but this definition could as easily be applied to expressions or functions.)

EXPRESSION STATEMENTS

Most of the statements in a C programme are *expression statements*. An expression statement is simply an expression followed by a semicolon.

The lines,

```
i = 0;
i = i + 1;
and
printf("Hello, world!\n");
```

are all expression statements. (In some languages, such as Pascal, the semicolon separates statements, such that the last statement is not followed by a semicolon. In C, however, the semicolon is a statement terminator; all simple statements are followed by semicolons. The semicolon is also used for a few other things in C; we’ve already seen that it terminates declarations, too.)

Expression statements do all of the real work in a C programme. Whenever you need to compute new values for variables, you’ll typically use expression statements (and they’ll typically contain assignment operators). Whenever you want your programme to do something visible, in the real world, you’ll typically call a function (as part of an expression statement). We’ve already seen the most basic example: calling the function `printf` to print text to the screen. But anything else you might do—read or write a disk file, talk to a modem or printer, draw pictures on the screen—will also involve function calls. Expressions and expression statements can be arbitrarily complicated. They don’t have to consist of exactly one simple function call, or of one simple assignment to a variable. For one thing, many functions return values, and the values they return can then be used by other parts of the expression. For example, C provides a `sqrt` (square root) function, which we might use to compute the hypotenuse of a right triangle like this:

```
c = sqrt(a*a + b*b);
```

To be useful, an expression statement must do something; it must have some lasting effect on the state of the programme. (Formally, a useful statement must have at least one *side effect*.) The first two sample expression statements in this section (above) assign new values to the variable `i`, and the third one calls `printf` to print something out, and these are good examples of statements that do something useful. (To make the distinction clear, we may note that degenerate constructions such as

```
0;
i;
```

```
or
i + 1;
```

are syntactically valid statements—they consist of an expression followed by a semicolon—but in each case, they compute a value without doing anything with it, so the computed value is discarded, and the statement is useless. But if the “degenerate” statements in this paragraph don’t make much sense to you, don’t worry; it’s because they, frankly, don’t make much sense.) It’s also possible for a single expression to have multiple side effects, but it’s easy for such an expression to be (a) confusing or (b) undefined. For now, we’ll only be looking at expressions (and, therefore, statements) which do one well-defined thing at a time.

IF STATEMENTS

The simplest way to modify the control flow of a programme is with an if statement, which in its simplest form looks like this:

```
if(x > max)
    max = x;
```

Even if you didn’t know any C, it would probably be pretty obvious that what happens here is that if *x* is greater than *max*, *x* gets assigned to *max*. (We’d use code like this to keep track of the maximum value of *x* we’d seen—for each new *x*, we’d compare it to the old maximum value *max*, and if the new value was greater, we’d update *max*.)

More generally, we can say that the syntax of an if statement is:

```
if(expression)
    statement
```

where *expression* is any expression and *statement* is any statement. What if you have a series of statements, all of which should be executed together or not at all depending on whether some condition is true? The answer is that you enclose them in braces:

```
if(expression)
{
    statement<sub>1</sub>
    statement<sub>2</sub>
    statement<sub>3</sub>
}
```

As a general rule, anywhere the syntax of C calls for a statement, you may write a series of statements enclosed by braces. (You do not need to, and should not, put a semicolon after the closing brace, because the series of statements enclosed by braces is not itself a simple expression statement.)

An if statement may also optionally contain a second statement, the “else clause,” which is to be executed if the condition is not met.

Here is an example:

```
if(n > 0)
    average = sum/n;
else {
    printf("can't compute average\n");
```

```
average = 0;
}
```

The first statement or block of statements is executed if the condition *is* true, and the second statement or block of statements (following the keyword *else*) is executed if the condition is *not* true. In this example, we can compute a meaningful average only if *n* is greater than 0; otherwise, we print a message saying that we cannot compute the average. The general syntax of an if statement is therefore,

```
if(expression)
statement<sub>1</sub>
else
statement<sub>2</sub>
```

(where both *statement₁* and *statement₂* may be lists of statements enclosed in braces).

It's also possible to nest one if statement inside another. (For that matter, it's in general possible to nest any kind of statement or control flow construct within another.) For example, here is a little piece of code which decides roughly which quadrant of the compass you're walking into, based on an *x* value which is positive if you're walking east, and a *y* value which is positive if you're walking north:

```
if(x > 0)
{
    if(y > 0)
        printf("Northeast.\n");
    else    printf("Southeast.\n");
}
else {
    if(y > 0)
        printf("Northwest.\n");
    else    printf("Southwest.\n");
}
```

When you have one if statement (or loop) nested inside another, it's a very good idea to use explicit braces {}, as shown, to make it clear (both to you and to the compiler) how they're nested and which else goes with which if. It's also a good idea to indent the various levels, also as shown, to make the code more readable to humans. Why do both? You use indentation to make the code visually more readable to yourself and other humans, but the compiler doesn't pay attention to the indentation (since all whitespace is essentially equivalent and is essentially ignored). Therefore, you also have to make sure that the punctuation is right. Here is an example of another common arrangement of if and else. Suppose we have a variable *grade* containing a student's numeric grade, and we want to print out the corresponding letter grade.

Here is code that would do the job:

```
if(grade >= 90)
    printf("A");
else if(grade >= 80)
    printf("B");
else if(grade >= 70)
```

```
printf("C");  
else if(grade >= 60)  
printf("D");  
else    printf("F");
```

What happens here is that exactly one of the five `printf` calls is executed, depending on which of the conditions is true. Each condition is tested in turn, and if one is true, the corresponding statement is executed, and the rest are skipped. If none of the conditions is true, we fall through to the last one, printing “F”. In the cascaded `if/else/if/else/...` chain, each `else` clause is another `if` statement. This may be more obvious at first if we reformat the example, including every set of braces and indenting each `if` statement relative to the previous one:

```
if(grade >= 90)  
{  
    printf("A");  
}  
else {  
    if(grade >= 80)  
    {  
        printf("B");  
    }  
    else {  
        if(grade >= 70)  
        {  
            printf("C");  
        }  
        else {  
            if(grade >= 60)  
            {  
                printf("D");  
            }  
            else {  
                printf("F");  
            }  
        }  
    }  
}
```

By examining the code this way, it should be obvious that exactly one of the `printf` calls is executed, and that whenever one of the conditions is found true, the remaining conditions do not need to be checked and none of the later statements within the chain will be executed. But once you’ve convinced yourself of this and learned to recognize the idiom, it’s generally preferable to arrange the statements as in the first example, without trying to indent each successive `if` statement one tabstop further out.

PROGRAMME STRUCTURE

We’ll have more to say later about programme structure, but for now let’s observe a few basics. A programme consists of one or more functions; it may also contain global

variables. (Our two example programmes so far have contained one function apiece, and no global variables.) At the top of a source file are typically a few boilerplate lines such as `#include <stdio.h>`, followed by the definitions (*i.e.* code) for the functions. (It's also possible to split up the several functions making up a larger programme into several source files, as we'll see in a later chapter.)

Each function is further composed of *declarations* and *statements*, in that order. When a sequence of statements should act as one (for example, when they should all serve together as the body of a loop) they can be enclosed in braces (just as for the outer body of the entire function). The simplest kind of statement is an *expression statement*, which is an expression (presumably performing some useful operation) followed by a semicolon. Expressions are further composed of *operators*, *objects* (variables), and *constants*.

C source code consists of several *lexical elements*. Some are words, such as `for`, `return`, `main`, and `i`, which are either *keywords* of the language (`for`, `return`) or *identifiers* (names) we've chosen for our own functions and variables (`main`, `i`). There are *constants* such as `1` and `10` which introduce new values into the programme. There are *operators* such as `=`, `+`, and `>`, which manipulate variables and values. There are other punctuation characters (often called *delimiters*), such as parentheses and squiggly braces `{ }`, which indicate how the other elements of the programme are grouped. Finally, all of the preceding elements can be separated by *whitespace*: spaces, tabs, and the "carriage returns" between lines.

The source code for a C programme is, for the most part, "free form." This means that the compiler does not care how the code is arranged: how it is broken into lines, how the lines are indented, or whether whitespace is used between things like variable names and other punctuation. (Lines like `#include <stdio.h>` are an exception; they must appear alone on their own lines, generally unbroken. Only lines beginning with `#` are affected by this rule; we'll see other examples later.) You can use whitespace, indentation, and appropriate line breaks to make your programmes more readable for yourself and other people (even though the compiler doesn't care). You can place explanatory *comments* anywhere in your programme—any text between the characters `/*` and `*/` is ignored by the compiler. (In fact, the compiler pretends that all it saw was whitespace.) Though comments are ignored by the compiler, well-chosen comments can make a programme *much* easier to read.

The usage of whitespace is our first *style* issue. It's typical to leave a blank line between different parts of the programme, to leave a space on either side of operators such as `+` and `=`, and to *indent* the bodies of loops and other control flow constructs.

Typically, we arrange the indentation so that the subsidiary statements controlled by a loop statement (the "loop body," such as the `printf` call in our second example programme) are all aligned with each other and placed one tab stop (or some consistent number of spaces) to the right of the controlling statement.

This indentation (like all whitespace) is not required by the compiler, but it makes programs *much* easier to read. (However, it can also be misleading, if used incorrectly or in the face of inadvertent mistakes. The compiler will decide what “the body of the loop” is based on its own rules, not the indentation, so if the indentation does not match the compiler’s interpretation, confusion is inevitable.)

To drive home the point that the compiler doesn’t care about indentation, line breaks, or other whitespace, here are a few (extreme) examples: The fragments,

```
for(i = 0; i < 10; i = i + 1)
printf("%d\n", i);
and
for(i = 0; i < 10; i = i + 1) printf("%d\n", i);
and
for(i=0;i<10;i=i+1)printf("%d\n",i);
and
for(i = 0; i < 10; i = i + 1)
printf("%d\n", i);
and
for ( i
= 0 ;
i < 10
; i =
i + 1
) printf (
"%d\n" , i
) ;
and
for
(i=0;
i<10;i=
i+1)printf
("%d\n", i);
```

are all treated exactly the same way by the compiler. Some programmers argue forever over the best set of “rules” for indentation and other aspects of programming style, calling to mind the old philosopher’s debates about the number of angels that could dance on the head of a pin. Style issues (such as how a programme is laid out) *are* important, but they’re not something to be too dogmatic about, and there are also other, deeper style issues besides mere layout and typography:

Although C compilers do not care about how a programme looks, proper indentation and spacing are critical in making programmes easy for people to read. We recommend writing only one statement per line, and using blanks around operators to clarify grouping. The position of braces is less important, although people hold passionate beliefs. We have chosen one of several popular styles. Pick a style that suits you, then use it consistently.

There is some value in having a reasonably standard style (or a few standard styles) for code layout. Please don’t take the above advice to “pick a style that suits you” as an invitation to invent your own brand-new style. If (perhaps after you’ve been programming

in C for a while) you have specific objections to specific facets of existing styles, you're welcome to modify them, but if you don't have any particular leanings, you're probably best off copying an existing style at first. (If you want to place your own stamp of originality on the programmes that you write, there are better avenues for your creativity than inventing a bizarre layout; you might instead try to make the logic easier to follow, or the user interface easier to use, or the code freer of bugs.)

Object Oriented Programming

Object oriented programming is different from procedural programming languages like C, Pascal in several ways. Everything in OOPs is grouped as “Objects”. OOP’s, defined in the purest sense, is implemented by sending messages to Objects. The different set of components or characteristics of OOP as follows.

- Object Entity.
- Abstraction.
- Encapsulation.
- Inheritance.
- Polymorphism.

OBJECT ENTITY

An Object is an instance of a class or it can be considered a “thing that can perform a set of activities”. In this world everything is Object. Object is an identifiable entity which has some characteristics as well as behaviour. For example a car is an object which has characteristics like its shape, colour, weight, size, model and behaviour *i.e.* moveability of car.

ABSTRACTION

Abstraction is an act to representing the essential features of an object without including the internal details or explanations. It is the most powerful paradigm of OOP’s. In other words Abstraction means hiding something but exist in reality.

For example there is a switch board having various switches and ask to student to switch on the fan. When the student press the switch of fan, he does not knows what is happening inside the switchboard.

He just knows about the essential features to switch on the fan. What is happening inside that is abstraction. The Abstraction is used in C++ by concept. The Class is the combination of data members and member functions.

The characteristics of an Object is represented in Class as its variables which we can define under the section *i.e.* Private, Public and Protected and behaviour is represented in class as member function.

ENCAPSULATION

The encapsulation can be implemented in C++ by using Class concept. As we knows a Class is the combination of data member and member functions, the member functions always operate on the data members which is to be declared in the same class as a single unit is called Encapsulation. In other words Encapsulation means wrapping up of data members and member functions into a single unit.

INHERITANCE

The idea of Inheritance derived from the concept of Classes. It is the process by which the object of one class acquire the property of object of an another class. Like the children always inherit some of properties from their parents.

The properties which are inherited from the parent class is called Base class and the child class which acquires these properties is called derived class.

There are five types of Inheritance:

- Single Inheritance.
- Multilevel Inheritance.
- Multiple Inheritance.
- Hierarchical Inheritance.
- Hybrid Inheritance.

POLYMORPHISM

Polymorphism is an another important concept of OOP's. The word *Poly* originated from Greek word many and *morphism* from the Greek word forms and thus polymorphism means many forms. Polymorphism allows many different types of Objects to perform the same operation by responding to the same message.

It is the process of defining number of objects of different classes into a group and call the methods to carry out the operation of the objects using different function calls. This concept is implemented in C++ by using Function Overloading, Operator Overloading, Virtual Functions etc.

EXAMPLE

Let's think about the types of parts that make up a deck and those are the cards. Since all of the cards are very similar in structure, we could use a struct to represent a single card:

```
enum Suit = {Clubs, Spades, Diamonds, Hearts};
struct Card
{
    Suit suit;
    char digit;
};
```

A note on the digit. If digit ≤ 10 , then it is a number, else it is the letter of the card (J, Q, K, A). You could also use it as a number 1(ace) through 13(king) as well, perhaps if you were using the card value in additions or such.

A simple object, the card deck only has one type of item. Now let's think about the types of actions you can perform on the deck, and then make a class declaration out of this list, as well as using the previously declared data.

```
class Deck
{
public:
    void CreateDeck(); //Fills array with legal cards
    void Shuffle(); //Shuffles those cards
    Card DrawCard(); //Gets a card from the deck
private:
    Card cards[52];
};
```

Now we have considered all of the things we may need to use a card deck. The programmer first sets up the deck with CreateDeck(), then whenever needed can Shuffle() the deck, and when the dealer deals a card, it can be picked up using DrawCard(), and then perhaps placed in the players hand (which could also could be a class too) or whatever the programmer needs to do with it.

A constructor is declared by creating a function by the same name as the class. The destructor has the same name, except with a ~ (the tilde key, next to the 1) in front of it. Below is the modified Deck class which takes advantage of C++ constructors and destructors.

The array change to a pointer is to allow for dynamic memory allocation using the new command, to show a very common use of the constructor and destructor:

```
class Deck
{
public:
    Deck();           //Constructor
    ~Deck();          //Destructor
    void CreateDeck(); //Fills array with legal cards
    void Shuffle();    //Shuffles those cards
    Card DrawCard();  //Gets a card from the deck
};
```

```
private:
    Card* cards;
};
```

Notice that obviously the constructor does not return anything, since it is an “invisible function” which is automatically called when you make the statement `Deck MyCardDeck;`. The same is true for the destructor, which is called when the variable goes out of scope. Below is an example of how to define and code a constructor, where the array is allocated and the data is initialized, and the destructor complement, which cleans up what the constructor did:

```
Deck::Deck()
{
    cards = new Card[52]; //Allocate memory
    CreateDeck(); //Set up the deck
}
Deck::~~Deck()
{
    delete[] cards; //Deallocate memory
}
```

Notice that all methods and members of the class are available in the constructor and destructor, just as in every other function. `Intclass.cc`, executable in any DOS compiler or console mode win 32 compiler shows exactly when these functions are called, and also shows many other concepts which will be explained later in this tutorial.

Let’s return to our earlier counter example which did not have a constructor but which really needed one, or at least a `Set Counter()` function. We can create a constructor to start the counter at zero, as well as starting it off at a value the user can enter:

```
class Counter
{
public:
    Counter();           //basic constructor
    Counter(int x);      //set the counter
    SetCount(int x);     //sets the count
    void Count() {CurrentCount++;}
    int ReadDisplay() {return CurrentCount;}
private:
    int CurrentCount;
};
```

Some of you who don’t know about *overloaded* functions may think two `Counter()` functions is an error, but it’s not! In C++, two functions with the same name can be declared (this applies everywhere, not just classes), as long as the compiler can discern between which function is being called.

In the case of the constructor, you must use this feature since the constructor must be the same name as the class. Also note that a destructor was not declared since there is nothing we need to clean up—the compiler already has a default destructor for the `int` data type (`CurrentCount`).

Let's see how we should write the code for these functions:

```
Counter::Counter()
{
    SetCount(0);
}
Counter::Counter(int x)
{
    SetCount(x);
}
Counter::SetCount(int x)
{
    CurrentCount = x;
}
```

After seeing it in code, things become a little clearer. You can overload as many times as you want to pass different things into the constructor, just be sure that they have a different number of parameters.

It is technically possible to have them with the same number, but since some basic data types can change into others (like int to float or similar), the function call becomes *ambiguous*, and the compiler doesn't know which to use and generates an error—so it's best just to steer clear of overloaded functions of the same number of parameters.

VARIABLES

A variable is a symbolic name for a memory location in which data can be stored and subsequently recalled. Variables are used for holding data values so that they can be utilized in various computations in a programme.

The value of variable can change during the execution of programme.

All variables have two important attributes:

- A type which is established when the variable is defined (*e.g.*, integer, real, character). Once defined, the type of a C++ variable cannot be changed.
- A value which can be changed by assigning a new value to the variable. The kind of values a variable can assume depends on its type. For example, an integer variable can only take integer values (*e.g.*, 2, 100, -12). Listing 1.2 illustrates the uses of some simple variable.

```
1 # include <iostream.h>
2 int main (void)
3 {
4     int workDays;
5     float workHours, payRate, weeklyPay;
6     workDays = 5;
7     workHours = 7.5;
8     payRate = 38.55;
9     weeklyPay = workDays * workHours * payRate;
```

```

10      cout << "Weekly Pay = ";
11      cout << weeklyPay;
12      cout << '\n';
13  }
```

ANNOTATION

4. This line defines an int (integer) variable called workDays, which will represent the number of working days in a week. As a general rule, a variable is defined by specifying its type first, followed by the variable name, followed by a semicolon.
5. This line defines three float (real) variables which, respectively, represent the work hours per day, the hourly pay rate, and the weekly pay. As illustrated by this line, multiple variables of the same type can be defined at once by separating them with commas.
6. This line is an assignment statement. It assigns the value 5 to the variable workDays. Therefore, after this statement is executed, workDays denotes the value 5.
7. This line assigns the value 7.5 to the variable workHours.
8. This line assigns the value 38.55 to the variable payRate.
9. This line calculates the weekly pay as the product of workDays, workHours, and payRate (* is the multiplication operator). The resulting value is stored in weeklyPay.
- 10-12. These lines output three items in sequence: the string "Weekly Pay = ", the value of the variable weeklyPay, and a newline character. When run, the programme will produce the following output:

Weekly Pay = 1445.625

When a variable is defined, its value is undefined until it is actually assigned one. For example, weeklyPay has an undefined value/unpredictable value (*i.e.*, whatever happens to be in the memory location which the variable denotes at the time) until line 9 is executed.

INITIALIZATION

It is important to ensure that a variable is initialized before it is used in any computation. It is possible to define a variable and initialize it at the same time. This is considered a good programming practice, because it pre-empts the possibility of using the variable prior to it being initialized. For all intents and purposes, the two programmes are equivalent.

```

1  #include <iostream.h>
2  int main (void)
3  {
4      int workDays = 5;
5      float workHours = 7.5;
```

```

6   float payRate = 38.55;
7   float weeklyPay = workDays * workHours * payRate;
8   cout << "Weekly Pay = ";
9   cout << weeklyPay;
10      cout << '\n';
11  }

```

OBJECTS

The abstract class at the top of the class hierarchy is called Object. With the exception of the Ownership class, all the other classes in the hierarchy are derived from this class. Programme gives the declaration of the Object class. Altogether only six member functions are declared: the destructor, IsNull, Hash, Put, Compare and Compare To.

Programme: Object Class Definition The Object class destructor is declared virtual. It is essential to declare it so, because the Object is an abstract class, and instances of derived class objects are very likely to be accessed through the base class interface. In particular, it is necessary for the destructor of the derived class to be called through the base class interface—this is precisely what the virtual keyword achieves.

The is Null function is a pure virtual member function which returns a Boolean value. This function is used in conjunction with the Null Object concrete classes described below in Section . The is Null function shall return false for all object instances derived from Object except if the object instance is the Null Object. We will see later that certain functions return object references. For example, a function which searches through a data structure for a particular object returns a reference to that object if it is found. If the object is not found, the search returns a reference to the Null Object instance. By using the is Null function, the programmer can test whether the search was successful. This is analogous to the NULL pointer in the C programming language and to the pointer value 0 in C++.

The Hash function is a pure virtual member function which returns a Hash Value. The Hash function is used in the implementation of hash tables which are discussed. We have put the Hash function in the Object class interface, so that it is possible to store any object derived from Object in a hash table. What the Hash function computes is unspecified. The only requirement is that it is *idempotent* . I.e., given an object instance obj, obj.Hash() does not modify the object in any way, and as long as obj is not modified in the interim, repeated calls always give exactly the same result.

Two functions for comparing objects are declared—Compare and Compare To. Compare is a public member function. That takes a const reference to Object and returns an int. Given two objects obj1 and obj2, calling obj1.Compare (obj2) *compares* the value of obj1 with the value of obj2. The result is equal to zero if; less than zero if; and, greater than zero if.

It is not necessary that the two objects compared using Compare have the same type. As long as they are instances of classes derived from Object, they can be compared.

The Compare function is used in the overloading of the comparison operators `operator==`, `operator!=`, `operator<`, `operator<=`, `operator>`, and `operator>=`, as shown in Programme .

The second comparison function, `CompareTo`, is a pure virtual function. An implementation for this function must be given in every concrete class derived from `Object`. The purpose of this function is to compare two objects that are both instances of the same derived class.

The purpose of the `Put` member function of the `Object` class is to output a human-readable representation of the object on the specified output stream. The `Put` function is a pure virtual function. However, since it has the side-effect of output, it is not strictly idempotent. Nevertheless, it is a `const` member function which means that calling the `Put` member function does not modify the object in any way. The use of the `Put` in the overloading of the ostream insertion operator, `operator<<`, is shown in Programme .

The use of polymorphism in the way shown gives the programmer enormous leverage. The fact almost all objects will be derived from the `Object` base class, together with the fact that the `Put` and `Compare` to member functions are virtual functions, ensures that the overloaded operators work as we expect for all derived class instances.

Only three functions need to be implemented to complete the definition of the `Object` class. These are shown in Programme. The `Object` class destructor is trivial. Since there are no member variables, nothing remains to be cleaned up. It is necessary to define the destructor in any event. As explained above, the destructor needs to be virtual and because in C++ there is no such thing as a pure virtual destructor.

The implementation of the `IsNull` member function is trivial. It simply returns the Boolean value `false`. The `Compare` member function makes use of *runtime type information* to ensure that the `CompareTo` member function is only invoked for objects which are of the same class. For objects which are not instances of the same class, the `before` function is used to order those objects based on their types. The C++ runtime library provides an implementation for the `before` function which returns true if the left hand type precedes the right hand type in the implementation's collation order.

The declaration of the `Null Object` class is given in Programme. The `Null Object` class is a concrete class derived from the `Object` abstract base class. The `Null Object` class is also a *singleton* class. *i.e.*, it is a class of which there will only ever be exactly one instance. The one instance is in fact a static member variable of the class itself. In order to ensure that no other instances can be created, the default constructor is declared to be a private member function. The static member function `Instance` returns a reference to the one and only `Null Object` instance.

Programme shows how the member functions of the `NullObject` class are defined. The most important characteristic of the `Null Object` class is that its `IsNull` member function returns the value `true`.

The remaining functions are trivial: The constructor does nothing; the two member functions `Hash` and `CompareTo` both return zero; and the `Put` function simply prints "Null Object". Finally, the `Instance` static member function returns a reference to the

static member variable instance which is the one and only instance of the Null Object class. One of the design goals of the C++ language is to treat user-defined data types and the built-in data types as equally as possible. However, the built-in data types of C++ do have one shortcoming with respect to user-defined ones—a built-in data type cannot be used as the base class from which other classes are derived. *E.g.*, it is not possible to write:

```
class D: public int//Wrong!
```

Consequently, it is not possible to extend the functionality of a built-in type using inheritance.

The usual design pattern for dealing with this deficiency is to put instances of the built-in types inside a *wrapper* class and to use the wrapped objects in place of the built-in types. Programme illustrates this idea. The class Wrapper <T> is a concrete class which is derived from the abstract base class Object. It is also a generic class, the purpose of which is to encapsulate an object of type T. Each instance of the Wrapper <T> class contains a single member variable of type T called datum. A *class* is an expanded concept of a data structure: instead of holding only data, it can hold both data and functions. An *object* is an instantiation of a class. In terms of variables, a class would be the type, and an object would be the variable.

Classes are generally declared using the keyword class, with the following format:

```
class class_name {  
access_specifier_1:  
member1;  
access_specifier_2:  
member2;  
...  
} object_names;
```

Where class_name is a valid identifier for the class, object_names is an optional list of names for objects of this class. The body of the declaration can contain members, that can be either data or function declarations, and optionally access specifier. All is very similar to the declaration on data structures, except that we can now include also functions and members, but also this new thing called *access specifier*. An access specifier is one of the following three keywords: private, public or protected.

These specifier modify the access rights that the members following them acquire:

- Private members of a class are accessible only from within other members of the same class or from their *friends*.
- Protected members are accessible from members of their same class and from their friends, but also from members of their derived classes.
- Finally, public members are accessible from anywhere where the object is visible.

By default, all members of a class declared with the class keyword have private access for all its members. Therefore, any member that is declared before one other class specifier automatically has private access.

For example:

```
class CRectangle {
int x, y;
public:
void set_values (int,int);
int area (void);
} rect;
```

Declares a class (*i.e.*, a type) called CRectangle and an object (*i.e.*, a variable) of this class called rect. This class contains four members: two data members of type int (member x and member y) with private access (because private is the default access level) and two member functions with public access: set_values() and area(), of which for now we have only included their declaration, not their definition.

Notice the difference between the class name and the object name: In the previous example, CRectangle was the class name (*i.e.*, the type), whereas rect was an object of type CRectangle. It is the same relationship int and a have in the following declaration:

```
int a;
```

where int is the type name (the class) and a is the variable name (the object). After the previous declarations of CRectangle and rect, we can refer within the body of the programme to any of the public members of the object rect as if they were normal functions or normal variables, just by putting the object's name followed by a dot (.) and then the name of the member. All very similar to what we did with plain data structures before.

For example:

```
rect.set_values (3,4);
myarea = rect.area();
```

The only members of rect that we cannot access from the body of our programme outside the class are x and y, since they have private access and they can only be referred from within other members of that same class.

Here is the complete example of class CRectangle:

```
//classes example
#include <iostream>
using namespace std;
class CRectangle {
int x, y;
public:
void set_values (int,int);
int area () {return (x*y);}
};
void CRectangle::set_values (int a, int b) {
x = a;
y = b;
}
int main () {
CRectangle rect;
rect.set_values (3,4);
cout << "area: " << rect.area();
```

```
return 0;
} area: 12
```

The most important new thing in this code is the operator of scope (::, two colons) included in the definition of `set_values()`. It is used to define a member of a class from outside the class declaration itself.

You may notice that the definition of the member function `area()` has been included directly within the definition of the `CRectangle` class given its extreme simplicity, whereas `set_values()` has only its prototype declared within the class, but its definition is outside it. In this outside declaration, we must use the operator of scope (::) to specify that we are defining a function that is a member of the class `CRectangle` and not a regular global function.

The scope operator (::) specifies the class to which the member being declared belongs, granting exactly the same scope properties as if this function definition was directly included within the class definition. For example, in the function `set_values()` of the previous code, we have been able to use the variables `x` and `y`, which are private members of class `CRectangle`, which means they are only accessible from other members of their class.

The only difference between defining a class member function completely within its class and to include only the prototype and later its definition, is that in the first case the function will automatically be considered an inline member function by the compiler, while in the second it will be a normal (not-inline) class member function, which in fact supposes no difference in behaviour.

Members `x` and `y` have private access (remember that if nothing else is said, all members of a class defined with keyword `class` have private access). By declaring them private we deny access to them from anywhere outside the class. This makes sense, since we have already defined a member function to set values for those members within the object: the member function `set_values()`. Therefore, the rest of the programme does not need to have direct access to them. Perhaps in a so simple example as this, it is difficult to see an utility in protecting those two variables, but in greater projects it may be very important that values cannot be modified in an unexpected way (unexpected from the point of view of the object).

One of the greater advantages of a class is that, as any other type, we can declare several objects of it. For example, following with the previous example of class `CRectangle`, we could have declared the object `rectb` in addition to the object `rect`:

```
//example: one class, two objects
#include <iostream>
using namespace std;
class CRectangle {
int x, y;
public:
void set_values (int,int);
int area () {return (x*y);}
};
```

```

void CRectangle::set_values (int a, int b) {
    x = a;
    y = b;
}
int main () {
    CRectangle rect, rectb;
    rect.set_values (3,4);
    rectb.set_values (5,6);
    cout << "rect area: " << rect.area() << endl;
    cout << "rectb area: " << rectb.area() << endl;
    return 0;
} rect area: 12
rectb area: 30

```

In this concrete case, the class (type of the objects) to which we are talking about is `CRectangle`, of which there are two instances or objects: `rect` and `rectb`. Each one of them has its own member variables and member functions.

Notice that the call to `rect.area()` does not give the same result as the call to `rectb.area()`. This is because each object of class `CRectangle` has its own variables `x` and `y`, as they, in some way, have also their own function members `set_value()` and `area()` that each uses its object's own variables to operate.

That is the basic concept of *object-oriented programming*: Data and functions are both members of the object. We no longer use sets of global variables that we pass from one function to another as parameters, but instead we handle objects that have their own data and functions embedded as members. Notice that we have not had to give any parameters in any of the calls to `rect.area` or `rectb.area`. Those member functions directly used the data members of their respective objects `rect` and `rectb`.

OBJECT-ORIENTATION

While the exact definition of the term is highly variable depending upon who you ask, there are several qualities that most will agree an Object-Oriented language should have:

- Encapsulation/Information Hiding.
- Inheritance.
- Polymorphism/Dynamic Binding.
- All pre-defined types are Objects.
- All operations performed by sending messages to Objects.
- All user-defined types are Objects.

For the purposes of this discussion, a language is considered to be a “pure” Object-Oriented languages if it satisfies all of these qualities. A “hybrid” language may support some of these qualities, but not all. In particular, many languages support the first three qualities, but not the final three.

C++ is considered to be a multi-paradigm language, of which one paradigm it supports is Object-Orientation. Thus, C++ is not (nor does it contend to be) a pure Object-Oriented language.

Python is often heralded as an Object-Oriented language, but its support for Object-Orientation seems to have been tacked on. Some operations are implemented as methods, while others are implemented as global functions. Also, the need for an explicit "self" parameter for methods is awkward. Some complain about Python's lack of "private" or "hidden" attributes, which goes against the Encapsulation/Information Hiding principle, while others feel that Python's "privateness is by convention" approach offers all of the practical benefits as language-enforced encapsulation without the hassle. The Ruby language, on the other hand, was created in part as a reaction to Python.

TYPES OF TYPING

The debate between static and dynamic typing has raged in Object-Oriented circles for many years with no clear conclusion. Proponents of dynamic typing contend that it is more flexible and allows for increased productivity. Those who prefer static typing argue that it enforces safer, more reliable code, and increases efficiency of the resulting product.

It is futile to attempt to settle this debate here except to say that a statically-typed language requires a very well-defined type system in order to remain as flexible as its dynamically-typed counterparts. Without the presence of genericity (templates, to use the C++ patois) and multiple type inheritance (not necessarily the same as multiple implementation inheritance), a static type system may severely inhibit the flexibility of a language. In addition, the presence of "casts" in a language can undermine the ability of the compiler to enforce type constraints.

A dynamic type system doesn't require variables to be declared as a specific type. Any variable can contain any value or object. Smalltalk and Ruby are two pure Object-Oriented languages that use dynamic typing. In many cases this can make the software more flexible and amenable to change. However, care must be taken that variables hold the expected kind of object. Typically, if a variable contains an object of a different type than a user of the object expects, some sort of "message not understood" error is raised at runtime. Users of dynamically-typed languages claim that this type of error is infrequent in practice.

Statically-typed languages require that all variables are declared with a specific type. The compiler will then ensure that at any given time the variable contains only an object compatible with that type. (We say "compatible with that type" rather than "exactly that type" since the inheritance relationship enables subtyping, in which a class that inherits from another class is said to have an IS-A relationship with the class from which it inherits, meaning that instances of the inheriting class can be considered to be of a compatible type with instances of the inherited class.)

By enforcing the type constraint on objects contained or referred to by the variable, the compiler can ensure a "message not understood" error can never occur at runtime. On the other hand, a static type system can hinder evolution of software in some circumstances. For example, if a method takes an object as a parameter, changing the

type of the object requires changing the signature of the method so that it is compatible with the new type of the object being passed.

If this same object is passed to many such methods, all of them must be updated accordingly, which could potentially be an arduous task. One must remember, though, that this ripple effect could occur even a dynamically-typed language.

If the type of the object is not what it was originally expected to be, it may not understand the messages being sent to it. Perhaps even worse is that it could understand the message but interpret it in a way not compatible with the semantics of the calling method. A statically-typed language can flag these errors at compilation-time, pointing out the precise locations of potential errors. A user of a dynamically-typed language must rely on extensive testing to ensure that all improper uses of the object are tracked down.

Eiffel is a statically-typed language that manages to remain nearly as flexible as its dynamic counterparts. Eiffel's generic classes and unprecedentedly flexible inheritance model allow it to achieve the safety and reliability of a static type system while still remaining nearly as flexible as a dynamic type system, all without requiring (nor allowing) the use of type casts. C++ also offers generic classes (known as "templates" in the C++ parlance), as well as multiple inheritance. Unfortunately, the presence of type casts and implicit type conversions can sometimes undermine the work of the compiler by allowing type errors to go undetected until runtime. Java is seriously hindered by a lack of generic classes.

This is alleviated to a degree by Java's singly-rooted type hierarchy (*i.e.* every class descends directly or indirectly from the class `Object`), but this scheme leaves much to be desired in terms of type-safety. Forthcoming versions of Java will address this shortcoming when generic classes are introduced in Java 1.5 or later.

GENERIC CLASSES

Generic classes, and more generally parametric type facilities, refer to the ability to parameterize a class with specific data types. A common example is a stack class that is parameterized by the type of elements it contains. This allows the stack to simultaneously be compile-time type safe and yet generic enough to handle any type of elements.

The primary benefit of parameterized types is that it allows statically typed languages to retain their compile-time type safety yet remain nearly as flexible as dynamically typed languages. Eiffel in particular uses generics extensively as a mechanism for type safe generic containers and algorithms. C++ templates are even more flexible, having many uses apart from simple generic containers, but also much more complex.

As already mentioned in the previous section, Java's lack of generic classes is a severe hole in the Java type system. When one considers that most living objects in a programme are stored in container classes, and that containers in Java are untyped due to lack of generics, it is questionable whether Java's type system provides any benefit over the more flexible dynamic counterparts. See also this article by Dave Thomas for

a discussion of Java's type system in regards to its lack of generics. Dynamically typed languages do not need parameterized types in order to support generic programming. Types are checked at runtime and thus dynamically typed languages support generic programming inherently.

INHERITANCE

Inheritance is the ability for a class or object to be defined as an extension or specialization of another class or object. Most object-oriented languages support class-based inheritance, while others such as SELF and JavaScript support object-based inheritance. A few languages, notably Python and Ruby, support both class- and object-based inheritance, in which a class can inherit from another class and individual objects can be extended at run time with the capabilities of other objects. For the remainder of this discussion, we'll be dealing primarily with class-based inheritance since it is by far the most common model.

Although commonly thought of as simple subtyping mechanism, there are actually many different uses of inheritance. In his landmark book *Object-Oriented Software Construction*, Bertrand Meyer identified and classified as many as 17 different forms of inheritance. Even so, most languages provide only a few syntactic constructs for inheritance which are general enough to allow inheritance to be used in many different ways. The most important distinction that can be made between various languages' support for inheritance is whether it supports single or multiple inheritance. Multiple inheritance is the ability for a class to inherit from more than one super (or base) class. For example, an application object called Persistent Shape might inherit from both Graphical Object and Persistent Object in order to be used as both a graphical object that can be displayed on the screen as well as a persistent object that can be stored in a database.

Multiple inheritance would appear to be an essential feature for a language to support for cases such as the above when two or more distinct hierarchies must be merged into one application domain. However, there are other issues to consider before making such an assertion.

First, we must consider that multiple inheritance introduces some complications into a programming language supporting it. Issues such as name clashes and ambiguities introduced in the object model must be resolved by the language in order for multiple inheritance and this leads to additional complexity in the language. Eiffel is known for its carefully and thoroughly well-designed support for multiple inheritance, which features feature renaming and fine-grained control over the manner in which multiply-inherited features are selected and applied to the inheriting class. The mechanisms C++ provides for multiple inheritance are more complicated and less flexible leading many people to (mistakenly) believe that multiple inheritance is inherently ill-conceived and complex.

Next, we must distinguish between implementation inheritance and interface/subtype inheritance. Subtype inheritance (also known loosely as interface inheritance) is the

most common form of inheritance, in which a subclass is considered to be a subtype of its super class, commonly referred to as an IS-A relationship. What this means is that the language considers an object to conform to the type of its class *or any of its super classes*. For example, a Circle IS-A Shape, so anywhere a Shape is used in a programme, a Circle may be used as well. This conformance notion is only applicable to statically typed languages since it is a feature used by the compiler to determine type correctness.

Implementation inheritance is the ability for a class to inherit part or all of its implementation from another class. For example, a Stack class that is implemented using an array might inherit from an Array class in order to define the Stack in terms of the Array. In this way, the Stack class could use any features from the Array to support its own implementation. With pure implementation inheritance, the fact that the Stack inherits its implementation from Array would not be visible to code using the Stack; the inheritance would be strictly an implementation matter. C++ supports this notion directly with “private inheritance”, in which methods from the base class are made private in the derived class. Recent versions of Eiffel also support this form of pure implementation inheritance using what is known as non-conforming inheritance. Most languages, on the other hand, do not support pure implementation inheritance so a class that inherits from another class is always considered to be a subtype of its super class(es).

Returning to the issue of multiple inheritance, we can see that a language’s support for multiple inheritance is not a boolean condition; a language can support one or more different forms of multiple inheritance in the same way it can support different forms of single inheritance (*e.g.* implementation and subtype inheritance). We’ve already seen that C++ and Eiffel independently support pure implementation inheritance as well as subtype inheritance. Both of these languages also support multiple inheritance in both forms. Java, while it does not support *pure* implementation inheritance, provides two separate inheritance mechanisms. The `extends` keyword is used for a combination of implementation and subtype inheritance while the `implements` keyword is used for pure subtype (interface) inheritance.

Subtype inheritance is less important in dynamic languages since type conformance is not generally an issue, so multiple implementation inheritance is preferred over multiple subtype inheritance (although most languages still consider any class inheriting from another to be a subtype). Smalltalk supports only a single notion of inheritance: single inheritance of both interface and implementation. This means that a class may only inherit from one other class and it inherits both implementation and interface.

Python similarly supports one form of inheritance (both implementation and subtype) but allows multiple inheritance and is thus more flexible in this regard than Smalltalk. Ruby lies somewhere in between the two approaches by allowing a class to inherit from only one class but also allowing a class to “mix in” the implementation of an arbitrary number of modules. This model is a slightly restricted version of the model provided by Python, but the restrictions can be overcome by Ruby’s ability to support a prototype-based approach using object-based inheritance.

FEATURE RENAMING

Feature renaming is the ability for a class or object to rename one of its features (a term we'll use to collectively refer to attributes and methods) that it inherited from a super class.

There are two important ways in which this can be put to use:

- Provide a feature with a more natural name for its new context
- Resolve naming ambiguities when a name is inherited from multiple inheritance paths

As an example of the first use, consider again a stack implemented by inheriting from an array. The array might provide an operation called `remove_last` to remove the last element of the array. In the stack, this operation is more appropriately named `pop`. Eiffel and Ruby both provide support for feature renaming. Ruby provides an alias method that allows you to alias any arbitrary method. Eiffel also provides support for feature renaming, although it is slightly more limited than in Ruby because you can only rename a feature in an inheritance clause.

METHOD OVERLOADING

Method overloading (also referred to as parametric polymorphism) is the ability for a class, module, or other scope to have two or more methods with the same name. Calls to these methods are disambiguated by the number and/or type of arguments passed to the method at the call site. For example, a class may have multiple `print` methods, one for each type of thing to be printed. The alternative to overloading in this scenario is to have a different name for each print method, such as `print_string` and `print_integer`.

Java and C++ both support method overloading in a similar fashion. Complexities in the mechanism to disambiguate calls to overloaded methods have lead some language designers to avoid overloading in their languages. None of the other languages under consideration support method overloading. Default argument values provide a subset of the behaviour for which method overloading is used, and some languages such as Ruby and Python have chosen this route instead.

OPERATOR OVERLOADING

Operator overloading is the ability for a programmer to define an operator (such as `+`, or `*`) for user-defined types. This allows the operator to be used in infix, prefix, or postfix form, rather than the standard functional form. For example, a user-defined `Matrix` type might provide a `*` infix operator to perform matrix multiplication with the familiar notation: `matrix1 * matrix2`.

Some (correctly) consider operator overloading to be mere syntactic “sugar” rather than an essential feature, while others (also correctly) point to the need for such syntactic sugar in numerical and other applications. Both points are valid, but it is clear that, when used appropriately, operator overloading can lead to much more readable

programmes. When abused, it can lead to cryptic, obfuscated code. Consider that in the presence of operator overloading, it may not be clear whether a given operator is built in to the language or defined by the user.

For any language that supports operator overloading, two things are necessary to alleviate such obfuscation:

- All operations must be messages to objects, and thus all operators are always method calls.
- Operators must have an equivalent functional form, so that using the operator as a method call will behave precisely the same as using it in infix, prefix, or postfix form.

This second point is subtle. It means that given any operator, it must be possible to invoke that operator in functional form. For example, the following two expressions should be equivalent: $1 + 2$ and $1.+(2)$. This ensures that no implicit behaviour is taking place that may not be immediately obvious from examining the source text.

HIGH ORDER FUNCTIONS

Higher order functions are, in the simplest sense, functions that can be treated as if they were data objects. In other words, they can be bound to variables (including the ability to be stored in collections), they can be passed to other functions as parameters, and they can be returned as the result of other functions. Due to this ability, higher order functions may be viewed as a form of deferred execution, wherein a function may be defined in one context, passed to another context, and then later invoked by the second context. This is different from standard functions in that higher order functions represent anonymous lambda functions, so that the invoking context need not know the name of the function being invoked.

Lexical closures (also known as static closures, or simply closures) take this one step further by bundling up the lexical (static) scope surrounding the function with the function itself, so that the function carries its surrounding environment around with it wherever it may be used. This means that the closure can access local variables or parameters, or attributes of the object in which it is defined, and will continue to have access to them even if it is passed to another module outside of its scope.

Among the languages we're considering, Smalltalk and Ruby have supported both higher order functions and lexical closures from the beginning in the form of blocks. A block is an anonymous function that may be treated as any other data object, and is also a lexical closure. Eiffel has recently added support for higher order functions using the "agent" mechanism. The inline variant of Eiffel agents forms a lexical closure. Python, which has long supported higher order functions in the form of lambda expressions, has recently added support for closures using its improved support for nested static scopes.

While neither Java nor C++ support higher order functions directly, both provide mechanisms for mimicking their behaviour. Java's anonymous classes allow a function to be bundled with an object that can be treated much as a higher order function can. It

can be bound to variables, passed to other functions as an argument, and can be returned as the result of a function. However, the function itself is named and thus cannot be treated in a generic fashion as true higher order functions can. C++ similarly provides partial support for higher order functions using function objects (or “functors”), and add the further benefit that the function call operator may be overloaded so that functors may be treated generically. Neither C++ nor Java, however, provide any support for lexical closures.

LANGUAGE IMPLEMENTATION

Garbage collection is a mechanism allowing a language implementation to free memory of unused objects on behalf of the programmer, thus relieving the burden on the programmer to do so. The alternative is for the programmer to explicitly free any memory that is no longer needed. There are several strategies for garbage collection that exist in various language implementations.

Reference counting is the simplest scheme and involves the language keeping track of how many references there are to a particular object in memory, and deleting that object when that reference count becomes zero. This scheme, although it is simple and deterministic, is not without its drawbacks, the most important being its inability to handle cycles. Cycles occur when two objects reference each other, and thus there reference counts will never become zero even if neither object is referenced by any other part of the programme. This is the scheme that is utilized by Python and Visual Basic, although in the case of Python an extra step is taken to ensure that cycles are handled appropriately.

“Mark and sweep” garbage collection is another scheme that overcomes this limitation. A mark and sweep garbage collector works in a two phase process, not surprisingly known as the mark phase and the sweep phase. The mark phase works by first starting at the “root” objects (objects on the stack, global objects, etc.), marking them as live, and recursively marking any objects referenced from them. These marked objects are the set of live objects in programme, and any objects that were not marked in this phase are unreferenced and therefore candidates for collection. In the sweep phase, any objects in memory that were not marked as live by the mark phase are deleted from memory.

The primary drawback of mark and sweep collection is that it is non-deterministic, meaning that objects are deleted at an unspecified time during the execution of the programme. This is the most common form of garbage collection, and the one that is supported by most implementations of Eiffel, Smalltalk, Ruby, and Java.

Generational garbage collection works in a similar fashion to mark and sweep garbage collection, except it capitalizes on the statistical probability that objects that have been alive the longest tend to stay alive longer than objects that were newly created. Thus a generational garbage collector will divide objects into “generations” based upon how long they’ve been alive. This division can be used to reduce the time spent in the mark

and sweep phases because the oldest generation of objects will not need to be collected as frequently. Generational garbage collectors are not as common as the other forms but may be found in some implementations of Eiffel, Smalltalk, Ruby, and Java.

C++ does not provide any sort of garbage collection, the reasons for which are discussed at length in Bjarne Stroustrup's *The Design and Evolution of C++*. It is possible, however, with some effort to layer reference counting garbage collection onto C++ using smart pointers. In addition there exist garbage collectors that can be integrated into C++ programmes, though their use has not caught on to any great degree within the C++ community.

NORMAL ACCESS

The Uniform Access Principle seeks to eliminate this needless coupling. A language supporting the Uniform Access Principle does not exhibit any notational differences between accessing a feature regardless of whether it is an attribute or a function. Thus, in our earlier example, access to `bar` would always be in the form of `foo.bar`, regardless of how `bar` is implemented. This makes clients of `Foo` more resilient to change.

Among our languages, only Eiffel and Ruby directly support the Uniform Access Principle, although Smalltalk renders the distinction moot by not allowing any access to attributes from clients.

CLASS VARIABLES/METHODS

Class variables and methods are owned by a class, and not any particular instance of a class. This means that for however many instances of a class exist at any given point in time, only one copy of each class variable/method exists and is shared by every instance of the class.

Smalltalk and Ruby support the most advanced notion of class variables and methods, due to their use of meta-classes and the fact that even classes are objects in these languages. Java and C++ provide “static” members which are effectively the same thing, yet more limited since they cannot be inherited. Python, surprisingly, does not support class methods or variables, but its advanced notion of a module allows workarounds for this limitation. Eiffel also does not provide direct support for class variables or methods, but it does provide similar, but limited, functionality in the form of “once” functions. Once functions are evaluated once only, and subsequent uses use a cached result.

REFLECTION

Reflection is the ability for a programme to determine various pieces of information about an object at runtime. This includes the ability to determine the type of the object, its inheritance structure, and the methods it contains, including the number and types of parameters and return types. It might also include the ability for

determining the names and types of attributes of the object. Most object-oriented languages support some form of reflection. Smalltalk, Ruby, and Python in particular have very powerful and flexible reflection mechanisms. Java also supports reflection, but not in as flexible and dynamic fashion as the others. C++ does not support reflection as we've defined it here, but it does supported a limited form of runtime type information that allows a programme to determine the type of an object at runtime. Eiffel also has support for a limited form of reflection, although it is much improved in the most recent versions of Eiffel, including the ability to determine the features contained in an object.

ACCESS CONTROL

Access control refers to the ability for a modules implementation to remain hidden behind its public interface. Access control is closely related to the encapsulation/information hiding principle of object-oriented languages. For example, a class Person may have methods such as name and e-mail, that return the person's name and e-mail address respectively. How these methods work is an implementation detail that should not be available to users of the Person class. These methods may, for example, connect to a database to retrieve the values. The database connection code that is used to do this is not relevant to client code and should not be exposed. Language-enforced access control allows us to enforce this.

Most object-oriented languages provide at least two levels of access control: public and protected. Protected features are not available outside of the class in which they are contained, except for subclasses. This is the scheme supported by Smalltalk, in which all methods are public and all attributes are protected. There are no protected methods in Smalltalk, so Smalltalk programmers resort to the convention of placing methods that should be protected into a "private protocol" of the class. See this discussion for the benefits and drawbacks of this approach. Visual Basic also supports these two levels of access control, although since there is no inheritance in Visual Basic, protected features are effectively private to the class in which they are declared. Some languages, notably Java and C++, provide a third level of access control known as "private". Private features are not available outside of the class in which they are declared, even for subclasses. Note, however, that this means that objects of a particular class can access the private features of other objects of that same class. Ruby also provides these three levels of access control, but they work slightly differently. Private in Ruby means that the feature cannot be accessed through a receiver, meaning that the feature will be available to subclasses, but not other instances of the same class. Java provides a fourth level of, known as "package private" access control which allows other classes in the same package to access such features.

Eiffel provides the most powerful and flexible access control scheme of any of these languages with what is known as "selective export". All features of an Eiffel class are by default public. However, any feature in an Eiffel class may specify an export clause

which lists explicitly what other classes may access that feature. The special class `NONE` may be used to indicate that *no* other class may access that feature. This includes attributes, but even public attributes are read only so an attribute can never be written to directly in Eiffel. In order to better support the Open-Closed principle, all features of a class are always available to subclasses in Eiffel, so there is no notion of private as there is in Java and C++. Python, curiously, does not provide any enforced access control. Instead, it provides a mechanism of name mangling: any feature that begins with underscores will have its name mangled by the Python interpreter. Although this does not prevent client code from using such features, it is a clear indicator that the feature is not intended for use outside the class and convention dictates that these features are “use at your own risk”.

FEATURES OF DBC

- *Preconditions*: Which are conditions that must be true before a method is invoked
- *Post-conditions*: Which are conditions guaranteed to be true after the invocation of a method
- *Invariants*: Which are conditions guaranteed to be true at any stable point during the lifetime of an object

There is much more to DBC than these simple facilities, including the manner in which precondition, post-conditions, and invariants are inherited in compliance with the Liskov Substitution Principle. However, at least these facilities must be present to support the central notions of DBC.

OBJECT-ORIENTED LANGUAGE FEATURES

Abstract data types inheritance object identity Object-Oriented Database Features: persistence support of transactions simple querying of bulk data concurrent access resilience security

WHY OBJECT-ORIENTED DATABASE

- *Industry Trends*: Integration and Sharing.
- Seamless integration of operating systems, databases, languages, spreadsheets, word processors, AI expert system shells.
- Sharing of data, information, software components, products, computing environments.
- *Referential sharing*: Multiple applications, products, or objects share common sub-objects. (Hypermedia links are then used to navigate from one object to another) Object-oriented databases allows referential sharing through the support of object identity and inheritance.

STRATEGY FOR DEFINING IMMUTABLE OBJECTS

The following rules define a simple strategy for creating immutable objects. Not all classes documented as “immutable” follow these rules. This does not necessarily mean the creators of these classes were sloppy — they may have good reason for believing that instances of their classes never change after construction.

However, such strategies require sophisticated analysis and are not for beginners:

- Don’t provide “setter” methods — methods that modify fields or objects referred to by fields.
- Make all fields final and private.
- Don’t allow subclasses to override methods. The simplest way to do this is to declare the class as final. A more sophisticated approach is to make the constructor private and construct instances in factory methods.
- If the instance fields include references to mutable objects, don’t allow those objects to be changed:
 - Don’t provide methods that modify the mutable objects.
 - Don’t share references to the mutable objects. Never store references to external, mutable objects passed to the constructor; if necessary, create copies, and store references to the copies. Similarly, create copies of your internal mutable objects when necessary to avoid returning the originals in your methods.

Applying this strategy to SynchronizedRGB results in the following steps:

- There are two setter methods in this class. The first one, set, arbitrarily transforms the object, and has no place in an immutable version of the class. The second one, invert, can be adapted by having it create a new object instead of modifying the existing one.
- All fields are already private; they are further qualified as final.
- The class itself is declared final.
- Only one field refers to an object, and that object is itself immutable. Therefore, no safeguards against changing the state of “contained” mutable objects are necessary.

After these changes, we have ImmutableRGB:

```
final public class ImmutableRGB {
    // Values must be between 0 and 255.
    final private int red;
    final private int green;
    final private int blue;
    final private String name;
    private void check(int red,
                       int green,
                       int blue) {
        if (red < 0 || red > 255
            || green < 0 || green > 255
```



```

        || blue < 0 || blue > 255) {
            throw new IllegalArgumentException();
        }
    }
    public ImmutableRGB(int red,
                        int green,
                        int blue,
                        String name) {
        check(red, green, blue);
        this.red = red;
        this.green = green;
        this.blue = blue;
        this.name = name;
    }
    public int getRGB() {
        return ((red << 16) | (green << 8) | blue);
    }
    public String getName() {
        return name;
    }
    public ImmutableRGB invert() {
        return new ImmutableRGB(255 - red,
                                255 - green,
                                255 - blue,
                                "Inverse of " + name);
    }
}

```

HIGH LEVEL CONCURRENCY OBJECTS

So far, this lesson has focused on the low-level APIs that have been part of the Java platform from the very beginning. These APIs are adequate for very basic tasks, but higher-level building blocks are needed for more advanced tasks.

This is especially true for massively concurrent applications that fully exploit today's multiprocessor and multi-core systems.

In this section we'll look at some of the high-level concurrency features introduced with version 5. 0 of the Java platform. Most of these features are implemented in the new `java. util. concurrent` packages.

There are also new concurrent data structures in the Java Collections Framework:

- Lock objects support locking idioms that simplify many concurrent applications.
- Executors define a high-level API for launching and managing threads. Executor implementations provided by `java. util. concurrent` provide thread pool management suitable for large-scale applications.
- Concurrent collections make it easier to manage large collections of data, and can greatly reduce the need for synchronization.

- Atomic variables have features that minimize synchronization and help avoid memory consistency errors.
- `ThreadLocalRandom` (in JDK 7) provides efficient generation of pseudorandom numbers from multiple threads.

LOCK OBJECTS

Synchronized code relies on a simple kind of re-entrant lock. This kind of lock is easy to use, but has many limitations. More sophisticated locking idioms are supported by the `java.util.concurrent.locks` package. We won't examine this package in detail, but instead will focus on its most basic interface, `Lock`.

Lock objects work very much like the implicit locks used by synchronized code. As with implicit locks, only one thread can own a `Lock` object at a time. Lock objects also support a wait/notify mechanism, through their associated `Condition` objects.

The biggest advantage of `Lock` objects over implicit locks is their ability to back out of an attempt to acquire a lock. The `tryLock` method backs out if the lock is not available immediately or before a timeout expires (if specified). The `lockInterruptibly` method backs out if another thread sends an interrupt before the lock is acquired. Let's use `Lock` objects to solve the deadlock problem we saw in `Liveness`. Alphonse and Gaston have trained themselves to notice when a friend is about to bow. We model this improvement by requiring that our `Friend` objects must acquire locks for both participants before proceeding with the bow. Here is the source code for the improved model, `Safelock`. To demonstrate the versatility of this idiom, we assume that Alphonse and Gaston are so infatuated with their new-found ability to bow safely that they can't stop bowing to each other:

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.Random;
public class Safelock {
    static class Friend {
        private final String name;
        private final Lock lock = new ReentrantLock();
        public Friend(String name) {
            this.name = name;
        }
        public String getName() {
            return this.name;
        }
        public boolean impendingBow(Friend bower) {
            Boolean myLock = false;
            Boolean yourLock = false;
            try {
                myLock = lock.tryLock();
                yourLock = bower.lock.tryLock();
            } finally {
```

```

        if (! (myLock && yourLock)) {
            if (myLock) {
                lock.unlock();
            }
            if (yourLock) {
                bower.lock.unlock();
            }
        }
        return myLock && yourLock;
    }
    public void bow(Friend bower) {
        if (impendingBow(bower)) {
            try {
                System.out.format("%s: %s has"
                    + " bowed to me!\n",
                    this.name, bower.getName());
                bower.bowBack(this);
            } finally {
                lock.unlock();
                bower.lock.unlock();
            }
        } else {
            System.out.format("%s: %s started"
                + " to bow to me, but saw that"
                + " I was already bowing to"
                + " him.\n",
                this.name, bower.getName());
        }
    }
    public void bowBack(Friend bower) {
        System.out.format("%s: %s has" +
            " bowed back to me!\n",
            this.name, bower.getName());
    }
}
static class BowLoop implements Runnable {
    private Friend bower;
    private Friend bowee;
    public BowLoop(Friend bower, Friend bowee) {
        this.bower = bower;
        this.bowee = bowee;
    }
    public void run() {
        Random random = new Random();
        for (;;) {
            try {
                Thread.sleep(random.nextInt(10));
            } catch (InterruptedException e) {}
            bowee.bow(bower);
        }
    }
}

```

```

        }
    }
}

public static void main(String[] args) {
    final Friend alphonse =
        new Friend("Alphonse");
    final Friend gaston =
        new Friend("Gaston");
    new Thread(new BowLoop(alphonse, gaston)).start();
    new Thread(new BowLoop(gaston, alphonse)).start();
}
}

```

EXECUTORS

In all of the previous examples, there's a close connection between the task being done by a new thread, as defined by its Runnable object, and the thread itself, as defined by a Thread object. This works well for small applications, but in large-scale applications, it makes sense to separate thread management and creation from the rest of the application. Objects that encapsulate these functions are known as executors.

The following subsections describe executors in detail:

- Executor Interfaces define the three executor object types.
- Thread Pools are the most common kind of executor implementation.
- Fork/Join is a framework (new in JDK 7) for taking advantage of multiple processors.

EXECUTOR INTERFACES

The java.util.concurrent package defines three executor interfaces:

1. Executor, a simple interface that supports launching new tasks.
2. ExecutorService, a subinterface of Executor, which adds features that help manage the life cycle, both of the individual tasks and of the executor itself.
3. ScheduledExecutorService, a subinterface of ExecutorService, supports future and/or periodic execution of tasks.

Typically, variables that refer to executor objects are declared as one of these three interface types, not with an executor class type.

The Executor Interface

The Executor interface provides a single method, `execute`, designed to be a drop-in replacement for a common thread-creation idiom. If `r` is a Runnable object, and `e` is an Executor object you can replace

```

(new Thread(r)).start();
with
e.execute(r);

```

However, the definition of `execute` is less specific. The low-level idiom creates a new thread and launches it immediately. Depending on the `Executor` implementation, `execute` may do the same thing, but is more likely to use an existing worker thread to run `r`, or to place `r` in a queue to wait for a worker thread to become available.

(We'll describe worker threads in the section on Thread Pools.) The executor implementations in `java.util.concurrent` are designed to make full use of the more advanced `ExecutorService` and `ScheduledExecutorService` interfaces, although they also work with the base `Executor` interface.

The `ExecutorService` Interface

The `ExecutorService` interface supplements `execute` with a similar, but more versatile `submit` method. Like `execute`, `submit` accepts `Runnable` objects, but also accepts `Callable` objects, which allow the task to return a value. The `submit` method returns a `Future` object, which is used to retrieve the `Callable` return value and to manage the status of both `Callable` and `Runnable` tasks. `ExecutorService` also provides methods for submitting large collections of `Callable` objects.

Finally, `ExecutorService` provides a number of methods for managing the shutdown of the executor. To support immediate shutdown, tasks should handle interrupts correctly.

The `ScheduledExecutorService` Interface

The `ScheduledExecutorService` interface supplements the methods of its parent `ExecutorService` with `schedule`, which executes a `Runnable` or `Callable` task after a specified delay. In addition, the interface defines `scheduleAtFixedRate` and `scheduleWithFixedDelay`, which execute specified tasks repeatedly, at defined intervals.

THREAD POOLS

Most of the executor implementations in `java.util.concurrent` use thread pools, which consist of worker threads. This kind of thread exists separately from the `Runnable` and `Callable` tasks it executes and is often used to execute multiple tasks.

Using worker threads minimizes the overhead due to thread creation. Thread objects use a significant amount of memory, and in a large-scale application, allocating and deallocating many thread objects creates a significant memory management overhead.

One common type of thread pool is the fixed thread pool. This type of pool always has a specified number of threads running; if a thread is somehow terminated while it is still in use, it is automatically replaced with a new thread. Tasks are submitted to the pool via an internal queue, which holds extra tasks whenever there are more active tasks than threads.

An important advantage of the fixed thread pool is that applications using it degrade gracefully. To understand this, consider a web server application where each HTTP request is handled by a separate thread. If the application simply creates a new thread

for every new HTTP request, and the system receives more requests than it can handle immediately, the application will suddenly stop responding to all requests when the overhead of all those threads exceed the capacity of the system.

With a limit on the number of the threads that can be created, the application will not be servicing HTTP requests as quickly as they come in, but it will be servicing them as quickly as the system can sustain.

A simple way to create an executor that uses a fixed thread pool is to invoke the `newFixedThreadPool` factory method in `java.util.concurrent`.

Executors this class also provides the following factory methods:

- The `newCachedThreadPool` method creates an executor with an expandable thread pool. This executor is suitable for applications that launch many short-lived tasks.
- The `newSingleThreadExecutor` method creates an executor that executes a single task at a time.
- Several factory methods are Scheduled Executor Service versions of the above executors.

FORK/JOIN

New in the Java SE 7 release, the fork/join framework is an implementation of the `ExecutorService` interface that helps you take advantage of multiple processors. It is designed for work that can be broken into smaller pieces recursively. The goal is to use all the available processing power to make your application wicked fast.

As with any `ExecutorService`, the fork/join framework distributes tasks to worker threads in a thread pool. The fork/join framework is distinct because it uses a work-stealing algorithm. Worker threads that run out of things to do can steal tasks from other threads that are still busy. The centre of the fork/join framework is the `ForkJoinPool` class, an extension of `AbstractExecutorService`. `ForkJoinPool` implements the core work-stealing algorithm and can execute `ForkJoinTasks`.

Basic Use

Using the fork/join framework is simple. The first step is to write some code that performs a segment of the work. Your code should look similar to this:

```
if (my portion of the work is small enough)
do the work directly
else
split my work into two pieces
invoke the two pieces and wait for the results
```

Wrap this code as a `ForkJoinTask` subclass, typically as one of its more specialized types `RecursiveTask` (which can return a result) or `RecursiveAction`. After your `ForkJoinTask` is ready, create one that represents all the work to be done and pass it to the `invoke()` method of a `ForkJoinPool` instance.

Blurring for Clarity

To help you understand how the fork/join framework works, consider a simple example. Suppose you want to perform a simple blur on an image. The original source image is represented by an array of integers, where each integer contains the colour values for a single pixel. The blurred destination image is also represented by an integer array with the same size as the source.

Performing the blur is accomplished by working through the source array one pixel at a time.

Each pixel is averaged with its surrounding pixels (the red, green, and blue components are averaged), and the result is placed in the destination array. Here is one possible implementation:

```
public class ForkBlur extends RecursiveAction {
    private int[] mSource;
    private int mStart;
    private int mLength;
    private int[] mDestination;
    private int mBlurWidth = 15; // Processing window size, should be odd.
    public ForkBlur(int[] src, int start, int length, int[] dst) {
        mSource = src;
        mStart = start;
        mLength = length;
        mDestination = dst;
    }
    // Average pixels from source, write results into destination.
    protected void computeDirectly() {
        int sidePixels = (mBlurWidth - 1) / 2;
        for (int index = mStart; index < mStart + mLength; index++) {
            // Calculate average.
            float rt = 0, gt = 0, bt = 0;
            for (int mi = -sidePixels; mi <= sidePixels; mi++) {
                int minindex = Math.min(Math.max(mi + index, 0),
                    mSource.length - 1);
                int pixel = mSource[minindex];
                rt += (float) ((pixel & 0x00ff0000) >> 16) /
                    mBlurWidth;
                gt += (float) ((pixel & 0x0000ff00) >> 8) /
                    mBlurWidth;
                bt += (float) ((pixel & 0x000000ff) >> 0) /
                    mBlurWidth;
            }

            // Re-assemble destination pixel.
            int dpixel = (0xff000000)
                | (((int) rt) << 16)
                | (((int) gt) << 8)
                | (((int) bt) << 0);
            mDestination[index] = dpixel;
        }
    }
}
```

```

    }
}
protected static int sThreshold = 10000;

```

Now you implement the abstract `compute()` method, which either performs the blur directly or splits it into two smaller tasks. A simple array length threshold helps determine whether the work is performed or split.

```

@Override
protected void compute() {
    if (mLength < sThreshold) {
        computeDirectly();
        return;
    }
    int split = mLength / 2;
    invokeAll(new ForkBlur(mSource, mStart, split, mDestination),
        new ForkBlur(mSource, mStart + split, mLength - split,
            mDestination));
}

```

If the previous methods are in a subclass of the `Recursive Action` class, setting it up to run in a `ForkJoin Pool` is straight forward. Create a task that represents all of the work to be done.

```

// source image pixels are in src
// destination image pixels are in dst
ForkBlur fb = new ForkBlur(src, 0, src.length, dst);

```

Create the `ForkJoinPool` that will run the task.

```
ForkJoinPool pool = new ForkJoinPool();
```

Run the task.

```
pool.invoke(fb);
```

For the full source code, including some extra code that shows the source and destination images in windows, see the `ForkBlurclass`.

MULTITHREADING

Multithreading is the ability for a single process to process two or more tasks concurrently. (We say concurrently rather than simultaneously because, in the absence of multiple processors, the tasks cannot run simultaneously but rather are interleaved in very small time slices and thus exhibit the appearance and semantics of concurrent execution.) The use of multithreading is becoming increasingly more common as operating system support for threads has become near ubiquitous.

Among the languages under discussion, nearly all support multithreading either directly within the language or through libraries. Ruby is somewhat unique in that its threading capabilities are built in to the interpreter itself, rather than wrappers around the operating system threading operations. This has the disadvantage that any operating system calls will block the entire interpreter, but has the advantage of being completely portable even to systems that do not support multithreading, such as MS-DOS.

REGULAR EXPRESSIONS

Regular expressions are pattern matching constructs capable of recognizing the class of languages known as regular languages. They are frequently used for text processing systems as well as for general applications that must use pattern recognition for other purposes. Libraries with regular expression support exist for nearly every language, but ever since the advent of Perl it has become increasingly important for a language to support regular expressions natively. This allows tighter integration with the rest of the language and allows more convenient syntax for use of regular expressions. Perl was the model for this kind of built-in support and Ruby, a close descendant of Perl, continues the tradition. Python, and recently Java, have included regular expression libraries as part of the standard base library distributed with the language implementation.

POINTER ARITHMETIC

Pointer arithmetic is the ability for a language to directly manipulate memory addresses and their contents. While, due to the inherent unsafety of direct memory manipulation, this ability is not often considered appropriate for high-level languages, it is essential for low-level systems applications. Thus, while object-oriented languages strive to remain at a fairly high level of abstraction, to be suitable for systems programming a language must provide such features or relegate such low-level tasks to a language with which it can interact. Most object-oriented languages have foregone support of pointer arithmetic in favour of providing integration with C. This allows low-level routines to be implemented in C while the majority of the application is written in the higher level language.

C++ on the other hand provides direct support for pointer arithmetic, both for compatibility with C and to allow C++ to be used for systems programming without the need to drop down to a lower level language. This is the source both of C++'s great flexibility as well as much of its complexity.

LANGUAGE INTEGRATION

For various reasons, including integration with existing systems, the need to interact with low level modules, or for sheer speed, it is important for a high level language (particularly interpreted languages) to be able to integrate seamlessly with other languages. Nearly every language to come along since C was first introduced provides such integration with C. This allows high level languages to remain free of the low level constructs that make C great for systems programming, but add much complexity.

All of the languages under discussion integrate tightly with C, except for Visual Basic, which can only do so through DCOM. Eiffel and Ruby provide particularly easy-to-use interfaces to C as well as callbacks to the language runtime. Python, Perl, Java, and Smalltalk provide similar interfaces, though they aren't quite as easy to use. C++, naturally, integrates quite transparently with C.

BUILT-IN SECURITY

Built-in security refers to a language implementation's ability to determine whether or not a piece of code comes from a "trusted" source (such as the user's hard disk), limiting the permissions of the code if it does not. For example, Java applets are considered untrusted, and thus they are limited in the actions they can perform when executed from a user's browser. They may not, for example, read or write from or to the user's hard disk, and they may not open a network connection to anywhere but the originating host.

Several languages, including Java, Ruby, and Perl, provide this ability "out of the box". Most languages defer this protection to the user's operating environment.

CAPERS JONES LANGUAGE LEVEL

The Capers Jones Language Level is a study that attempts to identify the number of source lines of code is necessary in a given language to implement a single function point. The higher the language level, the fewer lines of code it takes to implement a function point, and thus presumably is an indicator of the productivity levels achievable using the language.

Functional Programming

CODE FRAGMENTATION

Pulling together the code fragments from Section 3.6, the whole programme looks like this:

```
def newLine():
    print
def threeLines():
    newLine()
    newLine()
    newLine()
print "First Line."
threeLines()
print "Second Line."
```

This programme contains two function definitions: newLine and threeLines. Function definitions get executed just like other statements, but the effect is to create the new function. The statements inside the function do not get executed until the function is called, and the function definition generates no output.

As you might expect, you have to create a function before you can execute it. In other words, the function definition has to be executed before the first time it is called.

As an exercise, move the last three lines of this programme to the top, so the function calls appear before the definitions. Run the programme and see what error message you get.

As another exercise, start with the working version of the programme and move the definition of `newLine` after the definition of `threeLines`. What happens when you run this programme?

FLOW OF EXECUTION

In order to ensure that a function is defined before its first use, you have to know the order in which statements are executed, which is called the flow of execution.

Execution always begins at the first statement of the programme. Statements are executed one at a time, in order from top to bottom. Function definitions do not alter the flow of execution of the programme, but remember that statements inside the function are not executed until the function is called. Although it is not common, you can define one function inside another. In this case, the inner definition isn't executed until the outer function is called.

Function calls are like a detour in the flow of execution. Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

That sounds simple enough, until you remember that one function can call another. While in the middle of one function, the programme might have to execute the statements in another function. But while executing that new function, the programme might have to execute yet another function! Fortunately, Python is adept at keeping track of where it is, so each time a function completes, the programme picks up where it left off in the function that called it. When it gets to the end of the programme, it terminates. What's the moral of this sordid tale? When you read a programme, don't read from top to bottom. Instead, follow the flow of execution.

PARAMETERS AND ARGUMENTS

Some of the built-in functions you have used require arguments, the values that control how the function does its job. For example, if you want to find the sine of a number, you have to indicate what the number is. Thus, `sin` takes a numeric value as an argument.

Some functions take more than one argument. For example, `pow` takes two arguments, the base and the exponent. Inside the function, the values that are passed get assigned to variables called parameters.

Here is an example of a user-defined function that has a parameter:

```
def printTwice(bruce):  
    print bruce, bruce
```

This function takes a single argument and assigns it to a parameter named `bruce`. The value of the parameter (at this point we have no idea what it will be) is printed twice, followed by a newline. The name `bruce` was chosen to suggest that the name you give a parameter is up to you, but in general, you want to choose something more illustrative than *bruce*.

The function printTwice works for any type that can be printed:

```
>>> printTwice('Spam')
Spam Spam
printTwice(5)
5 5
printTwice(3.14159)
3.14159 3.14159
```

In the first function call, the argument is a string. In the second, it's an integer. In the third, it's a float. The same rules of composition that apply to built-in functions also apply to user-defined functions, so we can use any kind of expression as an argument for printTwice:

```
printTwice('Spam'*4)
SpamSpamSpamSpam SpamSpamSpamSpam
printTwice(math.cos(math.pi))
-1.0 -1.0
```

As usual, the expression is evaluated before the function is run, so printTwice prints SpamSpamSpamSpam SpamSpamSpamSpam instead of 'Spam'*4 'Spam'*4.

As an exercise, write a call to printTwice that does print 'Spam'*4'Spam'*4.

Hint: Strings can be enclosed in either single or double quotes, and the type of quote not used to enclose the string can be used inside it as part of the string.

We can also use a variable as an argument:

```
michael = 'Eric, the half a bee.'
printTwice(michael)
Eric, the half a bee. Eric, the half a bee.
```

Notice something very important here. The name of the variable we pass as an argument (michael) has nothing to do with the name of the parameter (bruce). It doesn't matter what the value was called back home (in the caller); here in printTwice, we call everybody bruce.

VARIABLES AND PARAMETERS ARE LOCAL

When you create a local variable inside a function, it only exists inside the function, and you cannot use it outside.

For example:

```
def catTwice(part1, part2):
    cat = part1 + part2
    printTwice(cat)
```

This function takes two arguments, concatenate them, and then prints the result twice. We can call the function with two strings:

```
chant1 = "Pie Jesu domine, "
chant2 = "Dona eis requiem."
catTwice(chant1, chant2)
Pie Jesu domine, Dona eis requiem. Pie Jesu
domine, Dona eis requiem.
```

When catTwice terminates, the variable cat is destroyed. If we try to print it, we get an error:

```
print cat
NameError: cat
```

Parameters are also local. For example, outside the function `printTwice`, there is no such thing as `bruce`. If you try to use it, Python will complain.

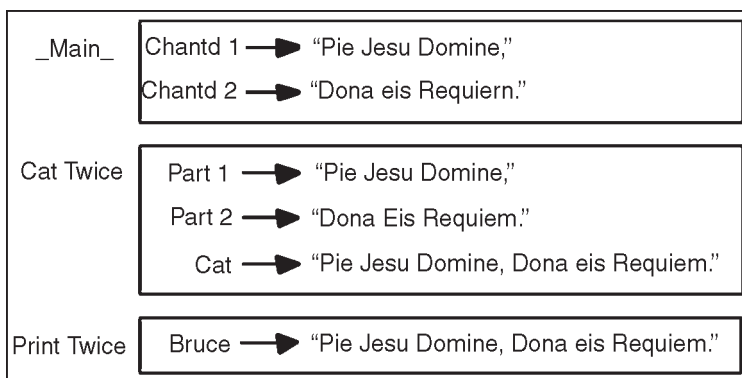
STACK DIAGRAMS

To keep track of which variables can be used where, it is sometimes useful to draw a stack diagram. Like state diagrams, stack diagrams show the value of each variable, but they also show the function to which each variable belongs.

Each function is represented by a frame. A frame is a box with the name of a function beside it and the parameters and variables of the function inside it.

The stack diagram for the previous example looks like this:

The order of the stack shows the flow of execution. `printTwice` was called by `catTwice`, and `catTwice` was called by `__main__`, which is a special name for the topmost function. When you create a variable outside of any function, it belongs to `__main__`. Each parameter refers to the same value as its corresponding argument. So, `part1` has the same value as `chant1`, `part2` has the same value as `chant2`, and `bruce` has the same value as `cat`.



If an error occurs during a function call, Python prints the name of the function, and the name of the function that called it, and the name of the function that called *that*, all the way back to `__main__`.

For example, if we try to access `cat` from within `printTwice`, we get a `NameError`:

```
Traceback (innermost last):
  File "test.py", line 13, in __main__
    catTwice(chant1, chant2)
  File "test.py", line 5, in catTwice
    printTwice(cat)
  File "test.py", line 9, in printTwice
    print cat
NameError: cat
```

This list of functions is called a traceback. It tells you what programme file the error occurred in, and what line, and what functions were executing at the time. It also shows the line of code that caused the error.

Notice the similarity between the traceback and the stack diagram. It's not a coincidence.

FUNCTIONS WITH RESULTS

You might have noticed by now that some of the functions we are using, such as the math functions, yield results. Other functions, like `newLine`, perform an action but don't return a value.

That raises some questions:

- What happens if you call a function and you don't do anything with the result (*i.e.*, you don't assign it to a variable or use it as part of a larger expression)?
- What happens if you use a function without a result as part of an expression, such as `newLine() + 7`?
- Can you write functions that yield results, or are you stuck with simple function like `newLine` and `printTwice`?

STATEMENTS

A statement is an instruction that the Python interpreter can execute. We have seen two kinds of statements: `print` and assignment. When you type a statement on the command line, Python executes it and displays the result, if there is one. The result of a `print` statement is a value. Assignment statements don't produce a result. A script usually contains a sequence of statements. If there is more than one statement, the results appear one at a time as the statements execute.

For example, the script

```
print 1
x = 2
print x
```

produces the output

```
1
2
```

Again, the assignment statement produces no output.

EVALUATING EXPRESSIONS

An expression is a combination of values, variables, and operators. If you type an expression on the command line, the interpreter evaluates it and displays the result:

```
1 + 1
2
```

Although expressions contain values, variables, and operators, not every expression contains all of these elements. A value all by itself is considered an expression, and so is a variable.

```
17
17
```

```
>>> x
2
```

Confusingly, evaluating an expression is not quite the same thing as printing a value.

```
message = 'Hello, World!'
message
'Hello, World!'
print message
Hello, World!
```

When the Python interpreter displays the value of an expression, it uses the same format you would use to enter a value. In the case of strings, that means that it includes the quotation marks. But if you use a print statement, Python displays the contents of the string without the quotation marks.

In a script, an expression all by itself is a legal statement, but it doesn't do anything.

```
17
3.2
'Hello, World!'
1 + 1
```

The script produces no output at all. How would you change the script to display the values of these four expressions?

OPERATORS AND OPERANDS

Operators are special symbols that represent computations like addition and multiplication. The values the operator uses are called operands.

The following are all legal Python expressions whose meaning is more or less clear:

```
20+32 hour-1 hour*60+minute minute/60 5**2
(5+9)*(15-7)
```

The symbols +, −, and /, and the use of parenthesis for grouping, mean in Python what they mean in mathematics. The asterisk (*) is the symbol for multiplication, and ** is the symbol for exponentiation.

When a variable name appears in the place of an operand, it is replaced with its value before the operation is performed. Addition, subtraction, multiplication, and exponentiation all do what you expect, but you might be surprised by division.

The following operation has an unexpected result:

```
minute = 59
minute/60
0
```

The value of minute is 59, and in conventional arithmetic 59 divided by 60 is 0.98333, not 0. The reason for the discrepancy is that Python is performing integer division.

When both of the operands are integers, the result must also be an integer, and by convention, integer division always rounds *down*, even in cases like this where the next integer is very close.

A possible solution to this problem is to calculate a percentage rather than a fraction:

```
minute*100/60
98
```

ORDER OF OPERATIONS

When more than one operator appears in an expression, the order of evaluation depends on the rules of precedence. Python follows the same precedence rules for its mathematical operators that mathematics does.

The acronym PEMDAS is a useful way to remember the order of operations:

- Parentheses have the highest precedence and can be used to force an expression to evaluate in the order you want. Since expressions in parentheses are evaluated first, $2 * (3-1)$ is 4, and $(1+1)**(5-2)$ is 8. You can also use parentheses to make an expression easier to read, as in $(\text{minute} * 100)/60$, even though it doesn't change the result.
- Exponentiation has the next highest precedence, so $2**1+1$ is 3 and not 4, and $3*1**3$ is 3 and not 27.
- Multiplication and Division have the same precedence, which is higher than Addition and Subtraction, which also have the same precedence. So $2*3-1$ yields 5 rather than 4, and $2/3-1$ is -1 , not 1 (remember that in integer division, $2/3=0$).
- Operators with the same precedence are evaluated from left to right. So in the expression $\text{minute}*100/60$, the multiplication happens first, yielding $5900/60$, which in turn yields 98. If the operations had been evaluated from right to left, the result would have been $59*1$, which is 59, which is wrong.

OPERATIONS ON STRINGS

In general, you cannot perform mathematical operations on strings, even if the strings look like numbers. The following are illegal (assuming that `message` has type `string`):

```
message-1 'Hello'/123 message*'Hello' '15'+2
```

Interestingly, the `+` operator does work with strings, although it does not do exactly what you might expect. For strings, the `+` operator represents concatenation, which means joining the two operands by linking them end-to-end. For example:

```
fruit = 'banana'
bakedGood = ' nut bread'
print fruit + bakedGood
```

The output of this programme is `banana nut bread`. The space before the word `nut` is part of the string, and is necessary to produce the space between the concatenated strings.

The `*` operator also works on strings; it performs repetition. For example, `'Fun'*3` is `'FunFunFun'`. One of the operands has to be a string; the other has to be an integer.

On one hand, this interpretation of `+` and `*` makes sense by analogy with addition and multiplication. Just as $4*3$ is equivalent to $4+4+4$, we expect `'Fun'*3` to be the same as `'Fun'+'Fun'+'Fun'`, and it is. On the other hand, there is a significant way in which string concatenation and repetition are different from integer addition and multiplication. Can you think of a property that addition and multiplication have that string concatenation and repetition do not?

COMPOSITION

So far, we have looked at the elements of a programme variables, expressions, and statements in isolation, without talking about how to combine them.

One of the most useful features of programming languages is their ability to take small building blocks and compose them. For example, we know how to add numbers and we know how to print; it turns out we can do both at the same time:

```
print 17 + 3
20
```

In reality, the addition has to happen before the printing, so the actions aren't actually happening at the same time. The point is that any expression involving numbers, strings, and variables can be used inside a print statement.

You've already seen an example of this:

```
print 'Number of minutes since midnight: ',
hour*60+minute
```

You can also put arbitrary expressions on the right-hand side of an assignment statement:

```
percentage = (minute * 100) / 60
```

This ability may not seem impressive now, but you will see other examples where composition makes it possible to express complex computations neatly and concisely. Warning: There are limits on where you can use certain expressions. For example, the left-hand side of an assignment statement has to be a *variable* name, not an expression. So, the following is illegal: `minute+1 = hour`.

FUNCTION PROTOTYPES

In modern C programming, it is considered good practice to use prototype declarations for all functions that you call. As we mentioned, these prototypes help to ensure that the compiler can generate correct code for calling the functions, as well as allowing the compiler to catch certain mistakes you might make. Strictly speaking, however, prototypes are optional. If you call a function for which the compiler has not seen a prototype, the compiler will do the best it can, assuming that you're calling the function correctly.

If prototypes are a good idea, and if we're going to get in the habit of writing function prototype declarations for functions we call that we've written (such as `multbyt看`), what happens for library functions such as `printf`? Where are their prototypes? The answer is in that boilerplate line

```
#include <stdio.h>
```

we've been including at the top of all of our programmes. `stdio.h` is conceptually a file full of external declarations and other information pertaining to the "Standard I/O" library functions, including `printf`. The `#include` directive (which we'll meet formally in a later chapter) arranges that all of the declarations within `stdio.h` are considered by the compiler, rather as if we'd typed them all in ourselves. Somewhere within these

declarations is an external function prototype declaration for `printf`, which satisfies the rule that there should be a prototype for each function we call. (For other standard library functions we call, there will be other “header files” to include.) Finally, one more thing about external function prototype declarations. We’ve said that the distinction between external declarations and defining instances of normal variables hinges on the presence or absence of the keyword `extern`. The situation is a little bit different for functions. The “defining instance” of a function is the function, including its body (that is, the brace-enclosed list of declarations and statements implementing the function). An external declaration of a function, even without the keyword `extern`, looks nothing like a function declaration. Therefore, the keyword `extern` is optional in function prototype declarations. If you wish, you can write,

```
int multbyt看wo(int);
```

and this is just as good an external function prototype declaration as,

```
extern int multbyt看wo(int);
```

(In the first form, without the `extern`, as soon as the compiler sees the semicolon, it knows it’s not going to see a function body, so the declaration can’t be a definition.) You may want to stay in the habit of using `extern` in all external declarations, including function declarations, since “`extern` = external declaration” is an easier rule to remember.

FUNCTION PHILOSOPHY

What makes a good function? The most important aspect of a good “building block” is that have a single, well-defined task to perform. When you find that a programme is hard to manage, it’s often because it has not been designed and broken up into functions cleanly. Two obvious reasons for moving code down into a function are because:

- It appeared in the main programme several times, such that by making it a function, it can be written just once, and the several places where it used to appear can be replaced with calls to the new function.
- The main programme was getting too big, so it could be made (presumably) smaller and more manageable by lopping part of it off and making it a function.

These two reasons are important, and they represent significant benefits of well-chosen functions, but they are not sufficient to automatically identify a good function. As we’ve been suggesting, a good function has at least these two additional attributes:

- It does just one well-defined task, and does it well.
- Its interface to the rest of the programme is clean and narrow.

Attribute 3 is just a restatement of two things we said above. Attribute 4 says that you shouldn’t have to keep track of too many things when calling a function. If you know what a function is supposed to do, and if its task is simple and well-defined, there should be just a few pieces of information you have to give it to act upon, and one or just a few pieces of information which it returns to you when it’s done. If you find yourself having to pass lots and lots of information to a function, or remember details

of its internal implementation to make sure that it will work properly this time, it's often a sign that the function is not sufficiently well-defined. (A poorly-defined function may be an arbitrary chunk of code that was ripped out of a main programme that was getting too big, such that it essentially has to have access to all of that main function's local variables.)

The whole point of breaking a programme up into functions is so that you don't have to think about the entire programme at once; ideally, you can think about just one function at a time. We say that a good function is a “black box,” which is supposed to suggest that the “container” it's in is opaque—callers can't see inside it (and the function inside can't see out). When you call a function, you only have to know what it does, not how it does it.

When you're *writing* a function, you only have to know what it's supposed to do, and you don't have to know why or under what circumstances its caller will be calling it. (When designing a function, we should perhaps think about the callers just enough to ensure that the function we're designing will be easy to call, and that we aren't accidentally setting things up so that callers will have to think about any internal details.)

Some functions may be hard to write (if they have a hard job to do, or if it's hard to make them do it truly well), but that difficulty should be compartmentalized along with the function itself. Once you've written a “hard” function, you should be able to sit back and relax and watch it do that hard work on call from the rest of your programme. It should be pleasant to notice (in the ideal case) how much easier the rest of the programme is to write, now that the hard work can be deferred to this workhorse function.

In fact, if a difficult-to-write function's interface is well-defined, you may be able to get away with writing a quick-and-dirty version of the function first, so that you can begin testing the rest of the programme, and then go back later and rewrite the function to do the hard parts. As long as the function's original interface anticipated the hard parts, you won't have to rewrite the rest of the programme when you fix the function.

If you find that difficulties pervade a programme, that the hard parts can't be buried inside black-box functions and then forgotten about; if you find that there are hard parts which involve complicated interactions among multiple functions, then the programme probably needs redesigning.

For the purposes of explanation, we've been seeming to talk so far only about “main programmes” and the functions they call and the rationale behind moving some piece of code down out of a “main programme” into a function. But in reality, there's obviously no need to restrict ourselves to a two-tier scheme. Any function we find ourselves writing will often be appropriately written in terms of sub-functions, sub-sub-functions, etc. (Furthermore, the “main programme,” `main()`, is itself just a function.)

SEPARATE COMPILATION LOGISTICS

When a programme consists of many functions, it can be convenient to split them up into several source files. Among other things, this means that when a change is made,

only the source file containing the change has to be recompiled, not the whole programme. The job of putting the pieces of a programme together and producing the final executable falls to a tool called the *linker*. (We may or may not need to invoke the linker explicitly; a compiler often invokes it automatically, as needed.) The linker looks through all of the pieces making up the programme, sorting out the external declarations and defining instances. The compiler has noted the definitions made by each source file, as well as the declarations of things used by each source file but (presumably) defined elsewhere. For each thing (global variable or function) used but not defined by one piece of the programme, the linker looks for another piece which does define that thing.

The logistics of writing a programme in several source files, and then compiling and linking all of the source files together, depend on the programming environment you're using. We'll cover two possibilities, depending on whether you're using a traditional command-line compiler or a newer integrated development environment (IDE) or other graphical user interface (GUI) compiler.

When using a command-line compiler, there are usually two main steps involved in building an executable programme from one or more source files. First, each source file is compiled, resulting in an *object file* containing the machine instructions (generated by the compiler) corresponding to just the code in that source file. Second, the various object files are *linked* together, with each other and with *libraries* containing code for functions which you did not write (such as `printf`), to produce a final, executable programme.

Under Unix, the `cc` command can perform one or both steps. So far, we've been using extremely simple invocations of `cc` such as,

```
cc -o hello hello.c
```

This invocation compiles a single source file, `hello.c`, links it, and places the executable in a file named `hello`. Suppose we have a programme which we're trying to build from three separate source files, `x.c`, `y.c`, and `z.c`. We could compile all three of them, and link them together, all at once, with the command

```
cc -o myprog x.c y.c z.c
```

Alternatively, we could compile them separately: the `-c` option to `cc` tells it to compile only, but not to link. Instead of building an executable, it merely creates an object file, with a name ending in `.o`, for each source file compiled.

So the three commands,

```
cc -c x.c
```

```
cc -c y.c
```

```
cc -c z.c
```

would compile `x.c`, `y.c`, and `z.c` and create object files `x.o`, `y.o`, and `z.o`. Then, the three object files could be linked together using

```
cc -o myprog x.o y.o z.o
```

When the `cc` command is given an `.o` file, it knows that it does not have to compile it (it's an object file, already compiled); it just sends it through to the link process.

Above we mentioned that the second, linking step also involves pulling in library functions. Normally, the functions from the Standard C library are linked in automatically. Occasionally, you must request a library manually; one common situation under Unix is that the math functions tend to be in a separate math library, which is requested by using `-lm` on the command line. Since the libraries must typically be searched *after* your program's own object files are linked (so that the linker knows which library functions your programme uses), any `-l` option must appear *after* the names of your files on the command line. For example, to link the object file `mymath.o` (previously compiled with `cc -c mymath.c`) together with the math library, you might use

```
cc -o mymathprog mymath.o -lm
```

(The `l` in the `-l` option is the lower case ell, for library; it is *not* the digit 1.)

Everything we've said about `cc` also applies to most other Unix C compilers. (Many of you will be using `gcc`, the FSF's GNU C Compiler.) There are command-line compilers for MS-DOS systems which work similarly. For example, the Microsoft C compiler comes with a `CL` ("compile and link") command, which works almost the same as Unix `cc`.

You can compile and link in one step:

```
cl hello.c
```

or you can compile only:

```
cl/c hello.c
```

creating an object file named `hello.obj` which you can link later. The preceding has all been about command-line compilers. If you're using some kind of integrated development environment, such as Borland's Turbo C or the Microsoft Programmer's Workbench or Visual C or Think C or Codewarrior, most of the mechanical details are taken care of for you. Typically you define a "project," and there's a way to specify the list of files which make up your project.

READING LINES

It's often convenient for a programme to process its input not a character at a time but rather a line at a time, that is, to read an entire line of input and then act on it all at once. The standard C library has a couple of functions for reading lines, but they have a few awkward features, so we're going to learn more about character input (and about writing functions in general) by writing our own function to read one line. Here it is:

```
#include <stdio.h>

/* Read one line from standard input, */
/* copying it to line array (but no more
than max chars). */
/* Does not place terminating \n in line
array. */
/* Returns line length, or 0 for empty
line, or EOF for end-of-file.
*/ int getline(char line[], int max)
{
```

```

int nch = 0;
int c;
max = max - 1; /* leave room for '\0' */
while((c = getchar()) != EOF)
{
    if(c == '\n')
        break;
    if(nch < max)
    {
        line[nch] = c;
        nch = nch + 1;
    }
}
if(c == EOF && nch == 0)
    return EOF;
line[nch] = '\0';
return nch;
}

```

As the comment indicates, this function will read one line of input from the standard input, placing it into the line array.

The size of the line array is given by the `max` argument; the function will never write more than `max` characters into line.

The main body of the function is a `getchar` loop, much as we used in the character-copying programme. In the body of this loop, however, we're storing the characters in an array. Also, we're only reading one line of characters, then stopping and returning.

First of all, the `getline` function accepts an array as a parametre. As we've said, array parametres are an exception to the rule that functions receive copies of their arguments—in the case of arrays, the function *does* have access to the actual array passed by the caller, and *can* modify it. Since the function is accessing the caller's array, not creating a new one to hold a copy, the function does not have to declare the argument array's size; it's set by the caller. However, so that we won't overflow the caller's array by reading too long a line into it, we allow the caller to pass along the size of the array, which we promise not to exceed.

Second, we see an example of the `break` statement. The top of the loop looks like our earlier character-copying loop—it stops when it reaches EOF—but we only want this loop to read one line, so we also stop (that is, `break` out of the loop) when we see the `\n` character signifying end-of-line. An equivalent loop, without the `break` statement, would be,

```

while((c = getchar()) != EOF && c != '\n')
{
    if(nch < max)
    {
        line[nch] = c;
        nch = nch + 1;
    }
}

```

We haven't learned about the internal representation of strings yet, but it turns out that strings in C are simply arrays of characters, which is why we are reading the line into an array of characters. The end of a string is marked by the special character, `'\0'`. To make sure that there's always room for that character, on our way in we subtract 1 from `max`, the argument that tells us how many characters we may place in the line array. When we're done reading the line, we store the end-of-string character `'\0'` at the end of the string we've just built in the line array.

Finally, there's one subtlety in the code which isn't too important for our purposes now but which you may wonder about: it's arranged to handle the possibility that a few characters (*i.e.* the apparent beginning of a line) are read, followed immediately by an EOF, without the usual `\n` end-of-line character.

In any case, the function returns the length (number of characters) of the line it read, not including the `\n`. (Therefore, it returns 0 for an empty line.) Like `getchar`, it returns EOF when there are no more lines to read. (It happens that EOF is a negative number, so it will never match the length of a line that `getline` has read.)

Here is an example of a test programme which calls `getline`, reading the input a line at a time and then printing each line back out:

```
#include <stdio.h>
extern int getline(char [], int);
main()
{
    char line[256];
    while(getline(line, 256) != EOF)
        printf("you typed \"%s\"\n", line);
    return 0;
}
```

The notation `char []` in the function prototype for `getline` says that `getline` accepts as its first argument an array of `char`. When the programme calls `getline`, it is careful to pass along the actual size of the array. You might notice a potential problem: since the number 256 appears in two places, if we ever decide that 256 is too small, and that we want to be able to read longer lines, we could easily change one of the instances of 256, and forget to change the other one.

READING NUMBERS

The `getline` function of the previous section reads one line from the user, as a *string*. What if we want to read a number? One straightforward way is to read a string as before, and then immediately convert the string to a number. The standard C library contains a number of functions for doing this. The simplest to use are `atoi()`, which converts a string to an integer, and `atof()`, which converts a string to a floating-point number. (Both of these functions are declared in the header `<stdlib.h>`, so you should `#include` that header at the top of any file using these functions.) You could read an integer from the user like this:


```
#include <stdlib.h>
char line[256];
int n;
printf("Type an integer:\n");
getline(line, 256);
n = atoi(line);
```

Now the variable `n` contains the number typed by the user. (This assumes that the user *did* type a valid number, and that `getline` did not return EOF.)

Reading a floating-point number is similar:

```
#include <stdlib.h>
char line[256];
double x;
printf("Type a floating-point number:\n");
getline(line, 256);
x = atof(line);
```

(`atof` is actually declared as returning type `double`, but you could also use it with a variable of type `float`, because in general, C automatically converts between `float` and `double` as needed.) Another way of reading in numbers, which you're likely to see in other books on C, involves the `scanf` function, but it has several problems, so we won't discuss it for now. (Superficially, `scanf` seems simple enough, which is why it's often used, especially in textbooks. The trouble is that to perform input reliably using `scanf` is not nearly as easy as it looks, especially when you're not sure what the user is going to type.)

FORMATTED I/O

There are a number of related functions used for formatted I/O, each one determining the format of the I/O from a *format string*. For output, the format string consists of plain text, which is output unchanged, and embedded *format specifications* which call for some special processing of one of the remaining arguments to the function. On input, the plain text must match what is seen in the input stream; the format specifications again specify what the meaning of remaining arguments is. Each format specification is introduced by a `%` character, followed by the rest of the specification.

OUTPUT: PRINT F FAMILY

For those functions performing output, the format specification takes the following form, with optional parts enclosed in brackets:

```
%<flags><field width><precision><length> conversion
```

The meaning of *flags*, *field width*, *precision*, *length*, and *conversion* are given below, although tersely. For more detail, it is worth looking at what the Standard says.

Flags

Zero or more of the following:

–

Left justify the conversion within its field.

+

A signed conversion will always start with a plus or minus sign.

Space

If the first character of a signed conversion is not a sign, insert a space. Overridden by + if present.

#

Forces an alternative form of output. The first digit of an octal conversion will always be a 0; inserts 0X in front of a non-zero hexadecimal conversion; forces a decimal point in all floating point conversions even if one is not necessary; does not remove trailing zeros from g and G conversions.

0

Pad d, i, o, u, x, X, e, E, f, F and G conversions on the left with zeros up to the field width. Overridden by the - flag. If a precision is specified for the d, i, o, u, x or X conversions, the flag is ignored. The behaviour is undefined for other conversions.

Field Width

A decimal integer specifying the minimum output field width. This will be exceeded if necessary. If an asterisk is used here, the next argument is converted to an integer and used for the value of the field width; if the value is negative it is treated as a - flag followed by a positive field width.

Output that would be less than the field width is padded with spaces (zeros if the *field width* integer starts with a zero) to fit. The padding is on the left unless the left-adjustment flag is specified.

Precision

This starts with a period '.'. It specifies the minimum number of digits for d, i, o, u, x, or X conversions; the number of digits after the decimal point for e, E, f conversions; the maximum number of digits for g and G conversions; the number of characters to be printed from a string for s conversion. The amount of padding overrides the field width. If an asterisk is used here, the next argument is converted to an integer and used for the value of the field width. If the value is negative, it is treated as if it were missing. If only the period is present, the precision is taken to be zero.

Length

h preceding a specifier to print an integral type causes it to be treated as if it were a short. (Note that the various sorts of short are always promoted to one of the flavours of int when passed as an argument.) l works like h but applies to a long integral argument. L is used to indicate that a long double argument is to be printed, and only

applies to the floating-point specifiers. These are cause undefined behaviour if they are used with the ‘wrong’ type of conversion.

Conversion

Table Conversions

Specifier	Effect precision	Default
d	signed decimal	1
i	signed decimal	1
u	unsigned decimal	1
–	unsigned octal	1
x	unsigned hexadecimal (0–f)	1
X	unsigned hexadecimal (0–F)	1
	<i>Precision</i> specifies minimum number of digits, expanded with leading zeros if necessary. Printing a value of zero with zero precision outputs no characters.	
f	Print a double with <i>precision</i> digits (rounded) after the decimal point. To suppress the decimal point use a <i>precision</i> of explicitly zero. Otherwise, at least one digit appears in front of the point.	6
e, E	Print a double in exponential format, rounded, with one digit before the decimal point, <i>precision</i> after it. A <i>precision</i> of zero suppresses the decimal point. There will be at least two digits in the exponent, which is printed as 1.23e15 in e format, or 1.23E15 in E format.	6
g, G	Use style f, or e (E with G) depending unspecified on the exponent. If the exponent is less than “4 or e” <i>precision</i> , f is not used. Trailing zeros are suppressed, a decimal point is only printed if there is a following digit.	
c	The int argument is converted to an unsigned char and the resultant character printed.	
s	Print a string up to <i>precision</i> digits long. infinite If <i>precision</i> is not specified, or is greater than the length of the string, the string must be NUL terminated.	

p	Display the value of a (void *) pointer in a system-dependent way.
n	The argument must be a pointer to an integer. The number of characters output so far by this call will be written into the integer.
%	A %

The functions that use these formats are described in Table. All need the inclusion of `<stdio.h>`. Their declarations are as shown.

```
#include <stdio.h>
int fprintf(FILE *stream, const char *format...);
int printf(const char *format...);
int sprintf(char *s, const char *format...);
#include <stdarg.h> /* as well as stdio.h */
int vfprintf(FILE *stream, const char *format, va list arg);
int vprintf(const char *format, va list arg);
int vsprintf(char *s, const char *format, va list arg);
```

Table Functions performing formatted output

Name	Purpose
fprintf	General formatted output as described. Output is written to the file indicated by stream.
printf	Identical to fprintf with a first argument equal to stdout.
sprintf	Identical to fprintf except that the output is not written to a file, but written into the character array pointed to by s.
vfprintf	Formatted output as for fprintf, but with the variable argument list replaced by arg which must have been initialized by va_start.va_end is not called by this function.
vprintf	Identical to vfprintf with a first argument equal to stdout.
vsprintf	Formatted output as for sprintf, but with the variable argument list replaced by arg which must have been initialized by va_start.va_end is not called by this function.

All of the above functions return the number of characters output, or a negative value on error. The trailing null is not counted by `sprintf` and `vsprintf`. Implementations must permit at least 509 characters to be produced by any single conversion.

INPUT SCANF

A number of functions exist analogous to the `printf` family, but for the purposes of input instead. The most immediate difference between the two families is that the `scanf` group needs to be passed *pointers* to their arguments, so that the values read can be assigned to the proper destinations. Forgetting to pass a pointer is a very common error, and one which the compiler cannot detect—the variable argument list prevents it. The format string is used to control interpretation of a stream of input data, which

generally contains values to be assigned to the objects pointed to by the remaining arguments to `scanf`. The contents of the format string may contain:

White Space

This causes the input stream to be read up to the next non-white-space character.

Ordinary Character

Anything except white-space or `%` characters. The next character in the input stream must match this character.

Conversion Specification

This is a `%` character, followed by an optional `*` character (which suppresses the conversion), followed by an optional non-zero decimal integer specifying the maximum field width, an optional `h`, `l` or `L` to control the length of the conversion and finally a non-optional conversion specifier. Note that use of `h`, `l`, or `L` will affect the type of pointer which must be used.

Except for the specifier `c`, `n` and `[]`, a field of input is a sequence of non-space characters starting at the first non-space character in the input. It terminates at the first conflicting character or when the input field width is reached. The result is put into wherever the corresponding argument points, unless the assignment is suppressed using the `*` mentioned already. The following conversion specifier may be used:

`diox`

Convert a signed integer, a signed integer in a form acceptable to `strtol`, an octal integer, an unsigned integer and a hexadecimal integer respectively.

`efg`

Convert a float (not a double).

`s`

Read a string, and add a null at the end. The string is terminated by whitespace on input (which is not read as part of the string).

`[]`

Read a string. A list of characters, called the *scan set* follows the `[]`. A `]` delimits the list. Characters are read until (but not including) the first character which is not in the scan set. If the first character in the list is a circumflex `^`, then the scan set includes any character not in the list. If the initial sequence is `[^]` or `[]`, the `]` is not a delimiter, but part of the list and another `]` will be needed to end the list. If there is a minus sign (`-`) in the list, it must be either the first or the last character; otherwise the meaning is implementation defined.

`c`

Read a single character; white space is significant here. To read the first non-white space character, use `%1s`. A field width indicates that an array of characters is to be read.

p

Read a (void *) pointer previously written out using the %p of one of the printf's.

%

A % is expected in the input, no assignment is made.

n

Return as an integer the number of characters read by this call so far.

The size specifier have the effect shown in Table.

Table. Size Specifier

Specifier	Modifies	Converts
l	d i o u x	long int
h	d i o u x	short int
l	e f	double
L	e f	long double

The functions are described below, with the following declarations:

```
#include <stdio.h>
```

```
int fscanf(FILE *stream, const char *format...);
```

```
int sscanf(const char *s, const char *format...);
```

```
int scanf(const char *format...);
```

Fscanf takes its input from the designated stream, scanf is identical to f scanf with a first argument of stdin, and sscanf takes its input from the designated character array.

If an input failure occurs before any conversion, EOF is returned. Otherwise, the number of successful conversions is returned: this may be zero if no conversions are performed.

An input failure is caused by reading EOF or reaching the end of the input string (as appropriate). A conversion failure is caused by a failure to match the proper pattern for a particular conversion.

I/O MODEL

The I/O model does not distinguish between the types of physical devices supporting the I/O. Each source or sink of data is treated in the same way, and is viewed as a *stream* of bytes. Since the smallest object that can be represented in C is the character, access to a file is permitted at any character boundary. Any number of characters can be read or written from a movable point, known as the *file position indicator*. The characters will be read, or written, in sequence from this point, and the position indicator moved accordingly. The position indicator is initially set to the beginning of a file when it is opened, but can also be moved by means of positioning requests. Opening a file in append mode has an implementation defined effect on the stream's file position indicator.

The overall effect is to provide sequential reads or writes unless the stream was opened in append mode, or the file position indicator is explicitly moved. There are two types of file, *text files* and *binary files*, which, within a programme, are manipulated

as *text streams* and *binary streams* once they have been opened for I/O. The `studio` package does not permit operations on the contents of files ‘directly’, but only by viewing them as streams.

TEXT STREAMS

The Standard specifies what is meant by the term *text stream*, which essentially considers a file to contain lines of text. A line is a sequence of zero or more characters terminated by a newline character. It is quite possible that the actual representation of lines in the external environment is different from this and there may be transformations of the data stream on the way in and out of the programme; a common requirement is to translate the ‘\n’ line-terminator into the sequence ‘\r\n’ on output, and do the reverse on input. Other translations may also be necessary.

Data read in from a text stream is guaranteed to compare equal to the data that was earlier written out to the file if the data consists only of complete lines of printable characters and the control characters horizontal-tab and newline, no newline character is immediately preceded by space characters and the last character is a newline.

It is guaranteed that, if the last character written to a text file is a newline, it will read back as the same. It is implementation defined whether the last line written to a text file must terminate with a newline character; this is because on some implementations text files and binary files are the same. Some implementations may strip the leading space from lines consisting only of a space followed by a newline, or strip trailing spaces at the end of a line!

An implementation must support text files with lines containing at least 254 characters, including the terminating newline. Opening a text stream in update mode may result in a binary stream in some implementations. Writing on a text stream may cause some implementations to truncate the file at that point—any data beyond the last byte of the current write being discarded.

BINARY STREAMS

A binary stream is a sequence of characters that can be used to record a program’s internal data, such as the contents of structures or arrays in binary form. Data read in from a binary stream will always compare equal to data written out earlier to the same stream, under the same implementation. In some circumstances, an implementation-defined number of NUL characters may be appended to a binary stream.

THE `STDIO.H` HEADER FILE

To provide support for streams of the various kinds, a number of functions and macros exist. The `<stdio.h>` header file contains the various declarations necessary for the functions, together with the following macro and type declarations:

FILE

The type of an object used to contain stream control information. Users of studio never need to know the contents of these objects, but simply manipulate pointers to them. It is not safe to copy these objects within the programme; sometimes their addresses may be ‘magic’.

fpos_t

A type of object that can be used to record unique values of a stream’s file position indicator.

_IOFBF _IOLBF _IONBF

Values used to control the buffering of a stream in conjunction with the `setvbuf` function.

BUFSIZ

The size of the buffer used by the `setbuf` function. An integral constant expression whose value is at least 256.

EOF

A negative integral constant expression, indicating the end-of-file condition on a stream *i.e.* that there is no more input.

FILENAME_MAX

The maximum length which a filename can have, if there is a limit, or otherwise the recommended size of an array intended to hold a file name.

FOPEN_MAX

The minimum number of files that the implementation guarantees may be held open concurrently; at least eight are guaranteed. Note that three predefined streams exist and may need to be closed if a programme needs to open more than five files explicitly.

L_tmpnam

The maximum length of the string generated by `tmpnam`; an integral constant expression.

SEEK_CUR SEEK_END SEEK_SET

Integral constant expressions used to control the actions of `fseek`.

TMP_MAX

The minimum number of unique filenames generated by `tmpnam`; an integral constant expression with a value of at least 25.

stdin stdout stderr

Predefined objects of type `(FILE *)` referring to the standard input, output and error streams respectively. These streams are automatically open when a programme starts execution.

FUNCTIONS OF STREAMS

OPENING

A stream is connected to a file by means of the `fopen`, `freopen` or `tmpfile` functions. These functions will, if successful, return a pointer to a `FILE` object. Three streams are

available without any special action; they are normally all connected to the physical device associated with the executing programme: usually your terminal.

They are referred to by the names `stdin`, the *standard input*, `stdout`, the *standard output*, and `stderr`, the *standard error* streams. Normal keyboard input is from `stdin`, normal terminal output is to `stdout`, and error messages are directed to `stderr`.

The separation of error messages from normal output messages allows the `stdout` stream to be connected to something other than the terminal device, and still to have error messages appear on the screen in front of you, rather than to be redirected to this file. These files are only fully buffered if they do not refer to interactive devices. As mentioned earlier, the file position indicator may or may not be movable, depending on the underlying device. It is not possible, for example, to move the file position indicator on `stdin` if that is connected to a terminal, as it usually is. All non-temporary files must have a *file name*, which is a string. The rules for what constitutes valid file names are implementation defined. Whether a file can be simultaneously open multiple times is also implementation defined. Opening a new file may involve creating the file. Creating an existing file causes its previous contents to be discarded.

CLOSING

Files are closed by explicitly calling `fclose`, `exit` or by returning from `main`. Any buffered data is flushed. If a programme stops for some other reason, the status of files which it had open is undefined.

BUFFERING

There are three types of buffering:

- *Unbuffered:* Minimum internal storage is used by `stdio` in an attempt to send or receive data as soon as possible.
- *Line buffered:* Characters are processed on a line-by-line basis. This is commonly used in interactive environments, and internal buffers are flushed only when full or when a newline is processed.
- *Fully buffered:* Internal buffers are only flushed when full.

The buffering associated with a stream can always be flushed by using `fflush` explicitly. Support for the various types of buffering is implementation defined, and can be controlled within these limits using `setbuf` and `setvbuf`.

DIRECT FILE MANIPULATION

A number of functions exist to operate on files directly.

```
#include <stdio.h>
int remove(const char *filename);
int rename(const char *old, const char *new);
char *tmpnam(char *s);
FILE *tmpfile(void);
remove
```


Causes a file to be removed. Subsequent attempts to open the file will fail, unless it is first created again. If the file is already open, the operation of remove is implementation defined. The return value is zero for success, any other value for failure.

rename

Changes the name of the file identified by old to new. Subsequent attempts to open the original name will fail, unless another file is created with the old name. As with remove, rename returns zero for a successful operation, any other value indicating a failure. If a file with the new name exists prior to calling rename, the behaviour is implementation defined. If rename fails for any reason, the original file is unaffected.

tmpnam

Generates a string that may be used as a file name and is guaranteed to be different from any existing file name. It may be called repeatedly, each time generating a new name. The constant TMP_MAX is used to specify how many times tmpnam may be called before it can no longer find a unique name. TMP_MAX will be at least 25. If tmpnam is called more than this number of times, its behaviour is undefined by the Standard, but many implementations offer no practical limit. If the argument s is set to NULL, then tmpnam uses an internal buffer to build the name, and returns a pointer to that. Subsequent calls may alter the same internal buffer. The argument may instead point to an array of at least L_tmpnam characters, in which case the name will be filled into the supplied buffer. Such a file name may then be created, and used as a temporary file. Since the name is generated by the function, it is unlikely to be very useful in any other context. Temporary files of this nature are not removed, except by direct calls to the remove function. They are most often used to pass temporary data between two separate programmes.

tmpfile

Creates a temporary binary file, opened for update, and returns a pointer to the stream of that file. The file will be removed when the stream is closed. If no file could be opened, tmpfile returns a null pointer.

FREOPEN

The freopen function is used to take an existing stream pointer and associate it with another named file:

```
#include <stdio.h>
FILE *freopen(const char *pathname,
               const char *mode, FILE *stream);
```

The mode argument is the same as for fopen. The stream is closed first, and any errors from the close are ignored. On error, NULL is returned, otherwise the new value for stream is returned.

CLOSING FILES

An open file is closed using fclose.

```
#include <stdio.h>
int fclose(FILE *stream);
```

Any unwritten data buffered for stream is flushed out and any unread data is thrown away. If a buffer had been automatically allocated for the stream, it is freed. The file is then closed. Zero is returned on success, EOF if any error occurs.

SETBUF, SETVBUF

These two functions are used to change the buffering strategy for an open stream:

```
#include <stdio.h>
int setvbuf(FILE *stream, char *buf,
int type, size_t size);
void setbuf(FILE *stream, char *buf);
```

They must be used *before* the file is either read from or written to. The type argument defines how the stream will be buffered.

Table. Table Type of buffering.

Value	Effect
_IONBF	Do not buffer I/O
_IOFBF	Fully buffer I/O
_IOLBF	Line buffer: flush buffer when full, when newline is written or when a read is requested.

The buf argument can be a null pointer, in which case an array is automatically allocated to hold the buffered data. Otherwise, the user can provide a buffer, but should ensure that its lifetime is at least as long as that of the stream: a common mistake is to use automatic storage allocated inside a compound statement; in correct usage it is usual to obtain the storage from malloc instead. The size of the buffer is specified by the size argument.

A call of setbuf is exactly the same as a call of setvbuf with IOFBF for the type argument, and BUFSIZ for the size argument. If buf is a null pointer, the value _IONBF is used for type instead. No value is returned by setbuf, setvbuf returns zero on success, non-zero if invalid values are provided for type or size, or the request cannot be complied with.

FFLUSH

```
#include <stdio.h>
int fflush(FILE *stream);
```

If stream refers to a file opened for output or update, any unwritten data is ‘written’ out. Exactly what that means is a function of the host environment, and C cannot guarantee, for example, that data immediately reaches the surface of a disk which might be supporting the file.

If the stream is associated with a file opened for input or update, any preceding ungetc operation is forgotten. The most recent operation on the stream must have been an output operation; if not, the behaviour is undefined.

A call of `fflush` with an argument of zero flushes every output or update stream. Care is taken to avoid those streams that have not had an output as their last operation, thus avoiding the undefined behaviour mentioned above. EOF is returned if an error occurs, otherwise zero.

Variables and Expressions

VALUES AND TYPES

A value is one of the fundamental things like a letter or a number that a programme manipulates. The values we have seen so far are 2 (the result when we added $1 + 1$), and ‘Hello, World!’. These values belong to different types: 2 is an integer, and ‘Hello, World!’ is a string, so-called because it contains a “string” of letters. You (and the interpreter) can identify strings because they are enclosed in quotation marks. The print statement also works for integers.

```
print 4
4
```

If you are not sure what type a value has, the interpreter can tell you.

```
type('Hello, World!')
type 'str'>
type(17)
type 'int'>
```

Not surprisingly, strings belong to the type str and integers belong to the type int. Less obviously, numbers with a decimal point belong to a type called float, because these numbers are represented in a format called floating-point.

```
type(3. 2)
type 'float'>
```

What about values like ‘17’ and ‘3. 2’? They look like numbers, but they are in quotation marks like strings.

```

type('17')
type 'str'>
type('3. 2')
type 'str'>

```

They're strings: When you type a large integer, you might be tempted to use commas between groups of three digits, as in 1, 000, 000.

This is not a legal integer in Python, but it is a legal expression:

```

print 1, 000, 000
1 0 0

```

Well, that's not what we expected at all! Python interprets 1, 000, 000 as a comma-separated list of three integers, which it prints consecutively. This is the first example we have seen of a semantic error: the code runs without producing an error message, but it doesn't do the "right" thing.

VARIABLES

One of the most powerful features of a programming language is the ability to manipulate variables. A variable is a name that refers to a value.

The assignment statement creates new variables and gives them values:

```

message = "What's up, Doc?"
n = 17
pi = 3. 14159

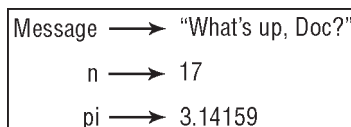
```

This example makes three assignments. The first assigns the string "What's up, Doc?" to a new variable named `message`. The second gives the integer 17 to `n`, and the third gives the floating-point number 3. 14159 to `pi`.

Notice that the first statement uses double quotes to enclose the string. In general, single and double quotes do the same thing, but if the string contains a single quote (or an apostrophe, which is the same character), you have to use double quotes to enclose it.

A common way to represent variables on paper is to write the name with an arrow pointing to the variable's value. This kind of figure is called a state diagram because it shows what state each of the variables is in (think of it as the variable's state of mind).

This diagram shows the result of the assignment statements:



The print statement also works with variables.

```

print message
What's up, Doc?
print n
17
print pi
3. 14159

```

In each case the result is the value of the variable. Variables also have types; again, we can ask the interpreter what they are.

```

type(message)
type `str`>
type(n)
type `int`>
type(pi)
type `float`>

```

The type of a variable is the type of the value it refers to.

VARIABLE NAMES AND KEYWORDS

Programmers generally choose names for their variables that are meaningful they document what the variable is used for. Variable names can be arbitrarily long. They can contain both letters and numbers, but they have to begin with a letter. Although it is legal to use uppercase letters, by convention we don't. If you do, remember that case matters. Bruce and bruce are different variables. The underscore character (_) can appear in a name. It is often used in names with multiple words, such as my_name or price_of_tea_in_china.

If you give a variable an illegal name, you get a syntax error:

```

76trombones = 'big parade'
SyntaxError: invalid syntax
more$ = 1000000
SyntaxError: invalid syntax
class = 'Computer Science 101'
SyntaxError: invalid syntax

```

76trombones is illegal because it does not begin with a letter. more\$ is illegal because it contains an illegal character, the dollar sign. But what's wrong with class?

It turns out that class is one of the Python keywords. Keywords define the language's rules and structure, and they cannot be used as variable names.

Python has twenty-nine keywords:

and	def	exec	if	not	return
assert	del	finally	import	or	try
break	elif	for	in	pass	while
class	else	from	is	print	yield
continue	except	global	lambda	raise	

If the interpreter complains about one of your variable names and you don't know.

ITERATION

MULTIPLE ASSIGNMENT

As you may have discovered, it is legal to make more than one assignment to the same variable. A new assignment makes an existing variable refer to a new value (and stop referring to the old value).

```

bruce = 5
print bruce,
bruce = 7
print bruce

```

The output of this programme is 5 7, because the first time bruce is printed, his value is 5, and the second time, his value is 7. The comma at the end of the first print statement suppresses the newline after the output, which is why both outputs appear on the same line.

With multiple assignment it is especially important to distinguish between an assignment operation and a statement of equality. Because Python uses the equal sign (=) for assignment, it is tempting to interpret a statement like `a = b` as a statement of equality. It is not!

First, equality is commutative and assignment is not. For example, in mathematics, if $a = 7$ then $7 = a$. But in Python, the statement `a = 7` is legal and `7 = a` is not.

Furthermore, in mathematics, a statement of equality is always true. If $a = b$ now, then a will always equal b .

In Python, an assignment statement can make two variables equal, but they don't have to stay that way:

```

a = 5
b = a      # a and b are now equal
a = 3      # a and b are no longer equal

```

The third line changes the value of `a` but does not change the value of `b`, so they are no longer equal. (In some programming languages, a different symbol is used for assignment, such as `←` or `:=`, to avoid confusion.)

Although multiple assignment is frequently helpful, you should use it with caution. If the values of variables change frequently, it can make the code difficult to read and debug.

THE WHILE STATEMENT

Computers are often used to automate repetitive tasks. Repeating identical or similar tasks without making errors is something that computers do well and people do poorly.

We have seen two programmes, `nLines` and `countdown`, that use recursion to perform repetition, which is also called iteration. Because iteration is so common, Python provides several language features to make it easier. The first feature we are going to look at is the `while` statement.

Here is what `countdown` looks like with a `while` statement:

```

def countdown(n):
    while n > 0:
        print n
        n = n-1
    print "Blastoff!"

```

Since we removed the recursive call, this function is not recursive. You can almost read the `while` statement as if it were English. It means, "While `n` is greater than 0,

continue displaying the value of n and then reducing the value of n by 1. When you get to 0, display the word Blastoff!”

More formally, here is the flow of execution for a while statement:

- Evaluate the condition, yielding 0 or 1.
- If the condition is false (0), exit the while statement and continue execution at the next statement.
- If the condition is true (1), execute each of the statements in the body and then go back to step 1.

The body consists of all of the statements below the header with the same indentation.

This type of flow is called a loop because the third step loops back around to the top. Notice that if the condition is false the first time through the loop, the statements inside the loop are never executed.

The body of the loop should change the value of one or more variables so that eventually the condition becomes false and the loop terminates. Otherwise the loop will repeat forever, which is called an infinite loop. An endless source of amusement for computer scientists is the observation that the directions on shampoo, “Lather, rinse, repeat,” are an infinite loop. In the case of countdown, we can prove that the loop terminates because we know that the value of n is finite, and we can see that the value of n gets smaller each time through the loop, so eventually we have to get to 0.

In other cases, it is not so easy to tell:

```
def sequence(n):
    while n != 1:
        print n,
        if n%2 == 0:          # n is even
            n = n/2
        else:                 # n is odd
            n = n*3+1
```

The condition for this loop is $n \neq 1$, so the loop will continue until n is 1, which will make the condition false. Each time through the loop, the programme outputs the value of n and then checks whether it is even or odd. If it is even, the value of n is divided by 2. If it is odd, the value is replaced by $n*3+1$. For example, if the starting value (the argument passed to `sequence`) is 3, the resulting sequence is 3, 10, 5, 16, 8, 4, 2, 1. Since n sometimes increases and sometimes decreases, there is no obvious proof that n will ever reach 1, or that the programme terminates. For some particular values of n , we can prove termination.

For example, if the starting value is a power of two, then the value of n will be even each time through the loop until it reaches 1. The previous example ends with such a sequence, starting with 16.

Particular values aside, the interesting question is whether we can prove that this programme terminates for *all positive values* of n . So far, no one has been able to prove it *or* disprove it! As an exercise, rewrite the function `nLines` from Section 4.9 using iteration instead of recursion.

TWO-DIMENSIONAL TABLES

A two-dimensional table is a table where you read the value at the intersection of a row and a column. A multiplication table is a good example. Let's say you want to print a multiplication table for the values from 1 to 6.

A good way to start is to write a loop that prints the multiples of 2, all on one line:

```
i = 1
while i <= 6:
    print 2*i, ' ',
    i = i + 1
print
```

The first line initializes a variable named *i*, which acts as a counter or loop variable. As the loop executes, the value of *i* increases from 1 to 6. When *i* is 7, the loop terminates. Each time through the loop, it displays the value of $2*i$, followed by three spaces.

Again, the comma in the print statement suppresses the newline. After the loop completes, the second print statement starts a new line.

The output of the programme is:

```
2           4           6           8           10          12
```

So far, so good. The next step is to encapsulate and generalize.

ENCAPSULATION AND GENERALIZATION

Encapsulation is the process of wrapping a piece of code in a function, allowing you to take advantage of all the things functions are good. Generalization means taking something specific, such as printing the multiples of 2, and making it more general, such as printing the multiples of any integer.

This function encapsulates the previous loop and generalizes it to print multiples of n:

```
def printMultiples(n):
    i = 1
    while i <= 6:
        print n*i, '\t',
        i = i + 1
    print
```

To encapsulate, all we had to do was add the first line, which declares the name of the function and the parameter list. To generalize, all we had to do was replace the value 2 with the parameter *n*.

If we call this function with the argument 2, we get the same output as before. With the argument 3, the output is:

```
3           6           9           12          15          18
```

With the argument 4, the output is:

```
4           8           12          16          20          24
```

By now you can probably guess how to print a multiplication table by calling `printMultiples` repeatedly with different arguments.

In fact, we can use another loop:

```
i = 1
while i ≤ 6:
    printMultiples(i)
    i = i + 1
```

Notice how similar this loop is to the one inside `printMultiples`. All we did was replace the `print` statement with a function call.

Table. The Output of This Programme is a Multiplication Table.

1	2	3	4	5	6
2	4	6	8	10	12
3	6	9	12	15	18
4	8	12	16	20	24
5	10	15	20	25	30
6	12	18	24	30	36

MORE ENCAPSULATION

To demonstrate encapsulation again, let's take the code from the end of Section 6.5 and wrap it up in a function:

```
def printMultTable():
    i = 1
    while i ≤ 6:
        printMultiples(i)
        i = i + 1
```

This process is a common development plan. We develop code by writing lines of code outside any function, or typing them in to the interpreter. When we get the code working, we extract it and wrap it up in a function.

This development plan is particularly useful if you don't know, when you start writing, how to divide the programme into functions. This approach lets you design as you go along.

LOCAL VARIABLES

You might be wondering how we can use the same variable, `i`, in both `printMultiples` and `printMultTable`. Doesn't it cause problems when one of the functions changes the value of the variable? The answer is no, because the `i` in `printMultiples` and the `i` in `printMultTable` are *not* the same variable. Variables created inside a function definition are local; you can't access a local variable from outside its "home" function. That means you are free to have multiple variables with the same name as long as they are not in the same function.

The stack diagram for this programme shows that the two variables named `i` are not the same variable. They can refer to different values, and changing one does not affect the other.

The value of `i` in `print MultTable` goes from 1 to 6. In the diagram it happens to be 3. The next time through the loop it will be 4. Each time through the loop, `print MultTable` calls `print Multiples` with the current value of `i` as an argument. That value gets assigned to the parametre `n`.

Inside `print Multiples`, the value of `i` goes from 1 to 6. In the diagram, it happens to be 2. Changing this variable has no effect on the value of `i` in `print MultTable`.

It is common and perfectly legal to have different local variables with the same name. In particular, names like `i` and `j` are used frequently as loop variables. If you avoid using them in one function just because you used them somewhere else, you will probably make the programme harder to read.

MORE GENERALIZATION

As another example of generalization, imagine you wanted a programme that would print a multiplication table of any size, not just the six-by-six table.

You could add a parametre to `printMultTable`:

```
def printMultTable(high):
    i = 1
    while i ≤ high:
        printMultiples(i)
        i = i + 1
```

We replaced the value 6 with the parametre `high`. If we call `print Mult Table` with the argument 7, it displays:

1	2	3	4	5	6
2	4	6	8	10	12
3	6	9	12	15	18
4	8	12	16	20	24
5	10	15	20	25	30
6	12	18	24	30	36
7	14	21	28	35	42

This is fine, except that we probably want the table to be square with the same number of rows and columns. To do that, we add another parametre `toprintMultiples` to specify how many columns the table should have.

Just to be annoying, we call this parametre `high`, demonstrating that different functions can have parametres with the same name (just like local variables).

Here's the whole programme:

```
def printMultiples(n, high):
    i = 1
    while i ≤ high:
        print n*i, '\t',
        i = i + 1
    print
def printMultTable(high):
    i = 1
    while i ≤ high:
```

```

    printMultiples(i, high)
    i = i + 1

```

Notice that when we added a new parameter, we had to change the first line of the function (the function heading), and we also had to change the place where the function is called in `printMultTable`.

**Table. As Expected, this Programme Generates
a Square Seven-by-seven.**

1	2	3	4	5	6	7
2	4	6	8	10	12	14
3	6	9	12	15	18	21
4	8	12	16	20	24	28
5	10	15	20	25	30	35
6	12	18	24	30	36	42
7	14	21	28	35	42	49

When you generalize a function appropriately, you often get a programme with capabilities you didn't plan. For example, you might notice that, because $ab = ba$, all the entries in the table appear twice.

You could save ink by printing only half the table. To do that, you only have to change one line of `printMultTable`. Change

```

    printMultiples(i, high) to
    printMultiples(i, i)
    and you get

```

1						
2	4					
3	6	9				
4	8	12	16			
5	10	15	20	25		
6	12	18	24	30	36	
7	14	21	28	35	42	49

As an exercise, trace the execution of this version of `printMultTable` and figure out how it works.

6. 9 Functions

A few times now, we have mentioned “all the things functions are good for.” By now, you might be wondering what exactly those things are.

Here are some of them:

- Giving a name to a sequence of statements makes your programme easier to read and debug.
- Dividing a long programme into functions allows you to separate parts of the programme, debug them in isolation, and then compose them into a whole.
- Functions facilitate both recursion and iteration.
- Well-designed functions are often useful for many programmes. Once you write and debug one, you can reuse it.

FUNCTIONS

FUNCTION CALLS

You have already seen one example of a function call:

```
>>> type("32")
<type 'str'>
```

The name of the function is `type`, and it displays the type of a value or variable. The value or variable, which is called the argument of the function, has to be enclosed in parentheses. It is common to say that a function “takes” an argument and “returns” a result. The result is called the return value.

Instead of printing the return value, we could assign it to a variable:

```
betty = type("32")
print betty
type 'str'>
```

As another example, the `id` function takes a value or a variable and returns an integer that acts as a unique identifier for the value:

```
id(3)
134882108
betty = 3
id(betty)
134882108
```

Every value has an `id`, which is a unique number related to where it is stored in the memory of the computer. The `id` of a variable is the `id` of the value to which it refers.

TYPE CONVERSION

Python provides a collection of built-in functions that convert values from one type to another. The `int` function takes any value and converts it to an integer, if possible, or complains otherwise:

```
int("32")
32
int("Hello")
ValueError: invalid literal for int(): Hello
```

int can also convert floating-point values to integers, but remember that it truncates the fractional part:

```
int(3.99999)
3
int(-2.3)
-2
```

The float function converts integers and strings to floating-point numbers:

```
float(32)
32.0
float("3.14159")
3.14159
```

Finally, the `str` function converts to type string:

```
str(32)
'32'
str(3.14149)
'3.14149'
```

It may seem odd that Python distinguishes the integer value 1 from the floating-point value 1.0. They may represent the same number, but they belong to different types. The reason is that they are represented differently inside the computer.

TYPE COERCION

Now that we can convert between types, we have another way to deal with integer division. Returning to the example from the previous chapter, suppose we want to calculate the fraction of an hour that has elapsed. The most obvious expression, `minute/60`, does integer arithmetic, so the result is always 0, even at 59 minutes past the hour.

One solution is to convert `minute` to floating-point and do floating-point division:

```
minute = 59
float(minute)/60
0.983333333333
```

Alternatively, we can take advantage of the rules for automatic type conversion, which is called type coercion. For the mathematical operators, if either operand is a float, the other is automatically converted to a float:

```
minute = 59
minute/60.0
0.983333333333
```

By making the denominator a float, we force Python to do floating-point division.

MATH FUNCTIONS

In mathematics, you have probably seen functions like `sin` and `log`, and you have learned to evaluate expressions like `sin(pi/2)` and `log(1/x)`. First, you evaluate the expression in parentheses (the argument). For example, `pi/2` is approximately 1.571, and `1/x` is 0.1 (if `x` happens to be 10.0). Then, you evaluate the function itself, either by looking it up in a table or by performing various computations. The `sin` of 1.571 is 1, and the `log` of 0.1 is `-1` (assuming that `log` indicates the logarithm base 10).

This process can be applied repeatedly to evaluate more complicated expressions like `log(1/sin(pi/2))`. First, you evaluate the argument of the innermost function, then evaluate the function, and so on.

Python has a `math` module that provides most of the familiar mathematical functions. A module is a file that contains a collection of related functions grouped together.

Before we can use the functions from a module, we have to import them:

```
import math
```

To call one of the functions, we have to specify the name of the module and the name of the function, separated by a dot, also known as a period. This format is called dot notation.

```
decibel = math. log10 (17. 0)
angle = 1. 5
height = math. sin(angle)
```

The first statement sets decibel to the logarithm of 17, base 10. There is also a function called log that takes logarithm base e. The third statement finds the sine of the value of the variable angle. sin and the other trigonometric functions (cos, tan, etc.) take arguments in radians. To convert from degrees to radians, divide by 360 and multiply by 2*pi. For example, to find the sine of 45 degrees, first calculate the angle in radians and then take the sine:

```
degrees = 45
angle = degrees * 2 * math. pi/360. 0
math. sin(angle)
0. 707106781187
```

The constant pi is also part of the math module. If you know your geometry, you can check the previous result by comparing it to the square root of two divided by two:

```
math. sqrt(2)/2. 0
0. 707106781187
```

COMPOSITION

Just as with mathematical functions, Python functions can be composed, meaning that you use one expression as part of another.

For example, you can use any expression as an argument to a function:

```
x = math. cos(angle + math. pi/2)
```

This statement takes the value of pi, divides it by 2, and adds the result to the value of angle. The sum is then passed as an argument to the cos function.

You can also take the result of one function and pass it as an argument to another:

```
x = math. exp(math. log(10. 0))
```

This statement finds the log base e of 10 and then raises e to that power. The result gets assigned to x.

ADDING NEW FUNCTIONS

So far, we have only been using the functions that come with Python, but it is also possible to add new functions. Creating new functions to solve your particular problems is one of the most useful things about a general-purpose programming language. In the context of programming, a function is a named sequence of statements that performs a desired operation. This operation is specified in a function definition. The functions we have been using so far have been defined for us, and these definitions have been hidden. This is a good thing, because it allows us to use the functions without worrying about the details of their definitions.

The syntax for a function definition is:

```
def NAME(LIST OF PARAMETRES): STATEMENTS
```

You can make up any names you want for the functions you create, except that you can't use a name that is a Python keyword. The list of parametres specifies what

information, if any, you have to provide in order to use the new function. There can be any number of statements inside the function, but they have to be indented from the left margin. In the examples in this book, we will use an indentation of two spaces.

The first couple of functions we are going to write have no parameters, so the syntax looks like this:

```
def newLine():
    print
```

This function is named newLine. The empty parentheses indicate that it has no parameters. It contains only a single statement, which outputs a newline character. (That's what happens when you use a print command without any arguments.)

The syntax for calling the new function is the same as the syntax for built-in functions:

```
print "First Line."
newLine()
print "Second Line."
```

The output of this programme is:

```
First line.
Second line.
```

Notice the extra space between the two lines. What if we wanted more space between the lines? We could call the same function repeatedly:

```
print "First Line."
newLine()
newLine()
newLine()
print "Second Line."
```

Or we could write a new function named threeLines that prints three new lines:

```
def threeLines():
    newLine()
    newLine()
    newLine()
    print "First Line."
    threeLines()
    print "Second Line."
```

This function contains three statements, all of which are indented by two spaces. Since the next statement is not indented, Python knows that it is not part of the function.

You should notice a few things about this programme:

- You can call the same procedure repeatedly. In fact, it is quite common and useful to do so.
- You can have one function call another function; in this case threeLines calls new Line.

So far, it may not be clear why it is worth the trouble to create all of these new functions.

Actually, there are a lot of reasons, but this example demonstrates two:

- Creating a new function gives you an opportunity to name a group of statements. Functions can simplify a programme by hiding a complex computation behind a single command and by using English words in place of arcane code.

- Creating a new function can make a programme smaller by eliminating repetitive code. For example, a short way to print nine consecutive new lines is to call `threeLines` three times.

As an exercise, write a function called `nine Lines` that uses `three Lines` to print nine blank lines. How would you print twenty-seven new lines?

Structure of Programming

Probably the best way to start learning a programming language is by writing a programme.

Therefore, here is our first programme:

```
// my first programme in C++
#include <iostream>
using namespace std;
int main ()
{
    cout << "Hello World!";
    return 0;
} Hello World!
```

The first panel shows the source code for our first programme. The second one shows the result of the programme once compiled and executed. The way to edit and compile a programme depends on the compiler you are using. Depending on whether it has a Development Interface or not and on its version. Consult the compilers section and the manual or help included with your compiler if you have doubts on how to compile a C++ console programme.

The previous programme is the typical programme that programmer apprentices write for the first time, and its result is the printing on screen of the “Hello World!” sentence.

It is one of the simplest programmes that can be written in C++, but it already contains the fundamental components that every C++ programme has. We are going to look line by line at the code we have just written:

```
// my first programme in C++
```

This is a comment line. All lines beginning with two slash signs (//) are considered comments and do not have any effect on the behaviour of the programme. The programmer can use them to include short explanations or observations within the source code itself. In this case, the line is a brief description of what our programme is.

```
#include <iostream>
```

Lines beginning with a pound sign (#) are directives for the preprocessor. They are not regular code lines with expressions but indications for the compiler's preprocessor. In this case the directive `#include <iostream>` tells the preprocessor to include the `iostream` standard file. This specific file (`iostream`) includes the declarations of the basic standard input-output library in C++, and it is included because its functionality is going to be used later in the programme.

```
Using namespace std;
```

All the elements of the standard C++ library are declared within what is called a namespace, the namespace with the name `std`. So in order to access its functionality we declare with this expression that we will be using these entities. This line is very frequent in C++ programmes that use the standard library, and in fact it will be included in most of the source codes included in these tutorials.

```
int main ()
```

This line corresponds to the beginning of the definition of the main function. The main function is the point by where all C++ programmes start their execution, independently of its location within the source code. It does not matter whether there are other functions with other names defined before or after it - the instructions contained within this function's definition will always be the first ones to be executed in any C++ programme. For that same reason, it is essential that all C++ programmes have a main function.

The word `main` is followed in the code by a pair of parentheses (`()`). That is because it is a function declaration: In C++, what differentiates a function declaration from other types of expressions are these parentheses that follow its name. Optionally, these parentheses may enclose a list of parameters within them.

Right after these parentheses we can find the body of the main function enclosed in braces (`{ }`). What is contained within these braces is what the function does when it is executed.

```
cout << "Hello World";
```

This line is a C++ statement. A statement is a simple or compound expression that can actually produce some effect. In fact, this statement performs the only action that generates a visible effect in our first programme.

`cout` represents the standard output stream in C++, and the meaning of the entire statement is to insert a sequence of characters (in this case the `Hello World` sequence of characters) into the standard output stream (which usually is the screen).

`cout` is declared in the `iostream` standard file within the `std` namespace, so that's why we needed to include that specific file and to declare that we were going to use this

specific namespace earlier in our code. Notice that the statement ends with a semicolon character (;). This character is used to mark the end of the statement and in fact it must be included at the end of all expression statements in all C++ programmes (one of the most common syntax errors is indeed to forget to include some semicolon after a statement).

```
return 0;
```

The return statement causes the main function to finish. return may be followed by a return code (in our example is followed by the return code 0). A return code of 0 for the main function is generally interpreted as the programme worked as expected without any errors during its execution. This is the most usual way to end a C++ console programme.

You may have noticed that not all the lines of this programme perform actions when the code is executed. There were lines containing only comments (those beginning by //). There were lines with directives for the compiler's preprocessor (those beginning by #). Then there were lines that began the declaration of a function (in this case, the main function) and, finally lines with statements (like the insertion into cout), which were all included within the block delimited by the braces ({ }) of the main function. The programme has been structured in different lines in order to be more readable, but in C++, we do not have strict rules on how to separate instructions in different lines. For example, instead of,

```
int main ()
{
    cout << " Hello World ";
    return 0;
}
```

We could have written:

```
int main () { cout << "Hello World"; return 0; }
```

All in just one line and this would have had exactly the same meaning as the previous code.

In C++, the separation between statements is specified with an ending semicolon (;) at the end of each one, so the separation in different code lines does not matter at all for this purpose. We can write many statements per line or write a single statement that takes many code lines. The division of code in different lines serves only to make it more legible and schematic for the humans that may read it.

Let us add an additional instruction to our first programme:

```
// my second programme in C++
#include <iostream>
using namespace std;
int main ()
{
    cout << "Hello World! ";
    cout << "I'm a C++ programme";
    return 0;
} Hello World! I'm a C++ programme
```

In this case, we performed two insertions into `cout` in two different statements. Once again, the separation in different lines of code has been done just to give greater readability to the programme, since `main` could have been perfectly valid defined this way:

```
int main(){cout << " Hello World! "; cout << " I'm a C++ programme ";
return 0; }
```

We were also free to divide the code into more lines if we considered it more convenient:

```
int main ()
{
    cout <<
        "Hello World!";
    cout
        << "I'm a C++ programme";
    return 0;
}
```

And the result would again have been exactly the same as in the previous examples.

IDENTIFIERS

A valid identifier is a sequence of one or more letters, digits or underscore characters (`_`). Neither spaces nor punctuation marks or symbols can be part of an identifier. Only letters, digits and single underscore characters are valid. In addition, variable identifiers always have to begin with a letter.

They can also begin with an underline character (`_`), but in some cases these may be reserved for compiler specific keywords or external identifiers, as well as identifiers containing two successive underscore characters anywhere. In no case they can begin with a digit.

Another rule that you have to consider when inventing your own identifiers is that they cannot match any keyword of the C++ language nor your compiler's specific ones, which are *reserved keywords*. The standard reserved keywords are:

`asm, auto, bool, break, case, catch, char, class, const, const_cast, continue, default, delete, do, double, dynamic_cast, else, enum, explicit, export, extern, false, float, for, friend, goto, if, inline, int, long, mutable, namespace, new, operator, private, protected, public, register, reinterpret_cast, return, short, signed, size of, static, static_cast, struct, switch, template, this, throw, true, try, typedef, typeid, typename, union, unsigned, using, virtual, void, volatile, wchar_t, while`

Additionally, alternative representations for some operators cannot be used as identifiers since they are reserved words under some circumstances: `and, and_eq, bitand, bitor, compl, not, not_eq, or, or_eq, xor, xor_eq`

Your compiler may also include some additional specific reserved keywords. Very important: The C++ language is a "case sensitive" language. That means that an identifier written in capital letters is not equivalent to another one with the same name but written

in small letters. Thus, for example, the `RESULT` variable is not the same as the `result` variable or the `Result` variable. These are three different variable identifiers.

Fundamental Data Types

When programming, we store the variables in our computer's memory, but the computer has to know what kind of data we want to store in them, since it is not going to occupy the same amount of memory to store a simple number than to store a single letter or a large number, and they are not going to be interpreted the same way.

The memory in our computers is organized in bytes. A byte is the minimum amount of memory that we can manage in C++. A byte can store a relatively small amount of data: one single character or a small integer (generally an integer between 0 and 255). In addition, the computer can manipulate more complex data types that come from grouping several bytes, such as long numbers or non-integer numbers.

Next you have a summary of the basic fundamental data types in C++, as well as the range of values that can be represented with each one:

Name	Description	Size	Range
char	Character or small integer.	1byte	signed: -128 to 127 unsigned: 0 to 255
short int	Short Integer. (short)	2bytes	signed: -32768 to 32767 unsigned: 0 to 65535
int	Integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
long int (long)	Long integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
bool	Boolean value. It can take one of two values: true or false.	1byte	true or false
float	Floating point number.	4bytes	3.4e +/- 38 (7 digits)
double	Double precision floating point number.	8bytes	1.7e +/- 308 (15 digits)
long double	Long double precision floating point number.	8bytes	1.7e +/- 308 (15 digits)
wchar_t	Wide character.	2bytes	1 wide character

The values of the columns `Size` and `Range` depend on the system the programme is compiled for. The values shown above are those found on most 32-bit systems. But for other systems, the general specification is that `int` has the natural size suggested by the system architecture (one “*word*”) and the four integer types `char`, `short`, `int` and `long` must each one be at least as large as the one preceding it, with `char` being always 1 byte in size.

The same applies to the floating point types `float`, `double` and `long double`, where each one must provide at least as much precision as the preceding one.

Declaration of Variables

In order to use a variable in C++, we must first declare it specifying which data type we want it to be. The syntax to declare a new variable is to write the specifier of the desired data type (like int, bool, float...) followed by a valid variable identifier.

For example:

```
int a;b
float mynumber;
```

These are two valid declarations of variables. The first one declares a variable of type int with the identifier a. The second one declares a variable of type float with the identifier mynumber. Once declared, the variables a and mynumber can be used within the rest of their scope in the programme.

If you are going to declare more than one variable of the same type, you can declare all of them in a single statement by separating their identifiers with commas.

For example:

```
int a, b, c;
```

This declares three variables (a, b and c), all of them of type int, and has exactly the same meaning as:

```
int a;
int b;
int c;
```

The integer data types char, short, long and int can be either signed or unsigned depending on the range of numbers needed to be represented. Signed types can represent both positive and negative values, whereas unsigned types can only represent positive values (and zero). This can be specified by using either the specifier signed or the specifier unsigned before the type name.

For example:

```
unsigned short int NumberOfSisters;
signed int MyAccountBalance;
```

By default, if we do not specify either signed or unsigned most compiler settings will assume the type to be signed, therefore instead of the second declaration above we could have written:

```
int MyAccountBalance;
```

with exactly the same meaning (with or without the keyword signed) An exception to this general rule is the char type, which exists by itself and is considered a different fundamental data type from signed char and unsigned char, thought to store characters. You should use either signed or unsigned if you intend to store numerical values in a char-sized variable. Short and long can be used alone as type specifier. In this case, they refer to their respective integer fundamental types: short is equivalent to short int and long is equivalent to long int. The following two variable declarations are equivalent:

```
short Year;
short int Year;
```

Finally, signed and unsigned may also be used as stand-alone type specifier, meaning the same as signed int and unsigned int respectively.

The following two declarations are equivalent:

```
unsigned Next Year;
unsigned int Next Year;
```

To see what variable declarations look like in action within a programme, we are going to see the C++ code of the example about your mental memory proposed at the beginning of this section:

```
//operating with variables
#include <iostream>
using namespace std;
int main ()
{
    //declaring variables:
    int a, b;
    int result;
    //process:
    a = 5;
    b = 2;
    a = a + 1;
    result = a-b;
    //print out the result:
    cout << result;
    //terminate the programme:
    return 0;
} 4
```

Do not worry if something else than the variable declarations themselves looks a bit strange to you. You will see the rest in detail in coming sections.

VARIABLES

All the variables that we intend to use in a programme must have been declared with its type specifier in an earlier point in the code, like we did in the previous code at the beginning of the body of the function main when we declared that a, b, and result were of type int.

```
#Include <iostream>
using namespace std;
```

```
int Integer;
char aCharacter;
char string [20];
unsigned int NumberOfSons;
```

Global variables

```
int main()
{
```

```
    unsigned short Age;
    float Anumber, AnotherOne;
```

Local variables

```
    count << "Enter your age:"
    cin >> Age;
    ...
```

Instructions

```
}
```


A variable can be either of global or local scope. A global variable is a variable declared in the main body of the source code, outside all functions, while a local variable is one declared within the body of a function or a block.

Global variables can be referred from anywhere in the code, even inside functions, whenever it is after its declaration. The scope of local variables is limited to the block enclosed in braces ({ }) where they are declared.

For example, if they are declared at the beginning of the body of a function (like in function main) their scope is between its declaration point and the end of that function. In the example above, this means that if another function existed in addition to main, the local variables declared in main could not be accessed from the other function and vice versa.

Initialization of Variables

When declaring a regular local variable, its value is by default undetermined. But you may want a variable to store a concrete value at the same moment that it is declared. In order to do that, you can initialize the variable.

There are two ways to do this in C++: The first one, known as c-like, is done by appending an equal sign followed by the value to which the variable will be initialized:
type identifier = initial_value;

For example, if we want to declare an int variable called a initialized with a value of 0 at the moment in which it is declared, we could write:

```
int a = 0;
```

The other way to initialize variables, known as constructor initialization, is done by enclosing the initial value between parentheses (()):

```
type identifier (initial_value);
```

For example:

```
int a (0);
```

Both ways of initializing variables are valid and equivalent in C++.

```
//initialization of variables
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main ()
```

```
{
```

```
    int a=5;//initial value = 5
```

```
    int b(2);//initial value = 2
```

```
    int result;//initial value undetermined
```

```
    a = a + 3;
```

```
    result = a-b;
```

```
    cout << result;
```

```
    return 0;
```

```
} 6
```

STRINGS

Variables that can store non-numerical values that are longer than one single character are known as strings. The C++ language library provides support for strings through the standard

string class. This is not a fundamental type, but it behaves in a similar way as fundamental types do in its most basic usage. A first difference with fundamental data types is that in order to declare and use objects (variables) of this type we need to include an additional header file in our source code: `<string>` and have access to the `std` namespace (which we already had in all our previous programmes thanks to the `using namespace std` statement).

```
//my first string
#include <iostream>
#include <string>
using namespace std;
int main ()
{
string mystring = "This is a string";
cout << mystring;
return 0;
}
```

output

This is a string

As you may see in the previous example, strings can be initialized with any valid string literal just like numerical type variables can be initialized to any valid numerical literal. Both initialization formats are valid with strings:

```
string mystring = "This is a string";
string mystring ("This is a string");
```

Strings can also perform all the other basic operations that fundamental data types can, like being declared without an initial value and being assigned values during execution:

```
//my first string
#include <iostream>
#include <string>
using namespace std;
int main ()
{
string mystring;
mystring = "This is the initial string content";
cout << mystring << endl;
mystring = "This is a different string content";
cout << mystring << endl;
return 0;
}
```

output

This is the initial string content

This is a different string content

For more details on C++ strings, you can have a look at the string class reference.

Literals

Literals are used to express particular values within the source code of a programme. We have already used these previously to give concrete values to variables or to express messages we wanted our programmes to print out, for example, when we wrote:

```
a = 5;
```

the 5 in this piece of code was a literal constant.

Literal constants can be divided in Integer Numerals, Floating-Point Numerals, Characters, Strings and Boolean Values.

Integer Numerals

```
1776
```

```
707
```

```
-273
```

They are numerical constants that identify integer decimal values. Notice that to express a numerical constant we do not have to write quotes (“”) nor any special character. There is no doubt that it is a constant: whenever we write 1776 in a programme, we will be referring to the value 1776.

In addition to decimal numbers (those that all of us are used to use every day) C++ allows the use as literal constants of octal numbers (base 8) and hexadecimal numbers (base 16). If we want to express an octal number we have to precede it with a 0 (zero character). And in order to express a hexadecimal number we have to precede it with the characters 0x (zero, x).

For example, the following literal constants are all equivalent to each other:

```
75//decimal
```

```
0113//octal
```

```
0x4b//hexadecimal
```

All of these represent the same number: 75 (seventy-five) expressed as a base-10 numeral, octal numeral and hexadecimal numeral, respectively.

Literal constants, like variables, are considered to have a specific data type. By default, integer literals are of type int. However, we can force them to either be unsigned by appending the u character to it, or long by appending l:

```
75//int
```

```
75u//unsigned int
```

```
75l//long
```

```
75ul//unsigned long
```

In both cases, the suffix can be specified using either upper or lower case letters.

Floating Point Numbers: They express numbers with decimals and/or exponents. They can include either a decimal point, an e character (that expresses “by ten at the Xth height”, where X is an integer value that follows the e character), or both a decimal point and an e character:

```
3.14159//3.14159
```

```
6.02e23//6.02 x 10^23
```

```
1.6e-19//1.6 x 10-19
```

```
3.0//3.0
```

These are four valid numbers with decimals expressed in C++. The first number is PI, the second one is the number of Avogadro, the third is the electric charge of an electron (an extremely small number) -all of them approximated- and the last one is the

number three expressed as a floating-point numeric literal. The default type for floating point literals is double. If you explicitly want to express a float or long double numerical literal, you can use the `f` or `l` suffixes respectively:

```
3.14159L//long double
6.02e23f//float
```

Any of the letters than can be part of a floating-point numerical constant (`e`, `f`, `l`) can be written using either lower or uppercase letters without any difference in their meanings.

Character and String Literals

There also exist non-numerical constants, like:

```
'z'
'p'
"Hello world"
"How do you do?"
```

The first two expressions represent single character constants, and the following two represent string literals composed of several characters. Notice that to represent a single character we enclose it between single quotes (`'`) and to express a string (which generally consists of more than one character) we enclose it between double quotes (`"`).

When writing both single character and string literals, it is necessary to put the quotation marks surrounding them to distinguish them from possible variable identifiers or reserved keywords.

Notice the difference between these two expressions:

```
x
'x'
```

`x` alone would refer to a variable whose identifier is `x`, whereas `'x'` (enclosed within single quotation marks) would refer to the character constant `'x'`.

Character and string literals have certain peculiarities, like the escape codes. These are special characters that are difficult or impossible to express otherwise in the source code of a programme, like newline (`\n`) or tab (`\t`). All of them are preceded by a backslash (`\`).

Here you have a list of some of such escape codes:

```
\n newline
\r carriage return
\t tab
\v vertical tab
\b backspace
\f form feed (page feed)
\a alert (beep)
\' single quote (')
\" double quote (")
\? question mark (?)
\\ backslash (\)
For example:
'\n'
```

```

'\t'
"Left \t Right"
"one\ntwo\nthree"

```

Additionally, you can express any character by its numerical ASCII code by writing a backslash character (\) followed by the ASCII code expressed as an octal (base-8) or hexadecimal (base-16) number. In the first case (octal) the digits must immediately follow the backslash (for example \23 or \40), in the second case (hexadecimal), an x character must be written before the digits themselves (for example \x20 or \x4A). String literals can extend to more than a single line of code by putting a backslash sign (\) at the end of each unfinished line.

```

"string expressed in \
two lines"

```

You can also concatenate several string constants separating them by one or several blank spaces, tabulators, newline or any other valid blank character:

```

"this forms" "a single" "string" "of characters"

```

Finally, if we want the string literal to be explicitly made of wide characters (wchar_t), instead of narrow characters (char), we can precede the constant with the L prefix:

```

L"This is a wide character string"

```

Wide characters are used mainly to represent non-English or exotic character sets.

Boolean Literals

There are only two valid Boolean values: true and false. These can be expressed in C++ as values of type bool by using the Boolean literals true and false.

Defined constants (#define): You can define your own names for constants that you use very often without having to resort to memory-consuming variables, simply by using the #define preprocessor directive. Its format is:

```

#define identifier value
For example:
#define PI 3.14159265
#define NEWLINE '\n'

```

This defines two new constants: PI and NEWLINE. Once they are defined, you can use them in the rest of the code as if they were any other regular constant, for example:

```

//defined constants: calculate circumference
#include <iostream>
using namespace std;
#define PI 3.14159
#define NEWLINE '\n'
int main ()
{
    double r=5.0; //radius
    double circle;
    circle = 2 * PI * r;
    cout << circle;
    cout << NEWLINE;
    return 0;
}

```

```

}
output
31.4159

```

In fact the only thing that the compiler preprocessor does when it encounters `#define` directives is to literally replace any occurrence of their identifier (in the previous example, these were `PI` and `NEWLINE`) by the code to which they have been defined (3.14159265 and `'\n'` respectively).

The `#define` directive is not a C++ statement but a directive for the preprocessor; therefore it assumes the entire line as the directive and does not require a semicolon (`;`) at its end. If you append a semicolon character (`;`) at the end, it will also be appended in all occurrences within the body of the programme that the preprocessor replaces.

Declared Constants (`const`)

With the `const` prefix you can declare constants with a specific type in the same way as you would do with a variable:

```

const int pathwidth = 100;
const char tabulator = '\t';

```

Here, `pathwidth` and `tabulator` are two typed constants. They are treated just like regular variables except that their values cannot be modified after their definition.

Operators

Once we know of the existence of variables and constants, we can begin to operate with them. For that purpose, C++ integrates operators. Unlike other languages whose operators are mainly keywords, operators in C++ are mostly made of signs that are not part of the alphabet but are available in all keyboards. This makes C++ code shorter and more international, since it relies less on English words, but requires a little of learning effort in the beginning. You do not have to memorize all the content of this page. Most details are only provided to serve as a later reference in case you need it.

Assignment (`=`)

The assignment operator assigns a value to a variable.

```
a = 5;
```

This statement assigns the integer value 5 to the variable `a`. The part at the left of the assignment operator (`=`) is known as the *lvalue* (left value) and the right one as the *rvalue* (right value). The *lvalue* has to be a variable whereas the *rvalue* can be either a constant, a variable, the result of an operation or any combination of these.

The most important rule when assigning is the *right-to-left* rule: The assignment operation always takes place from right to left, and never the other way:

```
a = b;
```

This statement assigns to variable `a` (the *lvalue*) the value contained in variable `b` (the *rvalue*). The value that was stored until this moment in `a` is not considered at all in

this operation, and in fact that value is lost. Consider also that we are only assigning the value of b to a at the moment of the assignment operation. Therefore a later change of b will not affect the new value of a. For example, let us have a look at the following code-I have included the evolution of the content stored in the variables as comments:

```
//assignment operator
#include <iostream>
using namespace std;
int main ()
{
    int a, b; //a:?, b:?
    a = 10; //a:10, b:?
    b = 4; //a:10, b:4
    a = b; //a:4, b:4
    b = 7; //a:4, b:7
    cout << "a:";
    cout << a;
    cout << " b:";
    cout << b;
    return 0;
}
output
a:4 b:7
```

This code will give us as result that the value contained in a is 4 and the one contained in b is 7. Notice how a was not affected by the final modification of b, even though we declared `a = b` earlier (that is because of the *right-to-left rule*).

A property that C++ has over other programming languages is that the assignment operation can be used as the rvalue (or part of an rvalue) for another assignment operation.

For example:

```
a = 2 + (b = 5);
```

is equivalent to:

```
b = 5;
```

```
a = 2 + b;
```

that means: first assign 5 to variable b and then assign to a the value 2 plus the result of the previous assignment of b (*i.e.* 5), leaving a with a final value of 7.

The following expression is also valid in C++:

```
a = b = c = 5;
```

It assigns 5 to all the three variables: a, b and c.

Arithmetic operators (+, -, *, /, %)

The five arithmetical operations supported by the C++ language are:

```
+ addition
- subtraction
* multiplication
/division
% modulo
```

Operations of addition, subtraction, multiplication and division literally correspond with their respective mathematical operators. The only one that you might not be so

used to see may be *modulo*; whose operator is the percentage sign (%). Modulo is the operation that gives the remainder of a division of two values.

For example, if we write:

```
a = 11 % 3;
```

the variable *a* will contain the value 2, since 2 is the remainder from dividing 11 between 3.

Compound assignment (**+=**, **-=**, ***=**, **/=**, **%=**, **>>=**, **<<=**, **&=**, **^=**, **|=**)

When we want to modify the value of a variable by performing an operation on the value currently stored in that variable we can use compound assignment operators:

Expression	is equivalent to
value += increase;	value = value + increase;
a -= 5;	a = a-5;
a/= b;	a = a/b;
price *= units + 1;	price = price * (units + 1);

and the same for all other operators.

For example:

```
//compound assignment operators
#include <iostream>
using namespace std;
int main ()
{
    int a, b=3;
    a = b;
    a+=2;//equivalent to a=a+2
    cout << a;
    return 0;
} 5
```

Increase and decrease (**++**, **-**)

Shortening even more some expressions, the increase operator (**++**) and the decrease operator (**--**) increase or reduce by one the value stored in a variable. They are equivalent to **+=1** and to **-=1**, respectively.

Thus:

```
c++;
c+=1;
c=c+1;
```

are all equivalent in its functionality: the three of them increase by one the value of *c*. In the early C compilers, the three previous expressions probably produced different executable code depending on which one was used. Nowadays, this type of code optimization is generally done automatically by the compiler, thus the three expressions should produce exactly the same executable code. A characteristic of this operator is that it can be used both as a prefix and as a suffix. That means that it can be written either before the variable identifier (**++a**) or after it (**a++**). Although in simple expressions like **a++** or **++a** both have exactly the same meaning, in other expressions in which the result of the increase or decrease operation is evaluated as a value in an outer expression

they may have an important difference in their meaning: In the case that the increase operator is used as a prefix (++a) the value is increased before the result of the expression is evaluated and therefore the increased value is considered in the outer expression; in case that it is used as a suffix (a++) the value stored in a is increased after being evaluated and therefore the value stored before the increase operation is evaluated in the outer expression. Notice the difference:

Example 1	Example 2
B=3;	B=3;
A=++B;	A=B++;
//A contains 4, B contains 4	//A contains 3, B contains 4s

In Example 1, B is increased before its value is copied to A. While in Example 2, the value of B is copied to A and then B is increased.

Relational and equality operators (==, !=, >, <, >=, <=)

In order to evaluate a comparison between two expressions we can use the relational and equality operators. The result of a relational operation is a Boolean value that can only be true or false, according to its Boolean result.

We may want to compare two expressions, for example, to know if they are equal or if one is greater than the other is. Here is a list of the relational and equality operators that can be used in C++:

```

==   Equal to
!=   Not equal to
>    Greater than
<    Less than
>=   Greater than or equal to
<=   Less than or equal to

```

Here there are some examples:

```

(7 == 5)//evaluates to false.
(5 > 4)//evaluates to true.
(3 != 2)//evaluates to true.
(6 >= 6)//evaluates to true.
(5 < 5)//evaluates to false.

```

Of course, instead of using only numeric constants, we can use any valid expression, including variables. Suppose that a=2, b=3 and c=6,

```

(a == 5)//evaluates to false since a is not equal to 5.
(a*b >= c)//evaluates to true since (2*3 >= 6) is true.
(b+4 > a*c)//evaluates to false since (3+4 > 2*6) is false.
((b=2) == a)//evaluates to true.

```

Be careful! The operator = (one equal sign) is not the same as the operator == (two equal signs), the first one is an assignment operator (assigns the value at its right to the variable at its left) and the other one (==) is the equality operator that compares whether both expressions in the two sides of it are equal to each other. Thus, in the last expression ((b=2) == a), we first assigned the value 2 to b and then we compared it to a, that also stores the value 2, so the result of the operation is true.

Logical operators (!, &&, ||)

The Operator `!` is the C++ operator to perform the Boolean operation NOT, it has only one operand, located at its right, and the only thing that it does is to inverse the value of it, producing false if its operand is true and true if its operand is false. Basically, it returns the opposite Boolean value of evaluating its operand.

For example:

```
!(5 == 5)//evaluates to false because the expression at its right (5 == 5) is true.
```

```
!(6 <= 4)//evaluates to true because (6 <= 4) would be false.
```

```
!true//evaluates to false
```

```
!false//evaluates to true.
```

The logical operators `&&` and `||` are used when evaluating two expressions to obtain a single relational result. The operator `&&` corresponds with Boolean logical operation AND. This operation results true if both its two operands are true, and false otherwise. The following panel shows the result of operator `&&` evaluating the expression `a && b`:

&& OPERATOR

a b a && b

true true true

true false false

false true false

false false false

The operator `||` corresponds with Boolean logical operation OR. This operation results true if either one of its two operands is true, thus being false only when both operands are false themselves.

Here are the possible results of `a || b`:

|| OPERATOR

a b a || b

true true true

true false true

false true true

false false false

For example:

```
((5 == 5) && (3 > 6))//evaluates to false (true && false).
```

```
((5 == 5) || (3 > 6))//evaluates to true (true || false).
```

Conditional operator (?)

The conditional operator evaluates an expression returning a value if that expression is true and a different one if the expression is evaluated as false.

Its format is:

condition ? result1: result2

If condition is true the expression will return result1, if it is not it will return result2.

```
7==5 ? 4: 3//returns 3, since 7 is not equal to 5.
```

```
7==5+2 ? 4: 3//returns 4, since 7 is equal to 5+2.
```

```
5>3 ? a: b//returns the value of a, since 5 is greater than 3.
```

```
a>b ? a: b//returns whichever is greater, a or b.
```

```
//conditional operator
```

```
#include <iostream>
```

```
using namespace std;
int main ()
{
    int a,b,c;
    a=2;
    b=7;
    c = (a>b) ? a: b;
    cout << c;
    return 0;
} 7
```

In this example a was 2 and b was 7, so the expression being evaluated (a>b) was not true, thus the first value specified after the question mark was discarded in favour of the second value (the one after the colon) which was b, with a value of 7.

Comma operator (,) The comma operator (,) is used to separate two or more expressions that are included where only one expression is expected. When the set of expressions has to be evaluated for a value, only the rightmost expression is considered.

For example, the following code:

```
a = (b=3, b+2);
```

Would first assign the value 3 to b, and then assign b+2 to variable a. So, at the end, variable a would contain the value 5 while variable b would contain value 3.

Bitwise Operators (&, |, ^, ~, <<, >>)

Bitwise operators modify variables considering the bit patterns that represent the values they store.

Operator	Asm equivalent	Description
&	AND	Bitwise AND
	OR	Bitwise Inclusive OR
^	XOR	Bitwise Exclusive OR
~	NOT	Unary complement (bit inversion)
<<	SHL	Shift Left
>>	SHR	Shift Right

Explicit type casting operator: Type casting operators allow you to convert a datum of a given type to another. There are several ways to do this in C++. The simplest one, which has been inherited from the C language, is to precede the expression to be converted by the new type enclosed between parentheses ():

```
int i;
float f = 3.14;
i = (int) f;
```

The previous code converts the float number 3.14 to an integer value (3), the remainder is lost. Here, the typecasting operator was (int). Another way to do the same thing in C++ is using the functional notation: preceding the expression to be converted by the type and enclosing the expression between parentheses:

```
i = int (f);
```

Both ways of type casting are valid in C++.

```
sizeof()
```

This operator accepts one parameter, which can be either a type or a variable itself and returns the size in bytes of that type or object:

```
a = sizeof (char);
```

This will assign the value 1 to a because char is a one-byte long type. The value returned by sizeof is a constant, so it is always determined before programme execution.

Other operators: Later in these tutorials, we will see a few more operators, like the ones referring to pointers or the specifics for object-oriented programming. Each one is treated in its respective section.

Precedence of operators: When writing complex expressions with several operands, we may have some doubts about which operand is evaluated first and which later. For example, in this expression:

```
a = 5 + 7 % 2
```

we may doubt if it really means:

```
a = 5 + (7 % 2) //with a result of 6, or
```

```
a = (5 + 7) % 2 //with a result of 0
```

The correct answer is the first of the two expressions, with a result of 6. There is an established order with the priority of each operator, and not only the arithmetic ones (those whose preference come from mathematics) but for all the operators which can appear in C++. From greatest to lowest priority, the priority order is as follows:

Level	Operator	Description	Grouping
1	::	scope	Left-to-right
2	() [] .-> ++—dynamic_ cast static_cast reinterpret _cast const_cast typeid	postfix	Left-to-right
3	++—~ !sizeof new delete * &	unary (prefix) indirection and reference (pointers)	Right-to-left Right-to-left
	+ -	unary sign operator	
4	(type)	type casting	Right-to-left
5	.* ->*	pointer-to-member	Left-to-right
6	*/%	multiplicative	Left-to-right
7	+ -	additive	Left-to-right
8	<< >>	shift	Left-to-right
9	< > <= >=	relational	Left-to-right
10	== !=	equality	Left-to-right
11	&	bitwise AND	Left-to-right
12	^	bitwise XOR	Left-to-right
13		bitwise OR	Left-to-right
14	&&	logical AND	Left-to-right
15		logical OR	Left-to-right
16	?:	conditional	Right-to-left
17	= *./= %+= += -= >>= <<= &= ^= =	assignment	Right-to-left
18	,	comma	Left-to-right

Grouping defines the precedence order in which operators are evaluated in the case that there are several operators of the same level in an expression.

All these precedence levels for operators can be manipulated or become more legible by removing possible ambiguities using parentheses signs (and), as in this example:

```
a = 5 + 7 % 2;
```

might be written either as:

```
a = 5 + (7 % 2);
```

or

```
a = (5 + 7) % 2;
```

depending on the operation that we want to perform. So if you want to write complicated expressions and you are not completely sure of the precedence levels, always include parentheses. It will also become a code easier to read.

Basic Input/Output: Until now, the example programmes of previous sections provided very little interaction with the user, if any at all. Using the standard input and output library, we will be able to interact with the user by printing messages on the screen and getting the user's input from the keyboard.

C++ uses a convenient abstraction called *streams* to perform input and output operations in sequential media such as the screen or the keyboard.

A stream is an object where a programme can either insert or extract characters to/from it. We do not really need to care about many specifications about the physical media associated with the stream—we only need to know it will accept or provide characters sequentially.

The standard C++ library includes the header file `iostream`, where the standard input and output stream objects are declared.

Standard Output (cout): By default, the standard output of a programme is the screen, and the C++ stream object defined to access it is `cout`. `cout` is used in conjunction with the insertion operator, which is written as `<<` (two “less than” signs).

```
cout << "Output sentence";//prints Output sentence on screen
cout << 120;//prints number 120 on screen
cout << x;//prints the content of x on screen
```

The `<<` operator inserts the data that follows it into the stream preceding it. In the examples above it inserted the constant string `Output sentence`, the numerical constant `120` and variable `x` into the standard output stream `cout`. Notice that the sentence in the first instruction is enclosed between double quotes (“”) because it is a constant string of characters. Whenever we want to use constant strings of characters we must enclose them between double quotes (“”) so that they can be clearly distinguished from variable names.

For example, these two sentences have very different results:

```
cout << "Hello";//prints Hello
cout << Hello;//prints the content of Hello variable
```

The insertion operator (`<<`) may be used more than once in a single statement:

```
cout << "Hello, " << "I am " << "a C++ statement";
```

This last statement would print the message `Hello, I am a C++ statement` on the screen. The utility of repeating the insertion operator (`<<`) is demonstrated when we

want to print out a combination of variables and constants or more than one variable:

```
cout << "Hello, I am " << age << " years old and my zipcode is " <<
zipcode;
```

If we assume the age variable to contain the value 24 and the zipcode variable to contain 90064 the output of the previous statement would be:

```
Hello, I am 24 years old and my zipcode is 90064
```

It is important to notice that cout does not add a line break after its output unless we explicitly indicate it, therefore, the following statements:

```
cout << "This is a sentence.";
cout << "This is another sentence.";
```

will be shown on the screen one following the other without any line break between them: This is a sentence. This is another sentence. even though we had written them in two different insertions into cout. In order to perform a line break on the output we must explicitly insert a new-line character into cout. In C++ a new-line character can be specified as \n (backslash, n):

```
cout << "First sentence.\n ";
cout << "Second sentence.\nThird sentence.";
```

This produces the following output:

```
First sentence.
Second sentence.
Third sentence.
```

Additionally, to add a new-line, you may also use the endl manipulator.

For example:

```
cout << "First sentence." << endl;
cout << "Second sentence." << endl;
```

would print out:

```
First sentence.
Second sentence.
```

The endl manipulator produces a newline character, exactly as the insertion of ‘\n’ does, but it also has an additional behaviour when it is used with buffered streams: the buffer is flushed. Anyway, cout will be an unbuffered stream in most cases, so you can generally use both the \n escape character and the endl manipulator in order to specify a new line without any difference in its behaviour.

Standard Input (cin): The standard input device is usually the keyboard. Handling the standard input in C++ is done by applying the overloaded operator of extraction (>>) on the cin stream. The operator must be followed by the variable that will store the data that is going to be extracted from the stream.

For example:

```
int age;
cin >> age;
```

The first statement declares a variable of type int called age, and the second one waits for an input from cin (the keyboard) in order to store it in this integer variable. cin can only process the input from the keyboard once the RETURN key has been pressed. Therefore, even if you request a single character, the extraction from cin will not process the input until the user presses RETURN after the character has been introduced.

You must always consider the type of the variable that you are using as a container with cin extractions. If you request an integer you will get an integer, if you request a character you will get a character and if you request a string of characters you will get a string of characters.

```
//i/o example
#include <iostream>
using namespace std;
int main ()
{
    int i;
    cout << "Please enter an integer value: ";
    cin >> i;
    cout << "The value you entered is " << i;
    cout << " and its double is " << i*2 << ".\n";
    return 0;
} Please enter an integer value: 702
The value you entered is 702 and its double is 1404.
```

The user of a programme may be one of the factors that generate errors even in the simplest programmes that use cin (like the one we have just seen). Since if you request an integer value and the user introduces a name (which generally is a string of characters), the result may cause your programme to misoperate since it is not what we were expecting from the user. So when you use the data input provided by cin extractions you will have to trust that the user of your programme will be cooperative and that he/she will not introduce his/her name or something similar when an integer value is requested. A little ahead, when we see the stringstream class we will see a possible solution for the errors that can be caused by this type of user input.

You can also use cin to request more than one datum input from the user:

```
cin >> a >> b;
is equivalent to:
cin >> a;
cin >> b;
```

In both cases the user must give two data, one for variable a and another one for variable b that may be separated by any valid blank separator: a space, a tab character or a newline. cin and strings We can use cin to get strings with the extraction operator (>>) as we do with fundamental data type variables:

```
cin >> mystring;
```

However, as it has been said, cin extraction stops reading as soon as it finds any blank space character, so in this case we will be able to get just one word for each extraction. This behaviour may or may not be what we want; for example if we want to get a sentence from the user, this extraction operation would not be useful. In order to get entire lines, we can use the function getline, which is the more recommendable way to get user input with cin:

```
//cin with strings
#include <iostream>
```

```

#include <string>
using namespace std;
int main ()
{
    string mystr;
    cout << "What's your name? ";
    getline (cin, mystr);
    cout << "Hello " << mystr << ".\n";
    cout << "What is your favourite team? ";
    getline (cin, mystr);
    cout << "I like " << mystr << " too!\n";
    return 0;
}

```

What's your name? Juan Soulié
Hello Juan Soulié.
What is your favourite team? The Isotopes
I like The Isotopes too!

Notice how in both calls to `getline` we used the same string identifier (`mystr`). What the programme does in the second call is simply to replace the previous content by the new one that is introduced. `stringstream`

The standard header file `<sstream>` defines a class called `stringstream` that allows a string-based object to be treated as a stream. This way we can perform extraction or insertion operations from/to strings, which is especially useful to convert strings to numerical values and vice versa. For example, if we want to extract an integer from a string we can write:

```

string mystr ("1204");
int myint;
stringstream(mystr) >> myint;

```

This declares a string object with a value of "1204", and an `int` object. Then we use `stringstream`'s constructor to construct an object of this type from the string object. Because we can use `stringstream` objects as if they were streams, we can extract an integer from it as we would have done on `cin` by applying the extractor operator (`>>`) on it followed by a variable of type `int`.

After this piece of code, the variable `myint` will contain the numerical value 1204.

```

//stringstreams
#include <iostream>
#include <string>
#include <sstream>
using namespace std;
int main ()
{
    string mystr;
    float price=0;
    int quantity=0;
    cout << "Enter price: ";
    getline (cin,mystr);
    stringstream(mystr) >> price;
    cout << "Enter quantity: ";

```



```
getline (cin,mystr);
stringstream(mystr) >> quantity;
cout << "Total price: " << price*quantity << endl;
return 0;
} Enter price: 22.25
Enter quantity: 7
Total price: 155.75
```

In this example, we acquire numeric values from the standard input indirectly. Instead of extracting numeric values directly from the standard input, we get lines from the standard input (cin) into a string object (mystr), and then we extract the integer values from this string into a variable of type int (myint).

Using this method, instead of direct extractions of integer values, we have more control over what happens with the input of numeric values from the user, since we are separating the process of obtaining input from the user (we now simply ask for lines) with the interpretation of that input. Therefore, this method is usually preferred to get numerical values from the user in all programmes that are intensive in user input.

Control Structures: A programme is usually not limited to a linear sequence of instructions. During its process it may bifurcate, repeat code or take decisions.

For that purpose, C++ provides control structures that serve to specify what has to be done by our programme, when and under which circumstances.

With the introduction of control structures we are going to have to introduce a new concept: the *compound-statement* or *block*. A block is a group of statements which are separated by semicolons (;) like all C++ statements, but grouped together in a block enclosed in braces: {}:

```
{statement1; statement2; statement3;}
```

Most of the control structures that we will see in this section require a generic statement as part of its syntax. A statement can be either a simple statement (a simple instruction ending with a semicolon) or a compound statement (several instructions grouped in a block), like the one just described. In the case that we want the statement to be a simple statement, we do not need to enclose it in braces ({}).

But in the case that we want the statement to be a compound statement it must be enclosed between braces ({}), forming a block.

Conditional structure: if and else: The if keyword is used to execute a statement or block only if a condition is fulfilled. Its form is: if (condition) statement

Where condition is the expression that is being evaluated. If this condition is true, statement is executed. If it is false, statement is ignored (not executed) and the programme continues right after this conditional structure.

For example, the following code fragment prints x is 100 only if the value stored in the x variable is indeed 100:

```
if (x == 100)
    cout << "x is 100";
```

If we want more than a single statement to be executed in case that the condition is true we can specify a block using braces {}:

```

if (x == 100)
{
    cout << "x is ";
    cout << x;
}

```

We can additionally specify what we want to happen if the condition is not fulfilled by using the keyword `else`. Its form used in conjunction with `if` is: `if (condition) statement1 else statement2`

For example:

```

if (x == 100)
    cout << "x is 100";
else

```

`cout << "x is not 100";`
prints on the screen `x is 100` if indeed `x` has a value of 100, but if it has not -and only if not- it prints out `x is not 100`.

The `if + else` structures can be concatenated with the intention of verifying a range of values. The following example shows its use telling if the value currently stored in `x` is positive, negative or none of them (*i.e.* zero):

```

if (x > 0)
    cout << "x is positive";
else if (x < 0)
    cout << "x is negative";
else
    cout << "x is 0";

```

Remember that in case that we want more than a single statement to be executed, we must group them in a block by enclosing them in braces `{ }`.

Iteration structures (loops): Loops have as purpose to repeat a statement a certain number of times or while a condition is fulfilled. The while loop

Its format is: `while (expression) statement` and its functionality is simply to repeat statement while the condition set in expression is true.

For example, we are going to make a programme to countdown using a while-loop:

```

//custom countdown using while
#include <iostream>
using namespace std;
int main ()
{
    int n;
    cout << "Enter the starting number > ";
    cin >> n;
    while (n>0) {
        cout << n << ", ";
        -n;
    }
    cout << "FIRE!\n";
    return 0;
}

```

Output

Enter the starting number > 8
8, 7, 6, 5, 4, 3, 2, 1, FIRE!

When the programme starts the user is prompted to insert a starting number for the countdown. Then the while loop begins, if the value entered by the user fulfills the condition $n > 0$ (that n is greater than zero) the block that follows the condition will be executed and repeated while the condition ($n > 0$) remains being true. The whole process of the previous programme can be interpreted according to the following script (beginning in main):

- User assigns a value to n
- The while condition is checked ($n > 0$). At this point there are two possibilities:
 condition is true: statement is executed (to step 3)
 condition is false: ignore statement and continue after it (to step 5)
- *Execute statement:* `cout << n << " "; --n;` (prints the value of n on the screen and decreases n by 1).
- End of block. Return automatically to step 2.
- *Continue the programme right after the block:* Print FIRE! and end programme.

When creating a while-loop, we must always consider that it has to end at some point, therefore we must provide within the block some method to force the condition to become false at some point, otherwise the loop will continue looping forever. In this case we have included `--n;` that decreases the value of the variable that is being evaluated in the condition (n) by one—this will eventually make the condition ($n > 0$) to become false after a certain number of loop iterations: to be more specific, when n becomes 0, that is where our while-loop and our countdown end.

Of course this is such a simple action for our computer that the whole countdown is performed instantly without any practical delay between numbers. The do-while loop.

Its format is: `do statement while (condition);`

Its functionality is exactly the same as the while loop, except that condition in the do-while loop is evaluated after the execution of statement instead of before, granting at least one execution of statement even if condition is never fulfilled. For example, the following example programme echoes any number you enter until you enter 0.

```
//number echoer
#include <iostream>
using namespace std;
int main ()
{
    unsigned long n;
    do {
        cout << "Enter number (0 to end): ";
        cin >> n;
        cout << "You entered: " << n << "\n";
    } while (n != 0);
    return 0;
} Enter number (0 to end): 12345
```

```

You entered: 12345
output
Enter number (0 to end): 160277
You entered: 160277
Enter number (0 to end): 0
You entered: 0

```

The do-while loop is usually used when the condition that has to determine the end of the loop is determined within the loop statement itself, like in the previous case, where the user input within the block is what is used to determine if the loop has to end. In fact if you never enter the value 0 in the previous example you can be prompted for more numbers forever. The for loop.

Its format is: for (initialization; condition; increase) statement; and its main function is to repeat statement while condition remains true, like the while loop. But in addition, the for loop provides specific locations to contain an initialization statement and an increase statement. So this loop is specially designed to perform a repetitive action with a counter which is initialized and increased on each iteration.

It works in the following way:

1. *Initialization is executed:* Generally it is an initial value setting for a counter variable. This is executed only once.
2. *Condition is checked:* If it is true the loop continues, otherwise the loop ends and statement is skipped (not executed).
3. *Statement is executed:* As usual, it can be either a single statement or a block enclosed in braces {}.
4. Finally, whatever is specified in the increase field is executed and the loop gets back to step 2.

Here is an example of countdown using a for loop:

```

//countdown using a for loop
#include <iostream>
using namespace std;
int main ()
{
    for (int n=10; n>0; n-) {
        cout << n << ", ";
    }
    cout << "FIRE!\n";
    return 0;
}
output
10, 9, 8, 7, 6, 5, 4, 3, 2, 1, FIRE!

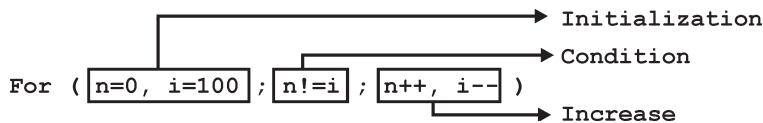
```

The initialization and increase fields are optional. They can remain empty, but in all cases the semicolon signs between them must be written. For example we could write: for (;n<10;) if we wanted to specify no initialization and no increase; or for (;n<10;n++) if we wanted to include an increase field but no initialization (maybe because the variable was already initialized before). Optionally, using the comma operator (,) we can specify more than one expression in any of the fields included in a for loop, like in initialization,

for example. The comma operator (,) is an expression separator, it serves to separate more than one expression where only one is generally expected. For example, suppose that we wanted to initialize more than one variable in our loop:

```
for (n=0, i=100; n!=i; n++, i-)
{
    //whatever here...
}
```

This loop will execute for 50 times if neither n or i are modified within the loop:



n starts with a value of 0, and i with 100, the condition is $n \neq i$ (that n is not equal to i). Because n is increased by one and i decreased by one, the loop's condition will become false after the 50th loop, when both n and i will be equal to 50. Jump statements.

The break statement: Using break we can leave a loop even if the condition for its end is not fulfilled. It can be used to end an infinite loop, or to force it to end before its natural end. For example, we are going to stop the count down before its natural end (maybe because of an engine check failure?):

```
//break loop example
#include <iostream>
using namespace std;
int main ()
{
    int n;
    for (n=10; n>0; n-)
    {
        cout << n << ", ";
        if (n==3)
        {
            cout << "countdown aborted!";
            break;
        }
    }
    return 0;
}
output
10, 9, 8, 7, 6, 5, 4, 3, countdown aborted!
```

The continue statement: The continue statement causes the programme to skip the rest of the loop in the current iteration as if the end of the statement block had been reached, causing it to jump to the start of the following iteration. For example, we are going to skip the number 5 in our countdown:

```
//continue loop example
#include <iostream>
using namespace std;
int main ()
{
```

```

    for (int n=10; n>0; n--) {
        if (n==5) continue;
        cout << n << ", ";
    }
    cout << "FIRE!\n";
    return 0;
}
output
10, 9, 8, 7, 6, 4, 3, 2, 1, FIRE!

```

The goto statement: goto allows to make an absolute jump to another point in the programme. You should use this feature with caution since its execution causes an unconditional jump ignoring any type of nesting limitations. The destination point is identified by a label, which is then used as an argument for the goto statement. A label is made of a valid identifier followed by a colon (:). Generally speaking, this instruction has no concrete use in structured or object oriented programming aside from those that low-level programming fans may find for it.

For example, here is our countdown loop using goto:

```

//goto loop example
#include <iostream>
using namespace std;
int main ()
{
    int n=10;
loop:
    cout << n << ", ";
    n--;
    if (n>0) goto loop;
    cout << "FIRE!\n";
    return 0;
}
output
10, 9, 8, 7, 6, 5, 4, 3, 2, 1, FIRE!

```

The exit function: exit is a function defined in the cstdlib library. The purpose of exit is to terminate the current programme with a specific exit code. Its prototype is:

```
void exit (int exitcode);
```

The exitcode is used by some operating systems and may be used by calling programmes. By convention, an exit code of 0 means that the programme finished normally and any other value means that some error or unexpected results happened.

The selective structure: switch: The syntax of the switch statement is a bit peculiar. Its objective is to check several possible constant values for an expression. Something similar to what we did at the beginning of this section with the concatenation of several if and else if instructions. Its form is the following:

```

switch (expression)
{
    case constant1:
        group of statements 1;
        break;

```

```

    case constant2:
    group of statements 2;
    break;
    .
    .
    .
    default:
    default group of statements
}

```

It works in the following way: switch evaluates expression and checks if it is equivalent to constant1, if it is, it executes group of statements 1 until it finds the break statement. When it finds this break statement the programme jumps to the end of the switch selective structure. If expression was not equal to constant1 it will be checked against constant2. If it is equal to this, it will execute group of statements 2 until a break keyword is found, and then will jump to the end of the switch selective structure. Finally, if the value of expression did not match any of the previously specified constants (you can include as many case labels as values you want to check), the programme will execute the statements included after the default: label, if it exists (since it is optional).

Both of the following code fragments have the same behaviour: switch example if-else equivalent

```

switch (x) {
    case 1:
        cout << "x is 1";
        break;
    case 2:
        cout << "x is 2";
        break;
    default:
        cout << "value of x unknown";
} if (x == 1) {
    cout << "x is 1";
}
else if (x == 2) {
    cout << "x is 2";
}
else {
    cout << "value of x unknown";
}

```

The switch statement is a bit peculiar within the C++ language because it uses labels instead of blocks. This forces us to put break statements after the group of statements that we want to be executed for a specific condition. Otherwise the remainder statements -including those corresponding to other labels- will also be executed until the end of the switch selective block or a break statement is reached.

For example, if we did not include a break statement after the first group for case one, the programme will not automatically jump to the end of the switch selective block and it would continue executing the rest of statements until it reaches either a

break instruction or the end of the switch selective block. This makes unnecessary to include braces { } surrounding the statements for each of the cases, and it can also be useful to execute the same block of instructions for different possible values for the expression being evaluated.

For example:

```
switch (x) {
    case 1:
    case 2:
    case 3:
        cout << "x is 1, 2 or 3";
    break;
    default:
        cout << "x is not 1, 2 nor 3";
}
```

Notice that switch can only be used to compare an expression against constants. Therefore we cannot put variables as labels (for example case n: where n is a variable) or ranges (case (1..3):) because they are not valid C++ constants. If you need to check ranges or values that are not constants, use a concatenation of if and else if statements.

Functions (I): Using functions we can structure our programmes in a more modular way, accessing all the potential that structured programming can offer to us in C++. A function is a group of statements that is executed when it is called from some point of the programme. The following is its format:

```
type name (parameter1, parameter2,...) {statements}
```

where:

- Type is the data type specifier of the data returned by the function.
- Name is the identifier by which it will be possible to call the function.
- Parameters (as many as needed): Each parameter consists of a data type specifier followed by an identifier, like any regular variable declaration (for example: int x) and which acts within the function as a regular local variable. They allow to pass arguments to the function when it is called. The different parameters are separated by commas.
- Statements is the function's body. It is a block of statements surrounded by braces { }.

Here you have the first function example:

```
//function example
#include <iostream>
using namespace std;
int addition (int a, int b)
{
    int r;
    r=a+b;
    return (r);
}
int main ()
{
```



```

int z;
z = addition (5,3);
cout << "The result is " << z;
return 0;
} The result is 8

```

In order to examine this code, first of all remember something said at the beginning of this tutorial: a C++ programme always begins its execution by the main function. So we will begin there.

We can see how the main function begins by declaring the variable `z` of type `int`. Right after that, we see a call to a function called `addition`. Paying attention we will be able to see the similarity between the structure of the call to the function and the declaration of the function itself some code lines above:

```

Int addition (int a, int b)
               ↑       ↑
z = addition (5,      3);

```

The parameters and arguments have a clear correspondence. Within the main function we called to `addition` passing two values: 5 and 3, that correspond to the `int a` and `int b` parameters declared for function `addition`. At the point at which the function is called from within `main`, the control is lost by `main` and passed to function `addition`. The value of both arguments passed in the call (5 and 3) are copied to the local variables `int a` and `int b` within the function.

Function `addition` declares another local variable (`int r`), and by means of the expression `r=a+b`, it assigns to `r` the result of `a` plus `b`. Because the actual parameters passed for `a` and `b` are 5 and 3 respectively, the result is 8.

The following line of code:

```
return (r);
```

finalizes function `addition`, and returns the control back to the function that called it in the first place (in this case, `main`). At this moment the programme follows its regular course from the same point at which it was interrupted by the call to `addition`. But additionally, because the `return` statement in function `addition` specified a value: the content of variable `r` (`return (r);`), which at that moment had a value of 8. This value becomes the value of evaluating the function call.

```

Int addition (int a, int b)
    ↓ 8
z = addition (5,      3);

```

So being the value returned by a function the value given to the function call itself when it is evaluated, the variable `z` will be set to the value returned by `addition (5, 3)`, that is 8. To explain it another way, you can imagine that the call to a function (`addition (5,3)`) is literally replaced by the value it returns (8).

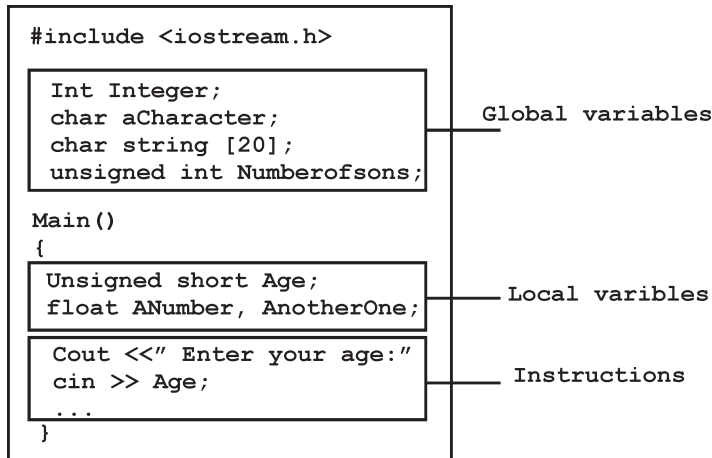
The following line of code in main is:

```
cout << "The result is " << z;
```

That, as you may already expect, produces the printing of the result on the screen.

Scope of variables: The scope of variables declared within a function or any other

inner block is only their own function or their own block and cannot be used outside of them. For example, in the previous example it would have been impossible to use the variables a, b or r directly in function main since they were variables local to function addition. Also, it would have been impossible to use the variable z directly within function addition, since this was a variable local to the function main.



Therefore, the scope of local variables is limited to the same block level in which they are declared. Nevertheless, we also have the possibility to declare global variables; These are visible from any point of the code, inside and outside all functions. In order to declare global variables you simply have to declare the variable outside any function or block; that means, directly in the body of the programme.

And here is another example about functions:

```

//function example
#include <iostream>
using namespace std;
int subtraction (int a, int b)
{
    int r;
    r=a-b;
    return (r);
}
int main ()
{
    int x=5, y=3, z;
    z = subtraction (7,2);
    cout << "The first result is " << z << '\n';
    cout << "The second result is " << subtraction (7,2) << '\n';
    cout << "The third result is " << subtraction (x,y) << '\n';
    z= 4 + subtraction (x,y);
    cout << "The fourth result is " << z << '\n';
    return 0;
}

```

output

```

The first result is 5
The second result is 5
The third result is 2
The fourth result is 6

```

In this case we have created a function called subtraction. The only thing that this function does is to subtract both passed parameters and to return the result.

Nevertheless, if we examine function main we will see that we have made several calls to function subtraction. We have used some different calling methods so that you see other ways or moments when a function can be called.

In order to fully understand these examples you must consider once again that a call to a function could be replaced by the value that the function call itself is going to return. For example, the first case (that you should already know because it is the same pattern that we have used in previous examples):

```

z = subtraction (7,2);
cout << "The first result is " << z;

```

If we replace the function call by the value it returns (i.e., 5), we would have:

```

z = 5;
cout << "The first result is " << z;

```

As well as:

```

cout << "The second result is " << subtraction (7,2);

```

has the same result as the previous call, but in this case we made the call to subtraction directly as an insertion parameter for cout. Simply consider that the result is the same as if we had written:

```

cout << "The second result is " << 5;

```

since 5 is the value returned by subtraction (7,2).

In the case of:

```

cout << "The third result is " << subtraction (x,y);

```

The only new thing that we introduced is that the parameters of subtraction are variables instead of constants. That is perfectly valid. In this case the values passed to function subtraction are the values of x and y, that are 5 and 3 respectively, giving 2 as result.

The fourth case is more of the same. Simply note that instead of:

```

z = 4 + subtraction (x,y);

```

we could have written:

```

z = subtraction (x,y) + 4;

```

with exactly the same result. I have switched places so you can see that the semicolon sign (;) goes at the end of the whole statement. It does not necessarily have to go right after the function call. The explanation might be once again that you imagine that a function can be replaced by its returned value:

```

z = 4 + 2;
z = 2 + 4;

```

Functions with no type. The use of void.

If you remember the syntax of a function declaration:

type name (argument1, argument2...) statement you will see that the declaration begins with a type, that is the type of the function itself (i.e., the type of the datum that will be

returned by the function with the return statement). But what if we want to return no value?

Imagine that we want to make a function just to show a message on the screen. We do not need it to return any value. In this case we should use the void type specifier for the function. This is a special specifier that indicates absence of type.

```
//void function example
#include <iostream>
using namespace std;
void printmessage ()
{
    cout << "I'm a function!";
}
int main ()
{
    printmessage ();
    return 0;
}
output
I'm a function!
```

Void can also be used in the function's parameter list to explicitly specify that we want the function to take no actual parameters when it is called. For example, function printmessage could have been declared as:

```
void printmessage (void)
{
    cout << "I'm a function!";
}
```

Although it is optional to specify void in the parameter list. In C++, a parameter list can simply be left blank if we want a function with no parameters.

What you must always remember is that the format for calling a function includes specifying its name and enclosing its parameters between parentheses. The non-existence of parameters does not exempt us from the obligation to write the parentheses. For that reason the call to printmessage is:

```
printmessage ();
```

The parentheses clearly indicate that this is a call to a function and not the name of a variable or some other C++ statement.

The following call would have been incorrect:

```
printmessage;
```

Functions (II): Arguments passed by value and by reference.

Until now, in all the functions we have seen, the arguments passed to the functions have been passed *by value*. This means that when calling a function with parameters, what we have passed to the function were copies of their values but never the variables themselves. For example, suppose that we called our first function addition using the following code:

```
int x=5, y=3, z;
z = addition (x, y);
```

What we did in this case was to call to function addition passing the values of x and y, *i.e.* 5 and 3 respectively, but not the variables x and y themselves.

```

    Int addition (int a, int b)
                ↑      ↑
    z = addition (5,      3);
  
```

This way, when the function addition is called, the value of its local variables a and b become 5 and 3 respectively, but any modification to either a or b within the function addition will not have any effect in the values of x and y outside it, because variables x and y were not themselves passed to the function, but only copies of their values at the moment the function was called.

But there might be some cases where you need to manipulate from inside a function the value of an external variable. For that purpose we can use arguments passed by reference, as in the function duplicate of the following example:

```

//passing parameters by reference
#include <iostream>
using namespace std;
void duplicate (int& a, int& b, int& c)
{
    a*=2;
    b*=2;
    c*=2;
}
int main ()
{
    int x=1, y=3, z=7;
    duplicate (x, y, z);
    cout << "x=" << x << ", y=" << y << ", z=" << z;
    return 0;
}
output
x=2, y=6, z=14
  
```

The first thing that should call your attention is that in the declaration of duplicate the type of each parameter was followed by an ampersand sign (&). This ampersand is what specifies that their corresponding arguments are to be passed *by reference* instead of *by value*. When a variable is passed by reference we are not passing a copy of its value, but we are somehow passing the variable itself to the function and any modification that we do to the local variables will have an effect in their counterpart variables passed as arguments in the call to the function.

```

Void duplicate (int& a, int& b, int& c)
                ↑x      ↑      ↑z
                ↓      ↓      ↓
    duplicate ( x,      y,      z);
  
```

To explain it in another way, we associate a, b and c with the arguments passed on the function call (x, y and z) and any change that we do on a within the function will

affect the value of x outside it. Any change that we do on b will affect y, and the same with c and z.

That is why our program's output, that shows the values stored in x, y and z after the call to duplicate, shows the values of all the three variables of main doubled.

If when declaring the following function:

```
void duplicate (int& a, int& b, int& c)
```

we had declared it this way:

```
void duplicate (int a, int b, int c)
```

i.e., without the ampersand signs (&), we would have not passed the variables by reference, but a copy of their values instead, and therefore, the output on screen of our programme would have been the values of x, y and z without having been modified.

Passing by reference is also an effective way to allow a function to return more than one value. For example, here is a function that returns the previous and next numbers of the first parameter passed.

```
//more than one returning value
#include <iostream>
using namespace std;
void prevnext (int x, int& prev, int& next)
{
    prev = x-1;
    next = x+1;
}
int main ()
{
    int x=100, y, z;
    prevnext (x, y, z);
    cout << "Previous=" << y << ", Next=" << z;
    return 0;
}
output
Previous=99, Next=101
```

Default values in parameters.

When declaring a function we can specify a default value for each parameter. This value will be used if the corresponding argument is left blank when calling to the function. To do that, we simply have to use the assignment operator and a value for the arguments in the function declaration. If a value for that parameter is not passed when the function is called, the default value is used, but if a value is specified this default value is ignored and the passed value is used instead. For example:

```
//default values in functions
#include <iostream>
using namespace std;
int divide (int a, int b=2)
{
    int r;
    r=a/b;
    return (r);
}
```

```

}
int main ()
{
cout << divide (12);
cout << endl;
cout << divide (20,4);
return 0;
}
output
6
5

```

As we can see in the body of the programme there are two calls to function divide. In the first one:

divide (12)

we have only specified one argument, but the function divide allows up to two. So the function divide has assumed that the second parameter is 2 since that is what we have specified to happen if this parameter was not passed (notice the function declaration, which finishes with `int b=2`, not just `int b`). Therefore the result of this function call is 6 (12/2).

In the second call:

divide (20,4)

there are two parameters, so the default value for `b` (`int b=2`) is ignored and `b` takes the value passed as argument, that is 4, making the result returned equal to 5 (20/4).

Overloaded functions: In C++ two different functions can have the same name if their parameter types or number are different. That means that you can give the same name to more than one function if they have either a different number of parameters or different types in their parameters. For example:

```

//overloaded function
#include <iostream>
using namespace std;
int operate (int a, int b)
{
    return (a*b);
}
float operate (float a, float b)
{
    return (a/b);
}
int main ()
{
    int x=5,y=2;
    float n=5.0,m=2.0;
    cout << operate (x,y);
    cout << "\n";
    cout << operate (n,m);
    cout << "\n";
    return 0;
}

```

```

}
output
10 2.5

```

In this case we have defined two functions with the same name, `operate`, but one of them accepts two parameters of type `int` and the other one accepts them of type `float`. The compiler knows which one to call in each case by examining the types passed as arguments when the function is called. If it is called with two `ints` as its arguments it calls to the function that has two `int` parameters in its prototype and if it is called with two `floats` it will call to the one which has two `float` parameters in its prototype.

In the first call to `operate` the two arguments passed are of type `int`, therefore, the function with the first prototype is called; This function returns the result of multiplying both parameters. While the second call passes two arguments of type `float`, so the function with the second prototype is called. This one has a different behaviour: it divides one parameter by the other. So the behaviour of a call to `operate` depends on the type of the arguments passed because the function has been *overloaded*. Notice that a function cannot be overloaded only by its return type. At least one of its parameters must have a different type.

Inline Functions.

The `inline` specifier indicates the compiler that inline substitution is preferred to the usual function call mechanism for a specific function. This does not change the behaviour of a function itself, but is used to suggest to the compiler that the code generated by the function body is inserted at each point the function is called, instead of being inserted only once and perform a regular call to it, which generally involves some additional overhead in running time.

The format for its declaration is:

```
inline type name (arguments...) {instructions...}
```

and the call is just like the call to any other function. You do not have to include the `inline` keyword when calling the function, only in its declaration.

Most compilers already optimize code to generate inline functions when it is more convenient. This specifier only indicates the compiler that inline is preferred for this function.

Recursivity: Recursivity is the property that functions have to be called by themselves. It is useful for many tasks, like sorting or calculate the factorial of numbers. For example, to obtain the factorial of a number ($n!$) the mathematical formula would be:

$$n! = n * (n-1) * (n-2) * (n-3) \dots * 1$$

more concretely, $5!$ (factorial of 5) would be:

$$5! = 5 * 4 * 3 * 2 * 1 = 120$$

and a recursive function to calculate this in C++ could be:

```
//factorial calculator
#include <iostream>
using namespace std;

```



```

long factorial (long a)
{
    if (a > 1)
        return (a * factorial (a-1));
    else
        return (1);
}
int main ()
{
    long number;
    cout << "Please type a number: ";
    cin >> number;
    cout << number << "! = " << factorial (number);
    return 0;
}

```

output

```

Please type a number: 9
9! = 362880

```

Notice how in function factorial we included a call to itself, but only if the argument passed was greater than 1, since otherwise the function would perform an infinite recursive loop in which once it arrived to 0 it would continue multiplying by all the negative numbers (probably provoking a stack overflow error on runtime).

This function has a limitation because of the data type we used in its design (long) for more simplicity. The results given will not be valid for values much greater than 10! or 15!, depending on the system you compile it.

Declaring functions: Until now, we have defined all of the functions before the first appearance of calls to them in the source code. These calls were generally in function main which we have always left at the end of the source code. If you try to repeat some of the examples of functions described so far, but placing the function main before any of the other functions that were called from within it, you will most likely obtain compiling errors. The reason is that to be able to call a function it must have been declared in some earlier point of the code, like we have done in all our examples.

But there is an alternative way to avoid writing the whole code of a function before it can be used in main or in some other function. This can be achieved by declaring just a prototype of the function before it is used, instead of the entire definition. This declaration is shorter than the entire definition, but significant enough for the compiler to determine its return type and the types of its parameters.

Its form is:

```

type name (argument_type1, argument_type2, ...);

```

It is identical to a function definition, except that it does not include the body of the function itself (*i.e.*, the function statements that in normal definitions are enclosed in braces { }) and instead of that we end the prototype declaration with a mandatory semicolon (;).

The parameter enumeration does not need to include the identifiers, but only the type specifier. The inclusion of a name for each parameter as in the function definition

is optional in the prototype declaration. For example, we can declare a function called proto function with two int parameters with any of the following declarations:

```
int proto function (int first, int second);
int proto function (int, int);
```

Anyway, including a name for each variable makes the prototype more legible.

```
//declaring functions prototypes
#include <iostream>
using namespace std;
void odd (int a);
void even (int a);
int main ()
{
    int i;
    do {
        cout << "Type a number (0 to exit): ";
        cin >> i;
        odd (i);
    } while (i!=0);
    return 0;
}
void odd (int a)
{
    if ((a%2)!=0) cout << "Number is odd.\n";
    else even (a);
}
void even (int a)
{
    if ((a%2)==0) cout << "Number is even.\n";
    else odd (a);
}
```

output

```
Type a number (0 to exit): 9
Number is odd.
Type a number (0 to exit): 6
Number is even.
Type a number (0 to exit): 1030
Number is even.
Type a number (0 to exit): 0
Number is even.
```

This example is indeed not an example of efficiency. I am sure that at this point you can already make a programme with the same result, but using only half of the code lines that have been used in this example. Anyway this example illustrates how prototyping works. Moreover, in this concrete example the prototyping of at least one of the two functions is necessary in order to compile the code without errors.

The first things that we see are the declaration of functions odd and even:

```
void odd (int a);
void even (int a);
```

This allows these functions to be used before they are defined, for example, in `main`, which now is located where some people find it to be a more logical place for the start of a programme: the beginning of the source code.

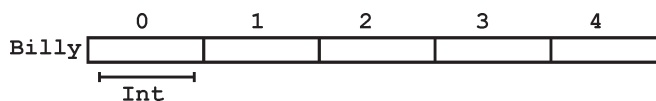
Anyway, the reason why this programme needs at least one of the functions to be declared before it is defined is because in `odd` there is a call to `even` and in `even` there is a call to `odd`. If none of the two functions had been previously declared, a compilation error would happen, since either `odd` would not be visible from `even` (because it has still not been declared), or `even` would not be visible from `odd` (for the same reason).

Having the prototype of all functions together in the same place within the source code is found practical by some programmers, and this can be easily achieved by declaring all functions prototypes at the beginning of a programme.

Arrays: An array is a series of elements of the same type placed in contiguous memory locations that can be individually referenced by adding an index to a unique identifier.

That means that, for example, we can store 5 values of type `int` in an array without having to declare 5 different variables, each one with a different identifier. Instead of that, using an array we can store 5 different values of the same type, `int` for example, with a unique identifier.

For example, an array to contain 5 integer values of type `int` called `billy` could be represented like this:



where each blank panel represents an element of the array, that in this case are integer values of type `int`. These elements are numbered from 0 to 4 since in arrays the first index is always 0, independently of its length.

Like a regular variable, an array must be declared before it is used. A typical declaration for an array in C++ is:

```
type name [elements];
```

where `type` is a valid type (like `int`, `float`...), `name` is a valid identifier and the `elements` field (which is always enclosed in square brackets `[]`), specifies how many of these elements the array has to contain.

Therefore, in order to declare an array called `billy` as the one shown in the above diagram it is as simple as:

```
int billy [5];
```

Note: The `elements` field within brackets `[]` which represents the number of elements the array is going to hold, must be a constant value, since arrays are blocks of non-dynamic memory whose size must be determined before execution. In order to create arrays with a variable length dynamic memory is needed, which is explained later in these tutorials.

Initializing arrays: When declaring a regular array of local scope (within a function, for example), if we do not specify otherwise, its elements will not be initialized to any value by default, so their content will be undetermined until we store some value in

them. The elements of global and static arrays, on the other hand, are automatically initialized with their default values, which for all fundamental types this means they are filled with zeros.

In both cases, local and global, when we declare an array, we have the possibility to assign initial values to each one of its elements by enclosing the values in braces `{}`. For example:

```
int billy [5] = {16, 2, 77, 40, 12071};
```

This declaration would have created an array like this:

	0	1	2	3	4
Billy	16	2	77	40	12071

The amount of values between braces `{}` must not be larger than the number of elements that we declare for the array between square brackets `[]`. For example, in the example of array `billy` we have declared that it has 5 elements and in the list of initial values within braces `{}` we have specified 5 values, one for each element.

When an initialization of values is provided for an array, C++ allows the possibility of leaving the square brackets empty `[]`. In this case, the compiler will assume a size for the array that matches the number of values included between braces `{}`:

```
int billy [] = {16, 2, 77, 40, 12071};
```

After this declaration, array `billy` would be 5 ints long, since we have provided 5 initialization values.

Accessing the values of an array: In any point of a programme in which an array is visible, we can access the value of any of its elements individually as if it was a normal variable, thus being able to both read and modify its value. The format is as simple as:

```
name[index]
```

Following the previous examples in which `billy` had 5 elements and each of those elements was of type `int`, the name which we can use to refer to each element is the following:

	Billy [0]	Billy [1]	Billy [2]	Billy [3]	Billy [4]
Billy					

For example, to store the value 75 in the third element of `billy`, we could write the following statement:

```
billy[2] = 75;
```

and, for example, to pass the value of the third element of `billy` to a variable called `a`, we could write:

```
a = billy[2];
```

Therefore, the expression `billy[2]` is for all purposes like a variable of type `int`.

Notice that the third element of `billy` is specified `billy[2]`, since the first one is `billy[0]`, the second one is `billy[1]`, and therefore, the third one is `billy[2]`. By this same reason, its last element is `billy[4]`. Therefore, if we write `billy[5]`, we would be accessing the sixth element of `billy` and therefore exceeding the size of the array. In C++ it is syntactically correct to exceed the valid range of indices for an array. This can create problems, since accessing out-of-range elements do not cause compilation errors but

can cause runtime errors. The reason why this is allowed will be seen further ahead when we begin to use pointers.

At this point it is important to be able to clearly distinguish between the two uses that brackets [] have related to arrays. They perform two different tasks: one is to specify the size of arrays when they are declared; and the second one is to specify indices for concrete array elements. Do not confuse these two possible uses of brackets [] with arrays.

```
int billy[5]; //declaration of a new array
billy[2] = 75; //access to an element of the array.
```

If you read carefully, you will see that a type specifier always precedes a variable or array declaration, while it never precedes an access.

Some other valid operations with arrays:

```
billy[0] = a;
billy[a] = 75;
b = billy [a+2];
billy[billy[a]] = billy[2] + 5;
//arrays example
#include <iostream>
using namespace std;
int billy [] = {16, 2, 77, 40, 12071};
int n, result=0;
int main ()
{
    for (n=0; n<5; n++)
    {
        result += billy[n];
    }
    cout << result;
    return 0;
}
output
12206
```

Multidimensional arrays: Multidimensional arrays can be described as “arrays of arrays”.

For example, a bidimensional array can be imagined as a bidimensional table made of elements, all of them of a same uniform data type.

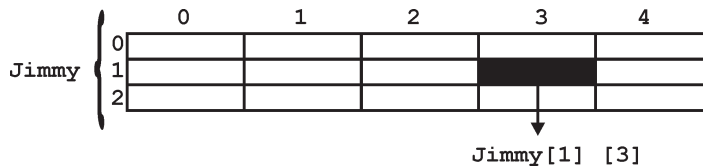
		0	1	2	3	4
Jimmy {	0					
	1					
	2					

jimmy represents a bidimensional array of 3 per 5 elements of type int. The way to declare this array in C++ would be:

```
int jimmy [3][5];
```

and, for example, the way to reference the second element vertically and fourth horizontally in an expression would be:

```
jimmy[1][3]
```



(remember that array indices always begin by zero).

Multidimensional arrays are not limited to two indices (*i.e.*, two dimensions). They can contain as many indices as needed. But be careful! The amount of memory needed for an array rapidly increases with each dimension. For example:

```
char century [100][365][24][60][60];
```

declares an array with a char element for each second in a century, that is more than 3 billion chars. So this declaration would consume more than 3 gigabytes of memory!

Multidimensional arrays are just an abstraction for programmers, since we can obtain the same results with a simple array just by putting a factor between its indices:

```
int jimmy [3][5]; //is equivalent to
int jimmy [15]; // (3 * 5 = 15)
```

With the only difference that with multidimensional arrays the compiler remembers the depth of each imaginary dimension for us. Take as example these two pieces of code, with both exactly the same result. One uses a bidimensional array and the other one uses a simple array: multidimensional array pseudo-multidimensional array

```
#define WIDTH 5
#define HEIGHT 3
int jimmy [HEIGHT][WIDTH];
int n,m;
int main ()
{
    for (n=0;n<HEIGHT;n++)
        for (m=0;m<WIDTH;m++)
        {
            jimmy[n][m]=(n+1)*(m+1);
        }
    return 0;
}

#define WIDTH 5
#define HEIGHT 3
int jimmy [HEIGHT * WIDTH];
int n,m;
int main ()
{
    for (n=0;n<HEIGHT;n++)
        for (m=0;m<WIDTH;m++)
        {
            jimmy[n*WIDTH+m]=(n+1)*(m+1);
        }
    return 0;
}
```

None of the two source codes above produce any output on the screen, but both assign values to the memory block called jimmy in the following way:

Jimmy

	0	1	2	3	4
0	1	2	3	4	5
1	2	4	6	8	10
2	3	6	9	12	15

We have used “defined constants” (#define) to simplify possible future modifications of the programme. For example, in case that we decided to enlarge the array to a height of 4 instead of 3 it could be done simply by changing the line:

```
#define HEIGHT 3
```

to:

```
#define HEIGHT 4
```

with no need to make any other modifications to the programme.

Arrays as parameters: At some moment we may need to pass an array to a function as a parameter.

In C++ it is not possible to pass a complete block of memory by value as a parameter to a function, but we are allowed to pass its address.

In practice this has almost the same effect and it is a much faster and more efficient operation.

In order to accept arrays as parameters the only thing that we have to do when declaring the function is to specify in its parameters the element type of the array, an identifier and a pair of void brackets [].

For example, the following function:

```
void procedure (int arg[])
```

accepts a parameter of type “array of int” called arg. In order to pass to this function an array declared as:

```
int myarray [40];
```

it would be enough to write a call like this:

```
procedure (myarray);
```

Here you have a complete example:

```
//arrays as parameters
#include <iostream>
using namespace std;
void printarray (int arg[], int length) {
    for (int n=0; n<length; n++)
        cout << arg[n] << " ";
    cout << "\n";
}
int main ()
{
    int firstarray[] = {5, 10, 15};
    int secondarray[] = {2, 4, 6, 8, 10};
    printarray (firstarray,3);
    printarray (secondarray,5);
    return 0;
}
output
5 10 15 2 4 6 8 10
```

As you can see, the first parameter (`int arg[]`) accepts any array whose elements are of type `int`, whatever its length. For that reason we have included a second parameter that tells the function the length of each array that we pass to it as its first parameter.

This allows the for loop that prints out the array to know the range to iterate in the passed array without going out of range.

COMMENTS

Comments are parts of the source code disregarded by the compiler. They simply do nothing. Their purpose is only to allow the programmer to insert notes or descriptions embedded within the source code.

C++ supports two ways to insert comments:

```
// line comment
/* block comment */
```

The first of them, known as line comment, discards everything from where the pair of slash signs (`//`) is found up to the end of that same line. The second one, known as block comment, discards everything between the `/*` characters and the first appearance of the `*/` characters, with the possibility of including more than one line.

We are going to add comments to our second programme:

```
/* my second programme in C++
with more comments */
#include <iostream>
using namespace std;
int main ()
{
    cout << "Hello World! "; // prints Hello World!
    cout << "I'm a C++ programme"; // prints I'm a C++ programme
    return 0;
} Hello World! I'm a C++ programme
```

If you include comments within the source code of your programmes without using the comment characters combinations `//`, `/*` or `*/`, the compiler will take them as if they were C++ expressions, most likely causing one or several error messages when you compile it.

VARIABLES

The usefulness of the “Hello World” programmes shown in the previous section is quite questionable. We had to write several lines of code, compile them, and then execute the resulting programme just to obtain a simple sentence written on the screen as result. It certainly would have been much faster to type the output sentence by ourselves. However, programming is not limited only to printing simple texts on the screen. In order to go a little further on and to become able to write programmes that perform useful tasks that really save us work we need to introduce the concept of variable.

Let us think that I ask you to retain the number 5 in your mental memory, and then I ask you to memorize also the number 2 at the same time. You have just stored two different values in your memory. Now, if I ask you to add 1 to the first number I said, you should be retaining the numbers 6 (that is 5+1) and 2 in your memory. Values that we could now for example subtract and obtain 4 as result.

The whole process that you have just done with your mental memory is a simile of what a computer can do with two variables.

The same process can be expressed in C++ with the following instruction set:

```
a = 5;  
b = 2;  
a = a + 1;  
result = a - b;
```

Obviously, this is a very simple example since we have only used two small integer values, but consider that your computer can store millions of numbers like these at the same time and conduct sophisticated mathematical operations with them.

Therefore, we can define a variable as a portion of memory to store a determined value.

Each variable needs an identifier that distinguishes it from the others, for example, in the previous code the variable identifiers were a, b and result, but we could have called the variables any names we wanted to invent, as long as they were valid identifiers.

SCOPE OF VARIABLES

All the variables that we intend to use in a programme must have been declared with its type specifier in an earlier point in the code, like we did in the previous code at the beginning of the body of the function main when we declared that a, b, and result were of type int. A variable can be either of global or local scope. A global variable is a variable declared in the main body of the source code, outside all functions, while a local variable is one declared within the body of a function or a block. Global variables can be referred from anywhere in the code, even inside functions, whenever it is after its declaration.

The scope of local variables is limited to the block enclosed in braces ({}) where they are declared. For example, if they are declared at the beginning of the body of a function (like in function main) their scope is between its declaration point and the end of that function. In the example above, this means that if another function existed in addition to main, the local variables declared in main could not be accessed from the other function and vice versa.

INITIALIZATION OF VARIABLES

When declaring a regular local variable, its value is by default undetermined. But you may want a variable to store a concrete value at the same moment that it is declared. In order to do that, you can initialize the variable. There are two ways to do this in C++:

The first one, known as c-like, is done by appending an equal sign followed by the value to which the variable will be initialized:

```
type identifier = initial_value ;
```

For example, if we want to declare an int variable called a initialized with a value of 0 at the moment in which it is declared, we could write:

```
int a = 0;
```

The other way to initialize variables, known as constructor initialization, is done by enclosing the initial value between parentheses (()):

```
type identifier (initial_value) ;
```

For example:

```
int a (0);
```

Both ways of initializing variables are valid and equivalent in C++.

```
// initialization of variables
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main ()
```

```
{
```

```
    int a=5; // initial value = 5
```

```
    int b(2); // initial value = 2
```

```
    int result; // initial value undetermined
```

```
    a = a + 3;
```

```
    result = a - b;
```

```
    cout << result;
```

```
    return 0;
```

```
} 6
```

Statements and Language Programming

Fortran (Formula Translation) is a very old computer language. It has undergone several evolutionary stages and at present fortran 95 is the standard fortran. The previous version (fortran 77) is also common among older people. It is believed that the demand for fortran is going down in favour of C++. Particularly, big laboratories (CERN for one) has decided to switch to C++. As a result, it is possible that fortran will be superseded by C++.

But, many times in the past, people have announced that the fortran will die within a short time and it has always bounced back. Over the time it has imported concepts from other computer languages and assimilated them. So it is possible that fortran will still survive.

The Fortran, as the name suggests, is best suited for numerical computations. That is why it is popular among physicists, chemists, mathematicians and engineers. The handling of alphanumeric strings is not so good in fortran but that is not such a handicap because one doesn't need this aspect very much while doing numerical computations. Like any language FORTRAN has its syntax and that is what we shall be learning here. We shall be following a utilitarian approach so we shall not dwell much on the structure of fortran per se but we shall learn fortran by using it.

So, in the following, shall introduce some concepts and immediately give examples to illustrate the concept. The way the fortran programmes are used on computers is as

follows. First one makes (writes) a fortran programme on computer. That is, one creates a file containing a collection of fortran statements. Invariably a fortran programme has .f or .f 77 as an ending character string (extension).

The computer cannot understand this fortran programme as such and it has to be converted into a programme that the computer understands. This is done by a fortran compiler and the process is called compilation of the programme.

The translated programme is in machine language and often called as a.out. But it can have any name. Before the compiler creates a machine language programme, it creates an intermediate file which has an ending.o. This file is called the object code. So, let us assume that you have made a programme called *foo.f*. On compilation, an object file *foo.o* and an executable a.out is created by the computer.

Other possible names of the compiler are fort, g77, f90, lf90 (only in the Institute of Physics and on some of the machines) etc. The output of the command is *foo.o* and a.out. To run the programme, type:

a.out

or

./a.out

at the command prompt. The programme will then be executed.

The steps involved in computing using fortran:

- Make a programme file (e.g. *foo.f*) using any editor of your choice
- Compile the file and make an object code (*foo.o*) and an executable file (usually *a.out*)
- Execute the programme (*a.out*)
- Check the results for errors, correct the errors in the fortran programme and go back to step 2

The Fortran compiler, by default, creates executable file named a.out after compilation. The problem with default is every compiled file is a.out so one doesn't know what it does if you have a number of programmes.

Also, every time you compile a new programme, old a.out is erased and new a.out is written in its place. So, it is convenient to follow a convention where the executable of a Fortran programme has a name similar to the name of the fortran programme. The convention I follow is to name the executable as *foo*. The command to generate this executable is

```
f77 -o foo foo.f
```

It appears that the object file *foo.o* is of no use. As mentioned above, the object file is a programme which is between a fortran programme and its executable code. It is useful when one has a number of subunits (subroutines, functions, libraries) in a fortran programme and if one is modifying only a part of a programme, it doesn't make sense to compile all the other programme units again and again. In this case, one can compile the programme that is changed and use link the object code thus created with the object codes of other programme units.

This saves a lot of compilation time, particularly if one has tens of other programme units. During the course of these lectures we will not be dealing with complicated and lengthy programmes and as a result we will not be using the object files in the course.

The C++ manual has introduced the C++ library. When using this library to solve a particular system it is necessary to write the C++ code that allows to deal with the system at hand.

The primary purpose of the Maple library was to allow the automatic generation of the C++ code being given the Maple description of an equations system and then to compile and run the generated code in order to get the result within the Maple session.

But when developing this library it appears that the use of symbolic computation may allow to increase the efficiency of the C++ solving procedure. Hence the procedures available in the Maple library may be divided into two categories:

- Procedures that allow to solve a given problem. In that case the procedure will:
 - Generate the necessary C++ code
 - Compile this code
 - Run the created programme
 - Return the result to Maple
- Procedures that allow to improve the efficiency of a solving procedure (which may eventually use C++ programme).

The Maple library ALIAS.m has initially been developed by Didier Bondyfalat. ALIAS-Maple is compiled for Maple V.5 and 9.5 and this version has 49356 lines of code. In this package there are numerous Maple procedures that may be used to solve or to help to solve a specific problem. The behaviour of these procedures may be modified by changing the values of some Maple variables (that will correspond to the parameters of the ALIAS-C++ procedures). An ALIAS-Maple variable is always defined using the following syntax:

`'ALIAS/XX'`

The use of some of these variables may need an understanding of the algorithms of ALIAS-C++. The C++ procedures created by ALIAS-Maple procedures have usually a fixed name (e.g. `_GS_` for the general purpose solving procedures) but this behaviour may be changed for some of them if the string variable `'ALIAS/ID'` is defined (and on the long term it is planned that all ALIAS-Maple procedures will propose this possibility): in that case all files created by a procedure have the `'ALIAS/ID'` string appended to their name.

Before explaining how to use this package very important remarks have to be done:

- An equation $f(x) = 0$ is defined in ALIAS-Maple by `f(x)` while an inequality is defined by `f(x) ≤ 0`
- Not all expression may interval evaluated. For example interval evaluating $1/x$ with x being an interval including 0 will cause the C++ programme created by ALIAS-Maple to crash. ALIAS-Maple provides procedures to deal with this problem.

- It has been noticed that for large expressions the compilation time of their C++ equivalent may be quite large (or even may be impossible). It is sometime possible to avoid this problem by turning off the optimization flag. Indeed by default the C++ programme will be compiled with the optimization flag -O and it is sometime better to turn off this option by setting the flag 'ALIAS/optimized' to 0. Another approach is to use some procedures provided in ALIAS-Maple that simplify the compilation by using dummy intervals or decompose expression into elementary components as will be seen later on. This problem may also be solved by using the ALIAS parser in order to avoid having to compile very large expressions.

CHARACTERS

A character variable is defined to be of type `char`. A character variable occupies a single byte which contains the *code* for the character. This code is a numeric value and depends on the *character coding system* being used (*i.e.*, is machine-dependent). The most common system is ASCII (American Standard Code for Information Interchange). For example, the character *A* has the ASCII code 65, and the character *a* has the ASCII code 97.

```
char   ch = 'A';
```

Like integers, a character variable may be specified to be signed or unsigned. By default (on most systems) `char` means signed `char`. However, on some systems it may mean unsigned `char`. A signed character variable can hold numeric values in the range -128 through 127. An unsigned character variable can hold numeric values in the range 0 through 255. As a result, both are often used to represent small integers in programmes (and can be assigned numeric values like integers):

```
signed char   offset = -88;
unsigned char row = 2, column = 26;
```

A literal character is written by enclosing the character between a pair of single quotes (*e.g.*, `'A'`). Non-printable characters are represented using escape sequences. For example:

```
'\n'   //new line
'\r'   //carriage return
'\t'   //horizontal tab
'\v'   //vertical tab
'\b'   //backspace
'\f'   //formfeed
```

Single and double quotes and the backslash character can also use the escape notation:

```
'\''   //single quote (\)
'\"'   //double quote (")
'\\'   //backslash (\)
```

Literal characters may also be specified using their numeric code value. The general escape sequence `\ooo` (*i.e.*, a backslash followed by up to three octal digits) is used for this purpose. For example (assuming ASCII):

```
'\12'   //newline (decimal code = 10)
'\11'   //horizontal tab (decimal code = 9)
```

```

'\101'  //'A' (decimal code = 65)
'\0'    //null (decimal code = 0)

```

STRINGS

A string is a consecutive sequence (*i.e.*, *array*) of characters which are terminated by a null character. A string variable is defined to be of type `char*` (*i.e.*, a *pointer* to character). A pointer is simply the address of a memory location. A string variable, therefore, simply contains the address of where the first character of a string appears.

For example, consider the definition:

```
char *str = "HELLO";
```

A literal string is written by enclosing its characters between a pair of double quotes (*e.g.*, "HELLO"). The compiler always appends a null character to a literal string to mark its end. The characters of a string may be specified using any of the notations for specifying literal characters.

For example:

```

"Name\tAddress\tTelephone"    //tab-separated words
"ASCII character 65: \101"     //'A' specified as '\101'

```

A long string may extend beyond a single line, in which case each of the preceding lines should be terminated by a backslash. For example:

```

"Example to show \
the use of backslash for \
writing a long string"

```

The backslash in this context means that the rest of the string is continued on the next line. The above string is equivalent to the single line string:

```
"Example to show the use of backslash for writing a long string"
```

A common programming error results from confusing a single-character string (*e.g.*, "A") with a single character (*e.g.*, 'A'). These two are *not* equivalent. The former consists of two bytes (the character 'A' followed by the character '\0'), whereas the latter consists of a single byte. The shortest possible string is the null string ("") which simply consists of the null character.

NAMES

Programming languages use names to refer to the various entities that make up a programme. We have already seen examples of an important category of such names (*i.e.*, variable names). Other categories include: function names, type names, and macro names, which will be described later in this book. Names are a programming convenience, which allow the programmer to organize what would otherwise be quantities of plain data into a meaningful and human-readable collection.

As a result, no trace of a name is left in the final executable code generated by a compiler. For example, a temperature variable eventually becomes a few bytes of memory which is referred to by the executable code by its address, not its name.

C++ imposes the following rules for creating valid names (also called identifiers). A name should consist of one or more characters, each of which may be a letter (*i.e.*, ‘A’-‘Z’ and ‘a’-‘z’), a digit (*i.e.*, ‘0’-‘9’), or an underscore character (‘_’), except that the first character may not be a digit. Upper and lower case letters are distinct.

For example: Salary//valid identifier

salary2 //valid identifier

2salary //invalid identifier

(begins with a digit)

_salary //valid identifier

Salary //valid but distinct from salary

C++ imposes no limit on the number of characters in an identifier. However, most implementation do. But the limit is usually so large that it should not cause a concern (*e.g.*, 255 characters).

SIMPLE INPUT/OUTPUT

The most common way in which a programme communicates with the outside world is through simple, character-oriented Input/Output (IO) operations. C++ provides two useful operators for this purpose: >> for input and << for output. We have already seen examples of output using <<. Listing below illustrates the use of >> for input.

```
1 #include <iostream.h>
2 int main (void)
3 {
4     int workDays = 5;
5     float workHours = 7.5;
6     float payRate, weeklyPay;
7     cout << "What is the hourly pay rate? ";
8     cin >> payRate;
9     weeklyPay = workDays * workHours * payRate;
10    cout << "Weekly Pay = ";
11    cout << weeklyPay;
12    cout << '\n';
13 }
```

ANNOTATION

7. This line outputs the prompt What is the hourly pay rate? to seek user input.
8. This line reads the input value typed by the user and copies it to payRate. The input operator >> takes an input stream as its left operand (cin is the standard C++ input stream which corresponds to data entered via the keyboard) and a variable (to which the input data is copied) as its right operand.
- 9-13. The rest of the programme is as before. When run, the programme will produce the following output (user input appears in bold):


```
What is the hourly pay rate? 33.55
Weekly Pay = 1258.125
```

Both << and >> return their left operand as their result, enabling multiple input or multiple output operations to be combined into one statement. This is illustrated by Listing below which now allows the input of both the daily work hours and the hourly pay rate.

```
1 #include <iostream.h>
2 int main (void)
3 {
4     int workDays = 5;
5     float workHours, payRate, weeklyPay;
6     cout << "What are the work hours and the hourly pay rate? ";
7     cin >> workHours >> payRate;
8     weeklyPay = workDays * workHours * payRate;
9     cout << "Weekly Pay = " << weeklyPay << '\n';
10 }
```

ANNOTATION

- This line reads two input values typed by the user and copies them to workHours and payRate, respectively. The two values should be separated by white space (*i.e.*, one or more space or tab characters). This statement is equivalent to:

```
(cin >> workHours) >> payRate;
```

Because the result of >> is its left operand, (cin >> workHours) evaluates to cin which is then used as the left operand of the next >> operator.

- This line is the result of combining lines 10-12 from Listing 1.4. It outputs “Weekly Pay =”, followed by the value of weeklyPay, followed by a newline character. This statement is equivalent to:

```
((cout << "Weekly Pay =") << weeklyPay) << '\n';
```

Because the result of << is its left operand, (cout << “Weekly Pay =”) evaluates to cout which is then used as the left operand of the next << operator, etc.

When run, the programme will produce the following output:

```
What are the work hours and the hourly pay rate? 7.5 33.55
Weekly Pay = 1258.125.
```

COMMENTS

A comment is a piece of descriptive text which explains some aspect of a programme. Programme comments are totally ignored by the compiler and are only intended for human readers. C++ provides two types of comment delimiters:

- Anything after //(until the end of the line on which it appears) is considered a comment.
- Anything enclosed by the pair/* and */is considered a comment.

Listing 1.6 illustrates the use of both forms.

Listing 1.6

```

1 #include <iostream.h>
2 /* This programme calculates the weekly gross pay for a worker,
3    based on the total number of hours worked and the hourly pay
4    rate. */
5 int main (void)
6 {
7     int workDays = 5;    //Number of work days per week
8     float   workHours = 7.5;    //Number of work hours per day
9     float   payRate = 33.50;    //Hourly pay rate
10    float   weeklyPay;    //Gross weekly pay
11    weeklyPay = workDays * workHours * payRate;
12    cout << "Weekly Pay = " << weeklyPay << '\n';
13 }
```

Comments should be used to enhance (not to hinder) the readability of a programme.

The following two points, in particular, should be noted:

- A comment should be easier to read and understand than the code which it tries to explain. A confusing or unnecessarily-complex comment is worse than no comment at all.
- Over-use of comments can lead to even less readability. A programme which contains so much comment that you can hardly see the code can by no means be considered readable.
- Use of descriptive names for variables and other entities in a programme, and proper indentation of the code can reduce the need for using comments. The best guideline for how to use comments is to simply apply common sense.

MEMORY

A computer provides a Random Access Memory (RAM) for storing executable programme code as well as the data that programme manipulates. This memory can be thought of as a contiguous sequence of bits, each of which is capable of storing a binary digit (0 or 1). Typically, the memory is also divided into groups of 8 consecutive bits (called bytes). The bytes are sequentially addressed. Therefore each byte can be uniquely identified by its address. The C++ compiler generates executable code which maps data entities to memory locations. For example, the variable definition `int salary = 65000;` causes the compiler to allocate a few bytes to represent salary. The exact number of bytes allocated and the method used for the binary representation of the integer depends on the specific C++ implementation, but let us say two bytes encoded as a 2's complement integer. The compiler uses the *address* of the first byte at which salary is allocated to refer to it. The above assignment causes the value 65000 to be stored as a 2's complement integer in the two bytes allocated.

While the exact binary representation of a data item is rarely of interest to a programmer, the general organization of memory and use of addresses for referring to data items (as we will see later) is very important.

DEFINED DATA TYPES (TYPEDEF)

C++ allows the definition of our own types based on other existing data types. We can do this using the keyword `typedef`, whose format is:

```
typedef existing_type new_type_name;
```

where `existing_type` is a C++ fundamental or compound type and `new_type_name` is the name for the new type we are defining.

For example:

```
typedef char C;
typedef unsigned int WORD;
typedef char * pChar;
typedef char field [50];
```

In this case we have defined four data types: `C`, `WORD`, `pChar` and `field` as `char`, `unsigned int`, `char*` and `char[50]` respectively, that we could perfectly use in declarations later as any other valid type:

```
C mychar, anotherchar, *ptc1;
WORD myword;
pChar ptc2;
field name;
```

`typedef` does not create different types. It only creates synonyms of existing types. That means that the type of `myword` can be considered to be either `WORD` or `unsigned int`, since both are in fact the same type. `typedef` can be useful to define an alias for a type that is frequently used within a programme. It is also useful to define types when it is possible that we will need to change the type in later versions of our programme, or if a type you want to use has a name that is too long or confusing.

Unions: Unions allow one same portion of memory to be accessed as different data types, since all of them are in fact the same location in memory. Its declaration and use is similar to the one of structures but its functionality is totally different:

```
union union_name {
    member_type1 member_name1;
    member_type2 member_name2;
    member_type3 member_name3;
    .
    .
} object_names;
```

All the elements of the union declaration occupy the same physical space in memory. Its size is the one of the greatest element of the declaration. For example:

```
union mytypes_t {
    char c;
    int i;
    float f;
```

```

    } mytypes;
    defines three elements:
mytypes.c
mytypes.i
mytypes.f

```

each one with a different data type. Since all of them are referring to the same location in memory, the modification of one of the elements will affect the value of all of them. We cannot store different values in them independent from each other.

One of the uses a union may have is to unite an elementary type with an array or structures of smaller elements.

For example:

```

union mix_t {
    long l;
    struct {
        short hi;
        short lo;
    } s;
    char c[4];
} mix;

```

Defines three names that allow to access the same group of 4 bytes: `mix.l`, `mix.s` and `mix.c` and which we can use according to how we want to access these bytes, as if they were a single long-type data, as if they were two short elements or as an array of char elements, respectively. For a *little-endian* system (most PC platforms), this union could be represented as:



The exact alignment and order of the members of a union in memory is platform dependant. Therefore be aware of possible portability issues with this type of use.

Anonymous unions: In C++ we have the option to declare anonymous unions. If we declare a union without any name, the union will be anonymous and we will be able to access its members directly by their member names.

For example, look at the difference between these two structure declarations: Structure with regular union structure with anonymous union

```

struct {
    char title[50];
    char author[50];
    union {
        float dollars;
        int yens;
    } price;
}

```

```

} book; struct {
    char title[50];
    char author[50];
    union {
        float dollars;
        int yens;
    };
} book;

```

The only difference between the two pieces of code is that in the first one we have given a name to the union (price) and in the second one we have not. The difference is seen when we access the members dollars and yens of an object of this type.

For an object of the first type, it would be:

```

book.price.dollars
book.price.yens

```

whereas for an object of the second type, it would be:

```

book.dollars
book.yens

```

Once again I remind you that because it is a union and not a struct, the members dollars and yens occupy the same physical space in the memory so they cannot be used to store two different values simultaneously. You can set a value for price in dollars or in yens, but not in both.

ENUMERATIONS (ENUM)

Enumerations create new data types to contain something different that is not limited to the values fundamental data types may take.

Its form is the following:

```

enum enumeration_name {
    value1,
    value2,
    value3,
    .
    .
} object_names;

```

For example, we could create a new type of variable called colour to store colours with the following declaration:

```

enum colours_t {black, blue, green, cyan, red, purple, yellow, white};

```

Notice that we do not include any fundamental data type in the declaration. To say it somehow, we have created a whole new data type from scratch without basing it on any other existing type. The possible values that variables of this new type colour_t may take are the new constant values included within braces.

For example, once the colours_t enumeration is declared the following expressions will be valid:

```

colours_t mycolour;
mycolor = blue;
if (mycolor == green) mycolor = red;

```

Enumerations are type compatible with numeric variables, so their constants are always assigned an integer numerical value internally. If it is not specified, the integer value equivalent to the first possible value is equivalent to 0 and the following ones follow a +1 progression. Thus, in our data type `colours_t` that we have defined above, black would be equivalent to 0, blue would be equivalent to 1, green to 2, and so on.

We can explicitly specify an integer value for any of the constant values that our enumerated type can take. If the constant value that follows it is not given an integer value, it is automatically assumed the same value as the previous one plus one.

For example:

```
enum months_t {january=1, february, march, april,
               may, june, july, august,
               september, october, november, december} y2k;
```

In this case, variable `y2k` of enumerated type `months_t` can contain any of the 12 possible values that go from january to december and that are equivalent to values between 1 and 12 (not between 0 and 11, since we have made january equal to 1).

CLASSES (I)

Class Hierarchy

The C++ class hierarchy which is used to represent the basic repertoire of abstract data types is shown in Figure . Two kinds of classes are; *abstract C++ classes* , which look like this *Abstract Class*, and *concrete C++ classes* , which look like this *Concrete Class*. Lines in the figure indicate derivation; base classes always appear to the left of derived classes.

An *abstract class* is a class which specifies an *interface* only. It is not possible create object instances of abstract classes. In C++ an abstract class typically has one or more *pure virtual member functions* . A *pure* virtual member function declares an interface only—there is no implementation defined. In effect, the interface specifies the set of operations without specifying the implementation.

An abstract class is intended to be used as the *base class* from which other classes are *derived* . Declaring the member functions *virtual* makes it possible to access the implementations provided by the derived classes through the base-class interface.

Consequently, we don't need to know how a particular object instance is implemented, nor do we need to know of which derived class it is an instance.

This design pattern uses the idea of *polymorphism* . Polymorphism literally means “having many forms.” The essential idea is that a single, common abstraction is used to define the set of values and the set of operations—the abstract data type. This interface is embodied in the C++ abstract class definition.

Then, various different implementations (*many forms*) of the abstract data type can be made. This is done in C++ by deriving concrete class instances from the abstract base class.

The remainder of this section presents the top levels of the class hierarchy. The top levels define those attributes of objects which are common to all of the classes in the hierarchy.

OPERATORS AND EXPRESSIONS

This chapter introduces the built-in C++ operators for composing expressions. An expression is any computation which yields a value. When discussing expressions, we often use the term evaluation. For example, we say that an expression evaluates to a certain value. Usually the final value is the only reason for evaluating the expression. However, in some cases, the expression may also produce side-effects. These are permanent changes in the programme state. In this sense, C++ expressions are different from mathematical expressions. C++ provides operators for composing arithmetic, relational, logical, bitwise, and conditional expressions. It also provides operators which produce useful side effects, such as assignment, increment, and decrement. We will look at each category of operators in turn. We will also discuss the precedence rules which govern the order of operator evaluation in a multi-operator expression.

ARITHMETIC OPERATORS

C++ provides five basic arithmetic operators.

Operator	Name	Example
+ //gives 16.9	Addition	12 + 4.9
- //gives -0.02	Subtraction	3.98-4
* //gives 6.8	Multiplication	2 * 3.4
/ //gives 4.5	Division	9/2.0
%//gives 1	Remainder	13 % 3

Except for remainder (%) all other arithmetic operators can accept a mix of integer and real operands. Generally, if both operands are integers then the result will be an integer. However, if one or both of the operands are reals then the result will be a real (or double to be exact).

When both operands of the division operator (/) are integers then the division is performed as an integer division and not the normal division we are used to. Integer division always results in an integer outcome.

```
9/2    //gives 4, not 4.5!
-9/2   //gives -5, not -4!
```

Unintended integer divisions are a common source of programming errors. To obtain a real division when both operands are integers, you should cast one of the operands to be real:

```
int cost = 100;
int volume = 80;
double unitPrice = cost/(double) volume;           //gives 1.25
```

The remainder operator (%) expects integers for both of its operands. It returns the remainder of integer-dividing the operands. For example $13\% 3$ is calculated by integer dividing 13 by 3 to give an outcome of 4 and a remainder of 1; the result is therefore 1.

It is possible for the outcome of an arithmetic operation to be too large for storing in a designated variable. This situation is called an overflow. The outcome of an overflow is machine-dependent and therefore undefined.

For example:

```
unsigned char k = 10 * 92; //overflow: 920 > 255
```

It is illegal to divide a number by zero. This results in a run-time *division-by-zero* failure which typically causes the programme to terminate.

RELATIONAL OPERATORS

C++ provides six relational operators for comparing numeric quantities. These are summarized in Table below. Relational operators evaluate to 1 (representing the *true* outcome) or 0 (representing the *false* outcome).

Table. Relational Operators.

Operator	Name	Example
==	Equality	5 == 5 //gives 1 t.e. true
!=	Inequality	5 != 5 //gives 0 t.e. false
<	Less than	5 < 5.5 //gives 1 t.e. true
<=	Less than or equal	5 <= 5 //gives 1 t.e. true
>	Greater than	5 > 5.5 //gives 0 t.e. false
>=	Greater than or equal	6.3 >= 5 //gives 1 t.e. true

Note that the <= and >= operators are only supported in the form shown. In particular, <= and >= are both invalid and do not mean anything.

The operands of a relational operator must evaluate to a number. Characters are valid operands since they are represented by numeric values. For example (assuming ASCII coding):

```
'A' < 'F' //gives 1 (is like 65 < 70) t.e. true
```

The relational operators should not be used for comparing strings, because this will result in the string *addresses* being compared, not the string contents.

For example, the expression:

```
"HELLO" < "BYE"
```

Causes the address of "HELLO" to be compared to the address of "BYE". As these addresses are determined by the compiler (in a machine-dependent manner), the outcome may be 0 or may be 1, and is therefore undefined.

C++ provides library functions (e.g., strcmp) for the lexicographic comparison of string. These will be described later in the book.

LOGICAL OPERATORS

C++ provides three logical operators for combining logical expression. These are summarized in table like the relational operators, logical operators evaluate to 1 or 0.

Table. Logical Operators.

Operator	Name	Example
!	Logical negation	!(5 == 5) //gives 0
&&	Logical and	5 < 6 && 6 < 6 //gives 1
	Logical or	5 < 6 6 < 5 //gives 1

Logical *negation* is a unary operator, which negates the logical value of its single operand. If its operand is non-zero it produce 0, and if it is 0 it produces 1. Logical *and* produces 0 if one or both of its operands evaluate to 0. Otherwise, it produces 1. Logical *or* produces 0 if both of its operands evaluate to 0. Otherwise, it produces 1.

Example:

Operand A	!A
0	1
1	0

Operand A	Operand B	A B
0	1	1
1	0	1
0	0	0
1	1	1

Operand A	Operand B	A&&B
0	1	0
1	0	0
0	0	0
1	1	1

Note that here we talk of zero and non-zero operands (not zero and 1). In general, any non-zero value can be used to represent the logical *true*, whereas only zero represents the logical *false*.

The following are, therefore, all valid logical expressions:

```
!20//gives 0
10 && 5      //gives 1
10 || 5.5    //gives 1
10 && 0      //gives 0
```

C++ does not have a built-in boolean type. It is customary to use the type `int` for this purpose instead.

For example:

```
int sorted = 0;    //false
int balanced = 1;  //true
```

BITWISE OPERATORS

C++ provides six bitwise operators for manipulating the individual bits in an integer quantity. These are summarized in table.

Table. Bitwise Operators.

Operator	Name	Example
~	Bitwise negation	~'\011' //gives '\366'
&	Bitwise and	'\011' & '\027' //gives '\001'
	Bitwise or	'\011' '\027' //gives '\037'
^	Bitwise exclusive Or	'\011' ^ '\027' //gives '\036'
<<	Bitwise left Shift	'\011' << 2 //gives '\044'
>>	Bitwise right Shift	'\011' >> 2 //gives '\002'

Bitwise operators expect their operands to be integer quantities and treat them as bit sequences. Bitwise *negation* is a unary operator which reverses the bits in its operands. Bitwise *and* compares the corresponding bits of its operands and produces a 1 when both bits are 1, and 0 otherwise.

Bitwise *or* compares the corresponding bits of its operands and produces a 0 when both bits are 0, and 1 otherwise. Bitwise *exclusive or* compares the corresponding bits of its operands and produces a 0 when both bits are 1 or both bits are 0, and 1 otherwise.

Bitwise *left shift* operator and bitwise *right shift* operator both take a bit sequence as their left operand and a positive integer quantity n as their right operand.

The former produces a bit sequence equal to the left operand but which has been shifted n bit positions to the left. The latter produces a bit sequence equal to the left operand but which has been shifted n bit positions to the right. Vacated bits at either end are set to 0.

To avoid worrying about the sign bit (which is machine dependent), it is common to declare a bit sequence as an unsigned quantity:

```
unsigned char x = '\011';
unsigned char y = '\027';
```

Table. Calculating Bite.

Example	Octal value	Bit Sequence
x	011	0 0 0 0 1 0 0 0 1
y	027	0 0 0 1 0 1 1 1 1
~x	366	1 1 1 1 0 1 1 0 0
x & y	001	0 0 0 0 0 0 0 0 1
x y	037	0 0 0 1 1 1 1 1 1
x ^ y	036	0 0 0 1 1 1 1 0 0
x << 2	044	0 0 1 0 0 1 0 0 0
x >> 2	002	0 0 0 0 0 0 1 0 0

INCREMENT/DECREMENT OPERATORS

The *auto increment* (++) and *auto decrement* (--) operators provide a convenient way of, respectively, adding and subtracting 1 from a numeric variable. These are summarized in Table below.

The examples assume the following variable definition:

```
int k = 5;
```

Table. Increment and Decrement Operators.

Operator Name	Example
++ Auto Increment (prefix)	++k + 10 //gives 16
++ Auto Increment (postfix)	k++ + 10 //gives 15
- Auto Decrement (prefix)	--k + 10 //gives 14
- Auto Decrement (postfix)	k-- + 10 //gives 15

Both operators can be used in prefix and postfix form. The difference is significant. When used in prefix form, the operator is first applied and the outcome is then used in the expression. When used in the postfix form, the expression is evaluated first and then the operator applied.

PROGRAMME

THE ARGUMENT VECTOR

C was written in order to implement Unix in a portable form. Unix was designed with a command language which was built up of independent programmes. These could be passed arguments on the command line. For instance:

```
ls -l /etc
cc -o programme prog.c
main (argc,argv)
int argc;
char *argv[];
{
}
```

The traditional names for the parameters are the argument count `argc` and the argument vector (array) `argv`. The operating system call which starts the C programme breaks up the command line into an array, where the first element `argv[0]` is the name of the command itself and the last argument `argv[argc-1]` is the last argument. For example, in the case of

```
cc -o programme prog.c
would result in the values
argv[0]
cc
argv[1]
```

```
-O
argv[2]
programme
argv[3]
prog.c
```

The following programme prints out the command line arguments:

```
main (argc,argv)
int argc;
char *argv[];
{ int i;
printf ("This programme is called %s\n",argv[0]);
if (argc > 1)
{
for (i = 1; i < argc; i++)
{
printf("argv[%d] = %s\n",i,argv[i]);
}
}
else
{
printf("Command has no arguments\n");
}
}
```

ENVIRONMENT VARIABLES

When we write a C programme which reads command line arguments, they are fed to us by the argument vector. Unix processes also a set of text variable associations called environment variables. Each child process inherits the environment of its parent. The static environment variables are stored in a special array which is also passed to main() and can be read if desired.

```
main (argc,argv,envp)
int argc;
char *argv[], *envp[];
{
}
```

The array of strings envp[] is a list of values of the *environment variables* of the system, formatted by

NAME=value

This gives C programmers access to the shell's global environment. In addition to the envp vector, it is possible to access the environment variables through the call getenv(). This is used as follows; suppose we want to access the shell environment variable \$HOME.

```
char *string;
string = getenv("HOME");
```

string is now a pointer to static but public data. You should not use string as if it were you're own property because it will be used again by the system. Copy it's contents to another string before using the data.

```
char buffer[500];  
strcpy (buffer, string);
```

SPECIAL LIBRARY FUNCTIONS AND MACROS

CHECKING CHARACTER

C provides a repertoire of standard library functions and macros for specialized purposes (and for the advanced user). These may be divided into various categories.

For instance,

- Character identification (ctype.h)
- String manipulation (string.h)
- Mathematical functions (math.h)

A programme generally has to #include special header files in order to use special functions in libraries. The names of the appropriate files can be found in particular compiler manuals. In the examples above the names of the header files are given in parentheses.

CHARACTER IDENTIFICATION

Some or all of the following functions/macros will be available for identifying and classifying single characters. The programmer ought to beware that it would be natural for many of these facilities to exist as macros rather than functions, so the usual remarks about macro parameters apply. An example of their use is given above. Assume that 'true' has any non-zero, integer value and that 'false' has the integer value zero. ch stands for some character, or char type variable.

isalpha(ch)

This returns true if ch is alphabetic and false otherwise. Alphabetic means a.z or A.Z.

isupper(ch)

Returns true if the character was upper case. If ch was not an alphabetic character, this returns false.

islower(ch)

Returns true if the character was lower case. If ch was not an alphabetic character, this returns false.

isdigit(ch)

Returns true if the character was a digit in the range 0.9.

isxdigit(ch)

Returns true if the character was a valid hexadecimal digit: that is, a number from 0.9 or a letter a.f or A.F.

isspace(ch)

Returns true if the character was a white space character, that is: a space, a TAB character or a newline.

ispunct(ch)

Returns true if *ch* is a punctuation character.

isalnum(*ch*)

Returns true if a character is alphanumeric: that is, alphabetic or digit.

isprint(*ch*)

Returns true if the character is printable: that is, the character is not a control character.

isgraph(*ch*)

Returns true if the character is graphic. *i.e.* if the character is printable (excluding the space)

isctrl(*ch*)

Returns true if the character is a control character. *i.e.* ASCII values 0 to 31 and 127.

isascii(*ch*)

Returns true if the character is a valid ASCII character: that is, it has a code in the range 0.127.

iscsym(*ch*)

Returns true if the character was a character which could be used in a C identifier.

toupper(*ch*)

This converts the character *ch* into its upper case counterpart. This does not affect characters which are already upper case, or characters which do not have a particular case, such as digits.

tolower(*ch*)

This converts a character into its lower case counterpart. It does not affect characters which are already lower case.

toascii(*ch*)

This strips off bit 7 of a character so that it is in the range 0.127: that is, a valid ASCII character.

Examples:

```
/* Demonstration of character utility functions */
/* prints out all the ASCII characters which give */
/* the value "true" for the listed character fns */
#include <stdio.h>
#include <ctype.h> /* contains character utilities */
#define ALLCHARS ch = 0; isascii(ch); ch++
main () /* A criminally long main programme! */
{ char ch;
printf ("VALID CHARACTERS FROM isalpha()\n\n");
for (ALLCHARS)
{
    if (isalpha(ch))
    {
        printf ("%c ",ch);
    }
}
printf ("\n\nINVALID CHARACTERS FROM isupper()\n\n");
for (ALLCHARS)
{
    if (isupper(ch))
    {
        printf ("%c ",ch);
    }
}
```

```
    }
}
printf ("\n\nINVALID CHARACTERS FROM islower()\n\n");
for (ALLCHARS)
{
    if (islower(ch))
    {
        printf ("%c ",ch);
    }
}
printf ("\n\nINVALID CHARACTERS FROM isdigit()\n\n");
for (ALLCHARS)
{
    if (isdigit(ch))
    {
        printf ("%c ",ch);
    }
}
printf ("\n\nINVALID CHARACTERS FROM isxdigit()\n\n");
for (ALLCHARS)
{
    if (isxdigit(ch))
    {
        printf ("%c ",ch);
    }
}
printf ("\n\nINVALID CHARACTERS FROM ispunct()\n\n");
for (ALLCHARS)
{
    if (ispunct(ch))
    {
        printf ("%c ",ch);
    }
}
printf ("\n\nINVALID CHARACTERS FROM isalnum()\n\n");
for (ALLCHARS)
{
    if (isalnum(ch))
    {
        printf ("%c ",ch);
    }
}
printf ("\n\nINVALID CHARACTERS FROM iscsym()\n\n");
for (ALLCHARS)
{
    if (iscsym(ch))
    {
        printf ("%c ",ch);
    }
}
}
```

Programme output:

```
VALID CHARACTERS FROM isalpha:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z a b c d e f
g h i j k l m n o p q r s t u v w x y z
VALID CHARACTERS FROM isupper:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
VALID CHARACTERS FROM islower:
a b c d e f g h i j k l m n o p q r s t u v w x y z
VALID CHARACTERS FROM isdigit:
0 1 2 3 4 5 6 7 8 9
VALID CHARACTERS FROM isxdigit:
0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f
VALID CHARACTERS FROM ispunct:
! " # $ % & ' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~
VALID CHARACTERS FROM isalnum:
0 1 2 3 4 5 6 7 8 9 A B C D E F G H I J K L M N O P Q R S T U V
W X Y Z
a b c d e f g h i j k l m n o p q r s t u v w x y z
VALID CHARACTERS FROM iscsym()
0 1 2 3 4 5 6 7 8 9 A B C D E F G H I J K L M N O P Q R S T U V
W X Y Z _ a b c d e f g h i j k l m n o p q r s t u v w x y z
```

STRING MANIPULATION

The following functions perform useful functions for string handling,.

strcat()

This function “concatenates” two strings: that is, it joins them together into one string. The effect of:

```
char *new,*this, onto[255];
new = strcat(onto,this);
```

is to join the string this onto the string onto. New is a pointer to the complete string; it is identical to onto. Memory is assumed to have been allocated for the starting strings. The string which is to be copied to must be large enough to accept the new string, tagged onto the end. If it is not then unpredictable effects will result. (In some programmes the user might get away without declaring enough space for the “onto” string, but in general the results will be garbage, or even a crashed machine.) To join two static strings together, the following code is required:

```
char *s1 = "string one";
char *s2 = "string two";
main ()
{ char buffer[255];
  strcat(buffer,s1);
  strcat(buffer,s2);
}
```

buffer would then contain “string onestring two”.

strlen()

This function returns a type int value, which gives the length or number of characters in a string, not including the NULL byte end marker. An example is:


```
int len;
char *string;
len = strlen (string);
strcpy()
```

This function copies a string from one place to another. Use this function in preference to custom routines: it is set up to handle any peculiarities in the way data are stored. An example is

```
char *to,*from;
to = strcpy (to,from);
```

Where to is a pointer to the place to which the string is to be copied and from is the place where the string is to be copied from.

```
strcmp()
```

This function compares two strings and returns a value which indicates how they compared. An example:

```
int value;
char *s1,*s2;
value = strcmp(s1,s2);
```

The value returned is 0 if the two strings were identical. If the strings were not the same, this function indicates the (ASCII) alphabetical order of the two. $s1 > s2$, alphabetically, then the value is > 0 . If $s1 < s2$ then the value is < 0 . Note that numbers come before letters in the ASCII code sequence and also that upper case comes before lower case.

There are also variations on the theme of the functions above which begin with strn instead of str. These enable the programmer to perform the same actions with the first n characters of a string:

```
strncat()
```

This function *concatenate* two strings by copying the first n characters of this to the end of the onto string.

```
char *onto,*new,*this;
new = strncat(onto,this,n);
strncpy()
```

This function copies the first n characters of a string from one place to another

```
char *to,*from;
int n;
to = strncpy (to,from,n);
strncmp()
```

This function compares the first n characters of two strings

```
int value;
char *s1,*s2;
value = strcmp(s1,s2,n);
```

The following functions perform conversions between strings and floating point/integer types, without needing to use sscanf(). They take a pre-initialized string and work out the value represented by that string.

```
atof()
```

ASCII to floating point conversion.

```
double x;
char *stringptr;
x = atof(stringptr);
atoi()
    ASCII to integer conversion.
int i;
char *stringptr;
i = atoi(stringptr);
atol()
    ASCII to long integer conversion.
long i;
char *stringptr;
i = atol(stringptr);
```

MATHEMATICAL FUNCTIONS

C has a library of standard mathematical functions which can be accessed by #including the appropriate header files (math.h etc.). It should be noted that all of these functions work with double or long float type variables. All of C's mathematical capabilities are written for long variable types. Here is a list of the functions which can be expected in the standard library file.

The variables used are all to be declared long:

```
int i; /* long int */
double x,y,result; /* long float */
```

The functions themselves must be declared long float or double (which might be done automatically in the mathematics library file, or in a separate file) and any constants must be written in floating point form: for instance, write 7.0 instead of just 7.

ABS()

Macro: Returns the unsigned value of the value in parentheses. See fabs() for a function version.

fabs()

Find the absolute or unsigned value of the value in parentheses:

```
result = fabs(x);
ceil()
```

Find out what the ceiling integer is: that is, the integer which is just above the value in parentheses. This is like rounding up.

```
i = ceil(x);
/* ceil (2.2) is 3 */
floor()
```

Find out what the floor integer is: that is, the integer which is just below the floating point value in parentheses

```
i = floor(x);
/* floor(2.2) is 2 */
exp()
```

Find the exponential value.

```
result = exp(x);
```

```
result = exp(2.7);
log()
```

Find the natural (Naperian) logarithm. The value used in the parentheses must be unsigned: that is, it must be greater than zero. It does not have to be declared specifically as unsigned. *e.g.*

```
result = log(x);
result = log(2.71828);
log10()
```

Find the base 10 logarithm. The value used in the parentheses must be unsigned: that is, it must be greater than zero. It does not have to be declared specifically as unsigned.

```
result = log10(x);
result = log10(10000);
pow()
```

Raise a number to the power.

```
result = pow(x,y); /*raise x to the power y */
result = pow(x,2); /*find x-squared */
result = pow(2.0,3.2); /* find 2 to the power 3.2 */
sqrt()
```

Find the square root of a number.

```
result = sqrt(x);
result = sqrt(2.0);
sin()
```

Find the sine of the angle in radians.

```
result = sin(x);
result = sin(3.14);
cos()
```

Find the cosine of the angle in radians.

```
result = cos(x);
result = cos(3.14);
tan()
```

Find the tangent of the angle in radians.

```
result = tan(x);
result = tan(3.14);
asin()
```

Find the arcsine or inverse sine of the value which must lie between +1.0 and -1.0.

```
result = asin(x);
result = asin(1.0);
acos()
```

Find the arccosine or inverse cosine of the value which must lie between +1.0 and -1.0.

```
result = acos(x);
result = acos(1.0);
atan()
```

Find the arctangent or inverse tangent of the value.

```
result = atan(x);
result = atan(200.0);
atan2()
```

This is a special inverse tangent function for calculating the inverse tangent of x divided by y . This function is set up to find this result more accurately than `atan()`.

```
result = atan2(x,y);
result = atan2(x/3.14);
sinh()
```

Find the hyperbolic sine of the value. (Pronounced “shine” or “sinch”)

```
result = sinh(x);
result = sinh(5.0);
cosh()
```

Find the hyperbolic cosine of the value.

```
result = cosh(x);
result = cosh(5.0);
tanh()
```

Find the hyperbolic tangent of the value.

```
result = tanh(x);
result = tanh(5.0);
```

MATHS ERRORS

Mathematical functions can be delicate animals. There exist mathematical functions which simply cannot produce sensible answers in all possible cases. Mathematical functions are not “user friendly”! One example of an unfriendly function is the inverse sine function `asin(x)` which only works for values of x in the range $+1.0$ to -1.0 . The reason for this is a mathematical one: namely that the sine function (of which `asin()` is the opposite) only has values in this range. The statement

```
y = asin (25.3);
```

is non-sense and it cannot possibly produce a value for y , because none exists. Similarly, there is no simple number which is the square root of a negative value, so an expression such as:

```
x = sqrt (-2.0);
```

would also be non-sense. This doesn’t stop the programmer from writing these statements though and it doesn’t stop a faulty programme from straying out of bounds. What happens then when an erroneous statement is executed? Some sort of error condition would certainly have to result.

In many languages, errors, like the ones above, are terminal: they cause a programme to stop without any option to recover the damage. In C, as the reader might have come to expect, this is not the case. It is possible (in principle) to recover from any error, whilst still maintaining firm control of a programme.

Errors like the ones above are called domain errors (the set of values which a function can accept is called the domain of the function). There are other errors which can occur too.

For example, division by zero is illegal, because dividing by zero is “mathematical non-sense” - it can be done, but the answer can be all the numbers which exist at the same time! Obviously a programme cannot work with any idea as vague as this.

Finally, in addition to these “pathological” cases, mathematical operations can fail just because the numbers they deal with get too large for the computer to handle, or too small, as the case may be.

Domain error: Illegal value put into function

Division: By zero: Dividing by zero is non-sense.

Overflow: Number became too large

Underflow: Number became too small.

Loss of accuracy: No meaningful answer could be calculated

Errors are investigated by calling a function called `matherr()`. The mathematical functions, listed above, call this function automatically when an error is detected. The function responds by returning a value which gives information about the error. The exact details will depend upon a given compiler. For instance a hypothetical example: if the error could be recovered from, `matherr()` returns 0, otherwise it returns -1. `matherr()` uses a “struct” type variable called an “exception” to diagnose faults in mathematical functions. This can be examined by programmes which trap their errors dutifully. Information about this structure must be found in a given compiler manual.

Although it is not possible to generalize, the following remarks about the behaviour of mathematical functions may help to avoid any surprises about their behaviour in error conditions.

- A function which fails to produce a sensible answer, for any of the reasons above, might simply return zero or it might return the maximum value of the computer. Be careful to check this. (Division by zero and underflow probably return zero, whereas overflow returns the maximum value which the computer can handle.)
- Some functions return the value NaN. Not a form of Indian unleavened bread, this stands for ‘Not a Number’, *i.e.* no sensible result could be calculated.
- Some method of signalling errors must clearly be used. This is the exception structure (a special kind of C variable) which gives information about the last error which occurred. Find out what it is and trap errors!
- Obviously, wherever possible, the programmer should try to stop errors from occurring in the first place.

Example

Here is an example for the mathematically minded. The programme below performs numerical integration by the simplest possible method of adding up the area under small strips of a graph of the function $f(y) = 2*y$. The integral is found between the limits 0 and 5 and the exact answer is 25. (See diagram.) The particular compiler used for this programme returns the largest number which can be represented by the computer when numbers overflow, although, in this simple case, it is impossible for the numbers to overflow.

```
/* Numerical Estimation of Integral */
#include <stdio.h>
#include <math.h>
```

```

#include <limits.h>
#define LIMIT 5
double inc = 0.001; /* Increment width - arbitrary */
double twopi;
/** LEVEL 0 */
main ()
{ double y,integrand();
  double integral = 0;
  twopi = 4 * asin(1.0);
  for (y = inc/2; y < LIMIT; y += inc)
  {
    integral += integrand (y) * inc;
  }
  printf ("Integral value =%.10f \n",integral);
}
/** LEVEL 1 */
double integrand (y)
double y;
{ double value;
  value = 2*y;
  if (value > 1e308)
  {
    printf ("Overflow error\n");
    exit (0);
  }
  return (value);
}

```

HIDDEN OPERATORS AND VALUES

CONCISE EXPRESSIONS

Many operators in C are more versatile than they appear to be, at first glance. Take, for example, the following operators

`= ++ - += -= etc.`

the assignment, increment and decrement operators. These innocent looking operators can be used in some surprising ways which make C source code very neat and compact.

The first thing to notice is that `++` and `--` are unary operators: that is, they are applied to a single variable and they affect that variable alone. They therefore produce one unique value each time they are used. The assignment operator, on the other hand, has the unusual position of being both unary, in the sense that it works out only one expression, and also binary or dyadic because it sits between two separate objects: an “lvalue” on the left hand side and an expression on the right hand side. Both kinds of operator have one thing in common however: both form statements which have values in their own right.

What does this mean? It means that certain kinds of statement, in C, do not have to be thought of as being complete and sealed off from the rest of a programme. To paraphrase a famous author: “In C, no statement is an island”.

A statement can be taken as a whole (as a “black box”) and can be treated as a single value, which can be assigned and compared to things!

The value of a statement is the result of the operation which was carried out in the statement.

Increment/decrement operator statements, taken as a whole, have a value which is one greater / or one less than the value of the variable which they act upon. So:

```
c = 5;
```

```
c++;
```

The second of these statement `c++`; has the value 6, and similarly:

```
c = 5;
```

```
c--;
```

The second of these statements `c--`; has the value 4. Entire assignment statements have values too. A statement such as:

```
c = 5;
```

has the value which is the value of the assignment. So the example above has the value 5. This has some important implications.

EXTENDED AND HIDDEN =

The idea that assignment statement has a value, can be used to make C programmes neat and tidy for one simple reason: it means that a whole assignment statement can be used in place of a value. For instance, the value `c = 0`; could be assigned to a variable `b`:

```
b = (c = 0);
```

or simply:

```
b = c = 0;
```

These equivalent statements set `b` and `c` to the value zero, provided `b` and `c` are of the same type! It is equivalent to the more usual:

```
b = 0;
```

```
c = 0;
```

Indeed, any number of these assignments can be strung together:

```
a = (b = (c = (d = (e = 5))))
```

or simply:

```
a = b = c = d = e = 5;
```

This very neat syntax compresses five lines of code into one single line! There are other uses for the valued assignment statement, of course: it can be used anywhere where a value can be used.

For instance:

- In other assignments (as above)
- As a parameter for functions
- Inside a comparison (`== > <` etc.)
- As an index for arrays.

The uses are manifold. Consider how an assignment statement might be used as a parameter to a function. The function below gets a character from the input stream stdin and passes it to a function called Process Character ():

```
Process Character (ch = getchar());
```

This is a perfectly valid statement in C, because the hidden assignment statement passes on the value which it assigns. The actual order of events is that the assignment is carried out first and then the function is called. It would not make sense the other way around, because, then there would be no value to pass on as a parameter. So, in fact, this is a more compact way of writing:

```
ch = getchar();  
Process Character (ch);
```

The two methods are entirely equivalent. If there is any doubt, examine a little more of this imaginary character processing programme:

```
Process Character(ch = getchar());  
if (ch == '*')  
{  
    printf ("Starry, Starry Night.");  
}
```

The purpose in adding the second statement is to impress the fact that ch has been assigned quite legitimately and it is still defined in the next statement and the one after. Until it is reassigned by a new assignment statement.

The fact that the assignment was hidden inside another statement does not make it any less valid. All the same remarks apply about the specialized assignment operators +=, *=, /= etc.

HIDDEN ++ AND –

The increment and decrement operators also form statements which have intrinsic values and, like assignment expressions, they can be hidden away in inconspicuous places. These two operators are slightly more complicated than assignments because they exist in two forms: as a postfix and as a prefix:

Postfix	Prefix
var++	++var
var–	–var

and these two forms have subtly different meanings. Look at the following example:

```
int i = 3;  
PrintNumber (i++);
```

The increment operator is hidden in the parameter list of the function PrintNumber(). This example is not as clear cut as the assignment statement examples however, because the variable i has, both a value before the ++ operator acts upon it, and a different value afterwards.

The question is then: which value is passed to the function? Is i incremented before or after the function is called? The answer is that this is where the two forms of the operator come into play.

If the operator is used as a prefix, the operation is performed *before* the function call. If the operator is used as a postfix, the operation is performed after the function call.

In the example above, then, the value 3 is passed to the function and when the function returns, the value of *i* is incremented to 4. The alternative is to write:

```
int i = 3;
PrintNumber (++i);
```

in which case the value 4 is passed to the function `PrintNumber()`. The same remarks apply to the decrement operator.

ARRAYS, STRINGS AND HIDDEN OPERATORS

Arrays and strings are one area of programming in which the increment and decrement operators are used a lot. Hiding operators inside array subscripts or hiding assignments inside loops can often make light work of tasks such as initialization of arrays. Consider the following example of a one dimensional array of integers.

```
#define SIZE 20
int i, array[SIZE];
for (i = 0; i < SIZE; array[i++] = 0)
{
}
```

This is a neat way of initializing an array to zero. Notice that the post fixed form of the increment operator is used. This prevents the element `array[0]` from assigning zero to memory which is out of the bounds of the array.

Strings too can benefit from hidden operators. If the standard library function `strlen()` (which finds the length of a string) were not available, then it would be a simple matter to write the function;

```
strlen (string) /* count the characters in a string */
char *string;
{ char *ptr;
  int count = 0;
  for (ptr = string; *(ptr++) != '\0'; count++)
  {
  }
  return (count);
}
```

This function increments count while the end of string marker `\0` is not found.

Example:

```
/* Hidden Operator Demo */
/* Any assignment or increment operator has a value */
/* which can be handed straight to printf(). */
/* Also compare the prefix / postfix forms of ++/-- */
#include <stdio.h>
main ()
{ int a,b,c,d,e;
  a = (b = (c = (d = (e = 0)))));
```

```

printf ("%d %d %d %d %d\n", a, b++, c--, d = 10, e + = 3);
a = b = c = d = e = 0;
printf ("%d %d %d %d %d\n", a, ++b, -c, d = 10, e + = 3);
}

/* end */
/* Hidden Operator demo #2 */
#include <stdio.h>
main () /* prints out zero! */
{
printf ("%d", Value());
}
Value() /* Check for zero. */
{ int value;
if ((value = GetValue()) == 0)
{
printf ("Value was zero\n");
}
return (value);
}
GetValue() /* Some function to get a value */
{
return (0);
}

/* end */

```

CAUTIONS ABOUT STYLE

Hiding operators away inside other statements can certainly make programmes look very elegant and compact, but, as with all neat tricks, it can make programmes harder to understand. Never forget that programming is communication to other programmers and be kind to the potential reader of a programme. (It could be you in years or months to come!)

Statements such as:

```

if ((i = (int)ch++) <= -comparison)
{
}

```

are not recommendable programming style and they are no more efficient than the more longwinded:

```

ch++;
i = (int)ch;
if (i <= comparison)
{
}
comparison--;

```

There is always a happy medium in which to settle on a readable version of the code. The statement above might perhaps be written as:

```

i = (int) ch++;
if (i <= -comparison)

```

```
{
}
```

Example:

```
/* Arrays and Hidden Operators */
#include <stdio.h>
#define SIZE 10
/* Level 0 */
main () /* Demo prefix and postfix ++ in arrays */
{ int i, array[SIZE];
  Initialize(array);
  i = 4;
  array[i++] = 8;
  Print (array);
  Initialize(array);
  i = 4;
  array[++i] = 8;
  Print(array);
}
/* Level 1 */
Initialize (array) /* set to zero */
int array[SIZE];
{ int i;
  for (i = 0; i < SIZE; array[i++] = 0)
  {
  }
}
Print (array) /* to stdout */
int array[SIZE];
{ int i = 0;
  while (i < SIZE)
  {
    printf ("%2d",array[i++]);
  }
  putchar ('\n');
}

/* end */
/* Hidden Operator */
#include <stdio.h>
#define MAXNO 20
main () /* Print out 5 x table */
{ int i, ctr = 0;
  for (i = 1; ++ctr <= MAXNO; i = ctr*5)
  {
    printf ("%3d",i);
  }
}
```

DATA TYPES

Since C allows you to define new data types we shall not be able to cover all of the possibilities, only the most important examples. The most important of these are

File: The type which files are classified under.

Enum: Enumerated type for abstract data.

Void: The “empty” type.

Volatile: New ANSI standard type for memory mapped I/O.

Const: New ANSI standard type for fixed data.

Struct: Groups of variables under a single name.

Union: Multi-purpose storage areas for dynamical memory allocation.

SPECIAL CONSTANT EXPRESSIONS

Constant expressions are often used without any thought, until a programmer needs to know how to do something special with them. Up to now the distinction between long and short integer types has largely been ignored. Constant values can be declared explicitly as long values, in fact, by placing the letter L after the constant.

```
long int variable = 23L;
variable = 236526598L;
```

Advanced programmers, writing systems software, often find it convenient to work with hexadecimal or octal numbers since these number bases have a special relationship to binary. A constant in one of these types is declared by placing either 0 (zero) or 0x in front of the appropriate value. If *ddd* is a value, then:

```
Octal number 0ddd
Hexadecimal number 0xddd
```

For example:

```
oct_value = 077; /* 77 octal */
hex_value = 0xFFEF; /* FFEF hex */
```

This kind of notation has already been applied to strings and single character constants with the backslash notation, instead of the leading zero character:

```
ch = '\ddd';
ch = '\xdd';
```

The values of character constants, like these, cannot be any greater than 255.

File: In all previous sections, the files stdin, stdout and stderr alone have been used in programmes.

These special files are always handled implicitly by functions like `printf()` and `scanf()`: The programmer never gets to know that they are, in fact, files. Programmes do not have to use these functions however: standard input/output files can be treated explicitly by general file handling functions just as well. Files are distinguished by file names and by file pointers. File pointers are variables which pass the location of files to file handling functions; being variables, they have to be declared as being some data type. That type is called `FILE` and file pointers have to be declared “pointer to `FILE`”.

For example:

```
FILE *fp;
FILE *fp = stdin;
FILE *fopen();
```

File handling functions which return file pointers must also be declared as pointers to files. Notice that, in contrast to all the other reserved words `FILE` is written in upper case: the reason for this is that `FILE` is not a simple data type such as `char` or `int`, but a *structure* which is only defined by the header file `stdio.h` and so, strictly speaking, it is not a reserved word itself. We shall return to look more closely at files soon.

Enum: Abstract data are usually the realm of exclusively high level languages such as Pascal. `enum` is a way of incorporating limited “high level” data facilities into C.

Enum is short for enumerated data: The user defines a type of data which is made up of a fixed set of words, instead of numbers or characters. These words are given substitute integer numbers by the compiler which are used to identify and compare `enum` type data. For example:

```
enum countries
{
    England,
    Scotland,
    Wales,
    Eire,
    Norge,
    Sverige,
    Danmark,
    Deutschland
};
main ()
{ enum countries variable;
  variable = England;
}
```

Example:

```
/* Enumerated Data */
#include <stdio.h>
enum countries
{
    England,
    Ireland,
    Scotland,
    Wales,
    Danmark,
    Island,
    Norge,
    Sverige
};
main () /* Electronic Passport Programme */
```

```

{ enum countries birthplace, getinfo();
printf ("Insert electronic passport\n");
birthplace = getinfo();
switch (birthplace)
{
    case England: printf ("Welcome home!\n");
    break;
    case Danmark:
    case Norge:      printf ("Velkommen til
England\n");
                    break;
}
}
enum countries getinfo() /* interrogate
    passport */
{
return (England);
}

/* end */

```

enum makes words into constant integer values for a programmer. Data which are declared enum are not the kind of data which it makes sense to do arithmetic with (even integer arithmetic), so in most cases it should not be necessary to know or even care about what numbers the compiler gives to the words in the list.

However, some compilers allow the programmer to force particular values on words. The compiler then tries to give the values successive integer numbers unless the programmer states otherwise.

For instance:

```

enum planets
{
Mercury,
Venus,
Earth = 12,
Mars,
Jupiter,
Saturn,
Uranus,
Neptune,
Pluto
};

```

This would probably yield values Mercury = 0, Venus = 1, Earth = 12, Mars = 13, Jupiter = 14 . etc. If the user tries to force a value which the compiler has already used then the compiler will complain.

The following example programme listing shows two points:

- Enum types can be local or global.
- The labels can be forced to have certain values

MACHINE LEVEL OPERATIONS

BITS AND BYTES

Down in the depths of your computer, below even the operating system are bits of memory. These days we are used to working at such a high level that it is easy to forget them. Bits (or binary digits) are the lowest level software objects in a computer: there is nothing more primitive. For precisely this reason, it is rare for high level languages to even acknowledge the existence of bits, let alone manipulate them. Manipulating bit patterns is usually the preserve of assembly language programmers. C, however, is quite different from most other high level languages in that it allows a programmer full access to bits and even provides high level operators for manipulating them.

BIT PATTERNS

- All computer data, of any type, are bit patterns. The only difference between a string and a floating point variable is the way in which we choose to interpret the patterns of bits in a computer's memory. For the most part, it is quite unnecessary to think of computer data as bit patterns; systems programmers, on the other hand, frequently find that they need to handle bits directly in order to make efficient use of memory when using flags. A flag is a message which is either one thing or the other: in system terms, the flag is said to be 'on' or 'off' or alternatively *set* or *cleared*. The usual place to find flags is in a status register of a CPU (central processor unit) or in a pseudo-register (this is a status register for an imaginary processor, which is held in memory). A status register is a group of bits (a byte perhaps) in which each bit signifies something special. In an ordinary byte of data, bits are grouped together and are interpreted to have a collective meaning; in a status register they are thought of as being independent. Programmers are interested to know about the contents of bits in these registers, perhaps to find out what happened in a programme after some special operation is carried out.

FLAGS, REGISTERS AND MESSAGES

A *register* is a place inside a computer processor chip, where data are worked upon in some way. A *status register* is a register which is used to return information to a programmer about the operations which took place in other registers.

Status registers contain flags which give yes or no answers to questions concerning the other registers. In advanced programming, there may be call for "pseudo registers" in addition to "real" ones.

A pseudo register is merely a register which is created by the programmer in computer memory (it does not exist inside a processor). Messages are just like pseudo status

registers: they are collections of flags which signal special information between different devices and/or different programmes in a computer system.

Messages do not necessarily have fixed locations: they may be passed a parameters. Messages are a very compact way of passing information to low level functions in a programme. Flags, registers, pseudo-registers and messages are all treated as bit patterns.

A programme which makes use of them must therefore be able to assign these objects to C variables for use. A bit pattern would normally be declared as a character or some kind of integer type in C, perhaps with the aid of a typedef statement.

```
typedef char byte;
typedef int bitpattern;
bitpattern variable;
byte message;
```

The flags or bits in a register/message. have the values 1 or 0, depending upon whether they are on or off (set or cleared). A programme can test for this by using combinations of the operators which C provides.

BIT OPERATORS AND ASSIGNMENTS

C provides the following operators for handling bit patterns:

<< Bit shift left (a specified number or bit positions)

>> Bit shift right (a specified number of bit positions)

| Bitwise Inclusive OR

^ Bitwise Exclusive OR

& Bitwise AND

~ Bitwise one's complement

&= And assign (variable = variable & value)

|= Exclusive OR assign (variable = variable | value)

^= Inclusive OR assign (variable = variable ^ value)

>>= Shift right assign (variable = variable >> value)

<<= Shift left assign (variable = variable << value)

BIT OPERATORS

Bitwise operations are not to be confused with logical operations (&&, ||.) A bit pattern is made up of 0s and 1s and bitwise operators operate individually upon each bit in the operand. Every 0 or 1 undergoes the operations individually. Bitwise operators (AND, OR) can be used in place of logical operators (&&,||), but they are less efficient, because logical operators are designed to reduce the number of comparisons made, in an expression, to the optimum: as soon as the truth or falsity of an expression is known, a logical comparison operator quits. A bitwise operator would continue operating to the last before the final result were known.

Below is a brief summary of the operations which are performed by the above operators on the bits of their operands.

SHIFT OPERATIONS

Imagine a bit pattern as being represented by the following group of boxes. Every box represents a bit; the numbers inside represent their values. The values written over the top are the common integer values which the whole group of bits would have, if they were interpreted collectively as an integer.

128	64	32	16	8	4	2	1
0	0	0	0	0	0	0	1

= 1

Shift operators move whole bit patterns left or right by shunting them between boxes.

The syntax of this operation is:

value << **number of positions**

value >> **number of positions**

So for example, using the boxed value (1) above:

1 << 1

would have the value 2, because the bit pattern would have been moved one place the left:

128	64	32	16	8	4	2	1
0	0	0	0	0	0	1	0

= 2

Similarly:

1 << 4

has the value 16 because the original bit pattern is moved by four places:

128	64	32	16	8	4	2	1
0	0	0	1	0	0	0	0

= 16

And:

6 << 2 == 12

128	64	32	16	8	4	2	1
0	0	0	0	0	1	1	0

= 6

Shift left 2 places:

128	64	32	16	8	4	2	1
0	0	0	0	1	1	0	0

= 12

Notice that every shift left multiplies by 2 and that every shift right would divide by two, integerwise. If a bit reaches the edge of the group of boxes then it falls out and is lost forever.

So:

1 >> 1 == 0

2 >> 1 == 1

2 >> 2 == 0

n >> n == 0

A common use of shifting is to scan through the bits of a bitpattern one by one in a loop: this is done by using *masks*.

TRUTH TABLES AND MASKING

The operations AND, OR (inclusive OR) and XOR/EOR (exclusive OR) perform comparisons or “masking” operations between two bits. They are binary or dyadic operators. Another operation called COMPLEMENT is a unary operator. The operations performed by these bitwise operators are best summarized by *truth tables*.

Truth tables indicate what the results of all possible operations are between two single bits. The same operation is then carried out for all the bits in the variables which are operated upon.

Complement ~

The complement of a number is the *logical opposite* of the number. C provides a “one’s complement” operator which simply changes all 1s into 0s and all 0s into 1s.

~1 has the value 0 (for each bit)

~0 has the value 1

As a truth table this would be summarized as follows:

~value	== result
0	1
1	0

AND &

This works between two values. *e.g.* (1 & 0)

value 1 &	value 2	== result
0	0	0
0	1	0
1	0	0
1	1	1

Both value 1 AND value 2 have to be 1 in order for the result or be 1.

OR |

This works between two values. *e.g.* (1 | 0)

value 1 	value 2	== result
0	0	0
0	1	1
1	0	1
1	1	1

The result is 1 if one OR the other OR both of the values is 1.

XOR/EOR ^

Operates on two values. *e.g.* (1 ^ 0)

value 1 ^	value 2	== result
0	0	0
0	1	1
1	0	1
1	1	0

The result is 1 if one OR the other (but not both) of the values is 1.

Bit patterns and logic operators are often used to make *masks*. A mask is as a thing which fits over a bit pattern and modifies the result in order perhaps to single out particular bits, usually to cover up part of a bit pattern.

This is particularly pertinent for handling flags, where a programmer wishes to know if one particular flag is set or not set and does not care about the values of the others.

This is done by deliberately inventing a value which only allows the particular flag of interest to have a non-zero value and then ANDing that value with the flag register. For example: in symbolic language:

```
MASK = 00000001
VALUE1 = 10011011
VALUE2 = 10011100
MASK & VALUE1 == 00000001
MASK & VALUE2 == 00000000
```

The zeros in the mask *masks off* the first seven bits and leave only the last one to reveal its true value. Alternatively, masks can be built up by specifying several flags:

```
FLAG1 = 00000001
FLAG2 = 00000010
FLAG3 = 00000100
MESSAGE = FLAG1 | FLAG2 | FLAG3
MESSAGE == 00000111
```

It should be emphasized that these expressions are only written in symbolic language: it is not possible to use binary values in C. The programmer must convert to hexadecimal, octal or denary first.

Example

A simple example helps to show how logical masks and shift operations can be combined. The first programme gets a denary number from the user and converts it into binary. The second programme gets a value from the user in binary and converts it into hexadecimal.

```
/* Bit Manipulation #1 */
/* Convert denary numbers into binary */
/* Keep shifting i by one to the left */
/* and test the highest bit. This does*/
/* NOT preserve the value of i */
#include <stdio.h>
#define NUMBEROFBITS 8
main ()
{ short i,j,bit;;
  short MASK = 0x80;
  printf ("Enter any number less than 128: ");
  scanf ("%h", &i);
  if (i > 128)
  {
    printf ("Too big\n");
    return (0);
  }
  printf ("Binary value = ");
  for (j = 0; j < NUMBEROFBITS; j++)
```

```

    {
        bit = i & MASK;
        printf ("%ld",bit/MASK);
        i <<= 1;
    }
printf ("\n");
}

/* end */

```

Output:

```

Enter any number less than 128: 56
Binary value = 00111000
Enter any value less than 128: 3
Binary value = 00000011

```

Example:

```

/* Bit Manipulation #2 */
/* Convert binary numbers into hex */
#include <stdio.h>
#define NUMBEROFBITS 8
main ()
{ short j,hex = 0;
  short MASK;
  char binary[NUMBEROFBITS];
printf ("Enter an 8-bit binary number: ");
for (j = 0; j < NUMBEROFBITS; j++)
{
    binary[j] = getchar();
}
for (j = 0; j < NUMBEROFBITS; j++)
{
    hex <<= 1;
    switch (binary[j])
    {
        case '1': MASK = 1;
        break;
        case '0': MASK = 0;
        break;
        default:    printf("Not binary\n");
                    return(0);
    }
    hex |= MASK;
}
printf ("Hex value = %1x\n",hex);
}

/* end */

```

Example:

```

Enter any number less than 128: 56
Binary value = 00111000
Enter any value less than 128: 3
Binary value = 00000011

```

FILES AND DEVICES

Files are places for reading data from or writing data to. This includes disk files and it includes devices such as the printer or the monitor of a computer. C treats all information which enters or leaves a programme as though it were a stream of bytes: a file. The most commonly used file streams are `stdin` (the keyboard) and `stdout` (the screen), but more sophisticated programmes need to be able to read or write to files which are found on a disk or to the printer etc.

An operating system allows a programme to see files in the outside world by providing a number of channels or ‘portals’ (‘inlets’ and ‘outlets’) to work through. In order to examine the contents of a file or to write information to a file, a programme has to *open* one of these portals. The reason for this slightly indirect method of working is that channels/portals hide operating system dependent details of filing from the programmer.

Think of it as a protocol. A programme which writes information does no more than pass that information to one of these portals and the operating system’s filing subsystem does the rest. A programme which reads data simply reads values from its file portal and does not have to worry about how they got there. This is extremely simple to work in practice.

To use a file then, a programme has to go through the following routine:

- Open a file for reading or writing.
- Read or write to the file using file handling functions provided by the standard library.
- Close the file to free the operating system “portal” for use by another programme or file.

A programme opens a file by calling a standard library function and is returned a file pointer, by the operating system, which allows a programme to address that particular file and to distinguish it from all others.

FILES GENERALLY

C provides two levels of file handling; these can be called high level and low level. High level files are all treated as text files. In fact, the data which go into the files are exactly what would be seen on the screen, character by character, except that they are stored in a file instead. This is true whether a file is meant to store characters, integers, floating point types. Any file, which is written to by high level file handling functions, ends up as a text file which could be edited by a text editor.

High level text files are also read back as character files, in the same way that input is acquired from the keyboard. This all means that high level file functions are identical in concept to keyboard/screen input/output.

The alternative to these high level functions, is obviously low level functions. These are more efficient, in principle, at filing data as they can store data in large lumps, in

raw memory format, without converting to text files first. Low level input/output functions have the disadvantage that they are less ‘programmer friendly’ than the high level ones, but they are likely to work faster.

FILE POSITIONS

When data are read from a file, the operating system keeps track of the current position of a programme within that file so that it only needs to make a standard library call to ‘read the next part of the file’ and the operating system obliges by reading some more and advancing its position within the file, until it reaches the end. Each single character which is read causes the position in a file to be advanced by one.

Although the operating system does a great deal of hand holding regarding file positions, a programme can control the way in which that position changes with functions such as `ungetc()` if need be. In most cases it is not necessary and it should be avoided, since complex movements within a file can cause complex movements of a disk drive mechanism which in turn can lead to wear on disks and the occurrence of errors.

HIGH LEVEL FILE HANDLING FUNCTIONS

Most of the high level input/output functions which deal with files are easily recognizable in that they start with the letter ‘f’. Some of these functions will appear strikingly familiar.

For instance:

```
fprintf()  
fscanf()  
fgets()  
fputs()
```

These are all generalized file handling versions of the standard input/output library. They work with generalized files, as opposed to the specific files `stdin` and `stdout` which `printf()` and `scanf()` use.

The file versions differ only in that they need an extra piece of information: the file pointer to a particular portal. This is passed as an extra parameter to the functions. They process data in an identical way to their standard I/O counterparts. Other filing functions will not look so familiar.

For example:

```
fopen()  
fclose()  
getc()  
ungetc();  
putc()  
fgetc()  
fputc()  
feof()
```

OPENING FILES

A file is opened by a call to the library function `fopen()`: this is available automatically when the library file `<stdio.h>` is included. There are two stages to opening a file: firstly a file portal must be found so that a programme can access information from a file at all.

Secondly the file must be physically located on a disk or as a device or whatever. The `fopen()` function performs both of these services and, if, in fact, the file it attempts to open does not exist, that file is created anew. The syntax of the `fopen()` function is:

```
FILE *returnpointer;
returnpointer = fopen("filename", "mode");
or
FILE returnpointer;
char *fname, *mode;
returnpointer = fopen(fname, mode);
```

The file name is a string which provides the name of the file to be opened. File names are system dependent so the details of this must be sought from the local operating system manual. The operation mode is also a string, chosen from one of the following:

- r Open file for reading
- w Open file for writing
- a Open file for appending
- rw Open file for reading and writing (some systems)

This mode string specifies the way in which the file will be used. Finally, `returnpointer` is a pointer to a `FILE` structure which is the whole object of calling this function. If the file (which was named) opened successfully when `fopen()` was called, `returnpointer` is a pointer to the file portal. If the file could not be opened, this pointer is set to the value `NULL`. This should be tested for, because it would not make sense to attempt to write to a file which could not be opened or created, for whatever reason.

A read only file is opened, for example, with some programme code such as:

```
FILE *fp;
if ((fp = fopen ("filename", "r")) == NULL)
{
    printf ("File could not be opened\n");
    error_handler();
}
```

A question which springs to mind is: what happens if the user has to type in the name of a file while the programme is running? The solution to this problem is quite simple. Recall the function `filename()` which was written in chapter 20.

```
char *filename()      /* return filename */
{ static char *filenm = "..";
do
{
    printf ("Enter filename:");
    scanf ("%24s", filenm);
    skipgarb();
```

```

    }
    while (strlen(filename) == 0);
    return (filename);
}

```

This function makes file opening simple. The programmer would now write something like:

```

FILE *fp;
char *filename();
if ((fp = fopen (filename(),"r")) == NULL)
{
    printf ("File could not be opened\n");
    error_handler();
}

```

and then the user of the programme would automatically be prompted for a filename. Once a file has been opened, it can be read from or written to using the other library functions (such as `fprintf()` and `fscanf()`) and then finally the file has to be closed again.

CLOSING A FILE

A file is closed by calling the function `fclose()`. `fclose()` has the syntax:

```

int returncode;
FILE *fp;
returncode = fclose (fp);

```

`fp` is a pointer to the file which is to be closed and `returncode` is an integer value which is 0 if the file was closed successfully. `fclose()` prompts the file manager to finish off its dealings with the named file and to close the portal which the operating system reserved for it. When closing a file, a programme needs to do something like the following:

```

if (fclose(fp) != 0)
{
    printf ("File did not exist.\n");
    error_handler();
}

```

`fprintf()`

This is the highest level function which writes to files. Its name is meant to signify “file-print-formatted” and it is almost identical to its stdout counterpart `printf()`. The form of the `fprintf()` statement is as follows:

```

fprintf (fp,"string",variables);

```

where `fp` is a file pointer, `string` is a control string which is to be formatted and the variables are those which are to be substituted into the blank fields of the format string.

For example, assume that there is an open file, pointed to by `fp`:

```

int i = 12;
float x = 2.356;
char ch = 's';
fprintf (fp, "%d %f %c", i, x, ch);

```


The conversion specifiers are identical to those for `printf()`. In fact `fprintf()` is related to `printf()` in a very simple way: the following two statements are identical.

```
printf ("Hello world %d", 1);
fprintf (stdout,"Hello world %d", 1);
fscanf ()
```

The analogue of `scanf()` is `fscanf()` and, as with `fprintf()`, this function differs from its standard I/O counterpart only in one extra parameter: a file pointer. The form of an `fscanf()` statement is:

```
FILE *fp;
int n;
n = fscanf (fp,"string",pointers);
```

where `n` is the number of items matched in the control string and `fp` is a pointer to the file which is to be read from. For example, assuming that `fp` is a pointer to an open file:

```
int i = 10;
float x = -2.356;
char ch = 'x';
fscanf (fp, "%d %f %c", &i, &x, &ch);
```

The remarks which were made about `scanf()` also apply to this function: `fscanf()` is a 'dangerous' function in that it can easily get out of step with the input data unless the input is properly formatted.

For comparison alone, `skipfilegarb()` is written below.

```
skipfilegarb(fp)
FILE *fp;
{
  while (getc(fp) != '\n')
  {
  }
}
```

SINGLE CHARACTER I/O

There are commonly four functions/macros which perform single character input/output to or from files. They are analogous to the functions/macros

```
getchar()
putchar()
```

for the standard I/O files and they are called:

```
getc()
ungetc();
putc()
fgetc()
fputc()
getc() and fgetc()
```

The difference between `getc()` and `fgetc()` will depend upon a particular system. It might be that `getc()` is implemented as a macro, whereas `fgetc()` is implemented as a function or vice versa. One of these alternatives may not be present at all in a library. Check the manual, to be sure! Both `getc()` and `fgetc()` fetch a single character from a file:

```
FILE *fp;
char ch;
/* open file */
ch = getc (fp);
ch = fgetc (fp);
```

These functions return a character from the specified file if they operated successfully, otherwise they return EOF to indicate the end of a file or some other error. Apart from this, these functions/macros are quite unremarkable.

ungetc()

ungetc() is a function which ‘un-gets’ a character from a file. That is, it reverses the effect of the last get operation. This is not like writing to a file, but it is like stepping back one position within the file. The purpose of this function is to leave the input in the correct place for other functions in a programme when other functions go too far in a file. An example of this would be a programme which looks for a word in a text file and processes that word in some way.

```
while (getc(fp) != ``)
{
}
```

The programme would skip over spaces until it found a character and then it would know that this was the start of a word. However, having used getc() to read the first character of that word, the position in the file would be the second character in the word! This means that, if another function wanted to read that word from the beginning, the position in the file would not be correct, because the first character would already have been read. The solution is to use ungetc() to move the file position back a character:

```
int returncode;
returncode = ungetc(fp);
```

The returncode is EOF if the operation was unsuccessful.

putc() and fputc()

These two functions write a single character to the output file, pointed to by fp. As with getc(), one of these may be a macro. The form of these statements is:

```
FILE *fp;
char ch;
int returncode;
returncode = fputc (ch,fp);
returncode = putc (ch,fp);
```

The returncode is the ascii code of the character sent, if the operation was successful, otherwise it is EOF.

fgets() and fputs()

Just as gets() and puts() fetched and sent strings to standard input/output files stdin and stdout, so fgets() and fputs() send strings to generalized files. The form of an fgets() statement is as follows:

```
char *strbuff,*returnval;
int n;
FILE *fp;
returnval = fgets (strbuff,n,fp);
```

strbuff is a pointer to an input buffer for the string; fp is a pointer to an open file. Returnval is a pointer to a string: if there was an error in fgets() this pointer is set to the value NULL, otherwise it is set to the value of "strbuff". No more than (n-1) characters are read by fgets() so the programmer has to be sure to set n equal to the size of the string buffer. (One byte is reserved for the NULL terminator.) The form of an fputs() statement is as follows:

```
char *str;
int returnval;
FILE *fp;
returnval = fputs (str,fp);
```

Where str is the NULL terminated string which is to be sent to the file pointed to by fp. returnval is set to EOF if there was an error in writing to the file.

PRINTER OUTPUT

Any serious application programme will have to be in full control of the output of a programme. For instance, it may need to redirect output to the printer so that data can be made into hard copies. To do this, one of three things must be undertaken:

- Stdout must be redirected so that it sends data to the printer device.
- A new "standard file" must be used (not all C compilers use this method.)
- A new file must be opened in order to write to the printer device

The first method is not generally satisfactory for applications programmes, because the standard files stdin and stdout can only easily be redirected from the operating system command line interpreter (when a programme is run by typing its name). Examples of this are:

```
type file > PRN
```

which send a text file to the printer device. The second method is reserved for only a few implementations of C in which another 'standard file' is opened by the local operating system and is available for sending data to the printer stream. This file might be called "stdprn" or "standard printer file" and data could be written to the printer by switching writing to the file like this:

```
fprintf (stdprn,"string %d.", integer);
```

The final method of writing to the printer is to open a file to the printer, personally. To do this, a programme has to give the "filename" of the printer device. This could be something like "PRT:" or "PRN" or "LPRT" or whatever. The filename (actually called a pseudo device name) is used to open a file in precisely the same way as any other file is opened: by using a call to fopen(). fopen() then returns a pointer to file (which is effectively "stdprn") and this is used to write data to a computer's printer driver. The programme code to do this should look something like the following:

```
FILE *stdprn;
if ((stdprn = fopen("PRT:", "w")) == NULL)
{
    printf ("Printer busy or disconnected\n");
    error_handler;
}
```

LOW LEVEL FILING OPERATIONS

Normally a programmer can get away with using the high level input/output functions, but there may be times when C's predilection for handling all high level input/output as text files, becomes a nuisance. A programme can then use a set of low level I/O functions which are provided by the standard library.

These are:

```
open()  
close()  
creat()  
read()  
write()  
rename()  
unlink()/remove()  
lseek()
```

These low level routines work on the operating system's end of the file portals. They should be regarded as being advanced features of the language because they are dangerous routines for bug ridden programmes. The data which they deal with is untranslated: that is, no conversion from characters to floating point or integers or any type at all take place. Data are treated as a raw stream of bytes. Low level functions should not be used on any file at the same time as high level routines, since high level file handling functions often make calls to the low level functions. Working at the low level, programmes can create, delete and rename files but they are restricted to the reading and writing of untranslated data: there are no functions such as `fprintf()` or `fscanf()` which make type conversions.

As well as the functions listed above a local operating system will doubtless provide special function calls which enable a programmer to make the most of the facilities offered by the particular operating environment. These will be documented, either in a compiler manual, or in an operating system manual, depending upon the system concerned.

FILE DESCRIPTORS

At the low level, files are not handled using file pointers, but with integers known as file handles or file descriptors. A file handle is essentially the number of a particular file portal in an array. In other words, for all the different terminology, they describe the same thing. For example:

int fd;

Would declare a file *handle* or *descriptor* or *portal* or whatever it is to be called.

open()

Open() is the low level file open function. The form of this function call is:

```
int fd, mode;
```

```
char *filename;
fd = open (filename, mode);
```

where filename is a string which holds the name of the file concerned, mode is a value which specifies what the file is to be opened for and fd is either a number used to distinguish the file from others, or -1 if an error occurred.

A programme can give more information to this function than it can to fopen() in order to define exactly what open() will do. The integer *mode* is a message or a pseudo register which passes the necessary information to open(), by using the following flags:

```
O_RDONLY    Read access only
O_WRONLY    Write access only
O_RDWR     Read/Write access
```

and on some compilers:

```
O_CREAT Create the file if it does not exist
O_TRUNC Truncate the file if it does exist
O_APPEND Find the end of the file before each
write
O_EXCL Exclude. Force create to fail if the file exists.
```

The macro definitions of these flags will be included in a library file: find out which one and #include it in the programme. The normal procedure is to open a file using one of the first three modes.

For example:

```
#define FAILED -1
main()
{ char *filename();
  int fd;
  fd = open(filename(), O_RDONLY);
  if (fd == FAILED)
  {
    printf ("File not found\n");
    error_handler (failed);
  }
}
```

This opens up a read-only file for low level handling, with error checking. Some systems allow a more flexible way of opening files. The four appended modes are values which can be bitwise ORed with one of the first three in order to get more mileage out of open(). The bitwise OR operator is the vertical bar “|”. For example, to emulate the fopen() function a programme could opt to create a file if it did not already exist:

```
fd = open (filename(), O_RDONLY | O_CREAT);
```

open() sets the file position to zero if the file is opened successfully.

Close()

close() releases a file portal for use by other files and brings a file completely up to date with regard to any changes that have been made to it. Like all other filing functions, it returns the value 0 if it performs successfully and the value -1 if it fails. *e.g.*

```
#define FAILED -1
if (close(fd) == FAILED)
{
    printf ("ERROR!");
}
```

Creat()

This function creates a new file and prepares it for access using the low level file handling functions. If a file which already exists is created, its contents are discarded. The form of this function call is:

```
int fd, pmode;
char *filename;
fd = creat(filename, pmode);
```

filename must be a valid filename; pmode is a flag which contains access-privilege mode bits (system specific information about allowed access) and fd is a returned file handle. In the absence of any information about pmode, this parameter can be set to zero. Note that, the action of creating a file opens it too. Thus after a call to creat, you should close the file descriptor.

Read()

This function gets a block of information from a file. The data are loaded directly into memory, as a sequence of bytes. The user must provide a place for them (either by making an array or by using malloc() to reserve space). read() keeps track of file positions automatically, so it actually reads the next block of bytes from the current file position. The following example reads *n* bytes from a file:

```
int returnvalue, fd, n;
char *buffer;
if ((buffer = malloc(size)) == NULL)
{
    puts ("Out of memory\n");
    error_handler ();
}
returnvalue = read (fd, buffer, n);
```

The return value should be checked. Its values are defined as follows:

End of file,

-1 Error occurred

n the number of bytes actually read. (If all went well this should be equal to *n*.)

Write()

This function is the opposite of read(). It writes a block of *n* bytes from a contiguous portion of memory to a file which was opened by open(). The form of this function is:

```
int returnvalue, fd, n;
char *buffer;
returnvalue = write (fd, buffer, n);
```

The return value should, again, be checked for errors:

-1 Error
 n Number of bytes written

Lseek()

Low level file handing functions have their equivalent of fseek() for finding a specific position within a file. This is almost identical to fseek() except that it uses the file handle rather than a file pointer as a parameter and has a different return value. The constants should be declared long int, or simply long.

```
#define FAILED -1L
long int pos, offset, fd;
int mode, returncode;
if ((pos = fseek (fd, offset, mode)) == FAILED)
{
    printf("Error!\n");
}
```

pos gives the new file position if successful, and -1 (long) if an attempt was made to read past the end of the file. The values which mode can take are:

- 0 Offset measured relative to the beginning of the file.
- 1 Offset measured relative to the current position.
- 2 Offset measured relative to the end of the file.

Unlink() and Remove()

These functions delete a file from disk storage. Once deleted, files are usually irretrievable. They return -1 if the action failed.

```
#define FAILED -1
int returnvalue;
char *filename;
if (unlink (filename) == FAILED)
{
    printf ("Can't delete %s\n", filename);
}
if (remove (filename) == FAILED)
{
    printf ("Can't delete %s\n", filename);
}
```

filename is a string containing the name of the file concerned. This function can fail if a file concerned is protected or if it is not found or if it is a device. (It is impossible to delete the printer!)

Rename()

This function renames a file. The programmer specifies two filenames: the old filename and a new file name. As usual, it returns the value -1 if the action fails. An example illustrates the form of the rename() call:

```
#define FAILED -1
char *old,*new;
if (rename(old,new) == FAILED)
{
    printf ("Can't rename %s as %s\n",old,new);
}
```

rename() can fail because a file is protected or because it is in use, or because one of the filenames given was not valid.

Integer Data Type

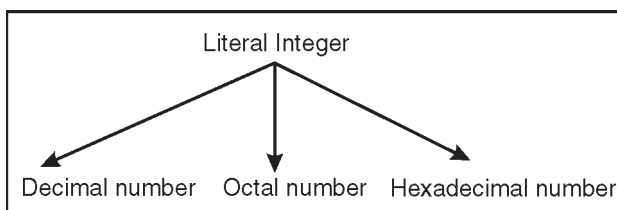
An integer variable may be defined to be of type short, int, or long. The only difference is that an int uses more or at least the same number of bytes as a short, and a long uses more or at least the same number of bytes as an int. For example, on the author's PC, a short uses 2 bytes, an int also 2 bytes, and a long 4 bytes.

```
short age = 20;  
int salary = 65000;  
long price = 4500000;
```

By default, an integer variable is assumed to be signed (*i.e.*, have a signed representation so that it can assume positive as well as negative values). However, an integer can be defined to be unsigned by using the keyword unsigned in its definition. The keyword signed is also allowed but is redundant.

```
unsigned short age = 20;  
unsigned int salary = 65000;  
unsigned long price = 4500000;
```

A literal integer (*e.g.*, 1984) is always assumed to be of type int, unless it has an L or l suffix, in which case it is treated as a long. Also, a literal integer can be specified to be unsigned using the suffix U or u. Literal integers can be expressed as follows:



The decimal number is the one we have been using so far. An integer is taken to be octal if it is preceded by a zero (0), and hexadecimal if it is preceded by a 0x or 0X. For example:

```
92      //decimal
0134    //equivalent octal
0x5C    //equivalent hexadecimal
```

Octal numbers use the base 8, and can therefore only use the digits 0-7.

Hexadecimal numbers use the base 16, and therefore use the digit 0-9 and letter A-F (or a-f) to represent, respectively, 10-15. Octal and hexadecimal numbers are calculated as follows:

$$0134 = 1 \times 8^2 + 3 \times 8^1 + 4 \times 8^0 = 64 + 24 + 4 = 92$$

$$0x5C = 5 \times 16^1 + 12 \times 16^0 = 80 + 12 = 92$$

FLOAT DATA TYPE

A real variable may be defined to be of type float or double. The latter uses more bytes and therefore offers a greater range and accuracy for representing real numbers. For example, on the author's PC, a float uses 4 and a double uses 8 bytes.

```
float interestRate = 0.06;
double pi = 3.141592654;
```

A literal real (*e.g.*, 0.06) is always assumed to be of type double, unless it has an F or f suffix, in which case it is treated as a float, or an L or l suffix, in which case it is treated as a long double. The latter uses more bytes than a double for better accuracy. For example:

```
0.06F 0.06f      3.141592654L 3.141592654l
```

In addition to the decimal notation used so far, literal reals may also be expressed in *scientific* notation. For example, 0.002164 may be written in the scientific notation as:

```
2.164E-3      or      2.164e-3
```

The letter E (or e) stands for *exponent*. The scientific notation is interpreted as follows:

$$2.164E-3 = 2.164 \times 10^{-3}$$

ASSIGNMENT OPERATOR

The assignment operator is used for storing a value at some memory location (typically denoted by a variable). Its left operand should be an lvalue, and its right operand may be an arbitrary expression. The latter is evaluated and the outcome is stored in the location denoted by the lvalue.

An lvalue (standing for left value) is anything that denotes a memory location in which a value may be stored. The only kind of lvalue we have seen so far in this book is a variable.

The assignment operator has a number of variants, obtained by combining it with the arithmetic and bitwise operators. These are summarized in Table. The examples assume that *n* is an integer variable.

Table. Assignment Operators.

Operator	Example	Equivalent to
=	n = 25	
+=	n += 25	n = n + 25
-=	n -= 25	n = n - 25
*=	n *= 25	n = n * 25
/=	n /= 25	n = n / 25
%=	n %= 25	n = n % 25
&=	n &= 0xF2F2	n = n & 0xF2F2
=	n = 0xF2F2	n = n 0xF2F2
^=	n ^= 0xF2F2	n = n ^ 0xF2F2
<<=	n <<= 4	n = n << 4
>>=	n >>= 4	n = n >> 4

An assignment operation is itself an expression whose value is the value stored in its left operand.

An assignment operation can therefore be used as the right operand of another assignment operation. Any number of assignments can be concatenated in this fashion to form one expression.

For example:

```
int m, n, p;
m = n = p = 100; //means: n = (m = (p = 100));
m = (n = p = 100) + 2; //means: m = (n = (p = 100)) + 2;
```

This is equally applicable to other forms of assignment.

For example:

```
m = 100;
m += n = p = 10; //means: m = m + (n = p = 10);
```

CONDITIONAL OPERATOR

The conditional operator takes three operands. It has the general form:

operand1* ? *operand2* : *operand3

First *operand1* is evaluated, which is treated as a logical condition. If the result is non-zero then *operand2* is evaluated and its value is the final result. Otherwise, *operand3* is evaluated and its value is the final result. For example:

```
int m = 1, n = 2;
int min = (m < n ? m : n); //min receives 1
```

Note that of the second and the third operands of the conditional operator only one is evaluated. This may be significant when one or both contain side-effects (*i.e.*, their evaluation causes a change to the value of a variable). For example, in `int min = (m < n ? m++ : n++)`; `m` is incremented because `m++` is evaluated but `n` is not incremented because `n++` is not evaluated.

Because a conditional operation is itself an expression, it may be used as an operand of another conditional operation, that is, conditional expressions may be nested.

For example:

```
int m = 1, n = 2, p = 3;
int min = (m < n ? (m < p ? m: p)
          : (n < p ? n: p));
```

COMMA OPERATOR

Multiple expressions can be combined into one expression using the comma operator. The comma operator takes two operands. It first evaluates the left operand and then the right operand, and returns the value of the latter as the final outcome.

For example:

```
int m, n, min;
int mCount = 0, nCount = 0;
//...
min = (m < n ? mCount++, m: nCount++, n);
```

Here when *m* is less than *n*, *mCount++* is evaluated and the value of *m* is stored in *min*. Otherwise, *nCount++* is evaluated and the value of *n* is stored in *min*.

SIZE OF OPERATOR

C++ provides a useful operator, *sizeof*, for calculating the size of any data item or type. It takes a single operand which may be a type name (*e.g.*, *int*) or an expression (*e.g.*, *100*) and returns the size of the specified entity in bytes. The outcome is totally machine-dependent. Listing below illustrates the use of *sizeof* on the built-in types we have encountered so far.

```
1 #include <iostream.h>
2 int main (void)
3 {
4     cout << "char      size = " << sizeof(char) << " bytes\n";
5     cout << "char*     size = " << sizeof(char*) << " bytes\n";
6     cout << "short     size = " << sizeof(short) << " bytes\n";
7     cout << "intsize = " << sizeof(int) << " bytes\n";
8     cout << "long      size = " << sizeof(long) << " bytes\n";
9     cout << "float      size = " << sizeof(float) << " bytes\n";
10    cout << "double     size = " << sizeof(double) << " bytes\n";
11    cout << "1.55       size = " << sizeof(1.55) << " bytes\n";
12    cout << "1.55L      size = " << sizeof(1.55L) << " bytes\n";
13    cout << "HELLO      size = " << sizeof("HELLO") << " bytes\n";
14 }
```

When run, the programme will produce the following output (on the author's PC):

```
char      size = 1 bytes
char*     size = 2 bytes
short     size = 2 bytes
intsize = 2 bytes
long      size = 4 bytes
float     size = 4 bytes
```

```
double size = 8 bytes
1.55 size = 8 bytes
1.55L size = 10 bytes
HELLO size = 6 bytes
```

OPERATOR PRECEDENCE

The order in which operators are evaluated in an expression is significant and is determined by precedence rules. These rules divide the C++ operators into a number of precedence levels. Operators in higher levels take precedence over operators in lower levels.

SIMPLE TYPE CONVERSION

A value in any of the built-in types we have seen so far can be converted (*type-cast*) to any of the other types.

For example:

```
(int) 3.14           //converts 3.14 to an int to give 3
(long) 3.14          //converts 3.14 to a long to give 3L
(double) 2           //converts 2 to a double to give 2.0
(char) 122           //converts 122 to a char whose code is 122
(unsigned short) 3.14 //gives 3 as an unsigned short
```

As shown by these examples, the built-in type identifiers can be used as type operators. Type operators are unary (*i.e.*, take one operand) and appear inside brackets to the left of their operand.

This is called explicit type conversion. When the type name is just one word, an alternate notation may be used in which the brackets appear around the operand:

```
int(3.14)           //same as: (int) 3.14
```

In some cases, C++ also performs implicit type conversion. This happens when values of different types are mixed in an expression.

For example:

```
double d = 1;       //d receives 1.0
inti = 10.5;        //i receives 10
i = i + d;          //means: i = int(double(i) + d)
```

In the last example, $i + d$ involves mismatching types, so i is first converted to double (*promoted*) and then added to d . The result is a double which does not match the type of i on the left side of the assignment, so it is converted to int (*demoted*) before being assigned to i . The above rules represent some simple but common cases for type conversion.

DATA TYPES AND OPERATORS

In this we'll learn about C's basic types, how to write constants and declare variables of these types, and what the basic operators are. As Kernighan and Ritchie say, "The

type of an object determines the set of values it can have and what operations can be performed on it.” This is a fairly formal, mathematical definition of what a type is, but it is traditional (and meaningful). There are several implications to remember:

- The “set of values” is finite. C’s int type can not represent *all* of the integers; its float type can not represent *all* floating-point numbers.
- When you’re using an object (that is, a variable) of some type, you may have to remember what values it can take on and what operations you can perform on it. For example, there are several operators which play with the binary (bit-level) representation of integers, but these operators are not meaningful for and may not be applied to floating-point operands.
- When declaring a new variable and picking a type for it, you have to keep in mind the values and operations you’ll be needing.

TYPES

There are only a few basic data types in C. The first ones we’ll be encountering and using are:

- Char a character
- Int an integer, in the range -32,767 to 32,767
- Long int a larger integer (up to +-2,147,483,647)
- Float a floating-point number
- Double a floating-point number, with more precision and perhaps greater range than float

If you can look at this list of basic types and say to yourself, “Oh, how simple, there are only a few types, I won’t have to worry much about choosing among them,” you’ll have an easy time with declarations. (Some masochists wish that the type system were more complicated so that they could specify more things about each variable, but those of us who would rather not have to specify these extra things each time are glad that we don’t have to.)

The ranges listed above for types int and long int are the guaranteed minimum ranges. On some systems, either of these types (or, indeed, any C type) may be able to hold larger values, but a programme that depends on extended ranges will not be as portable. Some programmers become obsessed with knowing exactly what the sizes of data objects will be in various situations, and go on to write programmes which depend on these exact sizes. Determining or controlling the size of an object is occasionally important, but most of the time we can sidestep size issues and let the compiler do most of the worrying. (From the ranges listed above, we can determine that type int must be at least 16 bits, and that type long int must be at least 32 bits. But neither of these sizes is exact; many systems have 32-bit ints, and some systems have 64-bit long ints.)

You might wonder how the computer stores characters. The answer involves a *character set*, which is simply a mapping between some set of characters and some set of small numeric codes. Most machines today use the ASCII character set, in which

the letter A is represented by the code 65, the ampersand and is represented by the code 38, the digit 1 is represented by the code 49, the space character is represented by the code 32, etc. (Most of the time, of course, you have *no* need to know or even worry about these particular code values; they're automatically translated into the right shapes on the screen or printer when characters are printed out, and they're automatically generated when you type characters on the keyboard. Eventually, though, we'll appreciate, and even take some control over, exactly when these translations—from characters to their numeric codes—are performed.) Character codes are usually small—the largest code value in ASCII is 126, which is the ~ (tilde or circumflex) character. Characters usually fit in a byte, which is usually 8 bits. In C, type `char` is defined as occupying one byte, so it is usually 8 bits.

Most of the simple variables in most programmes are of types `int`, `long int`, or `double`. Typically, we'll use `int` and `double` for most purposes, and `long int` any time we need to hold integer values greater than 32,767. As we'll see, even when we're manipulating individual characters, we'll usually use an `int` variable, for reasons to be discussed later. Therefore, we'll rarely use individual variables of type `char`; although we'll use plenty of arrays of `char`.

CONSTANTS

A *constant* is just an immediate, absolute value found in an expression. The simplest constants are decimal integers, *e.g.* 0, 1, 2, 123. Occasionally it is useful to specify constants in base 8 or base 16 (octal or hexadecimal); this is done by prefixing an extra 0 (zero) for octal, or 0x for hexadecimal: the constants 100, 0144, and 0x64 all represent the same number. (If you're not using these non-decimal constants, just remember not to use any leading zeroes. If you accidentally write 0123 intending to get one hundred and twenty three, you'll get 83 instead, which is 123 base 8.)

We write constants in decimal, octal, or hexadecimal for our convenience, not the compiler's. The compiler doesn't care; it always converts everything into binary internally, anyway. (There is, however, no good way to specify constants in source code in binary.)

A constant can be forced to be of type `long int` by suffixing it with the letter L (in upper or lower case, although upper case is strongly recommended, because a lower case l looks too much like the digit 1).

A constant that contains a decimal point or the letter e (or both) is a floating-point constant: 3.14, 10..01, 123e4, 123.456e7. The e indicates multiplication by a power of 10; 123.456e7 is 123.456 times 10 to the 7th, or 1,234,560,000. (Floating-point constants are of type `double` by default.)

We also have constants for specifying characters and strings. (Make sure you understand the difference between a character and a string: a character is exactly one character; a string is a set of zero or more characters; a string containing one character is distinct from a lone character.) A character constant is simply a single character

between single quotes: ‘A’, ‘.’, ‘%’. The numeric value of a character constant is, naturally enough, that character’s value in the machine’s character set. (In ASCII, for example, ‘A’ has the value 65.)

A *string* is represented in C as a sequence or array of characters. (We’ll have more to say about arrays in general, and strings in particular, later.) A string constant is a sequence of zero or more characters enclosed in double quotes: “apple”, “hello, world”, “this is a test”.

Within character and string constants, the backslash character \ is special, and is used to represent characters not easily typed on the keyboard or for various reasons not easily typed in constants.

The most common of these “character escapes” are:

<code>\n</code>	a “newline” character
<code>\b</code>	a backspace
<code>\r</code>	a carriage return (without a line feed)
<code>\’</code>	a single quote (e.g. in a character constant)
<code>\”</code>	a double quote (e.g. in a string constant)
<code>\\</code>	a single backslash

For example, “he said \”hi\”” is a string constant which contains two double quotes, and \’ is a character constant consisting of a (single) single quote. Notice once again that the character constant ‘A’ is very different from the string constant “A”.

DECLARATIONS

Informally, a *variable* (also called an *object*) is a place you can store a value. So that you can refer to it unambiguously, a variable needs a name.

You can think of the variables in your programme as a set of boxes or cubbyholes, each with a label giving its name; you might imagine that storing a value “in” a variable consists of writing the value on a slip of paper and placing it in the cubbyhole.

A *declaration* tells the compiler the name and type of a variable you’ll be using in your programme. In its simplest form, a declaration consists of the type, the name of the variable, and a terminating semicolon:

```
char c;
int i;
float f;
```

You can also declare several variables of the same type in one declaration, separating them with commas:

```
int i1, i2;
```

The placement of declarations is significant. You can’t place them just anywhere (*i.e.* they cannot be interspersed with the other statements in your programme). They must either be placed at the beginning of a function, or at the beginning of a brace-enclosed block of statements (which we’ll learn about in the next chapter), or outside of any function. Furthermore, the placement of a declaration, as well as its storage class, controls several things about its *visibility* and *life time*. You may wonder *why* variables must be declared before use.

There are two reasons:

- It makes things somewhat easier on the compiler; it knows right away what kind of storage to allocate and what code to emit to store and manipulate each variable; it doesn't have to try to intuit the programmer's intentions.
- It forces a bit of useful discipline on the programmer: you cannot introduce variables willy-nilly; you must think about them enough to pick appropriate types for them. (The compiler's error messages to you, telling you that you apparently forgot to declare a variable, are as often helpful as they are a nuisance: they're helpful when they tell you that you misspelled a variable, or forgot to think about exactly how you were going to use it.)

Although there are a few places where declarations can be omitted (in which case the compiler will assume an implicit declaration), making use of these removes the advantages of reason 2 above, so I recommend always declaring everything explicitly.

Most of the time, I recommend writing one declaration per line. For the most part, the compiler doesn't care what order declarations are in. You can order the declarations alphabetically, or in the order that they're used, or to put related declarations next to each other. Collecting all variables of the same type together on one line essentially orders declarations by type, which isn't a very useful order (it's only slightly more useful than random order).

A declaration for a variable can also contain an initial value. This *initializer* consists of an equals sign and an expression, which is usually a single constant:

```
int i = 1;
int i1 = 10, i2 = 20;
```

VARIABLE NAMES

Within limits, you can give your variables and functions any names you want. These names (the formal term is “identifiers”) consist of letters, numbers, and underscores. For our purposes, names must begin with a letter. Theoretically, names can be as long as you want, but extremely long ones get tedious to type after a while, and the compiler is not required to keep track of extremely long ones perfectly. (What this means is that if you were to name a variable, say, super supercalifragilisticexpialidocious, the compiler might get lazy and pretend that you'd named it super calafragalisticexpialidocio, such that if you later misspelled it supercalafagalisticexpialidociouz, the compiler wouldn't catch your mistake. Nor would the compiler necessarily be able to tell the difference if for some perverse reason you *deliberately* declared a second variable named supercalifragilisticoespialidoso.)

The capitalization of names in C is significant: the variable names `variable`, `Variable`, and `VARIABLE` (as well as silly combinations like `variAble`) are all distinct.

A final restriction on names is that you may not use *keywords* (the words such as `int` and `for` which are part of the syntax of the language) as the names of variables or functions (or as identifiers of any kind).

ARITHMETIC OPERATORS

The basic operators for performing arithmetic are the same in many computer languages:

```
+ addition
- subtraction
* multiplication
/ division
% modulus (remainder)
```

The `-` operator can be used in two ways: to subtract two numbers (as in $a - b$), or to negate one number (as in $-a$ or $a + -b$). When applied to integers, the division operator `/` discards any remainder, so $1/2$ is 0 and $7/4$ is 1. But when either operand is a floating-point quantity (type `float` or `double`), the division operator yields a floating-point result, with a potentially non-zero fractional part. So $1/2.0$ is 0.5, and $7.0/4.0$ is 1.75.

The *modulus* operator `%` gives you the remainder when two integers are divided: $1 \% 2$ is 1; $7 \% 4$ is 3. (The modulus operator can only be applied to integers.) An additional arithmetic operation you might be wondering about is exponentiation. Some languages have an exponentiation operator (typically `^` or `**`), but C doesn't. (To square or cube a number, just multiply it by itself.)

Multiplication, division, and modulus all have higher *precedence* than addition and subtraction. The term “precedence” refers to how “tightly” operators bind to their operands (that is, to the things they operate on). In mathematics, multiplication has higher precedence than addition, so $1 + 2 * 3$ is 7, not 9. In other words, $1 + 2 * 3$ is equivalent to $1 + (2 * 3)$. C is the same way.

All of these operators “group” from left to right, which means that when two or more of them have the same precedence and participate next to each other in an expression, the evaluation conceptually proceeds from left to right. For example, $1 - 2 - 3$ is equivalent to $(1 - 2) - 3$ and gives -4, not +2. (“Grouping” is sometimes called *associativity*, although the term is used somewhat differently in programming than it is in mathematics. Not all C operators group from left to right; a few group from right to left.) Whenever the default precedence or associativity doesn't give you the grouping you want, you can always use explicit parentheses.

For example, if you wanted to add 1 to 2 and then multiply the result by 3, you could write $(1 + 2) * 3$. By the way, the word “arithmetic” as used in the title of this section is an adjective, not a noun, and it's pronounced differently than the noun: the accent is on the third syllable.

ASSIGNMENT OPERATORS

The assignment operator `=` assigns a value to a variable. For example,

```
x = 1
sets x to 1, and
a = b
```

```
sets a to whatever b's value is. The expression
i = i + 1
```

is, as we've mentioned elsewhere, the standard programming idiom for increasing a variable's value by 1: this expression takes *i*'s old value, adds 1 to it, and stores it back into *i*. (C provides several “short-cut” operators for modifying variables in this and similar ways, which we'll meet later.)

We've called the `=` sign the “assignment operator” and referred to “assignment expressions” because, in fact, `=` is an operator just like `+` or `-`. C does not have “assignment statements”; instead, an assignment like `a = b` is an expression and can be used wherever any expression can appear. Since it's an expression, the assignment `a = b` has a value, namely, the same value that's assigned to *a*. This value can then be used in a larger expression; for example, we might write

```
c = a = b
which is equivalent to
c = (a = b)
```

and assigns *b*'s value to both *a* and *c*. (The assignment operator, therefore, groups from right to left.). It's usually a matter of style whether you initialize a variable with an initializer in its declaration or with an assignment expression near where you first use it. That is, there's no particular difference between,

```
int a = 10;
and
int a;
/* later... */
a = 10;
```

FUNCTION CALLS

We'll have much more to say about functions in a later chapter, but for now let's just look at how they're called. You call a function by mentioning its name followed by a pair of parentheses. If the function takes any arguments, you place the arguments between the parentheses, separated by commas.

These are all function calls:

```
printf("Hello, world!\n")
printf("%d\n", i)
sqrt(144.)
getchar()
```

The arguments to a function can be arbitrary expressions. Therefore, you don't have to say things like

```
int sum = a + b + c;
printf("sum = %d\n", sum);
```

if you don't want to; you can instead collapse it to

```
printf("sum = %d\n", a + b + c);
```

Many functions return values, and when they do, you can embed calls to these functions within larger expressions:

```

c = sqrt(a * a + b * b)
x = r * cos(theta)
i = f1(f2(j))

```

The first expression squares *a* and *b*, computes the square root of the sum of the squares, and assigns the result to *c*. (In other words, it computes $a * a + b * b$, passes that number to the `sqrt` function, and assigns `sqrt`'s return value to *c*.) The second expression passes the value of the variable *theta* to the `cos` (cosine) function, multiplies the result by *r*, and assigns the result to *x*.

The third expression passes the value of the variable *j* to the function `f2`, passes the return value of `f2` immediately to the function `f1`, and finally assigns `f1`'s return value to the variable *i*.

CONSTRUCTORS AND DESTRUCTORS

Objects generally need to initialize variables or assign dynamic memory during their process of creation to become operative and to avoid returning unexpected values during their execution. For example, what would happen if in the previous example we called the member function `area()` before having called function `set_values()`? Probably we would have gotten an undetermined result since the members *x* and *y* would have never been assigned a value. In order to avoid that, a class can include a special function called constructor, which is automatically called whenever a new object of this class is created. This constructor function must have the same name as the class, and cannot have any return type; not even void.

We are going to implement CRectangle including a constructor:

```

//example: class constructor
#include <iostream>
using namespace std;
class CRectangle {
int width, height;
public:
CRectangle (int,int);
int area () {return (width*height);}
};
CRectangle::CRectangle (int a, int b) {
width = a;
height = b;
}
int main () {
CRectangle rect (3,4);
CRectangle rectb (5,6);
cout << "rect area: " << rect.area() << endl;
cout << "rectb area: " << rectb.area() << endl;
return 0;
} rect area: 12
rectb area: 30

```

As you can see, the result of this example is identical to the previous one. But now we have removed the member function `set_values()`, and have included instead a constructor that performs a similar action: it initializes the values of `x` and `y` with the parameters that are passed to it.

Notice how these arguments are passed to the constructor at the moment at which the objects of this class are created:

```
CRectangle rect (3,4);
CRectangle rectb (5,6);
```

Constructors cannot be called explicitly as if they were regular member functions. They are only executed when a new object of that class is created.

You can also see how neither the constructor prototype declaration (within the class) nor the latter constructor definition include a return value; not even `void`.

The *destructor* fulfills the opposite functionality. It is automatically called when an object is destroyed, either because its scope of existence has finished (for example, if it was defined as a local object within a function and the function ends) or because it is an object dynamically assigned and it is released using the operator `delete`.

The destructor must have the same name as the class, but preceded with a tilde sign (`~`) and it must also return no value.

The use of destructors is especially suitable when an object assigns dynamic memory during its lifetime and at the moment of being destroyed we want to release the memory that the object was allocated.

```
//example on constructors and destructors
#include <iostream>
using namespace std;
class CRectangle {
int *width, *height;
public:
CRectangle (int,int);
~CRectangle ();
int area () {return (*width * *height);}
};
CRectangle::CRectangle (int a, int b) {
width = new int;
height = new int;
*width = a;
*height = b;
}
CRectangle::~~CRectangle () {
delete width;
delete height;
}
int main () {
CRectangle rect (3,4), rectb (5,6);
cout << "rect area: " << rect.area() << endl;
cout << "rectb area: " << rectb.area() << endl;
return 0;
```

```

} rect area: 12
rectb area: 30

```

Overloading Constructors: Like any other function, a constructor can also be overloaded with more than one function that have the same name but different types or number of parameters. Remember that for overloaded functions the compiler will call the one whose parameters match the arguments used in the function call. In the case of constructors, which are automatically called when an object is created, the one executed is the one that matches the arguments passed on the object declaration:

```

//overloading class constructors
#include <iostream>
using namespace std;
class CRectangle {
int width, height;
public:
CRectangle ();
CRectangle (int,int);
int area (void) {return (width*height);}
};
CRectangle::CRectangle () {
width = 5;
height = 5;
}
CRectangle::CRectangle (int a, int b) {
width = a;
height = b;
}
int main () {
CRectangle rect (3,4);
CRectangle rectb;
cout << "rect area: " << rect.area() << endl;
cout << "rectb area: " << rectb.area() << endl;
return 0;
} rect area: 12
rectb area: 25

```

In this case, rectb was declared without any arguments, so it has been initialized with the constructor that has no parameters, which initializes both width and height with a value of 5.

Important: Notice how if we declare a new object and we want to use its default constructor (the one without parameters), we do not include parentheses ():

```

CRectangle rectb;//right
CRectangle rectb();//wrong!

```

Default constructor

If you do not declare any constructors in a class definition, the compiler assumes the class to have a default constructor with no arguments. Therefore, after declaring a class like this one:

```

class CExample {
public:

```

```
int a,b,c;
void multiply (int n, int m) {a=n; b=m; c=a*b;};
};
```

The compiler assumes that CExample has a default constructor, so you can declare objects of this class by simply declaring them without any arguments:

```
CExample ex;
```

But as soon as you declare your own constructor for a class, the compiler no longer provides an implicit default constructor. So you have to declare all objects of that class according to the constructor prototypes you defined for the class:

```
class CExample {
public:
int a,b,c;
CExample (int n, int m) {a=n; b=m;};
void multiply () {c=a*b;};
};
```

Here we have declared a constructor that takes two parameters of type int. Therefore the following object declaration would be correct:

```
CExample ex (2,3);
```

But,

```
CExample ex;
```

Would not be correct, since we have declared the class to have an explicit constructor, thus replacing the default constructor. But the compiler not only creates a default constructor for you if you do not specify your own. It provides three special member functions in total that are implicitly declared if you do not declare your own. These are the *copy constructor*, the *copy assignment operator*, and the default destructor.

The copy constructor and the copy assignment operator copy all the data contained in another object to the data members of the current object. For CExample, the copy constructor implicitly declared by the compiler would be something similar to:

```
CExample::CExample (const CExample& rv) {
a=rv.a; b=rv.b; c=rv.c;
}
```

Therefore, the two following object declarations would be correct:

```
CExample ex (2,3);
CExample ex2 (ex); //copy constructor (data copied from ex)
```

Pointers to classes

It is perfectly valid to create pointers that point to classes. We simply have to consider that once declared, a class becomes a valid type, so we can use the class name as the type for the pointer. For example:

```
CRectangle * prect;
```

is a pointer to an object of class CRectangle.

As it happened with data structures, in order to refer directly to a member of an object pointed by a pointer we can use the arrow operator (->) of indirection.

Here is an example with some possible combinations:

```
//pointer to classes example
#include <iostream>
```

```

using namespace std;
class CRectangle {
int width, height;
public:
void set_values (int, int);
int area (void) {return (width * height);}
};
void CRectangle::set_values (int a, int b) {
width = a;
height = b;
}
int main () {
CRectangle a, *b, *c;
CRectangle * d = new CRectangle[2];
b= new CRectangle;
c= &a;
a.set_values (1,2);
b->set_values (3,4);
d->set_values (5,6);
d[1].set_values (7,8);
cout << "a area: " << a.area() << endl;
cout << "*b area: " << b->area() << endl;
cout << "*c area: " << c->area() << endl;
cout << "d[0] area: " << d[0].area() << endl;
cout << "d[1] area: " << d[1].area() << endl;
delete[] d;
delete b;
return 0;
} a area: 2 *b area: 12
*c area: 2
d[0] area: 30
d[1] area: 56

```

Next you have a summary on how can you read some pointer and class operators (*, &, -, [],) that appear in the previous example: expression can be read as

```

*x pointed by x
&x address of x
x.y member y of object x
x->y member y of object pointed by x
(*x).y member y of object pointed by x (equivalent to the previous
one)
x[0] first object pointed by x
x[1] second object pointed by x
x[n] (n+1)th object pointed by x

```

Be sure that you understand the logic under all of these expressions before proceeding with the next sections. If you have doubts, read again this section and/or consult the previous sections about pointers and data structures.

Classes defined with struct and union: Classes can be defined not only with keyword class, but also with keywords struct and union. The concepts of class and data structure are so similar that both keywords have in C++ the exact same functionality except that

the members of classes declared with keyword `struct` have public access by default, instead of private access, as classes declared with keyword `class` have. That is the only difference. For all other purposes both keywords are equivalent.

The concept under unions is different from than of `struct` and `class`, since unions only store one data member at a time, but they are also classes and thus can also hold function members. The default access in union classes is public.

CLASSES (II)

Overloading operators: C++ incorporates the option to use standard operators to perform operations with classes in addition to with fundamental types. For example:

```
int a, b, c;
a = b + c;
```

This is obviously valid code in C++, since the different variables of the addition are all fundamental types. Nevertheless, it is not so obvious that we could perform an operation similar to the following one:

```
struct {
    string product;
    float price;
} a, b, c;
a = b + c;
```

In fact, this will cause a compilation error, since we have not defined the behaviour our class should have with addition operations. However, thanks to the C++ feature to overload operators, we can design classes able to perform operations using standard operators. Here is a list of all the operators that can be overloaded:

Overloadable operators:

```
+ - * / = < > += -= *= /= << >>
<= >= == != <= >= ++ -- % & ^ ! |
~ &= ^= |= && || %= [] (), ->* -> new
delete new[] delete[]
```

To overload an operator in order to use it with classes we declare *operator functions*, which are regular functions whose names are the operator keyword followed by the operator sign that we want to overload. The format is:

```
type operator sign (parameters) { /*...*/ }
```

Here you have an example that overloads the addition operator (+). We are going to create a class to store bidimensional vectors and then we are going to add two of them: $a(3,1)$ and $b(1,2)$. The addition of two bidimensional vectors is an operation as simple as adding the two x coordinates to obtain the resulting x coordinate and adding the two y coordinates to obtain the resulting y. In this case the result will be $(3+1, 1+2) = (4,3)$.

```
//vectors: overloading operators example
#include <iostream>
using namespace std;
class CVector {
public:
```

```

int x,y;
CVector () {};
CVector (int,int);
CVector operator + (CVector);
};
CVector::CVector (int a, int b) {
x = a;
y = b;
}
CVector CVector::operator+ (CVector param) {
CVector temp;
temp.x = x + param.x;
temp.y = y + param.y;
return (temp);
}
int main () {
CVector a (3,1);
CVector b (1,2);
CVector c;
c = a + b;
cout << c.x << "," << c.y;
return 0;
} 4,3

```

It may be a little confusing to see so many times the CVector identifier. But, consider that some of them refer to the class name (type) CVector and some others are functions with that name (constructors must have the same name as the class). Do not confuse them:

```

CVector (int, int); //function name CVector (constructor)
CVector operator+ (CVector); //function returns a CVector

```

The function operator+ of class CVector is the one that is in charge of overloading the addition operator (+). This function can be called either implicitly using the operator, or explicitly using the function name:

```

c = a + b;
c = a.operator+ (b);

```

Both expressions are equivalent. Notice also that we have included the empty constructor (without parameters) and we have defined it with an empty block:

```

CVector () {};

```

This is necessary, since we have explicitly declared another constructor:

```

CVector (int, int);

```

And when we explicitly declare any constructor, with any number of parameters, the default constructor with no parameters that the compiler can declare automatically is not declared, so we need to declare it ourselves in order to be able to construct objects of this type without parameters. Otherwise, the declaration:

```

CVector c;

```

included in main() would not have been valid.

Anyway, I have to warn you that an empty block is a bad implementation for a constructor, since it does not fulfill the minimum functionality that is generally expected

from a constructor, which is the initialization of all the member variables in its class. In our case this constructor leaves the variables `x` and `y` undefined. Therefore, a more advisable definition would have been something similar to this:

```
CVector () {x=0; y=0;};
```

which in order to simplify and show only the point of the code I have not included in the example.

As well as a class includes a default constructor and a copy constructor even if they are not declared, it also includes a default definition for the assignment operator (`=`) with the class itself as parameter. The behaviour which is defined by default is to copy the whole content of the data members of the object passed as argument (the one at the right side of the sign) to the one at the left side:

```
CVector d (2,3);
CVector e;
e = d; //copy assignment operator
```

The copy assignment operator function is the only operator member function implemented by default. Of course, you can redefine it to any other functionality that you want, like for example, copy only certain class members or perform additional initialization procedures.

The overload of operators does not force its operation to bear a relation to the mathematical or usual meaning of the operator, although it is recommended. For example, the code may not be very intuitive if you use operator `+` to subtract two classes or operator `==` to fill with zeros a class, although it is perfectly possible to do so.

Although the prototype of a function operator `+` can seem obvious since it takes what is at the right side of the operator as the parameter for the operator member function of the object at its left side, other operators may not be so obvious. Here you have a table with a summary on how the different operator functions have to be declared (replace `@` by the operator in each case):

Expression Operator Member function Global function:

```
@a +-* & ! ~ ++--A::operator@() operator@(A)
a@ ++--A::operator@(int) operator@(A,int)
a@b +-*/% ^ & | < > == != <= >= << >> && ||, A::operator@ (B)
operator@(A,B)
a@b = += -= *= /= %= ^= &= |= <<= >>= [] A::operator@ (B) -
a(b, c...) () A::operator() (B, C...) -
a->x -> A::operator->() -
```

Where `a` is an object of class `A`, `b` is an object of class `B` and `c` is an object of class `C`. You can see in this panel that there are two ways to overload some class operators: as a member function and as a global function. Its use is indistinct, nevertheless I remind you that functions that are not members of a class cannot access the private or protected members of that class unless the global function is its friend (friendship is explained later).

The keyword `this` represents a pointer to the object whose member function is being executed. It is a pointer to the object itself. One of its uses can be to check if a parameter passed to a member function is the object itself. For example,

```
//this
#include <iostream>
using namespace std;
class CDummy {
public:
int isitme (CDummy& param);
};
int CDummy::isitme (CDummy& param)
{
if (&param == this) return true;
else return false;
}
int main () {
CDummy a;
CDummy* b = &a;
if (b->isitme(a))
cout << "yes, &a is b";
return 0;
} yes, &a is b
```

It is also frequently used in operator= member functions that return objects by reference (avoiding the use of temporary objects). Following with the vector's examples seen before we could have written an operator= function similar to this one:

```
CVector& CVector::operator= (const CVector& param)
{
x=param.x;
y=param.y;
return *this;
}
```

In fact this function is very similar to the code that the compiler generates implicitly for this class if we do not include an operator= member function to copy objects of this class.

Static members: A class can contain *static* members, either data or functions.

Static data members of a class are also known as “class variables”, because there is only one unique value for all the objects of that same class. Their content is not different from one object of this class to another.

For example, it may be used for a variable within a class that can contain a counter with the number of objects of that class that are currently allocated, as in the following example:

```
//static members in classes
#include <iostream>
using namespace std;
class CDummy {
public:
static int n;
CDummy () {n++;};
~CDummy () {n--};
};
int CDummy::n=0;
```

```

int main () {
    CDummy a;
    CDummy b[5];
    CDummy * c = new CDummy;
    cout << a.n << endl;
    delete c;
    cout << CDummy::n << endl;
    return 0;
} 7
6

```

In fact, static members have the same properties as global variables but they enjoy class scope. For that reason, and to avoid them to be declared several times, we can only include the prototype (its declaration) in the class declaration but not its definition (its initialization). In order to initialize a static data-member we must include a formal definition outside the class, in the global scope, as in the previous example:

```
int CDummy::n=0;
```

Because it is a unique variable value for all the objects of the same class, it can be referred to as a member of any object of that class or even directly by the class name (of course this is only valid for static members):

```
cout << a.n;
cout << CDummy::n;
```

These two calls included in the previous example are referring to the same variable: the static variable `n` within class `CDummy` shared by all objects of this class. Once again, I remind you that in fact it is a global variable. The only difference is its name and possible access restrictions outside its class.

Just as we may include static data within a class, we can also include static functions. They represent the same: they are global functions that are called as if they were object members of a given class. They can only refer to static data, in no case to non-static members of the class, as well as they do not allow the use of the keyword `this`, since it makes reference to an object pointer and these functions in fact are not members of any object but direct members of the class.

FRIENDSHIP AND INHERITANCE

Friend Functions

In principle, private and protected members of a class cannot be accessed from outside the same class in which they are declared. However, this rule does not affect *friends*.

Friends are functions or classes declared as such. If we want to declare an external function as friend of a class, thus allowing this function to have access to the private and protected members of this class, we do it by declaring a prototype of this external function within the class, and preceding it with the keyword `friend`:

```
//friend functions
#include <iostream>
```

```

using namespace std;
class CRectangle {
int width, height;
public:
void set_values (int, int);
int area () {return (width * height);}
friend CRectangle duplicate (CRectangle);
};
void CRectangle::set_values (int a, int b) {
width = a;
height = b;
}
CRectangle duplicate (CRectangle rectparam)
{
CRectangle rectres;
rectres.width = rectparam.width*2;
rectres.height = rectparam.height*2;
return (rectres);
}
int main () {
CRectangle rect, rectb;
rect.set_values (2,3);
rectb = duplicate (rect);
cout << rectb.area();
return 0;
} 24

```

The duplicate function is a friend of CRectangle. From within that function we have been able to access the members width and height of different objects of type CRectangle, which are private members. Notice that neither in the declaration of duplicate() nor in its later use in main() have we considered duplicate a member of class CRectangle. It isn't! It simply has access to its private and protected members without being a member.

The friend functions can serve, for example, to conduct operations between two different classes. Generally, the use of friend functions is out of an object-oriented programming methodology, so whenever possible it is better to use members of the same class to perform operations with them. Such as in the previous example, it would have been shorter to integrate duplicate() within the class CRectangle.

Friend classes: Just as we have the possibility to define a friend function, we can also define a class as friend of another one, granting that second class access to the protected and private members of the first one.

```

//friend class
#include <iostream>
using namespace std;
class CSquare;
class CRectangle {
int width, height;
public:
int area ()
{return (width * height);}
}

```

```

void convert (CSquare a);
};
class CSquare {
private:
int side;
public:
void set_side (int a)
{side=a;}
friend class CRectangle;
};
void CRectangle::convert (CSquare a) {
width = a.side;
height = a.side;
}
int main () {
CSquare sqr;
CRectangle rect;
sqr.set_side(4);
rect.convert(sqr);
cout << rect.area();
return 0;
} 16

```

In this example, we have declared CRectangle as a friend of CSquare so that CRectangle member functions could have access to the protected and private members of CSquare, more concretely to CSquare::side, which describes the side width of the square.

You may also see something new at the beginning of the programme: an empty declaration of class CSquare. This is necessary because within the declaration of CRectangle we refer to CSquare (as a parameter in convert()). The definition of CSquare is included later, so if we did not include a previous empty declaration for CSquare this class would not be visible from within the definition of CRectangle.

Consider that friendships are not corresponded if we do not explicitly specify so. In our example, CRectangle is considered as a friend class by CSquare, but CRectangle does not consider CSquare to be a friend, so CRectangle can access the protected and private members of CSquare but not the reverse way. Of course, we could have declared also CSquare as friend of CRectangle if we wanted to.

Another property of friendships is that they are *not transitive*: The friend of a friend is not considered to be a friend unless explicitly specified.

Inheritance between classes: A key feature of C++ classes is inheritance. Inheritance allows to create classes which are derived from other classes, so that they automatically include some of its “parent’s” members, plus its own.

For example, we are going to suppose that we want to declare a series of classes that describe polygons like our CRectangle, or like CTriangle. They have certain common properties, such as both can be described by means of only two sides: height and base.

This could be represented in the world of classes with a class CPolygon from which we would derive the two other ones: CRectangle and CTriangle.

The class CPolygon would contain members that are common for both types of polygon. In our case: width and height. And CRectangle and CTriangle would be its derived classes, with specific features that are different from one type of polygon to the other.

Classes that are derived from others inherit all the accessible members of the base class. That means that if a base class includes a member A and we derive it to another class with another member called B, the derived class will contain both members A and B.

In order to derive a class from another, we use a colon (:) in the declaration of the derived class using the following format:

```
class derived_class_name: public base_class_name
{ /*...*/};
```

Where derived_class_name is the name of the derived class and base_class_name is the name of the class on which it is based. The public access specifier may be replaced by any one of the other access specifiers protected and private. This access specifier describes the minimum access level for the members that are inherited from the base class.

```
//derived classes
#include <iostream>
using namespace std;
class CPolygon {
protected:
int width, height;
public:
void set_values (int a, int b)
{width=a; height=b;}
};
class CRectangle: public CPolygon {
public:
int area ()
{return (width * height);}
};
class CTriangle: public CPolygon {
public:
int area ()
{return (width * height/2);}
};
int main () {
CRectangle rect;
CTriangle trgl;
rect.set_values (4,5);
trgl.set_values (4,5);
cout << rect.area() << endl;
cout << trgl.area() << endl;
return 0;
} 20
10
```


The objects of the classes CRectangle and CTriangle each contain members inherited from CPolygon. These are: width, height and set_values().

The protected access specifier is similar to private. Its only difference occurs in fact with inheritance. When a class inherits from another one, the members of the derived class can access the protected members inherited from the base class, but not its private members. Since we wanted width and height to be accessible from members of the derived classes CRectangle and CTriangle and not only by members of CPolygon, we have used protected access instead of private.

We can summarize the different access types according to who can access them in the following way:

Access public protected private:

- Members of the same class yes yes yes
- Members of derived classes yes yes no
- Not members yes no no

Where “not members” represent any access from outside the class, such as from main(), from another class or from a function.

In our example, the members inherited by CRectangle and CTriangle have the same access permissions as they had in their base class CPolygon:

```
CPolygon::width//protected access
CRectangle::width//protected access
CPolygon::set_values()//public access
CRectangle::set_values()//public access
```

This is because we have used the public keyword to define the inheritance relationship on each of the derived classes:

```
class CRectangle: public CPolygon {...}
```

This public keyword after the colon (:) denotes the minimum access level for all the members inherited from the class that follows it (in this case CPolygon). Since public is the most accessible level, by specifying this keyword the derived class will inherit all the members with the same levels they had in the base class. If we specify a more restrictive access level like protected, all public members of the base class are inherited as protected in the derived class. Whereas if we specify the most restricting of all access levels: private, all the base class members are inherited as private.

For example, if daughter was a class derived from mother that we defined as:

```
class daughter: protected mother;
```

This would set protected as the maximum access level for the members of daughter that it inherited from mother. That is, all members that were public in mother would become protected in daughter. Of course, this would not restrict daughter to declare its own public members. That maximum access level is only set for the members inherited from mother. If we do not explicitly specify any access level for the inheritance, the compiler assumes private for classes declared with class keyword and public for those declared with struct.

What is inherited from the base class?

In principle, a derived class inherits every member of a base class except:

- Its constructor and its destructor
- Its operator=() members
- Its friends

Although the constructors and destructors of the base class are not inherited themselves, its default constructor (*i.e.*, its constructor with no parameters) and its destructor are always called when a new object of a derived class is created or destroyed.

If the base class has no default constructor or you want that an overloaded constructor is called when a new derived object is created, you can specify it in each constructor definition of the derived class:

```
derived_constructor_name (parameters): base_constructor_name  
(parameters) {...}
```

For example:

```
//constructors and derived classes  
#include <iostream>  
using namespace std;  
class mother {  
public:  
mother ()  
{cout << "mother: no parameters\n";}  
mother (int a)  
{cout << "mother: int parameter\n";}  
};  
class daughter: public mother {  
public:  
daughter (int a)  
{cout << "daughter: int parameter\n\n";}  
};  
class son: public mother {  
public:  
son (int a): mother (a)  
{cout << "son: int parameter\n\n";}  
};  
int main () {  
daughter cynthia (0);  
son daniel(0);  
return 0;  
} mother: no parameters  
daughter: int parameter  
mother: int parameter  
son: int parameter
```

Notice the difference between which mother's constructor is called when a new daughter object is created and which when it is a son object. The difference is because the constructor declaration of daughter and son:

```
daughter (int a)//nothing specified: call default  
son (int a): mother (a)//constructor specified: call this
```

Multiple inheritance: In C++ it is perfectly possible that a class inherits members from more than one class. This is done by simply separating the different base classes with commas in the derived class declaration. For example, if we had a specific class to print on screen (COutput) and we wanted our classes CRectangle and CTriangle to also inherit its members in addition to those of CPolygon we could write:

```
class CRectangle: public CPolygon, public COutput;
class CTriangle: public CPolygon, public COutput;
```

here is the complete example:

```
//multiple inheritance
#include <iostream>
using namespace std;
class CPolygon {
protected:
int width, height;
public:
void set_values (int a, int b)
{width=a; height=b;}
};
class COutput {
public:
void output (int i);
};
void COutput::output (int i) {
cout << i << endl;
}
class CRectangle: public CPolygon, public COutput {
public:
int area ()
{return (width * height);}
};
class CTriangle: public CPolygon, public COutput {
public:
int area ()
{return (width * height/2);}
};
int main () {
CRectangle rect;
CTriangle trgl;
rect.set_values (4,5);
trgl.set_values (4,5);
rect.output (rect.area());
trgl.output (trgl.area());
return 0;
} 20
10
```

Polymorphism: Before getting into this section, it is recommended that you have a proper understanding of pointers and class inheritance. If any of the following statements seem strange to you, you should review the indicated sections:

Statement: Explained in:

`int a::b(c) {};` **Classes**

`a->b` **Pointers**

`class a: public b;` **Friendship and inheritance**

Pointers to base class: One of the key features of derived classes is that a pointer to a derived class is type-compatible with a pointer to its base class. Polymorphism is the art of taking advantage of this simple but powerful and versatile feature, that brings Object Oriented Methodologies to its full potential. We are going to start by rewriting our programme about the rectangle and the triangle of the previous section taking into consideration this pointer compatibility property:

```
//pointers to base class
#include <iostream>
using namespace std;
class CPolygon {
protected:
int width, height;
public:
void set_values (int a, int b)
{width=a; height=b;}
};
class CRectangle: public CPolygon {
public:
int area ()
{return (width * height);}
};
class CTriangle: public CPolygon {
public:
int area ()
{return (width * height/2);}
};
int main () {
CRectangle rect;
CTriangle trgl;
CPolygon * ppoly1 = &rect;
CPolygon * ppoly2 = &trgl;
ppoly1->set_values (4,5);
ppoly2->set_values (4,5);
cout << rect.area() << endl;
cout << trgl.area() << endl;
return 0;
} 20
10
```

In function main, we create two pointers that point to objects of class CPolygon (ppoly1 and ppoly2). Then we assign references to rect and trgl to these pointers, and because both are objects of classes derived from CPolygon, both are valid assignments. The only limitation in using *ppoly1 and *ppoly2 instead of rect and trgl is that both *ppoly1 and *ppoly2 are of type CPolygon* and therefore we can only use these pointers to refer to the members that CRectangle and CTriangle inherit from CPolygon. For that

reason when we call the `area()` members at the end of the programme we have had to use directly the objects `rect` and `trgl` instead of the pointers `*ppoly1` and `*ppoly2`.

In order to use `area()` with the pointers to class `CPolygon`, this member should also have been declared in the class `CPolygon`, and not only in its derived classes, but the problem is that `CRectangle` and `CTriangle` implement different versions of `area`, therefore we cannot implement it in the base class. This is when virtual members become handy:

Virtual members: A member of a class that can be redefined in its derived classes is known as a virtual member. In order to declare a member of a class as virtual, we must precede its declaration with the keyword `virtual`:

```
//virtual members
#include <iostream>
using namespace std;
class CPolygon {
protected:
int width, height;
public:
void set_values (int a, int b)
{width=a; height=b;}
virtual int area ()
{return (0);}
};
class CRectangle: public CPolygon {
public:
int area ()
{return (width * height);}
};
class CTriangle: public CPolygon {
public:
int area ()
{return (width * height/2);}
};
int main () {
CRectangle rect;
CTriangle trgl;
CPolygon poly;
CPolygon * ppoly1 = &rect;
CPolygon * ppoly2 = &trgl;
CPolygon * ppoly3 = &poly;
ppoly1->set_values (4,5);
ppoly2->set_values (4,5);
ppoly3->set_values (4,5);
cout << ppoly1->area() << endl;
cout << ppoly2->area() << endl;
cout << ppoly3->area() << endl;
return 0;
} 20
10
0
```

Now the three classes (CPolygon, CRectangle and CTriangle) have all the same members: width, height, set_values() and area().

The member function area() has been declared as virtual in the base class because it is later redefined in each derived class. You can verify if you want that if you remove this virtual keyword from the declaration of area() within CPolygon, and then you run the programme the result will be 0 for the three polygons instead of 20, 10 and 0. That is because instead of calling the corresponding area() function for each object (CRectangle::area(), CTriangle::area() and CPolygon::area(), respectively), CPolygon::area() will be called in all cases since the calls are via a pointer whose type is CPolygon*.

Therefore, what the virtual keyword does is to allow a member of a derived class with the same name as one in the base class to be appropriately called from a pointer, and more precisely when the type of the pointer is a pointer to the base class but is pointing to an object of the derived class, as in the above example. A class that declares or inherits a virtual function is called a *polymorphic class*.

Note that despite of its virtuality, we have also been able to declare an object of type CPolygon and to call its own area() function, which always returns 0.

Abstract base classes: Abstract base classes are something very similar to our CPolygon class of our previous example. The only difference is that in our previous example we have defined a valid area() function with a minimal functionality for objects that were of class CPolygon (like the object poly), whereas in an abstract base classes we could leave that area() member function without implementation at all. This is done by appending =0 (equal to zero) to the function declaration.

An abstract base CPolygon class could look like this:

```
//abstract class CPolygon
class CPolygon {
protected:
int width, height;
public:
void set_values (int a, int b)
{width=a; height=b;}
virtual int area () =0;
};
```

Notice how we appended =0 to virtual int area () instead of specifying an implementation for the function. This type of function is called a *pure virtual function*, and all classes that contain at least one pure virtual function are *abstract base classes*.

The main difference between an abstract base class and a regular polymorphic class is that because in abstract base classes at least one of its members lacks implementation we cannot create instances (objects) of it. But a class that cannot instantiate objects is not totally useless; We can create pointers to it and take advantage of all its polymorphic abilities. Therefore a declaration like:

```
CPolygon poly;
```

would not be valid for the abstract base class we have just declared, because tries to instantiate an object. Nevertheless, the following pointers:

```
CPolygon * ppoly1;
CPolygon * ppoly2;
```

would be perfectly valid.

This is so for as long as CPolygon includes a pure virtual function and therefore it's an abstract base class. However, pointers to this abstract base class can be used to point to objects of derived classes.

Here you have the complete example:

```
//abstract base class
#include <iostream>
using namespace std;
class CPolygon {
protected:
int width, height;
public:
void set_values (int a, int b)
{width=a; height=b;}
virtual int area (void) =0;
};
class CRectangle: public CPolygon {
public:
int area (void)
{return (width * height);}
};
class CTriangle: public CPolygon {
public:
int area (void)
{return (width * height/2);}
};
int main () {
CRectangle rect;
CTriangle trgl;
CPolygon * ppoly1 = &rect;
CPolygon * ppoly2 = &trgl;
ppoly1->set_values (4,5);
ppoly2->set_values (4,5);
cout << ppoly1->area() << endl;
cout << ppoly2->area() << endl;
return 0;
} 20
10
```

If you review the programme you will notice that we refer to objects of different but related classes using a unique type of pointer (CPolygon*). This can be tremendously useful. For example, now we can create a function member of the abstract base class CPolygon that is able to print on screen the result of the area() function even though CPolygon itself has no implementation for this function:

```

//pure virtual members can be called
//from the abstract base class
#include <iostream>
using namespace std;
class CPolygon {
protected:
int width, height;
public:
void set_values (int a, int b)
{width=a; height=b;}
virtual int area (void) =0;
void printarea (void)
{cout << this->area() << endl;}
};
class CRectangle: public CPolygon {
public:
int area (void)
{return (width * height);}
};
class CTriangle: public CPolygon {
public:
int area (void)
{return (width * height/2);}
};
int main () {
CRectangle rect;
CTriangle trgl;
CPolygon * ppoly1 = &rect;
CPolygon * ppoly2 = &trgl;
ppoly1->set_values (4,5);
ppoly2->set_values (4,5);
ppoly1->printarea();
ppoly2->printarea();
return 0;
} 20
10

```

Virtual members and abstract classes grant C++ the polymorphic characteristics that make object-oriented programming such a useful instrument in big projects. Of course, we have seen very simple uses of these features, but these features can be applied to arrays of objects or dynamically allocated objects.

Let's end with the same example again, but this time with objects that are dynamically allocated:

```

//dynamic allocation and polymorphism
#include <iostream>
using namespace std;
class CPolygon {
protected:
int width, height;
public:

```



```

void set_values (int a, int b)
{width=a; height=b;}
virtual int area (void) =0;
void printarea (void)
{cout << this->area() << endl;}
};
class CRectangle: public CPolygon {
public:
int area (void)
{return (width * height);}
};
class CTriangle: public CPolygon {
public:
int area (void)
{return (width * height/2);}
};
int main () {
CPolygon * ppoly1 = new CRectangle;
CPolygon * ppoly2 = new CTriangle;
ppoly1->set_values (4,5);
ppoly2->set_values (4,5);
ppoly1->printarea();
ppoly2->printarea();
delete ppoly1;
delete ppoly2;
return 0;
} 20
10

```

Notice that the ppoly pointers:

```

CPolygon * ppoly1 = new CRectangle;
CPolygon * ppoly2 = new CTriangle;

```

are declared being of type pointer to CPolygon but the objects dynamically allocated have been declared having the derived class type directly.

Templates: In C++, *templates* are used to define generic functions and classes. A *generic* function or class is one which takes a *type* as an actual parameter. As a result, a single function or class definition gives rise to a family of functions or classes that are compiled from the same source code, but operate on different types.

E.g., consider the following function definition:

```

int Max (int x, int y)
{return x > y ? x: y;}

```

The Max function takes two arguments of type int and returns the larger of the two. To compute the maximum value of a pair doubles, we require a different Max function.

By using a template, we can define a generic Max function like this:

```

template <class T>
T Max (T x, T y)
{return x > y ? x: y;}

```

The template definition gives rise to a family of Max functions, one for every different type T. The C++ compiler automatically creates the appropriate Max functions as needed. *E.g.*, the code sequence

```
int i = Max (1, 2);
double d = Max (1.0, 2.0);
```

causes the creation of two Max functions, one for arguments of type int and the other for arguments of type double.

Templates can also be used to define generic classes. *E.g.*, consider the following class definition:

```
class Stack
{
//...
public:
void Push (int);
int Pop ();
};
```

This Stack class represents a stack of integers. The member functions Push and Pop are used to insert and to withdraw an int from the stack (respectively). If instead of ints a stack of doubles is required, we must write a different Stack class. By using a template as shown in Programme we can define a generic Stack class.

Programme: Stack Template Definition

The required classes are created automatically by the C++ compiler as needed. *E.g.*, the code sequence

```
Stack<int> s1;
s1.Push (1);
Stack<double> s2;
s2.Push (1.0);
```

causes the creation of two stack classes, Stack<int> and Stack<double>. This example also illustrates that the name of the class includes the actual type parameter used to create it.

Exceptions: Sometimes unexpected situations arise during the execution of a programme. Careful programmers write code that detects errors and deals with them appropriately. However, a simple algorithm can become unintelligible when error-checking is added because the error-checking code can obscure the normal operation of the algorithm.

Exceptions provide a clean way to detect and handle unexpected situations. When a programme detects an error, it *throws* an exception. When an exception is thrown, control is transferred to the appropriate *exception handler*. By defining a function that *catches* the exception, the programmer can write the code to handle the error. In C++, an exception is an object.

Typically exceptions are derived (directly or indirectly) from the base class called exception which is defined in the C++ standard library header file called <exception>.

A function throws an exception by using the throw statement:

```
class domain_error: public exception {};  
void f ()  
{  
    //...  
    throw domain_error ();  
}
```

The throw statement is similar to a return statement. A return statement represents the normal termination of a function and the object returned matches the return value of the function. A throw statement represents the abnormal termination of a function and the object thrown represents the type of error encountered.

Exception handlers are defined using a try block:

```
void g ()  
{  
    try {  
        f ();  
    }  
    catch (domain_error) {  
        //exception handler  
    }  
    catch (range_error) {  
        //exception handler  
    }  
}
```

The body of the try block is executed either until an exception is thrown or until it terminates normally. One or more exception handlers follow a try block. Each exception handler consists of a catch clause which specifies the exceptions to be caught, and a block of code, which is executed when the exception occurs. When the body of the try block throws an exception for which an exception is defined, control is transferred to the body of the exception handler.

In this example, the exception was thrown by the function f() and caught by the function g(). In general when an exception is thrown, the chain of functions called is searched in reverse (from caller to callee) to find the closest matching catch statement. As the stack of called functions unwinds, the destructors for the local variables declared in those functions are automatically executed. When a programme throws an exception that is not caught, the programme terminates.

Function templates: Function templates are special functions that can operate with *generic types*. This allows us to create a function template whose functionality can be adapted to more than one type or class without repeating the entire code for each type.

In C++ this can be achieved using *template parameters*. A template parameter is a special kind of parameter that can be used to pass a type as argument: just like regular function parameters can be used to pass values to a function, template parameters allow to pass also types to a function. These function templates can use these parameters as if they were any other regular type.

The format for declaring function templates with type parameters is:

```
template <class identifier> function_ declaration;
template <typename identifier> function_ declaration;
```

The only difference between both prototypes is the use of either the keyword `class` or the keyword `typename`. Its use is indistinct, since both expressions have exactly the same meaning and behave exactly the same way.

For example, to create a template function that returns the greater one of two objects we could use:

```
template <class myType>
myType GetMax (myType a, myType b) {
return (a>b?a:b);
}
```

Here we have created a template function with `myType` as its template parameter. This template parameter represents a type that has not yet been specified, but that can be used in the template function as if it were a regular type. As you can see, the function template `GetMax` returns the greater of two parameters of this still-undefined type.

To use this function template we use the following format for the function call:
function_name <type> (parameters);

For example, to call `GetMax` to compare two integer values of type `int` we can write:

```
int x,y;
GetMax <int> (x,y);
```

When the compiler encounters this call to a template function, it uses the template to automatically generate a function replacing each appearance of `myType` by the type passed as the actual template parameter (`int` in this case) and then calls it. This process is automatically performed by the compiler and is invisible to the programmer.

Here is the entire example:

```
//function template
#include <iostream>
using namespace std;
template <class T>
T GetMax (T a, T b) {
T result;
result = (a>b)? a: b;
return (result);
}
int main () {
int i=5, j=6, k;
long l=10, m=5, n;
k=GetMax<int>(i,j);
n=GetMax<long>(l,m);
cout << k << endl;
cout << n << endl;
return 0;
} 6
10
```

In this case, we have used T as the template parameter name instead of myType because it is shorter and in fact is a very common template parameter name. But you can use any identifier you like. In the example above we used the function template GetMax() twice. The first time with arguments of type int and the second one with arguments of type long. The compiler has instantiated and then called each time the appropriate version of the function. As you can see, the type T is used within the GetMax() template function even to declare new objects of that type:

```
T result;
```

Therefore, result will be an object of the same type as the parameters a and b when the function template is instantiated with a specific type. In this specific case where the generic type T is used as a parameter for GetMax the compiler can find out automatically which data type has to instantiate without having to explicitly specify it within angle brackets (like we have done before specifying <int> and <long>). So we could have written instead:

```
int i,j;
GetMax (i,j);
```

Since both i and j are of type int, and the compiler can automatically find out that the template parameter can only be int. This implicit method produces exactly the same result:

```
//function template II
#include <iostream>
using namespace std;
template <class T>
T GetMax (T a, T b) {
return (a>b?a:b);
}
int main () {
int i=5, j=6, k;
long l=10, m=5, n;
k=GetMax(i,j);
n=GetMax(l,m);
cout << k << endl;
cout << n << endl;
return 0;
} 6
10
```

Notice how in this case, we called our function template GetMax() without explicitly specifying the type between angle-brackets <>. The compiler automatically determines what type is needed on each call.

Because our template function includes only one template parameter (class T) and the function template itself accepts two parameters, both of this T type, we cannot call our function template with two objects of different types as arguments:

```
int i;
long l;
k = GetMax (i,l);
```

This would not be correct, since our `GetMax` function template expects two arguments of the same type, and in this call to it we use objects of two different types.

We can also define function templates that accept more than one type parameter, simply by specifying more template parameters between the angle brackets. For example:

```
template <class T, class U>
T GetMin (T a, U b) {
return (a<b?a:b);
}
```

In this case, our function template `GetMin()` accepts two parameters of different types and returns an object of the same type as the first parameter (`T`) that is passed. For example, after that declaration we could call `GetMin()` with:

```
int i,j;
long l;
i = GetMin<int,long> (j,l);
```

or simply:

```
i = GetMin (j,l);
```

even though `j` and `l` have different types, since the compiler can determine the appropriate instantiation anyway.

Class templates

We also have the possibility to write class templates, so that a class can have members that use template parameters as types. For example:

```
template <class T>
class mypair {
T values [2];
public:
mypair (T first, T second)
{
values[0]=first; values[1]=second;
}
};
```

The class that we have just defined serves to store two elements of any valid type. For example, if we wanted to declare an object of this class to store two integer values of type `int` with the values 115 and 36 we would write:

```
mypair<int> myobject (115, 36);
```

this same class would also be used to create an object to store any other type:

```
mypair<double> myfloats (3.0, 2.18);
```

The only member function in the previous class template has been defined inline within the class declaration itself. In case that we define a function member outside the declaration of the class template, we must always precede that definition with the template `<...>` prefix:

```
//class templates
#include <iostream>
using namespace std;
template <class T>
class mypair {
T a, b;
```

```

public:
mypair (T first, T second)
{a=first; b=second;}
T getmax ();
};
template <class T>
T mypair<T>::getmax ()
{
T retval;
retval = a>b? a: b;
return retval;
}
int main () {
mypair <int> myobject (100, 75);
cout << myobject.getmax();
return 0;
} 100

```

Notice the syntax of the definition of member function getmax:

```

template <class T>
T mypair<T>::getmax ()

```

Confused by so many T's? There are three T's in this declaration: The first one is the template parameter. The second T refers to the type returned by the function. And the third T (the one between angle brackets) is also a requirement: It specifies that this function's template parameter is also the class template parameter.

Template specialization: If we want to define a different implementation for a template when a specific type is passed as template parameter, we can declare a specialization of that template.

For example, let's suppose that we have a very simple class called mycontainer that can store one element of any type and that it has just one member function called increase, which increases its value. But we find that when it stores an element of type char it would be more convenient to have a completely different implementation with a function member uppercase, so we decide to declare a class template specialization for that type:

```

//template specialization
#include <iostream>
using namespace std;
//class template:
template <class T>
class mycontainer {
T element;
public:
mycontainer (T arg) {element=arg;}
T increase () {return ++element;}
};
//class template specialization:
template <>
class mycontainer <char> {

```

```

char element;
public:
mycontainer (char arg) {element=arg;}
char uppercase ()
{
if ((element>='a') && (element<='z'))
element+='A'-'a';
return element;
}
};
int main () {
mycontainer<int> myint (7);
mycontainer<char> mychar ('j');
cout << myint.increase() << endl;
cout << mychar.uppercase() << endl;
return 0;
} 8
J

```

This is the syntax used in the class template specialization:

```
template <> class mycontainer <char> {...};
```

First of all, notice that we precede the class template name with an empty template<> parameter list. This is to explicitly declare it as a template specialization.

But more important than this prefix, is the <char> specialization parameter after the class template name. This specialization parameter itself identifies the type for which we are going to declare a template class specialization (char). Notice the differences between the generic class template and the specialization:

```
template <class T> class mycontainer {...};
template <> class mycontainer <char> {...};
```

The first line is the generic template, and the second one is the specialization.

When we declare specializations for a template class, we must also define all its members, even those exactly equal to the generic template class, because there is no “inheritance” of members from the generic template to the specialization.

Non-type parameters for templates: Besides the template arguments that are preceded by the class or typename keywords, which represent types, templates can also have regular typed parameters, similar to those found in functions. As an example, have a look at this class template that is used to contain sequences of elements:

```

//sequence template
#include <iostream>
using namespace std;
template <class T, int N>
class mysequence {
T memblock [N];
public:
void setmember (int x, T value);
T getmember (int x);
};
template <class T, int N>

```



```

void mysequence<T,N>::setmember (int x, T value) {
memblock[x]=value;
}
template <class T, int N>
T mysequence<T,N>::getmember (int x) {
return memblock[x];
}
int main () {
mysequence <int,5> myints;
mysequence <double,5> myfloats;
myints.setmember (0,100);
myfloats.setmember (3,3.1416);
cout << myints.getmember(0) << '\n';
cout << myfloats.getmember(3) << '\n';
return 0;
} 100
3.1416

```

It is also possible to set default values or types for class template parameters. For example, if the previous class template definition had been:

```
template <class T=char, int N=10> class mysequence {...};
```

We could create objects using the default template parameters by declaring:

```
mysequence<> myseq;
```

Which would be equivalent to:

```
mysequence<char,10> myseq;
```

Templates and multiple-file projects: From the point of view of the compiler, templates are not normal functions or classes. They are compiled on demand, meaning that the code of a template function is not compiled until an instantiation with specific template arguments is required. At that moment, when an instantiation is required, the compiler generates a function specifically for those arguments from the template.

When projects grow it is usual to split the code of a programme in different source code files. In these cases, the interface and implementation are generally separated. Taking a library of functions as example, the interface generally consists of declarations of the prototypes of all the functions that can be called. These are generally declared in a “header file” with a.h extension, and the implementation (the definition of these functions) is in an independent file with c++ code. Because templates are compiled when required, this forces a restriction for multi-file projects: the implementation (definition) of a template class or function must be in the same file as its declaration. That means that we cannot separate the interface in a separate header file, and that we must include both interface and implementation in any file that uses the templates.

Since no code is generated until a template is instantiated when required, compilers are prepared to allow the inclusion more than once of the same template file with both declarations and definitions in a project without generating linkage errors.

Name spaces: Name spaces allow to group entities like classes, objects and functions under a name. This way the global scope can be divided in “sub-scopes”, each one with its own name.

```

Namespace identifier
{
  entities
}

```

Where identifier is any valid identifier and entities is the set of classes, objects and functions that are included within the namespace.

For example:

```

namespace myNamespace
{
  int a, b;
}

```

In this case, the variables a and b are normal variables declared within a namespace called myNamespace. In order to access these variables from outside the myNamespace namespace we have to use the scope operator::. For example, to access the previous variables from outside myNamespace we can write:

```

myNamespace::a
myNamespace::b

```

The functionality of namespaces is especially useful in the case that there is a possibility that a global object or function uses the same identifier as another one, causing redefinition errors.

For example:

```

//namespaces
#include <iostream>
using namespace std;
namespace first
{
  int var = 5;
}
namespace second
{
  double var = 3.1416;
}
int main () {
  cout << first::var << endl;
  cout << second::var << endl;
  return 0;
} 5
3.1416

```

In this case, there are two global variables with the same name: var. One is defined within the namespace first and the other one in second. No redefinition errors happen thanks to namespaces.

using

The keyword using is used to introduce a name from a namespace into the current declarative region. For example:

```

//using
#include <iostream>
using namespace std;

```

```

namespace first
{
int x = 5;
int y = 10;
}
namespace second
{
double x = 3.1416;
double y = 2.7183;
}
int main () {
using first::x;
using second::y;
cout << x << endl;
cout << y << endl;
cout << first::y << endl;
cout << second::x << endl;
return 0;
} 5
2.7183
10
3.1416

```

Notice how in this code, `x` (without any name qualifier) refers to `first::x` whereas `y` refers to `second::y`, exactly as our `using` declarations have specified. We still have access to `first::y` and `second::x` using their fully qualified names.

The keyword `using` can also be used as a directive to introduce an entire namespace:

```

//using
#include <iostream>
using namespace std;
namespace first
{
int x = 5;
int y = 10;
}
namespace second
{
double x = 3.1416;
double y = 2.7183;
}
int main () {
using namespace first;
cout << x << endl;
cout << y << endl;
cout << second::x << endl;
cout << second::y << endl;
return 0;
} 5
10
3.1416
2.7183

```

In this case, since we have declared that we were using namespace first, all direct uses of `x` and `y` without name qualifiers were referring to their declarations in namespace first.

Using and using namespace have validity only in the same block in which they are stated or in the entire code if they are used directly in the global scope. For example, if we had the intention to first use the objects of one namespace and then those of another one, we could do something like:

```
//using namespace example
#include <iostream>
using namespace std;
namespace first
{
    int x = 5;
}
namespace second
{
    double x = 3.1416;
}
int main () {
    {
        using namespace first;
        cout << x << endl;
    }
    {
        using namespace second;
        cout << x << endl;
    }
    return 0;
} 5
3.1416
```

Name space alias: We can declare alternate names for existing name spaces according to the following format:

```
namespace new_name = current_name;
```

Namespace std: All the files in the C++ standard library declare all of its entities within the `std` namespace. That is why we have generally included the `using namespace std;` statement in all programmes that used any entity defined in `iostream`.

Exceptions: Exceptions provide a way to react to exceptional circumstances (like runtime errors) in our programme by transferring control to special functions called *handlers*.

To catch exceptions we must place a portion of code under exception inspection. This is done by enclosing that portion of code in a *try block*. When an exceptional circumstance arises within that block, an exception is thrown that transfers the control to the exception handler. If no exception is thrown, the code continues normally and all handlers are ignored.

A exception is thrown by using the `throw` keyword from inside the `try` block. Exception handlers are declared with the keyword `catch`, which must be placed immediately after the `try` block:

```
//exceptions
#include <iostream>
using namespace std;
int main () {
try
{
throw 20;
}
catch (int e)
{
cout << "An exception occurred. Exception Nr. " << e << endl;
}
return 0;
} An exception occurred. Exception Nr. 20
```

The code under exception handling is enclosed in a try block. In this example this code simply throws an exception:

```
throw 20;
```

A throw expression accepts one parameter (in this case the integer value 20), which is passed as an argument to the exception handler. The exception handler is declared with the catch keyword. As you can see, it follows immediately the closing brace of the try block. The catch format is similar to a regular function that always has at least one parameter. The type of this parameter is very important, since the type of the argument passed by the throw expression is checked against it, and only in the case they match, the exception is caught. We can chain multiple handlers (catch expressions), each one with a different parameter type. Only the handler that matches its type with the argument specified in the throw statement is executed.

If we use an ellipsis (...) as the parameter of catch, that handler will catch any exception no matter what the type of the throw exception is. This can be used as a default handler that catches all exceptions not caught by other handlers if it is specified at last:

```
try {
//code here
}
catch (int param) {cout << "int exception";}
catch (char param) {cout << "char exception";}
catch (...) {cout << "default exception";}
```

In this case the last handler would catch any exception thrown with any parameter that is neither an int nor a char.

After an exception has been handled the programme execution resumes after the try-catch block, not after the throw statement!. It is also possible to nest try-catch blocks within more external try blocks. In these cases, we have the possibility that an internal catch block forwards the exception to its external level. This is done with the expression throw; with no arguments.

For example:

```
try {
try {
```

```
//code here
}
catch (int n) {
throw;
}
}
catch (...) {
cout << "Exception occurred";
}
}
```

Exception specifications: When declaring a function we can limit the exception type it might directly or indirectly throw by appending a throw suffix to the function declaration:

```
float myfunction (char param) throw (int);
```

This declares a function called myfunction which takes one argument of type char and returns an element of type float. The only exception that this function might throw is an exception of type int. If it throws an exception with a different type, either directly or indirectly, it cannot be caught by a regular int-type handler.

If this throw specifier is left empty with no type, this means the function is not allowed to throw exceptions. Functions with no throw specifier (regular functions) are allowed to throw exceptions with any type:

```
int myfunction (int param) throw();//no exceptions allowed
int myfunction (int param);//all exceptions allowed
```

Standard exceptions: The C++ Standard library provides a base class specifically designed to declare objects to be thrown as exceptions. It is called exception and is defined in the <exception> header file under the namespace std. This class has the usual default and copy constructors, operators and destructors, plus an additional virtual member function called what that returns a null-terminated character sequence (char *) and that can be overwritten in derived classes to contain some sort of description of the exception.

```
//standard exceptions
#include <iostream>
#include <exception>
using namespace std;
class myexception: public exception
{
virtual const char* what() const throw()
{
return "My exception happened";
}
} myex;
int main () {
try
{
throw myex;
}
catch (exception& e)
{
}
```

```
cout << e.what() << endl;
}
return 0;
} My exception happened.
```

We have placed a handler that catches exception objects by reference (notice the ampersand & after the type), therefore this catches also classes derived from exception, like our myex object of class myexception.

All exceptions thrown by components of the C++ Standard library throw exceptions derived from this `std::exception` class.

These are:

```
exception description
bad_alloc thrown by new on allocation failure
bad_cast thrown by dynamic_cast when fails with a referenced type
bad_exception thrown when an exception type doesn't match any catch
bad_typeid thrown by typeid
ios_base::failure thrown by functions in the iostream library
```

For example, if we use the operator `new` without (`nothrow`) and the memory cannot be allocated, an exception of type `bad_alloc` is thrown:

```
try
{
int * myarray= new int[1000];
}
catch (bad_alloc&)
{
cout << "Error allocating memory." << endl;
}
```

It is recommended to include all dynamic memory allocations within a try block that catches this type of exception to perform a clean action instead of an abnormal programme termination, which is what happens when this type of exception is thrown and not caught. If you want to force a `bad_alloc` exception to see it in action, you can try to allocate a huge array; On my system, trying to allocate 1 billion ints threw a `bad_alloc` exception.

Because `bad_alloc` is derived from the standard base class exception, we can handle that same exception by catching references to the exception class:

```
//bad_alloc standard exception
#include <iostream>
#include <exception>
using namespace std;
int main () {
try
{
int* myarray= new int[1000];
}
catch (exception& e)
{
cout << "Standard exception: " << e.what() << endl;
}
```

```
return 0;  
}
```

Type Casting: Converting an expression of a given type into another type is known as *type-casting*. We have already seen some ways to type cast:

Implicit conversion: Implicit conversions do not require any operator. They are automatically performed when a value is copied to a compatible type.

For example:

```
short a=2000;  
int b;  
b=a;
```

Here, the value of `a` has been promoted from `short` to `int` and we have not had to specify any type-casting operator. This is known as a standard conversion. Standard conversions affect fundamental data types, and allow conversions such as the conversions between numerical types (`short` to `int`, `int` to `float`, `double` to `int`...), to or from `bool`, and some pointer conversions. Some of these conversions may imply a loss of precision, which the compiler can signal with a warning. This can be avoided with an explicit conversion.

Computer Programmes and Programming Languages

Before a computer can be used to solve a given problem, it must first be programmed, that is, prepared for solving the problem by being given a set of instructions, or programme. The various programmes by which a computer controls aspects of its operations, such as those for translating data from one form to another, are known as software, as contrasted with hardware, which is the physical equipment comprising the installation. In most computers the moment-to-moment control of the machine resides in a special software programme called an operating system, or supervisor. Other forms of software include assemblers and compilers for programming languages and applications for business and home use. Software is of great importance; the usefulness of a highly sophisticated array of hardware can be severely compromised by the lack of adequate software.

Each instruction in the programme may be a simple, single step, telling the computer to perform some arithmetic operation, to read the data from some given location in the memory, to compare two numbers, or to take some other action. The programme is entered into the computer's memory exactly as if it were data, and on activation, the machine is directed to treat this material in the memory as instructions. Other data may then be read in and the computer can carry out the programme to solve the particular problem. Since computers are designed to operate with binary numbers, all data and instructions must be represented in this form; the machine language, in which the

computer operates internally, consists of the various binary codes that define instructions together with the formats in which the instructions are written.

Since it is time-consuming and tedious for a programmer to work in actual machine language, a programming language, or high-level language, designed for the programmer's convenience, is used for the writing of most programmes. The computer is programmed to translate this high-level language into machine language and then solve the original problem for which the programme was written. Certain high-level programming languages are universal, varying little from machine to machine.

DEVELOPMENT OF COMPUTERS

Although the development of digital computers is rooted in the abacus and early mechanical calculating devices, Charles Babbage is credited with the design of the first modern computer, the "analytical engine," during the 1830s.

American scientist Vannevar Bush built a mechanically operated device, called a differential analyser, in 1930; it was the first general-purpose analog computer. John Atanasoff constructed the first semielectronic digital computing device in 1939.

The first fully automatic calculator was the Mark I, or Automatic Sequence Controlled Calculator, begun in 1939 at Harvard by Howard Aiken, while the first all-purpose electronic digital computer, ENIAC (Electronic Numerical Integrator And Calculator), which used thousands of vacuum tubes, was completed in 1946 at the Univ. of Pennsylvania. UNIVAC (Universal Automatic Computer) became (1951) the first computer to handle both numeric and alphabetic data with equal facility; this was the first commercially available computer.

First-generation computers were supplanted by the transistorized computers of the late 1950s and early 60s, second-generation machines that were smaller, used less power, and could perform a million operations per second.

They, in turn, were replaced by the third-generation integrated-circuit machines of the mid-1960s and 1970s that were even smaller and were far more reliable. The 1980s and 90s were characterized by the development of the microprocessor and the evolution of increasingly smaller but powerful computers, such as the personal computer and personal digital assistant, which ushered in a period of rapid growth in the computer industry.

ANALOG COMPUTERS

An analog computer represents data as physical quantities and operates on the data by manipulating the quantities. It is designed to process data in which the variable quantities vary continuously; it translates the relationships between the variables of a problem into analogous relationships between electrical quantities, such as current and voltage, and solves the original problem by solving the equivalent problem, or analog, that is set up in its electrical circuits. Because of this feature, analog computers were

especially useful in the simulation and evaluation of dynamic situations, such as the flight of a space capsule or the changing weather patterns over a certain area.

The key component of the analog computer is the operational amplifier, and the computer's capacity is determined by the number of amplifiers it contains (often over 100). Although analog computers are commonly found in such forms as speedometers and watt-hour meters, they largely have been made obsolete for general-purpose mathematical computations and data storage by digital computers.

COMPUTER PROGRAMMING

Computer programming (often shortened to programming or coding) is the process of designing, writing, testing, debugging / troubleshooting, and maintaining the source code of computer programmes. This source code is written in a programming language. The purpose of programming is to create a programme that exhibits a certain desired behaviour. The process of writing source code often requires expertise in many different subjects, including knowledge of the application domain, specialized algorithms and formal logic.

DEFINITION

Hoc and Nguyen-Xuan define computer programming as “the process of transforming a mental plan in familiar terms into one compatible with the computer.” Said another way, programming is the craft of transforming requirements into something that a computer can execute.

OVERVIEW

Within software engineering, programming (the *implementation*) is regarded as one phase in a software development process. There is an ongoing debate on the extent to which the writing of programmes is an art, a craft or an engineering discipline. In general, good programming is considered to be the measured application of all three, with the goal of producing an efficient and evolvable software solution (the criteria for “efficient” and “evolvable” vary considerably). The discipline differs from many other technical professions in that programmers, in general, do not need to be licensed or pass any standardized (or governmentally regulated) certification tests in order to call themselves “programmers” or even “software engineers.”

However, representing oneself as a “Professional Software Engineer” without a license from an accredited institution is illegal in many parts of the world. However, because the discipline covers many areas, which may or may not include critical applications, it is debatable whether licensing is required for the profession as a whole. In most cases, the discipline is self-governed by the entities which require the programming, and sometimes very strict environments are defined (*e.g.* United States Air Force use of

AdaCore and security clearance). Another ongoing debate is the extent to which the programming language used in writing computer programmes affects the form that the final programme takes. This debate is analogous to that surrounding the Sapir–Whorf hypothesis in linguistics, which postulates that a particular spoken language’s nature influences the habitual thought of its speakers.

Different language patterns yield different patterns of thought. This idea challenges the possibility of representing the world perfectly with language, because it acknowledges that the mechanisms of any language condition the thoughts of its speaker community.

HISTORY

The Antikythera mechanism from ancient Greece was a calculator utilizing gears of various sizes and configuration to determine its operation, which tracked the metonic cycle still used in lunar-to-solar calendars, and which is consistent for calculating the dates of the Olympiads. Al-Jazari built programmable Automata in 1206. One system employed in these devices was the use of pegs and cams placed into a wooden drum at specific locations, which would sequentially trigger levers that in turn operated percussion instruments. The output of this device was a small drummer playing various rhythms and drum patterns. The Jacquard Loom, which Joseph Marie Jacquard developed in 1801, uses a series of pasteboard cards with holes punched in them. The hole pattern represented the pattern that the loom had to follow in weaving cloth. The loom could produce entirely different weaves using different sets of cards. Charles Babbage adopted the use of punched cards around 1830 to control his Analytical Engine. The synthesis of numerical calculation, predetermined operation and output, along with a way to organize and input instructions in a manner relatively easy for humans to conceive and produce, led to the modern development of computer programming. Development of computer programming accelerated through the Industrial Revolution. In the late 1880s, Herman Hollerith invented the recording of data on a medium that could then be read by a machine. Prior uses of machine readable media, above, had been for control, not data. “After some initial trials with paper tape, he settled on punched cards...”

To process these punched cards, first known as “Hollerith cards” he invented the tabulator, and the keypunch machines. These three inventions were the foundation of the modern information processing industry. In 1896 he founded the *Tabulating Machine Company* (which later became the core of IBM). The addition of a control panel (plugboard) to his 1906 Type I Tabulator allowed it to do different jobs without having to be physically rebuilt. By the late 1940s, there were a variety of plug-board programmable machines, called unit record equipment, to perform data-processing tasks (card reading). Early computer programmers used plug-boards for the variety of complex calculations requested of the newly invented machines. The invention of the von Neumann architecture allowed computer programmes to be stored in computer memory.

Early programmes had to be painstakingly crafted using the instructions (elementary operations) of the particular machine, often in binary notation. Every model of computer

would likely use different instructions (machine language) to do the same task. Later, assembly languages were developed that let the programmer specify each instruction in a text format, entering abbreviations for each operation code instead of a number and specifying addresses in symbolic form (*e.g.*, ADD X, TOTAL). Entering a programme in assembly language is usually more convenient, faster, and less prone to human error than using machine language, but because an assembly language is little more than a different notation for a machine language, any two machines with different instruction sets also have different assembly languages.

In 1954, FORTRAN was invented; it was the first high level programming language to have a functional implementation, as opposed to just a design on paper. (A high-level language is, in very general terms, any programming language that allows the programmer to write programmes in terms that are more abstract than assembly language instructions, *i.e.* at a level of abstraction “higher” than that of an assembly language.) It allowed programmers to specify calculations by entering a formula directly (*e.g.* $Y = X*2 + 5*X + 9$). The programme text, or *source*, is converted into machine instructions using a special programme called a compiler, which translates the FORTRAN programme into machine language. In fact, the name FORTRAN stands for “Formula Translation”.

Many other languages were developed, including some for commercial programming, such as COBOL. Programmes were mostly still entered using punched cards or paper tape. By the late 1960s, data storage devices and computer terminals became inexpensive enough that programmes could be created by typing directly into the computers. Text editors were developed that allowed changes and corrections to be made much more easily than with punched cards. (Usually, an error in punching a card meant that the card had to be discarded and a new one punched to replace it.)

As time has progressed, computers have made giant leaps in the area of processing power. This has brought about newer programming languages that are more abstracted from the underlying hardware. Although these high-level languages usually incur greater overhead, the increase in speed of modern computers has made the use of these languages much more practical than in the past. These increasingly abstracted languages typically are easier to learn and allow the programmer to develop applications much more efficiently and with less source code.

However, high-level languages are still impractical for a few programmes, such as those where low-level hardware control is necessary or where maximum processing speed is vital. Throughout the second half of the twentieth century, programming was an attractive career in most developed countries. Some forms of programming have been increasingly subject to offshore outsourcing (importing software and services from other countries, usually at a lower wage), making programming career decisions in developed countries more complicated, while increasing economic opportunities in less developed areas. It is unclear how far this trend will continue and how deeply it will impact programmer wages and opportunities.

MODERN PROGRAMMING

Quality Requirements

Whatever the approach to software development may be, the final programme must satisfy some fundamental properties. The following properties are among the most relevant:

- **Efficiency/performance:** the amount of system resources a programme consumes (processor time, memory space, slow devices such as disks, network bandwidth and to some extent even user interaction): the less, the better. This also includes correct disposal of some resources, such as cleaning up temporary files and lack of memory leaks.
- **Reliability:** how often the results of a programme are correct. This depends on conceptual correctness of algorithms, and minimization of programming mistakes, such as mistakes in resource management (*e.g.*, buffer overflows and race conditions) and logic errors (such as division by zero or off-by-one errors).
- **Robustness:** how well a programme anticipates problems not due to programmer error. This includes situations such as incorrect, inappropriate or corrupt data, unavailability of needed resources such as memory, operating system services and network connections, and user error.
- **Usability:** the ergonomics of a programme: the ease with which a person can use the programme for its intended purpose, or in some cases even unanticipated purposes. Such issues can make or break its success even regardless of other issues. This involves a wide range of textual, graphical and sometimes hardware elements that improve the clarity, intuitiveness, cohesiveness and completeness of a program's user interface.
- **Portability:** the range of computer hardware and operating system platforms on which the source code of a programme can be compiled/interpreted and run. This depends on differences in the programming facilities provided by the different platforms, including hardware and operating system resources, expected behaviour of the hardware and operating system, and availability of platform specific compilers (and sometimes libraries) for the language of the source code.
- **Maintainability:** the ease with which a programme can be modified by its present or future developers in order to make improvements or customizations, fix bugs and security holes, or adapt it to new environments. Good practices during initial development make the difference in this regard. This quality may not be directly apparent to the end user but it can significantly affect the fate of a programme over the long term.

Readability of Source Code

In computer programming, readability refers to the ease with which a human reader can comprehend the purpose, control flow, and operation of source code. It affects the aspects of quality above, including portability, usability and most importantly maintainability. Readability is important because programmers spend the majority of their time reading, trying to understand and modifying existing source code, rather than writing new source code.

Unreadable code often leads to bugs, inefficiencies, and duplicated code. A study found that a few simple readability transformations made code shorter and drastically reduced the time to understand it. Following a consistent programming style often helps readability.

However, readability is more than just programming style. Many factors, having little or nothing to do with the ability of the computer to efficiently compile and execute the code, contribute to readability. Some of these factors include:

- Different indentation styles (whitespace)
- Comments
- Decomposition
- Naming conventions for objects (such as variables, classes, procedures, etc.)

Algorithmic Complexity

The academic field and the engineering practice of computer programming are both largely concerned with discovering and implementing the most efficient algorithms for a given class of problem. For this purpose, algorithms are classified into *orders* using so-called Big O notation, $O(n)$, which expresses resource use, such as execution time or memory consumption, in terms of the size of an input. Expert programmers are familiar with a variety of well-established algorithms and their respective complexities and use this knowledge to choose algorithms that are best suited to the circumstances.

Methodologies

The first step in most formal software development projects is requirements analysis, followed by testing to determine value modeling, implementation, and failure elimination (debugging). There exist a lot of differing approaches for each of those tasks. One approach popular for requirements analysis is Use Case analysis.

Nowadays many programmers use forms of Agile software development where the various stages of formal software development are more integrated together into short cycles that take a few weeks rather than years. There are many approaches to the Software development process. Popular modeling techniques include Object-Oriented Analysis and Design (OOAD) and Model-Driven Architecture (MDA). The Unified Modeling Language (UML) is a notation used for both the OOAD and MDA. A similar technique

used for database design is Entity-Relationship Modeling (ER Modeling). Implementation techniques include imperative languages (object-oriented or procedural), functional languages, and logic languages.

Measuring Language Usage

It is very difficult to determine what are the most popular of modern programming languages. Some languages are very popular for particular kinds of applications (*e.g.*, COBOL is still strong in the corporate data center, often on large mainframes, FORTRAN in engineering applications, scripting languages in web development, and C in embedded applications), while some languages are regularly used to write many different kinds of applications. Also many applications use a mix of several languages in their construction and use.

Methods of measuring programming language popularity include: counting the number of job advertisements that mention the language, the number of books teaching the language that are sold (this overestimates the importance of newer languages), and estimates of the number of existing lines of code written in the language (this underestimates the number of users of business languages such as COBOL).

Debugging

Debugging is a very important task in the software development process, because an incorrect programme can have significant consequences for its users. Some languages are more prone to some kinds of faults because their specification does not require compilers to perform as much checking as other languages. Use of a static analysis tool can help detect some possible problems.

Debugging is often done with IDEs like Eclipse, Kdevelop, NetBeans, Code::Blocks, and Visual Studio. Standalone debuggers like gdb are also used, and these often provide less of a visual environment, usually using a command line.

PROGRAMMING LANGUAGE

A programming language is an artificial language designed to express computations that can be performed by a machine, particularly a computer. Programming languages can be used to create programmes that control the behaviour of a machine, to express algorithms precisely, or as a mode of human communication. The earliest programming languages predate the invention of the computer, and were used to direct the behaviour of machines such as Jacquard looms and player pianos. Thousands of different programming languages have been created, mainly in the computer field, with many more being created every year.

Most programming languages describe computation in an imperative style, *i.e.*, as a sequence of commands, although some languages, such as those that support functional

programming or logic programming, use alternative forms of description. A programming language is usually split into the two components of syntax (form) and semantics (meaning) and many programming languages have some kind of written specification of their syntax and/or semantics. Some languages are defined by a specification document, for example, the C programming language is specified by an ISO Standard, while other languages, such as Perl, have a dominant implementation that is used as a reference.

DEFINITIONS

A programming language is a notation for writing programmes, which are specifications of a computation or algorithm. Some, but not all, authors restrict the term “programming language” to those languages that can express *all* possible algorithms. Traits often considered important for what constitutes a programming language include:

- *Function and target:* A *computer programming language* is a language used to write computer programmes, which involve a computer performing some kind of computation or algorithm and possibly control external devices such as printers, disk drives, robots, and so on. For example PostScript programmes are frequently created by another programme to control a computer printer or display. More generally, a programming language may describe computation on some, possibly abstract, machine. It is generally accepted that a complete specification for a programming language includes a description, possibly idealized, of a machine or processor for that language. In most practical contexts, a programming language involves a computer; consequently programming languages are usually defined and studied this way. Programming languages differ from natural languages in that natural languages are only used for interaction between people, while programming languages also allow humans to communicate instructions to machines.
- *Abstractions:* Programming languages usually contain abstractions for defining and manipulating data structures or controlling the flow of execution. The practical necessity that a programming language support adequate abstractions is expressed by the abstraction principle; this principle is sometimes formulated as recommendation to the programmer to make proper use of such abstractions.
- *Expressive power:* The theory of computation classifies languages by the computations they are capable of expressing. All Turing complete languages can implement the same set of algorithms. ANSI/ISO SQL and Charity are examples of languages that are not Turing complete, yet often called programming languages.

Markup languages like XML, HTML or troff, which define structured data, are not generally considered programming languages. Programming languages may, however, share the syntax with markup languages if a computational semantics is defined. XSLT,

for example, is a Turing complete XML dialect. Moreover, LaTeX, which is mostly used for structuring documents, also contains a Turing complete subset.

The term *computer language* is sometimes used interchangeably with programming language. However, the usage of both terms varies among authors, including the exact scope of each. One usage describes programming languages as a subset of computer languages. In this vein, languages used in computing that have a different goal than expressing computer programmes are generically designated computer languages. For instance, markup languages are sometimes referred to as computer languages to emphasize that they are not meant to be used for programming. Another usage regards programming languages as theoretical constructs for programming abstract machines, and computer languages as the subset thereof that runs on physical computers, which have finite hardware resources. John C. Reynolds emphasizes that formal specification languages are just as much programming languages as are the languages intended for execution. He also argues that textual and even graphical input formats that affect the behaviour of a computer are programming languages, despite the fact they are commonly not Turing-complete, and remarks that ignorance of programming language concepts is the reason for many flaws in input formats.

ELEMENTS

All programming languages have some primitive building blocks for the description of data and the processes or transformations applied to them (like the addition of two numbers or the selection of an item from a collection). These primitives are defined by syntactic and semantic rules which describe their structure and meaning respectively.

Syntax

A programming language's surface form is known as its syntax. Most programming languages are purely textual; they use sequences of text including words, numbers, and punctuation, much like written natural languages. On the other hand, there are some programming languages which are more graphical in nature, using visual relationships between symbols to specify a programme.

The syntax of a language describes the possible combinations of symbols that form a syntactically correct programme. The meaning given to a combination of symbols is handled by semantics (either formal or hard-coded in a reference implementation). Since most languages are textual, this article discusses textual syntax.

Programming language syntax is usually defined using a combination of regular expressions (for lexical structure) and Backus–Naur Form (for grammatical structure). Below is a simple grammar, based on Lisp:

```
Wxpression ::= atom | list
Atom ::= number | symbol
Number ::= [+ -]? ['0' - '9'] +
```

Symbol ::= [*'A'-'Z'a'-'z'*].*

List ::= *(' expression* ')*

This grammar specifies the following:

- An *expression* is either an *atom* or a *list*;
- An *atom* is either a *number* or a *symbol*;
- A *number* is an unbroken sequence of one or more decimal digits, optionally preceded by a plus or minus sign;
- A *symbol* is a letter followed by zero or more of any characters (excluding whitespace); and
- A *list* is a matched pair of parentheses, with zero or more *expressions* inside it.

The following are examples of well-formed token sequences in this grammar: *'12345'*, *'()'*, *'(a b c232 (1))'*

Not all syntactically correct programmes are semantically correct. Many syntactically correct programmes are nonetheless ill-formed, per the language's rules; and may (depending on the language specification and the soundness of the implementation) result in an error on translation or execution. In some cases, such programmes may exhibit undefined behaviour. Even when a programme is well-defined within a language, it may still have a meaning that is not intended by the person who wrote it.

Using natural language as an example, it may not be possible to assign a meaning to a grammatically correct sentence or the sentence may be false:

- "Colorless green ideas sleep furiously." is grammatically well-formed but has no generally accepted meaning.
- "John is a married bachelor." is grammatically well-formed but expresses a meaning that cannot be true.

The following C language fragment is syntactically correct, but performs an operation that is not semantically defined (because *p* is a null pointer, the operations *p->real* and *p->im* have no meaning):

```
complex *p = NULL;
complex abs_p = sqrt (p->real * p->real + p->im * p->im);
```

If the type declaration on the first line were omitted, the programme would trigger an error on compilation, as the variable "p" would not be defined. But the programme would still be syntactically correct, since type declarations provide only semantic information.

The grammar needed to specify a programming language can be classified by its position in the Chomsky hierarchy. The syntax of most programming languages can be specified using a Type-2 grammar, *i.e.*, they are context-free grammars. Some languages, including Perl and Lisp, contain constructs that allow execution during the parsing phase. Languages that have constructs that allow the programmer to alter the behaviour of the parser make syntax analysis an undecidable problem, and generally blur the distinction between parsing and execution. In contrast to Lisp's macro system and Perl's

BEGIN blocks, which may contain general computations, C macros are merely string replacements, and do not require code execution.

Semantics

The term *semantics* refers to the meaning of languages, as opposed to their form (syntax).

Static Semantics

The static semantics defines restrictions on the structure of valid texts that are hard or impossible to express in standard syntactic formalisms. For compiled languages, static semantics essentially include those semantic rules that can be checked at compile time. Examples include checking that every identifier is declared before it is used (in languages that require such declarations) or that the labels on the arms of a case statement are distinct. Many important restrictions of this type, like checking that identifiers are used in the appropriate context (*e.g.* not adding a integer to a function name), or that subroutine calls have the appropriate number and type of arguments can be enforced by defining them as rules in a logic called a type system. Other forms of static analyses like data flow analysis may also be part of static semantics. Newer programming languages like Java and C# have definite assignment analysis, a form of data flow analysis, as part of their static semantics.

Dynamic Semantics

Once data has been specified, the machine must be instructed to perform operations on the data. For example, the semantics may define the strategy by which expressions are evaluated to values, or the manner in which control structures conditionally execute statements. The *dynamic semantics* (also known as *execution semantics*) of a language defines how and when the various constructs of a language should produce a programme behaviour. There are many ways of defining execution semantics. Natural language is often used to specify the execution semantics of languages commonly used in practice. A significant amount of academic research went into formal semantics of programming languages, which allow execution semantics to be specified in a formal manner. Results from this field of research have seen limited application to programming language design and implementation outside academia.

Type System

A type system defines how a programming language classifies values and expressions into *types*, how it can manipulate those types and how they interact. The goal of a type system is to verify and usually enforce a certain level of correctness in programmes written in that language by detecting certain incorrect operations. Any decidable type

system involves a trade-off: while it rejects many incorrect programmes, it can also prohibit some correct, albeit unusual programmes. In order to bypass this downside, a number of languages have *type loopholes*, usually unchecked casts that may be used by the programmer to explicitly allow a normally disallowed operation between different types. In most typed languages, the type system is used only to type check programmes, but a number of languages, usually functional ones, infer types, relieving the programmer from the need to write type annotations. The formal design and study of type systems is known as *type theory*.

Typed versus untyped languages: A language is *typed* if the specification of every operation defines types of data to which the operation is applicable, with the implication that it is not applicable to other types. For example, the data represented by “this text between the quotes” is a string. In most programming languages, dividing a number by a string has no meaning. Most modern programming languages will therefore reject any programme attempting to perform such an operation. In some languages, the meaningless operation will be detected when the programme is compiled (“static” type checking), and rejected by the compiler, while in others, it will be detected when the programme is run (“dynamic” type checking), resulting in a runtime exception. A special case of typed languages are the *single-type* languages. These are often scripting or markup languages, such as REXX or SGML, and have only one data type—most commonly character strings which are used for both symbolic and numeric data. In contrast, an *untyped language*, such as most assembly languages, allows any operation to be performed on any data, which are generally considered to be sequences of bits of various lengths. High-level languages which are untyped include BCPL and some varieties of Forth. In practice, while few languages are considered typed from the point of view of type theory (verifying or rejecting *all* operations), most modern languages offer a degree of typing. Many production languages provide means to bypass or subvert the type system.

Static versus dynamic typing: In *static typing* all expressions have their types determined prior to the programme being run (typically at compile-time). For example, 1 and (2+2) are integer expressions; they cannot be passed to a function that expects a string, or stored in a variable that is defined to hold dates.

Statically typed languages can be either *manifestly typed* or *type-inferred*. In the first case, the programmer must explicitly write types at certain textual positions (for example, at variable declarations). In the second case, the compiler *infers* the types of expressions and declarations based on context. Most mainstream statically typed languages, such as C++, C# and Java, are manifestly typed. Complete type inference has traditionally been associated with less mainstream languages, such as Haskell and ML. However, many manifestly typed languages support partial type inference; for example, Java and C# both infer types in certain limited cases.

Dynamic typing, also called *latent typing*, determines the type-safety of operations at runtime; in other words, types are associated with *runtime values* rather than *textual*

expressions. As with type-inferred languages, dynamically typed languages do not require the programmer to write explicit type annotations on expressions. Among other things, this may permit a single variable to refer to values of different types at different points in the programme execution. However, type errors cannot be automatically detected until a piece of code is actually executed, potentially making debugging more difficult. Ruby, Lisp, JavaScript, and Python are dynamically typed.

Weak and strong typing: *Weak typing* allows a value of one type to be treated as another, for example treating a string as a number. This can occasionally be useful, but it can also allow some kinds of programme faults to go undetected at compile time and even at runtime.

Strong typing prevents the above. An attempt to perform an operation on the wrong type of value raises an error. Strongly typed languages are often termed *type-safe* or *safe*.

An alternative definition for “weakly typed” refers to languages, such as Perl and JavaScript, which permit a large number of implicit type conversions. In JavaScript, for example, the expression `2 * x` implicitly converts `x` to a number, and this conversion succeeds even if `x` is null, undefined, an Array, or a string of letters. Such implicit conversions are often useful, but they can mask programming errors. *Strong* and *static* are now generally considered orthogonal concepts, but usage in the literature differs. Some use the term *strongly typed* to mean *strongly, statically typed*, or, even more confusingly, to mean simply *statically typed*. Thus C has been called both strongly typed and weakly, statically typed.

Standard Library and Run-time System

Most programming languages have an associated core library (sometimes known as the ‘standard library’, especially if it is included as part of the published language standard), which is conventionally made available by all implementations of the language. Core libraries typically include definitions for commonly used algorithms, data structures, and mechanisms for input and output.

A language’s core library is often treated as part of the language by its users, although the designers may have treated it as a separate entity. Many language specifications define a core that must be made available in all implementations, and in the case of standardized languages this core library may be required. The line between a language and its core library therefore differs from language to language. Indeed, some languages are designed so that the meanings of certain syntactic constructs cannot even be described without referring to the core library. For example, in Java, a string literal is defined as an instance of the `java.lang.String` class; similarly, in Smalltalk, an anonymous function expression (a “block”) constructs an instance of the library’s `BlockContext` class. Conversely, Scheme contains multiple coherent subsets that suffice to construct the rest of the language as library macros, and so the language designers do not even bother to say which portions of the language must be implemented as language constructs, and which must be implemented as parts of a library.

DESIGN AND IMPLEMENTATION

Programming languages share properties with natural languages related to their purpose as vehicles for communication, having a syntactic form separate from its semantics, and showing *language families* of related languages branching one from another. But as artificial constructs, they also differ in fundamental ways from languages that have evolved through usage. A significant difference is that a programming language can be fully described and studied in its entirety, since it has a precise and finite definition. By contrast, natural languages have changing meanings given by their users in different communities. While constructed languages are also artificial languages designed from the ground up with a specific purpose, they lack the precise and complete semantic definition that a programming language has.

Many languages have been designed from scratch, altered to meet new needs, combined with other languages, and eventually fallen into disuse. Although there have been attempts to design one “universal” programming language that serves all purposes, all of them have failed to be generally accepted as filling this role. The need for diverse programming languages arises from the diversity of contexts in which languages are used:

- Programmes range from tiny scripts written by individual hobbyists to huge systems written by hundreds of programmers.
- Programmers range in expertise from novices who need simplicity above all else, to experts who may be comfortable with considerable complexity.
- Programmes must balance speed, size, and simplicity on systems ranging from microcontrollers to super computers.
- Programmes may be written once and not change for generations, or they may undergo continual modification.
- Finally, programmers may simply differ in their tastes: they may be accustomed to discussing problems and expressing them in a particular language.

One common trend in the development of programming languages has been to add more ability to solve problems using a higher level of abstraction. The earliest programming languages were tied very closely to the underlying hardware of the computer. As new programming languages have developed, features have been added that let programmers express ideas that are more remote from simple translation into underlying hardware instructions. Because programmers are less tied to the complexity of the computer, their programmes can do more computing with less effort from the programmer. This lets them write more functionality per time unit. Natural language processors have been proposed as a way to eliminate the need for a specialized language for programming.

However, this goal remains distant and its benefits are open to debate. Edsger W. Dijkstra took the position that the use of a formal language is essential to prevent the introduction of meaningless constructs, and dismissed natural language programming as “foolish”. Alan Perlis was similarly dismissive of the idea. Hybrid approaches have

been taken in Structured English and SQL. A language's designers and users must construct a number of artifacts that govern and enable the practice of programming. The most important of these artifacts are the language *specification* and *implementation*.

Specification

The specification of a programming language is intended to provide a definition that the language users and the implementors can use to determine whether the behaviour of a programme is correct, given its source code. A programming language specification can take several forms, including the following:

- An explicit definition of the syntax, static semantics, and execution semantics of the language. While syntax is commonly specified using a formal grammar, semantic definitions may be written in natural language (*e.g.*, as in the C language), or a formal semantics (*e.g.*, as in Standard ML and Scheme specifications).
- A description of the behaviour of a translator for the language (*e.g.*, the C++ and Fortran specifications). The syntax and semantics of the language have to be inferred from this description, which may be written in natural or a formal language.
- A *reference* or *model* implementation, sometimes written in the language being specified (*e.g.*, Prolog or ANSI REXX). The syntax and semantics of the language are explicit in the behaviour of the reference implementation.

Implementation

An implementation of a programming language provides a way to execute that programme on one or more configurations of hardware and software. There are, broadly, two approaches to programming language implementation: *compilation* and *interpretation*. It is generally possible to implement a language using either technique. The output of a compiler may be executed by hardware or a programme called an interpreter. In some implementations that make use of the interpreter approach there is no distinct boundary between compiling and interpreting. For instance, some implementations of BASIC compile and then execute the source a line at a time. Programmes that are executed directly on the hardware usually run several orders of magnitude faster than those that are interpreted in software. One technique for improving the performance of interpreted programmes is just-in-time compilation. Here the virtual machine, just before execution, translates the blocks of bytecode which are going to be used to machine code, for direct execution on the hardware.

USAGE

Thousands of different programming languages have been created, mainly in the computing field. Programming languages differ from most other forms of human

expression in that they require a greater degree of precision and completeness. When using a natural language to communicate with other people, human authors and speakers can be ambiguous and make small errors, and still expect their intent to be understood. However, figuratively speaking, computers “do exactly what they are told to do”, and cannot “understand” what code the programmer intended to write. The combination of the language definition, a programme, and the program’s inputs must fully specify the external behaviour that occurs when the programme is executed, within the domain of control of that programme. A programming language provides a structured mechanism for defining pieces of data, and the operations or transformations that may be carried out automatically on that data. A programmer uses the abstractions present in the language to represent the concepts involved in a computation.

These concepts are represented as a collection of the simplest elements available (called primitives). *Programming* is the process by which programmers combine these primitives to compose new programmes, or adapt existing ones to new uses or a changing environment. Programmes for a computer might be executed in a batch process without human interaction, or a user might type commands in an interactive session of an interpreter. In this case the “commands” are simply programmes, whose execution is chained together. When a language is used to give commands to a software application (such as a shell) it is called a scripting language.

Measuring Language Usage

It is difficult to determine which programming languages are most widely used, and what usage means varies by context. One language may occupy the greater number of programmer hours, a different one have more lines of code, and a third utilize the most CPU time. Some languages are very popular for particular kinds of applications. For example, COBOL is still strong in the corporate data center, often on large mainframes; FORTRAN in scientific and engineering applications; C in embedded applications and operating systems; and other languages are regularly used to write many different kinds of applications.

Various methods of measuring language popularity, each subject to a different bias over what is measured, have been proposed:

- Counting the number of job advertisements that mention the language
- The number of books sold that teach or describe the language
- Estimates of the number of existing lines of code written in the language—which may underestimate languages not often found in public searches
- Counts of language references (*i.e.*, to the name of the language) found using a web search engine.

Combining and averaging information from various internet sites, langpop.com claims that in 2008 the 10 most cited programming languages are (in alphabetical order): C, C++, C#, Java, JavaScript, Perl, PHP, Python, Ruby, and SQL.

TAXONOMIES

There is no overarching classification scheme for programming languages. A given programming language does not usually have a single ancestor language. Languages commonly arise by combining the elements of several predecessor languages with new ideas in circulation at the time. Ideas that originate in one language will diffuse throughout a family of related languages, and then leap suddenly across familial gaps to appear in an entirely different family. The task is further complicated by the fact that languages can be classified along multiple axes.

For example, Java is both an object-oriented language (because it encourages object-oriented organization) and a concurrent language (because it contains built-in constructs for running multiple threads in parallel). Python is an object-oriented scripting language. In broad strokes, programming languages divide into *programming paradigms* and a classification by *intended domain of use*. Traditionally, programming languages have been regarded as describing computation in terms of imperative sentences, *i.e.* issuing commands. These are generally called imperative programming languages. A great deal of research in programming languages has been aimed at blurring the distinction between a programme as a set of instructions and a programme as an assertion about the desired answer, which is the main feature of declarative programming. More refined paradigms include procedural programming, object-oriented programming, functional programming, and logic programming; some languages are hybrids of paradigms or multi-paradigmatic. An assembly language is not so much a paradigm as a direct model of an underlying machine architecture. By purpose, programming languages might be considered general purpose, system programming languages, scripting languages, domain-specific languages, or concurrent/distributed languages (or a combination of these). Some general purpose languages were designed largely with educational goals. A programming language may also be classified by factors unrelated to programming paradigm. For instance, most programming languages use English language keywords, while a minority do not. Other languages may be classified as being esoteric or not.

HISTORY

Early Developments

The first programming languages predate the modern computer. The 19th century had “programmable” looms and player piano scrolls which implemented what are today recognized as examples of domain-specific languages. By the beginning of the twentieth century, punch cards encoded data and directed mechanical processing. In the 1930s and 1940s, the formalisms of Alonzo Church’s lambda calculus and Alan Turing’s Turing machines provided mathematical abstractions for expressing algorithms; the lambda calculus remains influential in language design. In the 1940s, the first electrically powered digital computers were created.

The first high-level programming language to be designed for a computer was Plankalkül, developed for the German Z3 by Konrad Zuse between 1943 and 1945. However, it was not implemented until 1998 and 2000. Programmers of early 1950s computers, notably UNIVAC I and IBM 701, used machine language programmes, that is, the first generation language (1GL). 1GL programming was quickly superseded by similarly machine-specific, but mnemonic, second generation languages (2GL) known as assembly languages or “assembler”.

Later in the 1950s, assembly language programming, which had evolved to include the use of macro instructions, was followed by the development of “third generation” programming languages (3GL), such as FORTRAN, LISP, and COBOL. 3GLs are more abstract and are “portable”, or at least implemented similarly on computers that do not support the same native machine code. Updated versions of all of these 3GLs are still in general use, and each has strongly influenced the development of later languages. At the end of the 1950s, the language formalized as ALGOL 60 was introduced, and most later programming languages are, in many respects, descendants of Algol. The format and use of the early programming languages was heavily influenced by the constraints of the interface.

Refinement

The period from the 1960s to the late 1970s brought the development of the major language paradigms now in use, though many aspects were refinements of ideas in the very first Third-generation programming languages:

- APL introduced *array programming* and influenced functional programming.
- PL/I (NPL) was designed in the early 1960s to incorporate the best ideas from FORTRAN and COBOL.
- In the 1960s, Simula was the first language designed to support *object-oriented programming*; in the mid-1970s, Smalltalk followed with the first “purely” object-oriented language.
- C was developed between 1969 and 1973 as a *system programming* language, and remains popular.
- Prolog, designed in 1972, was the first *logic programming* language.
- In 1978, ML built a polymorphic type system on top of Lisp, pioneering *statically typed functional programming* languages.

Each of these languages spawned an entire family of descendants, and most modern languages count at least one of them in their ancestry.

The 1960s and 1970s also saw considerable debate over the merits of *structured programming*, and whether programming languages should be designed to support it. Edsger Dijkstra, in a famous 1968 letter published in the Communications of the ACM, argued that GOTO statements should be eliminated from all “higher level” programming languages.

The 1960s and 1970s also saw expansion of techniques that reduced the footprint of a programme as well as improved productivity of the programmer and user. The card

deck for an early 4GL was a lot smaller for the same functionality expressed in a 3GL deck.

Consolidation and Growth

The 1980s were years of relative consolidation. C++ combined object-oriented and systems programming. The United States government standardized Ada, a systems programming language derived from Pascal and intended for use by defence contractors. In Japan and elsewhere, vast sums were spent investigating so-called “fifth generation” languages that incorporated logic programming constructs. The functional languages community moved to standardize ML and Lisp. Rather than inventing new paradigms, all of these movements elaborated upon the ideas invented in the previous decade.

One important trend in language design for programming large-scale systems during the 1980s was an increased focus on the use of *modules*, or large-scale organizational units of code. Modula-2, Ada, and ML all developed notable module systems in the 1980s, although other languages, such as PL/I, already had extensive support for modular programming. Module systems were often wedded to generic programming constructs.

The rapid growth of the Internet in the mid-1990s created opportunities for new languages. Perl, originally a Unix scripting tool first released in 1987, became common in dynamic web sites. Java came to be used for server-side programming, and bytecode virtual machines became popular again in commercial settings with their promise of “Write once, run anywhere” (UCSD Pascal had been popular for a time in the early 1980s). These developments were not fundamentally novel, rather they were refinements to existing languages and paradigms, and largely based on the C family of programming languages.

Programming language evolution continues, in both industry and research. Current directions include security and reliability verification, new kinds of modularity (mixins, delegates, aspects), and database integration such as Microsoft’s LINQ. The 4GLs are examples of languages which are domain-specific, such as SQL, which manipulates and returns sets of data rather than the scalar values which are canonical to most programming languages. Perl, for example, with its ‘here document’ can hold multiple 4GL programmes, as well as multiple JavaScript programmes, in part of its own perl code and use variable interpolation in the ‘here document’ to support multi-language programming.

HISTORY OF PROGRAMMING LANGUAGES

This article discusses the major developments in the history of programming languages. For a detailed timeline of events, see the timeline of programming languages.

BEFORE 1940

The first programming languages predate the modern computer. At first, the languages were codes. The Jacquard loom, invented in 1801, used holes in punched cards to

represent sewing loom arm movements in order to generate decorative patterns automatically. During a nine-month period in 1842-1843, Ada Lovelace translated the memoir of Italian mathematician Luigi Menabrea about Charles Babbage's newest proposed machine, the Analytical Engine. With the article, she appended a set of notes which specified in complete detail a method for calculating Bernoulli numbers with the Engine, recognized by some historians as the world's first computer programme.

Herman Hollerith realized that he could encode information on punch cards when he observed that train conductors encode the appearance of the ticket holders on the train tickets using the position of punched holes on the tickets. Hollerith then encoded the 1890 census data on punch cards. The first computer codes were specialized for their applications. In the first decades of the 20th century, numerical calculations were based on decimal numbers. Eventually it was realized that logic could be represented with numbers, not only with words. For example, Alonzo Church was able to express the lambda calculus in a formulaic way.

The Turing machine was an abstraction of the operation of a tape-marking machine, for example, in use at the telephone companies. Turing machines set the basis for storage of programmes as data in the von Neumann architecture of computers by representing a machine through a finite number. However, unlike the lambda calculus, Turing's code does not serve well as a basis for higher-level languages—its principal use is in rigorous analyses of algorithmic complexity. Like many “firsts” in history, the first modern programming language is hard to identify. From the start, the restrictions of the hardware defined the language.

Punch cards allowed 80 columns, but some of the columns had to be used for a sorting number on each card. FORTRAN included some keywords which were the same as English words, such as “IF”, “GOTO” (go to) and “CONTINUE”. The use of a magnetic drum for memory meant that computer programmes also had to be interleaved with the rotations of the drum. Thus the programmes were more hardware-dependent. To some people, what was the first modern programming language depends on how much power and human-readability is required before the status of “programming language” is granted. Jacquard looms and Charles Babbage's Difference Engine both had simple, extremely limited languages for describing the actions that these machines should perform. One can even regard the punch holes on a player piano scroll as a limited domain-specific language, albeit not designed for human consumption.

THE 1940S

In the 1940s, the first recognizably modern, electrically powered computers were created. The limited speed and memory capacity forced programmers to write hand tuned assembly language programmes. It was soon discovered that programming in assembly language required a great deal of intellectual effort and was error-prone. In 1948, Konrad Zuse published a paper about his programming language Plankalkül. However, it was not implemented in his lifetime and his original contributions were

isolated from other developments. Some important languages that were developed in this period include:

- 1943 - Plankalkül (Konrad Zuse), designed, but unimplemented for a half-century.
- 1943 - ENIAC coding system, machine-specific codeset appearing in 1948.
- 1949 - 1954 — a series of machine-specific mnemonic instruction sets, like ENIAC's, beginning in 1949 with C-10 for BINAC (which later evolved into UNIVAC). Each codeset, or instruction set, was tailored to a specific manufacturer.

THE 1950S AND 1960S

In the 1950s, the first three modern programming languages whose descendants are still in widespread use today were designed:

- FORTRAN (1955), the “FORMula TRANslator”, invented by John Backus et al.;
- LISP [1958], the “LIST Processor”, invented by John McCarthy et al.;
- COBOL, the COMmon Business Oriented Language, created by the Short Range Committee, heavily influenced by Grace Hopper.

Another milestone in the late 1950s was the publication, by a committee of American and European computer scientists, of “a new language for algorithms”; the *ALGOL 60 Report* (the “ALGOrithmic Language”). This report consolidated many ideas circulating at the time and featured two key language innovations:

- Nested block structure: code sequences and associated declarations could be grouped into blocks without having to be turned into separate, explicitly named procedures;
- Lexical scoping: a block could have its own private variables, procedures and functions, invisible to code outside that block, *i.e.* information hiding.

Another innovation, related to this, was in how the language was described:

- A mathematically exact notation, Backus-Naur Form (BNF), was used to describe the language's syntax. Nearly all subsequent programming languages have used a variant of BNF to describe the context-free portion of their syntax.

Algol 60 was particularly influential in the design of later languages, some of which soon became more popular. The Burroughs large systems were designed to be programmed in an extended subset of Algol. Algol's key ideas were continued, producing ALGOL 68:

- Syntax and semantics became even more orthogonal, with anonymous routines, a recursive typing system with higher-order functions, etc.;
- Not only the context-free part, but the full language syntax and semantics were defined formally, in terms of Van Wijngaarden grammar, a formalism designed specifically for this purpose.

Algol 68's many little-used language features (*e.g.* concurrent and parallel blocks) and its complex system of syntactic shortcuts and automatic type coercions made it unpopular with implementers and gained it a reputation of being *difficult*. Niklaus Wirth actually walked out of the design committee to create the simpler Pascal language. Some important languages that were developed in this period include:

- 1951 - Regional Assembly Language
- 1952 - Autocode
- 1954 - FORTRAN
- 1954 - IPL (forerunner to LISP)
- 1955 - FLOW-MATIC (forerunner to COBOL)
- 1957 - COMTRAN (forerunner to COBOL)
- 1958 - LISP
- 1958 - ALGOL 58
- 1959 - FACT (forerunner to COBOL)
- 1959 - COBOL
- 1962 - APL
- 1962 - Simula
- 1962 - SNOBOL
- 1963 - CPL (forerunner to C)
- 1964 - BASIC
- 1964 - PL/I
- 1967 - BCPL (forerunner to C).

1967-1978: ESTABLISHING FUNDAMENTAL PARADIGMS

The period from the late 1960s to the late 1970s brought a major flowering of programming languages. Most of the major language paradigms now in use were invented in this period:

- Simula, invented in the late 1960s by Nygaard and Dahl as a superset of Algol 60, was the first language designed to support object-oriented programming.
- C, an early systems programming language, was developed by Dennis Ritchie and Ken Thompson at Bell Labs between 1969 and 1973.
- Smalltalk (mid 1970s) provided a complete ground-up design of an object-oriented language.
- Prolog, designed in 1972 by Colmerauer, Roussel, and Kowalski, was the first logic programming language.
- ML built a polymorphic type system (invented by Robin Milner in 1973) on top of Lisp, pioneering statically typed functional programming languages.

Each of these languages spawned an entire family of descendants, and most modern languages count at least one of them in their ancestry. The 1960s and 1970s also saw considerable debate over the merits of “structured programming”, which essentially

meant programming without the use of Goto. This debate was closely related to language design: some languages did not include GOTO, which forced structured programming on the programmer. Although the debate raged hotly at the time, nearly all programmers now agree that, even in languages that provide GOTO, it is bad programming style to use it except in rare circumstances. As a result, later generations of language designers have found the structured programming debate tedious and even bewildering. Some important languages that were developed in this period include:

- 1968 - Logo
- 1969 - B (forerunner to C)
- 1970 - Pascal
- 1970 - Forth
- 1972 - C
- 1972 - Smalltalk
- 1972 - Prolog
- 1973 - ML
- 1975 - Scheme
- 1978 - SQL (initially only a query language, later extended with programming constructs).

THE 1980S: CONSOLIDATION, MODULES AND PERFORMANCE

The 1980s were years of relative consolidation in imperative languages. Rather than inventing new paradigms, all of these movements elaborated upon the ideas invented in the previous decade. C++ combined object-oriented and systems programming. The United States government standardized Ada, a systems programming language intended for use by defence contractors. In Japan and elsewhere, vast sums were spent investigating so-called fifth-generation programming languages that incorporated logic programming constructs. The functional languages community moved to standardize ML and Lisp. Research in Miranda, a functional language with lazy evaluation, began to take hold in this decade.

One important new trend in language design was an increased focus on programming for large-scale systems through the use of *modules*, or large-scale organizational units of code. Modula, Ada, and ML all developed notable module systems in the 1980s. Module systems were often wedded to generic programming constructs—generics being, in essence, parameterized modules.

Although major new paradigms for imperative programming languages did not appear, many researchers expanded on the ideas of prior languages and adapted them to new contexts. For example, the languages of the Argus and Emerald systems adapted object-oriented programming to distributed systems.

The 1980s also brought advances in programming language implementation. The RISC movement in computer architecture postulated that hardware should be designed for compilers rather than for human assembly programmers. Aided by processor speed

improvements that enabled increasingly aggressive compilation techniques, the RISC movement sparked greater interest in compilation technology for high-level languages. Language technology continued along these lines well into the 1990s. Some important languages that were developed in this period include:

- 1980 - C++ (as C with classes, name changed in July 1983)
- 1983 - Objective-C
- 1983 - Ada
- 1984 - Common Lisp
- 1985 - Eiffel
- 1986 - Erlang
- 1987 - Perl
- 1988 - Tcl
- 1989 - FL (Backus).

THE 1990S: THE INTERNET AGE

The 1990s saw no fundamental novelty in imperative languages, but much recombination and maturation of old ideas. This era began the spread of functional languages. A big driving philosophy was programmer productivity. Many “rapid application development” (RAD) languages emerged, which usually came with an IDE, garbage collection, and were descendants of older languages. All such languages were object-oriented. These included Object Pascal, Visual Basic, and Java. Java in particular received much attention. More radical and innovative than the RAD languages were the new scripting languages. These did not directly descend from other languages and featured new syntaxes and more liberal incorporation of features.

Many consider these scripting languages to be more productive than even the RAD languages, but often because of choices that make small programmes simpler but large programmes more difficult to write and maintain. Nevertheless, scripting languages came to be the most prominent ones used in connection with the Web. Some important languages that were developed in this period include:

- 1990 - Haskell
- 1991 - Python
- 1991 - Visual Basic
- 1993 - Ruby
- 1993 - Lua
- 1994 - CLOS (part of ANSI Common Lisp)
- 1995 - Java
- 1995 - Delphi (Object Pascal)
- 1995 - JavaScript
- 1995 - PHP
- 1997 - Rebol
- 1999 - D

CURRENT TRENDS

Programming language evolution continues, in both industry and research. Some of the current trends include:

- Mechanisms for adding security and reliability verification to the language: extended static checking, information flow control, static thread safety.
- Alternative mechanisms for modularity: mixins, delegates, aspects.
- Component-oriented software development.
- Constructs to support concurrent and distributed programming.
- Metaprogramming, reflection or access to the abstract syntax tree
- Increased emphasis on distribution and mobility.
- Integration with databases, including XML and relational databases.
- Support for Unicode so that source code (programme text) is not restricted to those characters contained in the ASCII character set; allowing, for example, use of non-Latin-based scripts or extended punctuation.
- XML for graphical interface (XUL, XAML).

Some important languages developed during this period include:

- 2001 - C#
- 2001 - Visual Basic .NET
- 2002 - F#
- 2003 - Scala
- 2003 - Factor
- 2006 - Windows Power Shell
- 2007 - Clojure
- 2007 - Groovy
- 2009 - Go.

PROMINENT PEOPLE IN THE HISTORY OF PROGRAMMING LANGUAGES

- John Backus, inventor of Fortran.
- Alan Cooper, developer of Visual Basic.
- Edsger W. Dijkstra, developed the framework for structured programming.
- James Gosling, developer of Oak, the precursor of Java.
- Anders Hejlsberg, developer of Turbo Pascal, Delphi and C#.
- Grace Hopper, developer of Flow-Matic, influencing COBOL.
- Kenneth E. Iverson, developer of APL, and co-developer of J along with Roger Hui.
- Bill Joy, inventor of vi, early author of BSD Unix, and originator of SunOS, which became Solaris.
- Alan Kay, pioneering work on object-oriented programming, and originator of Smalltalk.

- Brian Kernighan, coauthor of the first book on the C programming language with Dennis Ritchie, coauthor of the AWK and AMPL programming languages.
 - John McCarthy, inventor of LISP.
 - John von Neumann, originator of the operating system concept.
 - Dennis Ritchie, inventor of C (programming language). Unix Operating System, Plan 9 Operating System.
 - Bjarne Stroustrup, developer of C++.
 - Ken Thompson, inventor of B, Go Programming Language, Inferno Programming Language.
 - Niklaus Wirth, inventor of Pascal, Modula and Oberon.
 - Larry Wall, creator of Perl and Perl 6
 - Guido van Rossum, creator of Python
 - Yukihiro Matsumoto, creator of Ruby

PROGRAMMING LANGUAGE GENERATIONS

Programming languages have been classified into several programming language generations. Historically, this classification was used to indicate increasing power of programming styles. Later writers have somewhat redefined the meanings as distinctions previously seen as important became less significant to current practice.

HISTORICAL VIEW OF FIRST THREE GENERATIONS

The terms “first-generation” and “second-generation” programming language were not used prior to the coining of the term “third-generation.” In fact, none of these three terms are mentioned in an early compendium of programming language. The introduction of a third generation of computer technology coincided with the creation of a new generation of programming languages. The marketing for this generational shift in machines did correlate with several important changes in what were called high level programming languages, discussed below, giving technical content to the second/third-generation distinction among high level programming languages as well, and reflexively renaming assembler languages as first-generation.

First Generation

As Grace Hopper said about coding in machine language: “We were not programmers in those days. The word had not yet come over from England. We were coders.

The task of encoding an algorithm wasn’t thought of as writing in a language any more than was the task of wiring a plug-board. But even by the early 1950s, the assembly languages were seen as a distinct “epoch”. The distinguishing properties of these first generation programming languages are that:

- The code can be read and written by a programmer. To run on a computer it must be converted into a machine readable form, a process called assembly.
- The language is specific to a particular target machine or family of machines, directly reflecting their characteristics like instruction sets, registers, storage access models, etc., requiring and enabling the programmer to manage their use.
- Some assembler languages provide a macro-facility enabling the development of complex patterns of machine instructions, but these are not considered to change the basic nature of the language.

First-generation languages are sometimes used in kernels and device drivers, but more often find use in extremely intensive processing such as games, video editing, and graphic manipulation/rendering.

Second Generation

Second-generation programming languages, originally just called high level programming languages, were created to simplify the burden of programming by making its expression more like the normal mode of expression for thoughts used by the programmer. They were introduced in the late 1950s, with FORTRAN reflecting the needs of scientific programmers, ALGOL reflecting an attempt to produce a European/American standard view.

The most important issue faced by the developers of second-level languages was convincing customers that the code produced by the compilers performed well-enough to justify abandonment of assembler programming. In view of the widespread skepticism about the possibility of producing efficient programmes with an automatic programming system and the fact that inefficiencies could no longer be hidden, we were convinced that the kind of system we had in mind would be widely used only if we could demonstrate that it would produce programmes almost as efficient as hand coded ones and do so on virtually every job.

The FORTRAN compiler was seen as a tour-de-force in the production of high-quality code, even including "... a Monte Carlo simulation of its execution ... so as to minimize the transfers of items between the store and the index registers. Second-generation programming languages evolved through the decade. FORTRAN lost some of its machine-dependent features, like access to the lights and switches on the operator console.

Most second-generation languages employed a static storage model in which storage for data was allocated only once, when a programme is loaded, making recursion difficult, but Algol evolved to provide block-structured naming constructs and began to expand the set of features made available to programmers, like concurrency management. In this way (Algol 68) began the movement into a new generation of programming languages.

Third Generation

The introduction of a third generation of computer technology coincided with the creation of a new generation of programming languages. The third-generation languages emphasized:

- Expression of an algorithm in a way that was independent of the characteristics of the machine on which the algorithm would run.
- The rise of strong typing – by which typed languages deprecated or severely controlled access to the underlying storage representation of data. Complete prohibition of such access has never been a feature of major-use programming languages, which generally simply provide barriers to accidental access, *e.g.* coding them as “native” methods.
- Block structure and automated management of storage with a stack – introduced in the Algol family of languages and adopted rapidly by most other major modular languages
- Broad-spectrum applicability and greatly extended functionality – which was intended to service the needs of not only the previously separated commercial and scientific domains. The extended functionality often included concurrency features, creation and reference to non-stack data,

RE-CHARACTERIZATION OF FIRST THREE GENERATIONS

Since the 1990s, some authors have recharacterized the development of programming languages in a way that removed the (no longer topical) distinctions between early high-level languages like Fortran or Cobol and later ones, like

First Generation

In this categorization, a *first-generation programming language* is a machine-level programming language. Originally, no translator was used to compile or assemble the first-generation language. The first-generation programming instructions were entered through the front panel switches of the computer system.

The main benefit of programming in a first-generation programming language is that the code a user writes can run very fast and efficiently, since it is directly executed by the CPU. However, machine language is a lot more difficult to learn than higher generational programming languages, and it is far more difficult to edit if errors occur. In addition, if instructions need to be added into memory at some location, then all the instructions after the insertion point need to be moved down to make room in memory to accommodate the new instructions. Doing so on a front panel with switches can be very difficult.

Second Generation

Second-generation programming language is a generational way to categorize assembly languages. The term was coined to provide a distinction from higher level

third-generation programming languages (3GL) such as COBOL and earlier machine code languages. Second-generation programming languages have the following properties:

- The code can be read and written by a programmer. To run on a computer it must be converted into a machine readable form, a process called assembly.
- The language is specific to a particular processor family and environment.

Second-generation languages are sometimes used in kernels and device drivers (though C is generally employed for this in modern kernels), but more often find use in extremely intensive processing such as games, video editing, graphic manipulation/rendering. One method for creating such code is by allowing a compiler to generate a machine-optimized assembly language version of a particular function. This code is then hand-tuned, gaining both the brute-force insight of the machine optimizing algorithm and the intuitive abilities of the human optimizer.

Third Generation

A *third-generation programming language (3GL)* is a refinement of a second-generation programming language. Whereas a second generation language is more aimed to fix logical structure to the language, a third generation language aims to refine the usability of the language in such a way to make it more user friendly. This could mean restructuring categories of possible functions to make it more efficient, condensing the overall bulk of code via classes (*e.g.* Visual Basic). A third generation language improves over a second generation language by having more refinement on the usability of the language itself from the perspective of the user.

First introduced in the late 1950s, FORTRAN, ALGOL and COBOL are early examples of this sort of language.

Most “modern” languages (BASIC, C, C++, C#, Pascal, and Java) are also third-generation languages.

Most 3GLs support structured programming.

LATER GENERATIONS

“Generational” classification of these languages was abandoned after the third-generation languages, with the natural successors to the third-generation languages being termed object-oriented. C gave rise to C++ and later to Java and C#, Lisp to CLOS, ADA to ADA95, and even COBOL to COBOL2002, and new languages have emerged in that “generation” as well.

But significantly different languages and systems were already being called fourth and fifth generation programming languages by language communities with special interests. The manner in which these generations have been put forward tends to differ in character from those of earlier generations, and they represent software points-of-view leading away from the mainstream.

Bibliography

- Abelson, H. and G. J. Sussman with Julie Sussman.: *Structure and Interpretation of Computer Programs* (2nd ed.). Cambridge, MA: MIT Press, 1996.
- Aggoun, A. and N. Beldiceanu.: “Overview of the CHIP Compiler System,” in K. Furukawa (Ed.) *Logic Programming, Proceedings of the Eighth International Conference*, Paris, France, Cambridge: MIT Press, 1991.
- ANSI-X3.: *Information Technology—Programming Language—Common Lisp*. Washington, DC: American National Standards Institute, 1994.
- Appel, A. and D. MacQueen.: “Separate Compilation for Standard ML,” *SIGPLAN Conference on Programming Language Design and Implementation*, 1994.
- Backus, J. W. et al.: “The FORTRAN Automatic Coding System.” *Proceedings of the Western Joint Computing Conference*, Reprinted in Rosen, 1957.
- Barnes, J.: *Programming in Ada*, London: Pearson Education Limited, 2005-2006.
- Barron, D. W.: *An Introduction to the Study of Programming Languages*, Cambridge, UK: Cambridge University Press, 1977.
- Birnes, W. J. (Ed.): *High-Level Languages and Software Applications*. New York: McGraw-Hill, 1989.
- Bridges, D. and F. Richman.: *Varieties of Constructive Mathematics*, London Mathematical Society Lecture Note Series #97. Cambridge, UK: Cambridge University Press, 1987.
- Burstall, R., D. MacQueen, and D. Sanella.: *HOPE: An Experimental Applicative Language*, Report CSR-62-80, Computer Science Department, Edinburgh University, Scotland, 1980.

- Cardelli, L., J. Donahue, M. Jordan, B. Kaslow, and G. Nelson.: “*The Modula-3 Type System.*” *Sixteenth Annual ACM Symposium on Principles of Programming Languages*, 1989.
- Demers, A. J., J. E. Donahue, and G. Skinner.: “*Data Types as Values: Polymorphism, Type-checking, Encapsulation.*” *Conference Record of the Fifth Annual ACM Symposium on Principles of Programming Languages*, New York: ACM Press, 1978.
- Dijkstra, E. W. and C. S. Scholten.: *Predicate Calculus and Program Semantics (Texts and Monographs in Computer Science)*. New York: Springer-Verlag, 1990.
- Ellis, M. A. and B. Stroustrup.: *The Annotated C++ Reference Manual*, Reading, MA: Addison-Wesley, 1990.
- Feinberg, N., Ed.: *Dylan Programming: An Object-Oriented and Dynamic Language*, Reading, MA: Addison-Wesley, 1996.
- Friedman, D. P., C. T. Haynes, and E. Kohlbecker.: “*Programming with Continuations.*” In P. Pepper (Ed.), *Program Transformations and Programming Environments*. New York: Springer-Verlag, 1985.
- Futatsugi, K., J. Goguen, J.-P. Jouannaud, and J. Meseguer.: “*PRINCIPLES OF OBJ2*” In *Proceedings of the ACM Symposium on Principles of Programming Languages*, 1985.
- Goguen, J. and G. Malcolm (Eds.): *Software Engineering with OBJ: Algebraic Specification in Action*. Amsterdam: Kluwer Academic Publishers, 2000.
- Gunter, C. A.: *Semantics of Programming Languages: Structures and Techniques (Foundations of Computing Series)*. Cambridge, MA: MIT Press, 1992.
- Halstead, R. H., Jr.: “*Multilisp: A Language for Concurrent Symbolic Computation.*” *ACM Transactions on Programming Languages and Systems*, 1985.
- Hankin, C.: *Lambda Calculi: A Guide for Computer Scientists (Graduate Texts in Computer Science, No 3)*. Oxford: Clarendon Press, 1995.
- Harper, R. and Mitchell, J.: “*On the Type Structure of Standard ML.*” *ACM Transactions on Programming Languages and Systems*, 1993.
- Knuth, D. E. and L. Trabb Pardo.: “*Early Development of Programming Languages.*” In *Encyclopedia of Computer Science and Technology*, New York: Marcel Dekker, 1977.
- Koenig, A. and B. Stroustrup.: “*Exception Handling for C++.*” *Proceedings of the USENIX C++ Conference*, San Francisco, 1990.
- Lampson, B. W., J. J. Horning, R. L. London, J. G. Mitchell, and G. J. Popek.: “*Report on the Programming Language Euclid.*” *Technical Report CSL-81-12*. Palo Alto, CA: Xerox Palo Alto Research Center, 1981.
- Lesk, M. E.: “*Lex—A Lexical Analyzer Generator.*” *Computing Science Technical Report*, Murray Hill, NJ: AT&T Bell Laboratories, 1975.
- Liskov, B. and A. Snyder.: “*Exception Handling in CLU.*” *IEEE Transactions on Software Engineering* SE-5(6): 546–558. Also reprinted in Horowitz, 1979.

- Liskov, B., R. Atkinson, T. Bloom, E. Moss, J. C. Schaffert, R. Scheifler, and A. Snyder.: *CLU Reference Manual*, New York: Springer-Verlag, 1984.
- MacQueen, D. B.: “*The Implementation of Standard ML Modules.*” *ACM Conference on Lisp and Functional Programming*, New York: ACM Press, 1988.
- Minker J. (Ed.): *Logic-Based Artificial Intelligence The Kluwer International Series in Engineering and Computer Science*, Amsterdam: Kluwer Academic Publishers, 2000.
- Moon, D. A.: “*Garbage Collection in Large Lisp Systems.*” *Proceedings of the 1984 ACM Symposium on Lisp and Functional Programming*, New York: ACM Press, 1984.
- Pountain, D.: “*Constraint Logic Programming: A Child of Prolog Finds New Life Solving Programming’s Nastier Problems,*” *BYTE*, February, 1995.
- Rosser, J. B.: “*Highlights of the History of the Lambda Calculus.*” *Proceedings of the ACM Symposium on Lisp and Functional Programming*, New York: ACM Press, 1982.
- Rubin, F.: “*‘GOTO Statement Considered Harmful’ Considered Harmful*” *ACM* 30(3), pp. 195–196. Replies in the June, July, August, November, and December, 1987.
- Shapiro, E. and D. H. D. Warren (Eds.): “*Special Section: The Fifth Generation Project: Personal Perspectives, Launching the New Era, and Epilogue,*” *Communications of the ACM*, 1993.

Index

A

Accuracy 212, 223
Alias 165, 254
Allocate 38, 164, 223, 230, 242, 243, 257
Allocation 37, 242, 257
Analogous 41
Analogue 203
Annotation 40, 162, 163
Anonymous 166
Argument 24, 27, 81, 82, 84, 85, 86, 87, 91, 92, 93
Arithmetic Operators 220
Array 76, 81, 82, 83, 87, 91, 218
Assigning 187
Assignment Operators 220
Assignments 185, 186, 187, 194

B

Binary 184, 190, 193, 196, 197, 198
Binary Streams 88
Buffering 89, 90, 92

C

Coincidence 72
 Communication 188
 Compilation 25, 77, 118, 119, 120, 158, 227
 Compilation Logistics 77
 Compile 38, 39, 45, 118, 119, 120, 123, 124, 125, 126, 127, 133, 136, 161, 163, 164, 170,
 224, 225, 228, 230, 235, 243, 244, 246, 247, 248, 251, 258
 Computation 39, 40, 73, 75, 117, 119, 169
 Concatenation 74, 151, 152
 Concisely 75
 Concrete 41, 42, 43, 46, 129, 130, 150, 159, 168, 233
 Concurrency 58
 Constants 215, 217
 Contiguous 164
 Control Flow 27, 28, 29, 30, 32
 Convenience 161
 Convenient 123, 141, 173, 249
 Convention 73, 118, 150
 Conversion 82, 83, 84, 85, 86, 87

D

Data 176, 181, 191, 192, 193, 194, 195, 200, 201, 202, 205, 206, 207, 208
 Declarations 32, 75, 76, 78, 85, 88, 218, 219
 Definition 207
 Destructor 37, 38, 41, 42, 222, 223, 225, 236, 245, 256
 Dynamic Semantics 270
 Dynamical 190

E

Efficiency 119
 Encapsulate 61
 Encapsulation 35, 36
 Evaluated 70, 74, 120, 136, 138, 139, 140, 146, 148, 152, 154, 173, 212, 213, 214, 215
 Evaluation 74, 169, 213
 Execute 40, 68, 69, 72, 118, 121, 122, 124, 146, 147, 149, 151, 152, 153, 223, 224, 229,
 245, 255
 Execution 69, 71, 122, 130, 139, 148, 150, 153, 157, 222, 244, 255
 Explicit 126, 131, 132, 139, 142, 155, 168, 215, 223, 225, 228, 233, 235, 247, 250, 258
 Exponent 69, 73, 74, 131, 212
 Expression Statements 28

F

Floating 83

Fortran 117, 118, 119

Fraction 73

Freopen 91

Function 175, 176, 178, 179, 180, 182, 183, 185, 186, 187, 188, 190, 191, 194, 199, 200,
201, 202, 203, 204, 206, 207, 208, 209

Function Calls 221

Function Prototypes 75

G

Getchar 186, 198, 203

H

Hierarchy 41, 168, 169

I

Illegal 74, 75, 170

Implementation 58, 61, 62, 63, 64

Implemented 35, 36, 42, 168, 229

Implicit 45, 215, 225, 228, 230, 247, 258

Indistinct 229, 246

Inheritance 35, 36, 43, 56, 231, 233, 235, 237, 238, 250

Initialization 40, 129, 149, 158, 229, 231

Input 25, 79, 80, 81, 82, 85, 86, 87, 90, 92

Instantiation 43, 248, 251

Integral 83, 89

J

JavaScript 283

K

Keyboard 199

L

Legible 123, 141

Logical 194, 196, 197

Loops 32, 187

M

- Macros 175, 203, 204
- Manipulation 90
- Modern Programming 264
- Myexception 256, 257

O

- Output 22, 23, 24, 25, 82, 83, 85, 89, 90, 92, 93

P

- Parentheses 23, 24, 27, 32
- Polymorphism 35, 36, 42, 168, 237, 238, 242
- Priority 140
- Procedural 35
- Program Structure 31
- Programming Languages 278
- Prototypes 75
- Punctuation 125
- Putchar 189, 203

Q

- Quotation 73, 131

R

- Reading Lines 79
- Reading Numbers 81
- Repetition 74

S

- Scanf 179, 190, 197, 200, 201, 202, 203, 206
- Statements 24, 27, 28, 29, 30, 31, 32, 76, 218
- Static Semantics 270
- Strings 81, 218
- Symbolic 39, 119
- Syntactic 158
- Syntax 185, 195, 201, 202

T

- Text Streams 88

U

Unix 173, 174

V

Variable Names 219

Variables 26, 32, 76, 174, 180, 190, 194, 196, 202, 215, 217, 218, 219, 221

Versatile 184

Virtual 36, 41, 42, 126, 168, 239, 240, 241, 242, 243, 256
